Unsupervised Learning Handbook

Clustering, Mixture Models, Principal Component Analysis and Anomaly Detection: a self-contained guide, derived step by step

Udit Bhatia ES 335 / ES 678 Machine Learning

A self-contained handbook

How to use this handbook

This handbook develops unsupervised learning from first principles and can be read on its own. Its mathematical prerequisites are stated in Chapter 1, *Mathematical background*. Chapters 2 to 5 correspond to four classes: Chapter 2 (clustering with K-means), Chapter 3 (mixture models, hierarchical and density-based clustering, and the evaluation of a clustering), Chapter 4 (principal component analysis and maps for visualisation) and Chapter 5 (anomaly detection). Each of these chapters ends with fifteen practice problems, marked *Basic* (direct use of a definition or formula), *Standard* (a calculation or proof of several steps) or *Challenge* (a harder proof or a question that needs judgement); complete solutions are in Chapter 6. Figure 1 shows the three questions that the chapters answer.

Examples. The small examples are worked in full and can be checked with a calculator; the larger examples use data generated for illustration.

Sections marked *optional* may be skipped on a first reading.

Notebook. The classroom demonstrations run in Google Colab, in the browser, with nothing to install: open the notebook in Colab. Use *File, Save a copy in Drive* to run and change it.

Algorithms. Each method is stated as a numbered algorithm with its inputs and outputs, followed by its cost.

Proofs. The main results are proved: the convergence of Lloyd's algorithm, the linear boundaries of K-means clusters, the decomposition of the total sum of squares, Ward's merge cost, the monotonicity of EM and its M-step, the characterisation of principal components, the equivalence of maximum variance and minimum reconstruction error, and the properties of the Mahalanobis distance. Results stated without proof carry a reference.

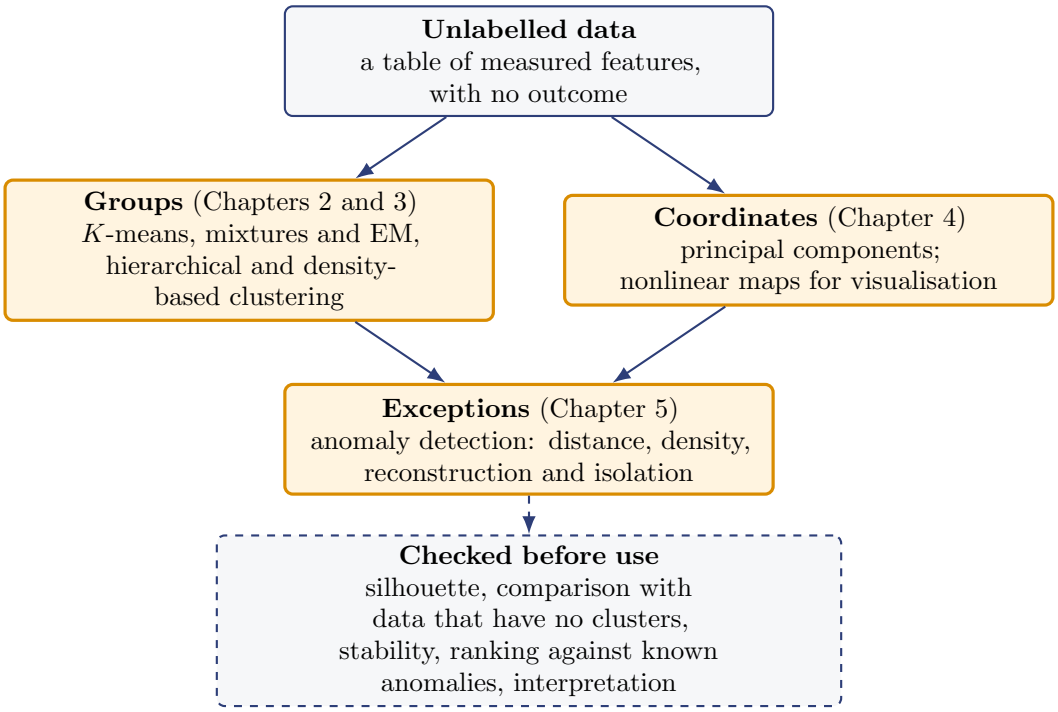


Figure 1: Structure of the handbook. Unsupervised learning asks three questions of a feature table: which examples belong together, which coordinates describe the data economically, and which examples do not fit.

Reading this book in 2026

Most of the data that machine-learning systems use carry no labels. A hospital stores millions of scans but has expert annotations for a few thousand; a factory logs every sensor every second but records a fault only when one is noticed; a telescope survey, a genome sequencer and a payments

network produce far more measurements than anyone will ever label. Unsupervised learning is the part of the subject that works with such data directly, and in 2026 it runs inside many systems whose visible purpose is supervised prediction.

In the sciences. Single-cell biology reduces the expression of thousands of genes in each of hundreds of thousands of cells with principal component analysis, clusters the cells into types, and inspects the result on a two-dimensional map of the kind described in Chapter 4. Climate science has used principal components for decades under the name *empirical orthogonal functions*, and groups daily weather patterns into a small number of regimes with K-means. Astronomy, materials science and genomics use the same tools to find groups and outliers in large surveys before any hypothesis is formulated.

In engineering and industry. Anomaly detection monitors machines, power grids, networks and payments, where faults, intrusions and fraud are rare and change form, so that examples of every failure can never be collected in advance (Chapter 5). Customer and product segmentation, the grouping of documents and images for search, and the removal of near-duplicate records all rest on clustering.

Inside modern machine-learning systems. The largest models in vision, speech, language and biology are now trained first without labels, by *self-supervised learning*, and the vectors they produce (embeddings) have become the usual input to the methods of this book (Section 4.5, *Unsupervised learning on learned representations*). Clustering embeddings is used to curate training data, by removing near-duplicates and balancing over-represented topics; search engines over millions of embeddings use K-means to divide the space and to compress vectors, so that nearest neighbours can be found without comparing every pair. Low-rank approximation, the idea behind principal component analysis, reappears in methods that fine-tune or compress large models by changing only a few directions of their weight matrices. Deployed models are monitored by asking whether incoming data still resemble the training data, which is anomaly detection applied to inputs and embeddings (Section 5.6, *Anomaly detection in deployed machine-learning systems*).

What has changed, and what has not. The data are larger, and the features are often learned vectors with hundreds or thousands of dimensions rather than a few measured quantities. Scale makes approximate and mini-batch versions of the algorithms common, and high dimension makes the questions of distance, scaling and the curse of dimensionality (Section 4.1, *Reducing the number of features*) more pressing, not less. The mathematics has not changed: a sum of squared distances to be minimised, an eigenvalue problem, a likelihood maximised by EM, a score with a threshold. Nor has the main difficulty. Without labels every method returns an answer, and the answer must be checked against data that have no structure, against resampling and against what the groups, directions or exceptions mean. This book develops the methods from first principles so that the reader can use them, adapt them, and judge when their results deserve trust.

Contents

Reading this book in 2026 . . . 1

1 Mathematical background **5**

1.1 Supervised and unsupervised learning . . . 5

1.2 Euclidean distance . . . 5

1.3 Mean, variance and standardisation . . . 6

1.4 Minimising a function of one variable . . . 6

1.5 The normal distribution . . . 7

1.6 Bayes' rule . . . 7

1.7 Likelihood and log-likelihood . . . 7

1.8 Projections and the covariance matrix . . . 7

1.9 Eigenvalues and eigenvectors of a symmetric matrix . . . 15

1.10 Constrained optimisation with a Lagrange multiplier . . . 19

1.11 Jensen's inequality for the logarithm . . . 20

1.12 Median and robust measures of spread . . . 20

2 Clustering with K-means **21**

2.1 Eight training runs . . . 21

2.2 The K-means objective . . . 22

2.3 Lloyd's algorithm . . . 24

2.4 A worked example . . . 29

2.5 Using K-means in practice . . . 31

2.6 Where clustering is used . . . 38

2.7 Practice problems for Chapter 2 . . . 39

3 Mixture models, hierarchical and density-based clustering **41**

3.1 Gaussian mixtures and the EM algorithm . . . 41

3.2 Agglomerative hierarchical clustering . . . 49

3.3 Density-based clustering: DBSCAN and HDBSCAN . . . 53

3.4 Evaluating a clustering . . . 56

3.5 Choosing a clustering method . . . 58

3.6 Practice problems for Chapter 3 . . . 58

4 Principal component analysis and maps for visualisation **60**
4.1 Reducing the number of features . . . 60
4.2 Principal components . . . 61
4.3 Using principal component analysis . . . 67
4.4 Nonlinear maps for visualisation . . . 70
4.5 Unsupervised learning on learned representations . . . 74
4.6 Practice problems for Chapter 4 . . . 75

5 Anomaly detection **77**
5.1 Anomalies and anomaly scores . . . 77
5.2 Scores for one feature . . . 78
5.3 Two features: distance, shape and neighbours . . . 80
5.4 Isolation forest . . . 83
5.5 Choosing a method, and evaluating with few labels . . . 85
5.6 Anomaly detection in deployed machine-learning systems . . . 87
5.7 Practice problems for Chapter 5 . . . 87

6 Solutions and reference **89**
6.1 Solutions to the practice problems . . . 89
6.2 Common questions . . . 96
6.3 Glossary . . . 97
6.4 Summary of formulas and results . . . 99

Chapter 1

Mathematical background

1.1 Supervised and unsupervised learning

Intuition: sorting LEGO pieces

A child given a box of LEGO pieces with no instructions still sorts them: by colour, by size, or by shape, whichever seems natural. If a parent says "wheels go here, windows go there" and shows a few examples, the child is learning with labels; this is supervised learning. Without such instructions the child must decide what makes pieces similar; this is unsupervised learning, and the groups that result are clusters.

An example is described by a list of **features**, numbers recorded about it: for one *training run* of a model, quantities such as its training time and its peak memory. In **supervised** learning each example also carries a **label** y, the outcome to be predicted, and the quality of a model is measured by how well its predictions agree with the labels. In **unsupervised** learning no labels are available, and the aim is to describe the structure of the features themselves.

Clustering is the unsupervised problem of dividing the examples into groups, called **clusters**, such that examples in the same cluster are similar to one another and examples in different clusters are not. Because there are no labels, there is no error rate with which to judge a clustering, and other measures are needed.

Use in machine learning. Every method in this book works without an outcome variable: K-means (Chapter 2, *Clustering with K-means*), mixture models, hierarchical and density-based clustering (Chapter 3, *Mixture models, hierarchical and density-based clustering*), principal component analysis and maps for visualisation (Chapter 4, *Principal component analysis and maps for visualisation*), and anomaly detection (Chapter 5, *Anomaly detection*). The same tools also serve supervised learning, for example to reduce the number of features before a predictor is trained (Section 4.3.4, *From PCA to clustering*).

1.2 Euclidean distance

An example with d features is a point $\mathbf{x} = (x_1, \ldots, x_d)$. The **Euclidean distance** between two points is

$$\|\mathbf{x} - \mathbf{y}\| = \sqrt{(x_1 - y_1)^2 + \cdots + (x_d - y_d)^2},$$

and its square, $\|\mathbf{x} - \mathbf{y}\|^2 = (x_1 - y_1)^2 + \cdots + (x_d - y_d)^2$, is the **squared distance**. For example, from $(4, 4)$ to $(1, 3)$ the squared distance is $(4-1)^2 + (4-3)^2 = 9 + 1 = 10$, and the distance is $\sqrt{10} \approx 3.162$. Since the square root is increasing, the point nearest to $\mathbf{x}$ is the same whether distances or squared distances are compared.

Use in machine learning. Distance is how these methods measure similarity. K-means assigns each point to the nearest centroid (Section 2.3.3, *Optimal partition for fixed centroids*), hierarchical clustering merges the nearest clusters (Section 3.2, *Agglomerative hierarchical clustering*), DBSCAN counts the neighbours within a radius (Section 3.3, *Density-based clustering: DBSCAN and HDBSCAN*), and nearest-neighbour anomaly scores measure how isolated a point is (Section 5.3, *Two features: distance, shape and neighbours*). Because the squared distance adds the contributions of the coordinates, a feature measured on a large numerical scale can dominate it (Section 2.5.3, *Dependence on the scale of the features*).

1.3 Mean, variance and standardisation

The **mean** of numbers $x_1, \dots, x_m$ is $\bar{x} = \frac{1}{m}\sum_j x_j$. Their **variance** is the mean squared deviation from the mean, $\frac{1}{m}\sum_j (x_j - \bar{x})^2$, and the square root of the variance is the **standard deviation** s, which is measured in the same units as the x_j. The mean of a set of points is computed coordinate by coordinate; for instance, the mean of $(4, 4)$ and $(4, 3)$ is $(4, 3.5)$.

A feature is **standardised** by subtracting its mean and dividing by its standard deviation, $z_j = (x_j - \bar{x})/s$. The standardised feature has mean 0, since $\frac{1}{m}\sum_j z_j = \frac{1}{s}\big(\frac{1}{m}\sum_j x_j - \bar{x}\big) = \frac{1}{s}(\bar{x} - \bar{x}) = 0$. Its variance is therefore the mean of its squares, $\frac{1}{m}\sum_j z_j^2 = \frac{1}{s^2}\cdot\frac{1}{m}\sum_j (x_j - \bar{x})^2 = \frac{s^2}{s^2} = 1$, so its standard deviation is 1. It has no units, because the numerator and the denominator are measured in the same units: a value of $z_j = 2$ means two standard deviations above the mean, whether the feature was originally measured in hours or in megabytes.

Use in machine learning. The mean is the best centroid of a cluster (Section 2.3.1, *Optimal centroids for a fixed partition*); variance is the quantity that principal component analysis keeps (Section 4.2.1, *Projection onto a direction and the variance it retains*); standardisation puts features on a common scale before clustering or principal component analysis (Section 2.5.3, *Dependence on the scale of the features*, Section 4.3.2, *The effect of scaling on PCA*); and the standard score is the simplest anomaly score (Section 5.2.1, *Standard scores and robust scores*).

1.4 Minimising a function of one variable

If a differentiable function $f(c)$ has a minimum at an interior point, its derivative is zero there, $f'(c) = 0$. Conversely, if $f'(c) = 0$ and $f''(c) > 0$, then c is a local minimum. A quadratic $f(c) = ac^2 + bc + e$ with $a > 0$ has exactly one stationary point, and it is the global minimum. Completing the square,

$$ac^2 + bc + e = a\Big(c^2 + \frac{b}{a}c\Big) + e = a\Big(c + \frac{b}{2a}\Big)^2 + e - \frac{b^2}{4a},$$

and since $a > 0$ the squared term is never negative and is zero only at $c = -b/(2a)$. This is also the only solution of $f'(c) = 2ac + b = 0$. The derivative of a sum is the sum of the derivatives, and since $(x - c)^2 = x^2 - 2xc + c^2$, its derivative with respect to c is $-2x + 2c = -2(x - c)$.

Use in machine learning. Setting a derivative to zero gives the best centroid of a cluster (Section 2.3.1, *Optimal centroids for a fixed partition*) and the updates of the M-step of EM (Section 3.1.3, *The M-step: updating the means*, Section 3.1.6, *General Gaussian mixtures*); the second-derivative test confirms that each of these stationary points is a minimum or a maximum.

1.5 The normal distribution

A quantity that varies around a central value μ with spread σ is often modelled by the **normal distribution**, whose density is

$$\mathcal{N}(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big).$$

The density is largest at $x = \mu$ and decreases symmetrically on either side; about 68% of the probability lies within one standard deviation σ of μ. A density is not itself a probability (probabilities are areas under it), but it measures how well a value is explained: a value close to μ has a high density, and a value several σ away has a very small one.

Use in machine learning. The normal distribution is the building block of a Gaussian mixture (Section 3.1.1, *A probabilistic model for clustering*), the model behind density-based anomaly scores (Section 5.2.2, *Scoring by density under a fitted model*), and the reference that calibrates the robust score (Section 5.2.1, *Standard scores and robust scores*); t-SNE also uses a Gaussian to define the neighbourhood of each point (Section 4.4.1, *t-SNE*).

1.6 Bayes' rule

Suppose an observation x was produced by one of several sources k, with prior probabilities $P(k)$ and densities $p(x \mid k)$. Then the probability of source k given the observation is

$$P(k \mid x) = \frac{P(k)\, p(x \mid k)}{\sum_j P(j)\, p(x \mid j)}.$$

The numerator is the prior probability of the source multiplied by how well the source explains x; the denominator ensures that the probabilities sum to one.

Use in machine learning. Bayes' rule gives the responsibilities of a mixture model, the probability that each component produced a given example (Section 3.1.2, *The E-step: responsibilities*). These soft memberships are what distinguish a mixture model from K-means.

1.7 Likelihood and log-likelihood

For independent observations $x_1, \ldots, x_n$, the **likelihood** of a model is the product $\prod_i p(x_i)$ of the densities it assigns to them. The **log-likelihood** $\sum_i \log p(x_i)$ is easier to differentiate and avoids numerical underflow. Since the logarithm is increasing, maximising the log-likelihood is equivalent to maximising the likelihood.

Use in machine learning. Fitting a mixture model means maximising its log-likelihood, which the EM algorithm increases at every iteration (Section 3.1.4, *The EM algorithm*, Section 3.1.5, *Why the log-likelihood never decreases (optional)*); log-densities under a fitted model are also anomaly scores (Section 5.2.2, *Scoring by density under a fitted model*).

1.8 Projections and the covariance matrix

Projection onto a line. Let $\mathbf{u}$ be a unit vector, $\|\mathbf{u}\| = 1$. The line through the origin in direction $\mathbf{u}$ consists of the points $t\,\mathbf{u}$ for real t: each is the vector $\mathbf{u}$ multiplied by the number t. For a point $\mathbf{x}$, the difference $\mathbf{x} - t\,\mathbf{u}$, taken coordinate by coordinate, is the vector from the point $t\,\mathbf{u}$ of the line to $\mathbf{x}$, and its length $\|\mathbf{x} - t\,\mathbf{u}\|$ is the distance between them; for example, with $\mathbf{x} = (2, 3)$, $\mathbf{u} = (0.6,\ 0.8)$ and $t = 2$, the point of the line is (1.2, 1.6), the difference is (0.8, 1.4) and the distance

is $\sqrt{0.8^2 + 1.4^2} = \sqrt{2.6} \approx 1.61$. The point of the line nearest to $\mathbf{x}$ is given by the value of t that makes this distance smallest. Since distances are nonnegative and squaring preserves the order of nonnegative numbers, the same t makes the *squared* distance smallest, and the squared distance is easier to work with because it contains no square root. Writing a squared length as a dot product with itself, $\|\mathbf{a}\|^2 = \mathbf{a}^\top\mathbf{a}$, and using $\mathbf{x}^\top\mathbf{u} = \mathbf{u}^\top\mathbf{x}$,

$$g(t) = \|\mathbf{x} - t\,\mathbf{u}\|^2 = (\mathbf{x} - t\,\mathbf{u})^\top(\mathbf{x} - t\,\mathbf{u}) = \mathbf{x}^\top\mathbf{x} - 2t\,\mathbf{u}^\top\mathbf{x} + t^2\,\mathbf{u}^\top\mathbf{u} = \|\mathbf{x}\|^2 - 2t\,\mathbf{u}^\top\mathbf{x} + t^2,$$

the last step because $\mathbf{u}^\top\mathbf{u} = \|\mathbf{u}\|^2 = 1$. This is a quadratic in t with positive leading coefficient, so its minimum is where the derivative vanishes: $g'(t) = -2\,\mathbf{u}^\top\mathbf{x} + 2t = 0$ gives $t = \mathbf{u}^\top\mathbf{x}$. The nearest point, $(\mathbf{u}^\top\mathbf{x})\,\mathbf{u}$, is the **projection** of $\mathbf{x}$ onto the line, and the number $\mathbf{u}^\top\mathbf{x}$, its signed position along the line, is the **score** of $\mathbf{x}$ in direction $\mathbf{u}$. The residual $\mathbf{r} = \mathbf{x} - (\mathbf{u}^\top\mathbf{x})\,\mathbf{u}$ is perpendicular to $\mathbf{u}$, because $\mathbf{u}^\top\mathbf{r} = \mathbf{u}^\top\mathbf{x} - (\mathbf{u}^\top\mathbf{x})(\mathbf{u}^\top\mathbf{u}) = 0$. Write $s = \mathbf{u}^\top\mathbf{x}$ for the score, which is a number, so that the definition of the residual reads $\mathbf{x} = s\,\mathbf{u} + \mathbf{r}$: the point is its projection plus its residual. The squared length of $\mathbf{x}$ is the dot product of this sum with itself. The transpose of a sum is the sum of the transposes, $(s\,\mathbf{u} + \mathbf{r})^\top = s\,\mathbf{u}^\top + \mathbf{r}^\top$, and multiplying out the two brackets term by term, exactly as $(a+b)(c+d) = ac + ad + bc + bd$ for numbers, with the number s moved in front of each product, gives

$$\begin{aligned}\|\mathbf{x}\|^2 &= (s\,\mathbf{u} + \mathbf{r})^\top(s\,\mathbf{u} + \mathbf{r}) \\ &= s^2\,\mathbf{u}^\top\mathbf{u} + s\,\mathbf{u}^\top\mathbf{r} + s\,\mathbf{r}^\top\mathbf{u} + \mathbf{r}^\top\mathbf{r}.\end{aligned}$$

The two middle terms are equal, because a dot product does not depend on the order of its factors, $\mathbf{r}^\top\mathbf{u} = \sum_j r_j u_j = \mathbf{u}^\top\mathbf{r}$; they combine into $2s\,\mathbf{u}^\top\mathbf{r}$. Now $\mathbf{u}^\top\mathbf{u} = \|\mathbf{u}\|^2 = 1$ because $\mathbf{u}$ is a unit vector, $\mathbf{u}^\top\mathbf{r} = 0$ because the residual is perpendicular to $\mathbf{u}$, and $\mathbf{r}^\top\mathbf{r} = \|\mathbf{r}\|^2$, so

$$\|\mathbf{x}\|^2 = s^2\,\mathbf{u}^\top\mathbf{u} + 2s\,\mathbf{u}^\top\mathbf{r} + \mathbf{r}^\top\mathbf{r} = s^2 \times 1 + 2s \times 0 + \|\mathbf{r}\|^2 = s^2 + \|\mathbf{r}\|^2.$$

The cross term vanishes, and this is the whole reason the squared lengths add. In the example above, $\mathbf{x} = (2, 3)$ and $\mathbf{u} = (0.6,\ 0.8)$ give $s = 0.6 \times 2 + 0.8 \times 3 = 3.6$, projection $s\,\mathbf{u} = (2.16,\ 2.88)$ and residual $\mathbf{r} = (2 - 2.16,\ 3 - 2.88) = (-0.16,\ 0.12)$. The residual is perpendicular to $\mathbf{u}$, since $\mathbf{u}^\top\mathbf{r} = 0.6 \times (-0.16) + 0.8 \times 0.12 = -0.096 + 0.096 = 0$, and indeed $s^2 + \|\mathbf{r}\|^2 = 12.96 + (0.0256 + 0.0144) = 12.96 + 0.04 = 13 = 2^2 + 3^2 = \|\mathbf{x}\|^2$. Substituting $s = \mathbf{u}^\top\mathbf{x}$ and $\mathbf{r} = \mathbf{x} - (\mathbf{u}^\top\mathbf{x})\,\mathbf{u}$ gives Pythagoras' theorem in d dimensions:

$$\|\mathbf{x}\|^2 = (\mathbf{u}^\top\mathbf{x})^2 + \big\|\mathbf{x} - (\mathbf{u}^\top\mathbf{x})\,\mathbf{u}\big\|^2.$$

This splitting of a squared length into a part along a direction and a part perpendicular to it is used throughout the book. In principal component analysis it shows that keeping as much variance as possible is the same as making the reconstruction error as small as possible (Section 4.2.3, *Scores and reconstruction error*), and it gives the reconstruction error when several components are kept (Section 4.2.4, *Several principal components*). With the cluster means in place of the line, the same expansion gives the decomposition of the total sum of squares in K-means (Section 2.3.2, *Within-cluster and between-cluster sums of squares*). In anomaly detection the perpendicular part is the distance of an example from the principal line, and in Mahalanobis distance the two parts are weighted separately (Section 5.3, *Two features: distance, shape and neighbours*). Figure 1.1 shows the split for run 6 of the eight training runs of Section 2.1, *Eight training runs*: the run, its projection onto a direction $\mathbf{u}$, and the residual, which meets the line at a right angle.

Projection onto any vector. A direction is often given by a vector $\mathbf{v}$ that does not have unit length. Dividing by its length gives the unit vector $\mathbf{u} = \mathbf{v}/\|\mathbf{v}\|$, and substituting into the projection,

$$(\mathbf{u}^\top\mathbf{x})\,\mathbf{u} = \frac{\mathbf{v}^\top\mathbf{x}}{\|\mathbf{v}\|}\,\frac{\mathbf{v}}{\|\mathbf{v}\|} = \frac{\mathbf{v}^\top\mathbf{x}}{\mathbf{v}^\top\mathbf{v}}\,\mathbf{v}.$$

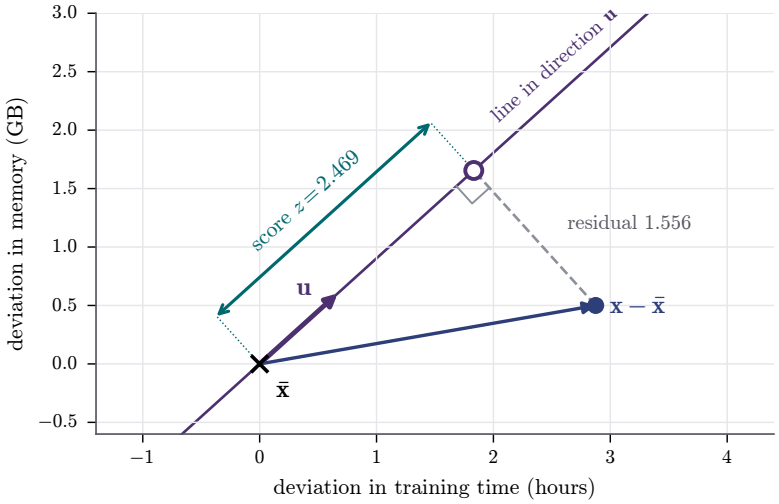


Figure 1.1: Projection of a centred point (run 6 of the eight training runs) onto a unit direction $\mathbf{u}$, here the first principal direction of the runs (Section 4.2.2, *The first principal direction*). The axes show deviations from the mean, so the mean $\bar{\mathbf{x}}$ is at the origin and the run is drawn at $\mathbf{x} - \bar{\mathbf{x}}$. The score is the signed length of the projection along $\mathbf{u}$; the residual is perpendicular to $\mathbf{u}$; the squared length of the point is the sum of the two squares.

Example: projecting $\mathbf{x} = (2, 3)$ onto $\mathbf{v} = (4, 1)$, step by step

1. **Unit vector.** $\|\mathbf{v}\| = \sqrt{4^2 + 1^2} = \sqrt{17} \approx 4.123$, so $\mathbf{u} = \mathbf{v}/\|\mathbf{v}\| = (4, 1)/\sqrt{17} \approx (0.970,\ 0.243)$, a vector of length 1 pointing the same way as $\mathbf{v}$.
2. **Score.** $\mathbf{u}^\top \mathbf{x} = (4 \times 2 + 1 \times 3)/\sqrt{17} = 11/\sqrt{17} \approx 2.668$: the foot of the perpendicular from $\mathbf{x}$ lies 2.668 units from the origin along the line.
3. **Projection.** $\mathbf{p} = (\mathbf{u}^\top \mathbf{x})\,\mathbf{u} = \dfrac{\mathbf{v}^\top \mathbf{x}}{\mathbf{v}^\top \mathbf{v}}\,\mathbf{v} = \frac{11}{17}(4, 1) = (\frac{44}{17},\ \frac{11}{17}) = (2.588,\ 0.647)$. Its length is the score: $\|\mathbf{p}\| = \frac{11}{17}\sqrt{17} = 11/\sqrt{17} \approx 2.668$.
4. **Residual.** $\mathbf{r} = \mathbf{x} - \mathbf{p} = (2 - \frac{44}{17},\ 3 - \frac{11}{17}) = (-\frac{10}{17},\ \frac{40}{17}) \approx (-0.588,\ 2.353)$. It is perpendicular to $\mathbf{v}$, since $\mathbf{v}^\top \mathbf{r} = 4 \times (-\frac{10}{17}) + 1 \times \frac{40}{17} = 0$, and its length is $\|\mathbf{r}\| = \sqrt{100 + 1600}/17 = \sqrt{1700}/17 \approx 2.425$.
5. **Pythagoras.** $(\mathbf{u}^\top \mathbf{x})^2 + \|\mathbf{r}\|^2 = \frac{121}{17} + \frac{1700}{289} = \frac{121}{17} + \frac{100}{17} = \frac{221}{17} = 13 = 2^2 + 3^2 = \|\mathbf{x}\|^2$; in decimals, $7.118 + 5.882 = 13$.

The score and the angle. The three points $\mathbf{0}$, $\mathbf{p}$ and $\mathbf{x}$ form a triangle with a right angle at $\mathbf{p}$, because the side from $\mathbf{p}$ to $\mathbf{x}$ is the residual and is perpendicular to the line (Figure 1.2). Its longest side, the hypotenuse, runs from $\mathbf{0}$ to $\mathbf{x}$ and has length $\|\mathbf{x}\|$; the side along the line has length equal to the score; the third side has length $\|\mathbf{r}\|$. Let θ be the angle at the origin between $\mathbf{x}$ and the line. In a right-angled triangle the cosine of an angle is the adjacent side divided by the hypotenuse, so

$$\cos\theta = \frac{\mathbf{u}^\top \mathbf{x}}{\|\mathbf{x}\|}, \qquad \text{that is,} \qquad \mathbf{u}^\top \mathbf{x} = \|\mathbf{x}\| \cos\theta.$$

In the example, $\cos\theta = 2.668/3.606 = 0.740$ and $\theta \approx 42.3°$. The score is therefore the length of the *shadow* that $\mathbf{x}$ casts on the line when light falls perpendicular to the line. If the angle between $\mathbf{x}$ and $\mathbf{u}$ exceeds 90°, the foot of the perpendicular lies on the other side of the origin, in the direction of $-\mathbf{u}$; then both the score and $\cos\theta$ are negative and the formula still holds. Because $|\cos\theta| \le 1$, a score is never longer than the vector itself. Substituting $\mathbf{u} = \mathbf{v}/\|\mathbf{v}\|$ gives the angle between two arbitrary vectors from dot products alone,

$$\cos\theta = \frac{\mathbf{v}^\top \mathbf{x}}{\|\mathbf{v}\|\,\|\mathbf{x}\|},$$

here $11/(\sqrt{17}\sqrt{13}) \approx 0.740$ again. This formula needs no picture and so holds in any number of dimensions; in machine learning it is the **cosine similarity** used to compare embedding vectors (Section 4.5, *Unsupervised learning on learned representations*).

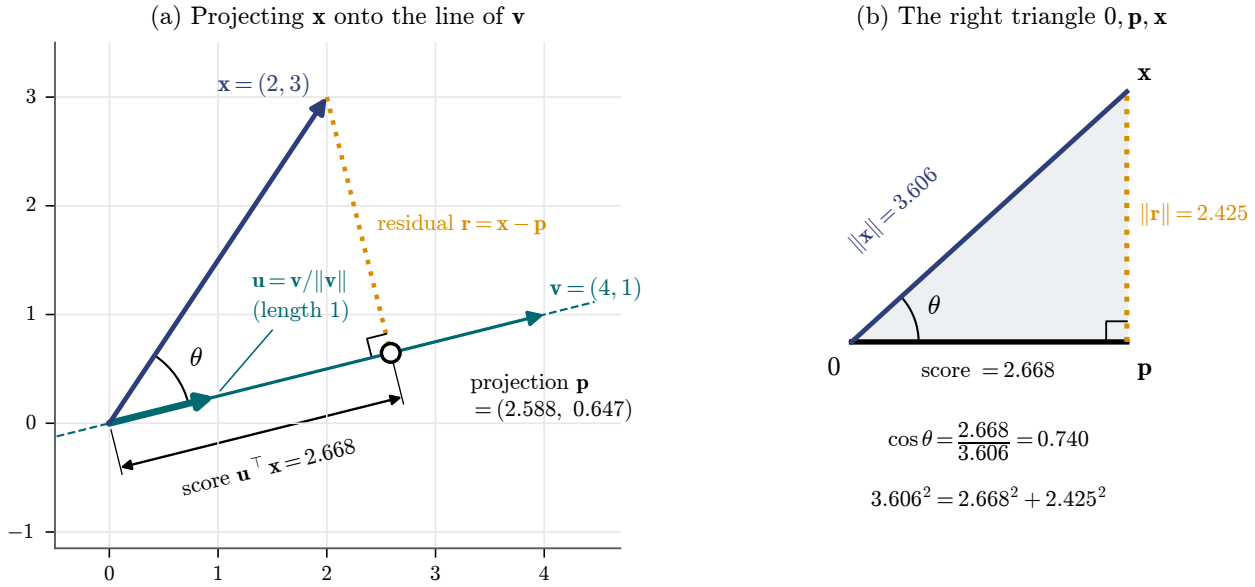


Figure 1.2: Projecting $\mathbf{x} = (2, 3)$ onto the line of $\mathbf{v} = (4, 1)$. (a) The unit vector $\mathbf{u}$ points along $\mathbf{v}$; the perpendicular from $\mathbf{x}$ meets the line at the projection $\mathbf{p}$; the score $\mathbf{u}^\top\mathbf{x} = 2.668$ is the distance from $\mathbf{0}$ to $\mathbf{p}$ along the line; the residual $\mathbf{r} = \mathbf{x} - \mathbf{p}$ meets the line at a right angle. (b) The same triangle turned so that the line is horizontal. The score is the side next to θ, so $\cos\theta = \text{score}/\|\mathbf{x}\|$, and Pythagoras gives $\|\mathbf{x}\|^2 = \text{score}^2 + \|\mathbf{r}\|^2$.

The score depends on the direction. Keep $\mathbf{x} = (2, 3)$ fixed and turn the line about the origin (Figure 1.3). A unit vector at angle φ from the first axis is $\mathbf{u} = (\cos\varphi,\ \sin\varphi)$; it has length 1 because $\cos^2\varphi + \sin^2\varphi = 1$. The score of $\mathbf{x}$ in this direction is

$$z(\varphi) = \mathbf{u}^\top\mathbf{x} = 2\cos\varphi + 3\sin\varphi.$$

At $\varphi = 0°$, $\mathbf{u} = (1, 0)$ and $z = 2$, the first coordinate of $\mathbf{x}$; at $\varphi = 90°$, $\mathbf{u} = (0, 1)$ and $z = 3$, the second coordinate. Projecting onto a coordinate axis therefore just reads off one feature, and projecting onto any other direction produces a weighted combination of the features, a new feature. To see how z changes with φ, write $\mathbf{x}$ through its length and its own angle α: $\mathbf{x} = \|\mathbf{x}\|(\cos\alpha,\ \sin\alpha)$ with $\|\mathbf{x}\| = \sqrt{13} \approx 3.606$, $\cos\alpha = 2/\sqrt{13} \approx 0.555$, $\sin\alpha = 3/\sqrt{13} \approx 0.832$, and $\alpha \approx 56.3°$. Then, by the identity $\cos(a - b) = \cos a\cos b + \sin a\sin b$,

$$z(\varphi) = \|\mathbf{x}\|\,(\cos\alpha\cos\varphi + \sin\alpha\sin\varphi) = \|\mathbf{x}\|\cos(\varphi - \alpha),$$

which is the shadow formula with $\theta = \varphi - \alpha$, the angle between $\mathbf{u}$ and $\mathbf{x}$. The score is largest, $\|\mathbf{x}\| \approx 3.606$, when the line points along $\mathbf{x}$ ($\varphi = \alpha$); it is zero when the line is perpendicular to $\mathbf{x}$ ($\varphi = \alpha + 90° \approx 146.3°$); it is negative when the angle between them exceeds $90°$, and reaches $-\|\mathbf{x}\|$ when $\mathbf{u}$ points directly away from $\mathbf{x}$ ($\varphi \approx 236.3°$). For a single point the best direction is simply the direction of the point. For a data set no single direction passes through every point, and choosing the direction in which the projected points are most spread out is the problem that principal component analysis solves.

Projecting a data set. Let $\mathbf{x}_1, \ldots, \mathbf{x}_n$ have mean $\bar{\mathbf{x}}$, and write $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$ for the deviations (read "$\mathbf{x}$ tilde"). They sum to zero: since $\bar{\mathbf{x}} = \frac{1}{n}\sum_i \mathbf{x}_i$, we have $\sum_i \mathbf{x}_i = n\bar{\mathbf{x}}$, and so $\sum_{i=1}^n \tilde{\mathbf{x}}_i = \sum_{i=1}^n \mathbf{x}_i - n\bar{\mathbf{x}} = n\bar{\mathbf{x}} - n\bar{\mathbf{x}} = \mathbf{0}$. Subtracting the mean is a shift of the coordinate system that moves $\bar{\mathbf{x}}$ to the origin: in the new coordinates the point $\mathbf{x}_i$ sits at $\tilde{\mathbf{x}}_i$ and the mean sits at $\mathbf{0}$. A line through the origin in these *centred* coordinates is therefore the line through $\bar{\mathbf{x}}$ in the original coordinates,

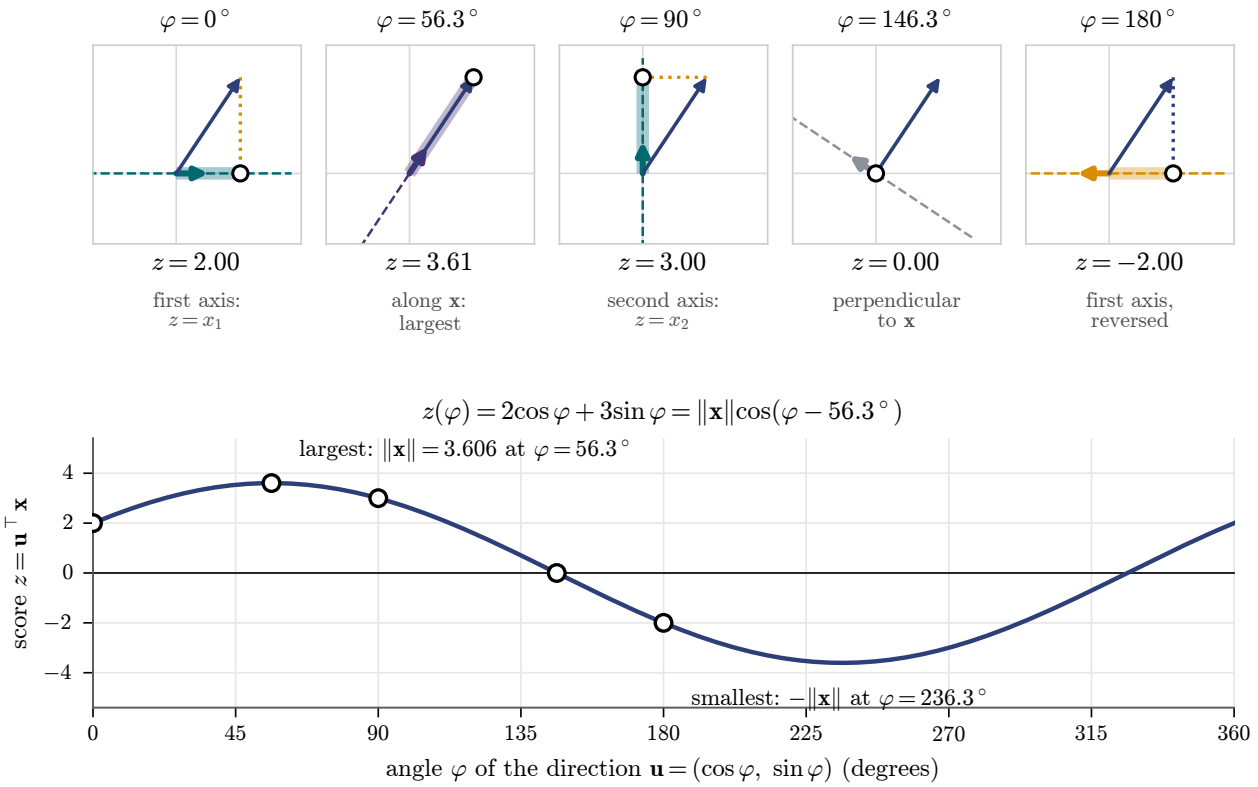


Figure 1.3: The score of $\mathbf{x} = (2, 3)$ as the direction turns. Top: five directions $\mathbf{u} = (\cos\varphi,\ \sin\varphi)$, each drawn as a short thick arrow on its dashed line, with the foot of the perpendicular (open circle) and the shaded projection. The first axis gives $z = x_1 = 2$, the second axis $z = x_2 = 3$, the direction of $\mathbf{x}$ the largest score $\|\mathbf{x}\| = 3.61$, the perpendicular direction $z = 0$, and the reversed first axis $z = -2$. Bottom: the score for every angle, the cosine curve $\|\mathbf{x}\|\cos(\varphi - 56.3°)$, with the five cases marked.

and the score $\mathbf{u}^\top\tilde{\mathbf{x}}_i$ measures how far along that line, from $\bar{\mathbf{x}}$, the point $\mathbf{x}_i$ projects. Projecting every deviation onto $\mathbf{u}$ replaces each point by one number,

$$z_i = \mathbf{u}^\top\tilde{\mathbf{x}}_i = u_1\tilde{x}_{i1} + u_2\tilde{x}_{i2} + \cdots + u_d\tilde{x}_{id},$$

so the data set becomes n numbers on a line. These numbers have mean zero, since $\frac{1}{n}\sum_i z_i = \mathbf{u}^\top\big(\frac{1}{n}\sum_i \tilde{\mathbf{x}}_i\big) = 0$, and therefore their variance is the mean of their squares,

$$s^2(\mathbf{u}) = \frac{1}{n}\sum_{i=1}^{n} z_i^2.$$

This is the spread of the data seen along the direction $\mathbf{u}$. It depends on $\mathbf{u}$, and the covariance matrix is exactly what describes that dependence.

Which direction to try. After the two coordinate axes, the simplest directions to try are the two diagonals. The **rising diagonal** is the line through the mean on which the two deviations are equal, $\tilde{x}_1 = \tilde{x}_2$: moving along it, one more hour of training goes with one more gigabyte of memory. The vector $(1, 1)$ points along it. Its length is $\sqrt{1^2 + 1^2} = \sqrt{2}$, so dividing by $\sqrt{2}$ gives the unit vector

$$\mathbf{u} = \tfrac{1}{\sqrt{2}}(1, 1) \approx (0.7071,\ 0.7071), \qquad \|\mathbf{u}\|^2 = \tfrac{1}{2} + \tfrac{1}{2} = 1.$$

The **falling diagonal**, on which $\tilde{x}_2 = -\tilde{x}_1$, has direction $(1, -1)$ and unit vector $\frac{1}{\sqrt{2}}(1, -1)$. It is perpendicular to the rising one, since $(1, 1)^\top(1, -1) = 1 - 1 = 0$. For the eight runs the rising diagonal is a natural first guess: longer runs use more memory, so the points lie close to a rising line. It is only a guess, however. The rising diagonal is not the direction of largest variance, which lies at 42.1° rather than 45°; Section 1.9, *Eigenvalues and eigenvectors of a symmetric matrix* shows how

to find it. Figure 1.4 shows the eight runs projected onto both diagonals. Along the rising diagonal the eight scores are spread widely; along the falling diagonal they are bunched near zero. The same eight points therefore have a large variance in one direction and a small variance in another; the two values, 11.7422 and 0.8672, are computed in the example on the eight runs below.

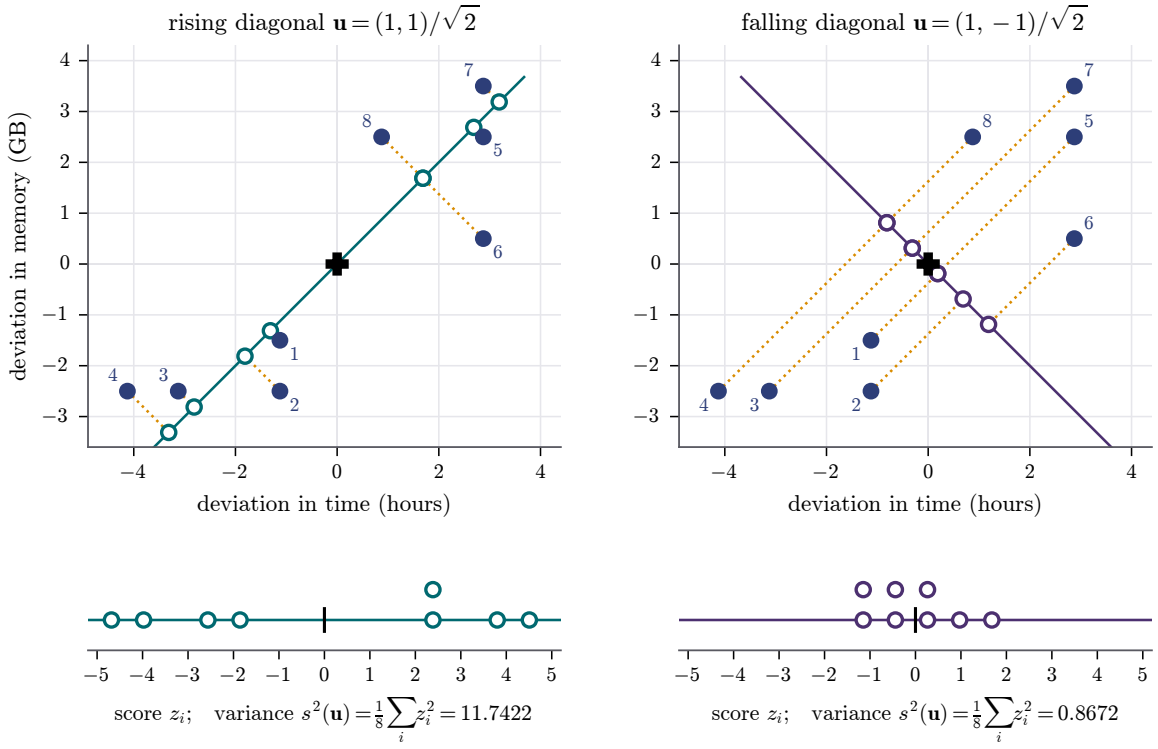


Figure 1.4: The eight centred runs (filled circles, numbered) projected onto two directions through the mean (cross). Each dotted segment is a residual, perpendicular to the line; each open circle is a projected point. Below each panel the eight scores are drawn on a number line (equal scores are stacked), with their variance. The rising diagonal follows the trend of the runs and gives a large variance; the falling diagonal cuts across it and gives a small one.

Example: projecting one run, step by step

Project run 6, $\mathbf{x} = (8,\ 6)$, onto the rising diagonal $\mathbf{u} = \frac{1}{\sqrt{2}}(1,1) \approx (0.7071,\ 0.7071)$. The mean of the eight runs is $\bar{\mathbf{x}} = (5.125,\ 5.500)$.

1. **Centre.** Subtract the mean: $\tilde{\mathbf{x}} = \mathbf{x} - \bar{\mathbf{x}} = (8 - 5.125,\ 6 - 5.500) = (2.875,\ 0.5)$. In these coordinates the mean is the origin.
2. **Score.** $z = \mathbf{u}^\top \tilde{\mathbf{x}} = \frac{1}{\sqrt{2}}(2.875 + 0.5) = 3.375/\sqrt{2} \approx 2.3865$. The run projects 2.3865 units along the line, measured from the mean.
3. **Projected point.** $z\,\mathbf{u} = 2.3865 \times (0.7071,\ 0.7071) = (1.6875,\ 1.6875)$; exactly $(3.375/2,\ 3.375/2)$.
4. **Residual.** $\mathbf{r} = \tilde{\mathbf{x}} - z\,\mathbf{u} = (2.875 - 1.6875,\ 0.5 - 1.6875) = (1.1875,\ -1.1875)$. It is perpendicular to the line: $\mathbf{u}^\top \mathbf{r} = \frac{1}{\sqrt{2}}(1.1875 + (-1.1875)) = 0$.
5. **Pythagoras.** $\|\tilde{\mathbf{x}}\|^2 = 2.875^2 + 0.5^2 = 8.265625 + 0.25 = 8.515625$, and $z^2 + \|\mathbf{r}\|^2 = 5.6953125 + 2.8203125 = 8.515625$.
6. **Original units.** Adding the mean back, the projected point is $\bar{\mathbf{x}} + z\,\mathbf{u} = (5.125 + 1.6875,\ 5.500 + 1.6875) = (6.8125,\ 7.1875)$, on the line through $\bar{\mathbf{x}}$ in direction $\mathbf{u}$.

Steps 1 and 2, repeated for every run, give the eight scores $z_1, \ldots, z_8$ whose variance is $s^2(\mathbf{u})$.

Figure 1.5 draws the steps, one panel each.

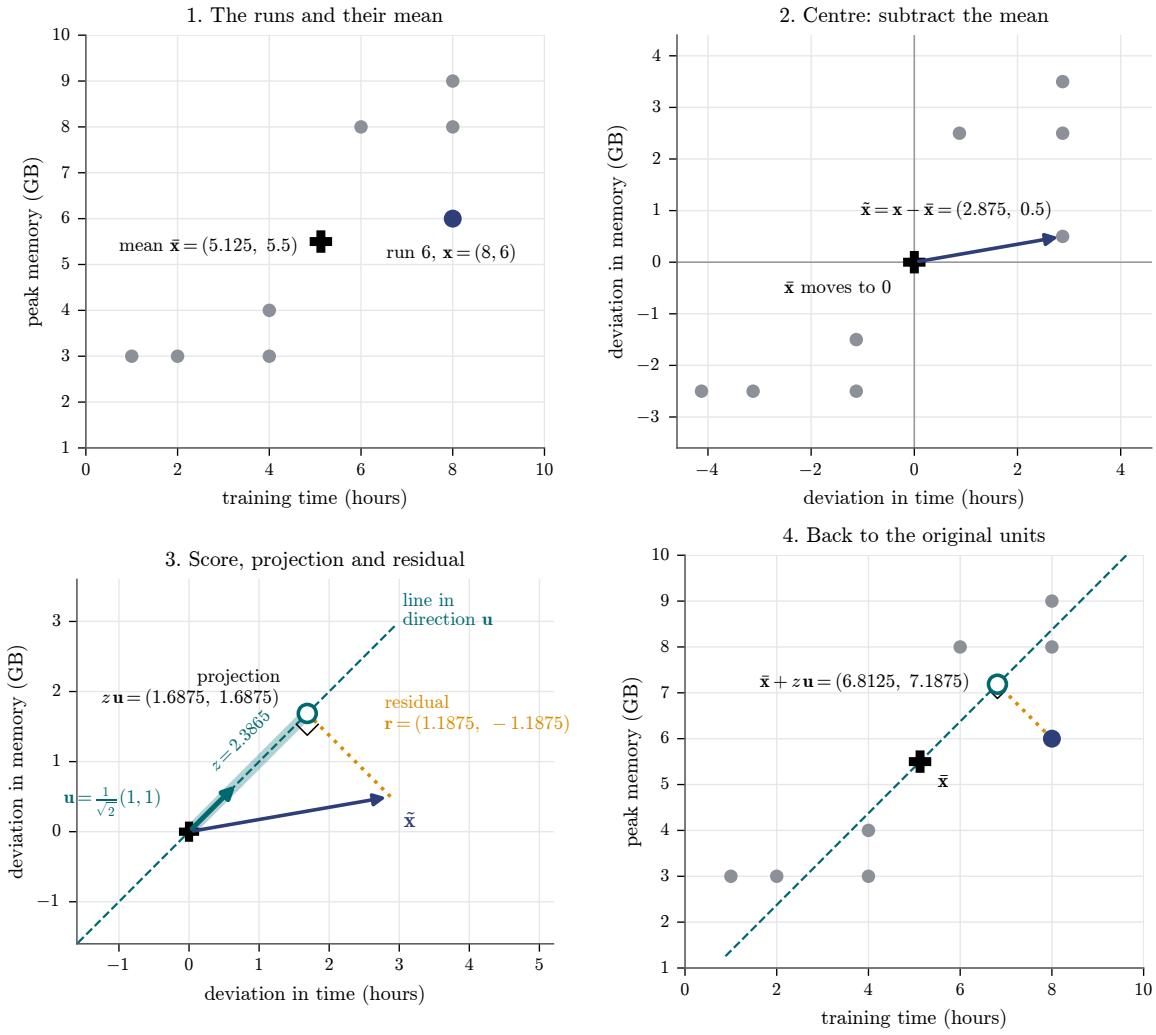


Figure 1.5: Projecting run 6 onto $\mathbf{u} = \frac{1}{\sqrt{2}}(1, 1)$, following the steps of the example. (1) The runs and their mean $\bar{\mathbf{x}}$ in the original units. (2) Subtracting the mean moves $\bar{\mathbf{x}}$ to the origin and run 6 to $\tilde{\mathbf{x}}$. (3) The score z is the position of the foot of the perpendicular along the line; the projection $z\,\mathbf{u}$ and the residual $\mathbf{r}$ meet at a right angle. (4) Adding the mean back places the projected point on the line through $\bar{\mathbf{x}}$.

Two features, written out. With two features and $\mathbf{u} = (u_1, u_2)$, squaring the score gives

$$z_i^2 = (u_1\tilde{x}_{i1} + u_2\tilde{x}_{i2})^2 = u_1^2\,\tilde{x}_{i1}^2 + 2u_1u_2\,\tilde{x}_{i1}\tilde{x}_{i2} + u_2^2\,\tilde{x}_{i2}^2.$$

Averaging over the n points, the three averages that appear are

$$S_{11} = \frac{1}{n}\sum_i \tilde{x}_{i1}^2, \qquad S_{22} = \frac{1}{n}\sum_i \tilde{x}_{i2}^2, \qquad S_{12} = \frac{1}{n}\sum_i \tilde{x}_{i1}\tilde{x}_{i2},$$

the variance of the first feature, the variance of the second, and their **covariance**. Hence

$$s^2(\mathbf{u}) = u_1^2\,S_{11} + 2u_1u_2\,S_{12} + u_2^2\,S_{22}.$$

The variance along every direction is fixed by these three numbers: the two variances, weighted by u_1^2 and u_2^2, and the covariance, weighted by $2u_1u_2$. Collecting them in the symmetric **covariance matrix** and multiplying out confirms that

$$S = \begin{pmatrix} S_{11} & S_{12} \\ S_{12} & S_{22} \end{pmatrix}, \qquad \mathbf{u}^\top S\mathbf{u} = \begin{pmatrix} u_1 & u_2 \end{pmatrix}\begin{pmatrix} S_{11}u_1 + S_{12}u_2 \\ S_{12}u_1 + S_{22}u_2 \end{pmatrix} = u_1^2S_{11} + 2u_1u_2S_{12} + u_2^2S_{22} = s^2(\mathbf{u}).$$

Any number of features. Since $z_i = \mathbf{u}^\top \tilde{\mathbf{x}}_i = \tilde{\mathbf{x}}_i^\top \mathbf{u}$, we have $z_i^2 = \mathbf{u}^\top \tilde{\mathbf{x}}_i \, \tilde{\mathbf{x}}_i^\top \mathbf{u}$, and averaging over i,

$$s^2(\mathbf{u}) = \mathbf{u}^\top \Big[\frac{1}{n}\sum_{i=1}^{n} \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^\top\Big]\mathbf{u} = \mathbf{u}^\top S\, \mathbf{u}, \qquad S = \frac{1}{n}\sum_{i=1}^{n} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top.$$

The (j,k) entry of the $d \times d$ matrix $\tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^\top$ is $\tilde{x}_{ij}\tilde{x}_{ik}$, so $S_{jk} = \frac{1}{n}\sum_i \tilde{x}_{ij}\tilde{x}_{ik}$: the variance of feature j when $j = k$, and the covariance of features j and k otherwise. If the deviations are stacked as the rows of an $n \times d$ matrix X_c, so that the entry in row i and column j is $\tilde{x}_{ij}$, then the transpose $X_c^\top$ has $\tilde{x}_{ij}$ in row j and column i. The (j,k) entry of the $d \times d$ product $X_c^\top X_c$ is row j of $X_c^\top$ multiplied by column k of X_c,

$$(X_c^\top X_c)_{jk} = \sum_{i=1}^{n} \tilde{x}_{ij}\, \tilde{x}_{ik} = n\, S_{jk},$$

and therefore $S = \frac{1}{n} X_c^\top X_c$.

The role of the covariance. Along a coordinate axis, $\mathbf{u} = (1,0)$, the formula gives $s^2 = S_{11}$, the variance of the first feature alone; the covariance plays no part. Along the two diagonals,

$$\mathbf{u} = \tfrac{1}{\sqrt{2}}(1,1): \quad s^2 = \tfrac{1}{2}S_{11} + S_{12} + \tfrac{1}{2}S_{22}, \qquad \mathbf{u} = \tfrac{1}{\sqrt{2}}(1,-1): \quad s^2 = \tfrac{1}{2}S_{11} - S_{12} + \tfrac{1}{2}S_{22}.$$

The two diagonals share the average of the two variances; the covariance is added along one and subtracted along the other. A positive covariance means that the two features tend to be above their means together, so the cloud of points is stretched along the rising diagonal and narrow across it; a negative covariance stretches it along the falling diagonal; with zero covariance no oblique direction is favoured beyond what the two variances give.

Example: the eight training runs

The mean is $\bar{\mathbf{x}} = (5.125,\ 5.500)$, and the deviations $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$ are

run i	1	2	3	4	5	6	7	8
$\tilde{x}_{i1}$ (hours)	−1.125	−1.125	−3.125	−4.125	2.875	2.875	2.875	0.875
$\tilde{x}_{i2}$ (GB)	−1.500	−2.500	−2.500	−2.500	2.500	0.500	3.500	2.500

The sums of squares and products of the deviations, taken run by run from the table, are

$$\begin{aligned}
\sum_i \tilde{x}_{i1}^2 &= 1.265625 + 1.265625 + 9.765625 + 17.015625 \\
&\quad + 8.265625 + 8.265625 + 8.265625 + 0.765625 = 54.875, \\
\sum_i \tilde{x}_{i1}\tilde{x}_{i2} &= 1.6875 + 2.8125 + 7.8125 + 10.3125 \\
&\quad + 7.1875 + 1.4375 + 10.0625 + 2.1875 = 43.500, \\
\sum_i \tilde{x}_{i2}^2 &= 2.25 + 6.25 + 6.25 + 6.25 + 6.25 + 0.25 + 12.25 + 6.25 = 46.000,
\end{aligned}$$

so, dividing by $n = 8$,

$$S = \begin{pmatrix} 6.8594 & 5.4375 \\ 5.4375 & 5.7500 \end{pmatrix}.$$

Along the rising diagonal $\mathbf{u} = \frac{1}{\sqrt{2}}(1,1)$, the score of run 6 is $(2.875 + 0.5)/\sqrt{2} \approx 2.3865$, as in the example above. Over all eight runs,

$$\sum_i z_i^2 = \tfrac{1}{2}\Big(\sum_i \tilde{x}_{i1}^2 + 2\sum_i \tilde{x}_{i1}\tilde{x}_{i2} + \sum_i \tilde{x}_{i2}^2\Big) = \tfrac{1}{2}(54.875 + 87.000 + 46.000) = 93.9375,$$

so $s^2 = 93.9375/8 = 11.7422$, which is $\frac{1}{2}S_{11} + \frac{1}{2}S_{22} + S_{12} = 6.3047 + 5.4375$. Along the falling diagonal $\sum_i z_i^2 = \frac{1}{2}(54.875 - 87.000 + 46.000) = 6.9375$ and $s^2 = 0.8672 = 6.3047 - 5.4375$. The

positive covariance makes the rising diagonal carry about 13.5 times the variance of the falling one: longer runs use more memory, and the runs lie close to a rising line.

The covariance of two projections. For two unit directions $\mathbf{u}$ and $\mathbf{v}$, the scores $\mathbf{u}^\top \tilde{\mathbf{x}}_i$ and $\mathbf{v}^\top \tilde{\mathbf{x}}_i$ both have mean zero, and their covariance is

$$\frac{1}{n}\sum_{i=1}^{n}(\mathbf{u}^\top \tilde{\mathbf{x}}_i)(\tilde{\mathbf{x}}_i^\top \mathbf{v}) = \mathbf{u}^\top S\,\mathbf{v}.$$

The single matrix S thus gives the variance of every projection of the data and the covariance between any two projections. Principal components are directions for which these covariances are zero (Section 4.2.4, *Several principal components*).

Positive semidefiniteness. Because $\mathbf{u}^\top S\mathbf{u}$ is a variance, an average of squares, $\mathbf{u}^\top S\mathbf{u} \geq 0$ for every vector $\mathbf{u}$: S is **positive semidefinite**. It is also symmetric, since $S_{jk} = S_{kj}$. Finding the unit vector that makes $\mathbf{u}^\top S\mathbf{u}$ as large as possible is the problem solved by principal component analysis (Section 4.2.2, *The first principal direction*); Section 4.2.1, *Projection onto a direction and the variance it retains* plots $s^2(\mathbf{u})$ against the angle of $\mathbf{u}$ for the eight runs.

Use in machine learning. Projection is the central operation of principal component analysis: the principal directions are the unit vectors onto which the projected data have the largest variance $\mathbf{u}^\top S\mathbf{u}$ (Section 4.2, *Principal components*). The covariance matrix also describes the shape of each component of a Gaussian mixture (Section 3.1.6, *General Gaussian mixtures*) and defines the Mahalanobis distance used to detect anomalies (Section 5.3, *Two features: distance, shape and neighbours*).

1.9 Eigenvalues and eigenvectors of a symmetric matrix

The question. Section 1.8, *Projections and the covariance matrix* reduced the spread of a data set along a unit direction $\mathbf{u}$ to one formula, $s^2(\mathbf{u}) = \mathbf{u}^\top S\mathbf{u}$. For the eight runs the rising diagonal gives 11.7422 and the falling diagonal 0.8672. Two questions remain: which direction gives the largest variance, and how large is that variance? Trying directions one at a time cannot settle this, and with many features it is hopeless. The answer is that the best direction is an *eigenvector* of S, and the largest variance is the matching *eigenvalue*. This section defines both and shows why they answer the question.

What a matrix does to a vector. Multiplying a vector by a matrix gives another vector, which in general points in a different direction. For the covariance matrix of the eight runs,

$$S = \begin{pmatrix} 6.8594 & 5.4375 \\ 5.4375 & 5.7500 \end{pmatrix},$$

the first axis $\mathbf{e}_1 = (1, 0)$ is sent to $S\mathbf{e}_1 = (6.8594,\ 5.4375)$, the first column of S, which points at 38.4° instead of 0°. The second axis $\mathbf{e}_2 = (0, 1)$ is sent to the second column, $(5.4375,\ 5.7500)$, at 46.6° instead of 90°. Both are turned towards the rising diagonal, the direction in which the runs are spread out. A few special directions are not turned at all: multiplying by S only stretches or shrinks them (Figure 1.6a).

Definition. A nonzero vector $\mathbf{v}$ is an **eigenvector** of a square matrix S with **eigenvalue** λ if

$$S\mathbf{v} = \lambda\mathbf{v}\ :$$

multiplying by S only rescales $\mathbf{v}$, by the factor λ. To find the eigenvalues, write $\lambda\mathbf{v} = \lambda I\mathbf{v}$, where I is the identity matrix; the condition becomes $(S - \lambda I)\mathbf{v} = \mathbf{0}$ with $\mathbf{v} \neq \mathbf{0}$. A square matrix sends

some nonzero vector to $\mathbf{0}$ exactly when it has no inverse, which is exactly when its determinant is zero. The eigenvalues are therefore the roots of $\det(S - \lambda I) = 0$. For a 2×2 matrix,

$$\begin{aligned}\det(S - \lambda I) &= \det \begin{pmatrix} S_{11} - \lambda & S_{12} \\ S_{21} & S_{22} - \lambda \end{pmatrix} = (S_{11} - \lambda)(S_{22} - \lambda) - S_{12}S_{21} \\ &= \lambda^2 - (S_{11} + S_{22})\,\lambda + (S_{11}S_{22} - S_{12}S_{21}),\end{aligned}$$

and since $S_{11} + S_{22} = \operatorname{tr} S$, the sum of the diagonal entries (the **trace**), and $S_{11}S_{22} - S_{12}S_{21} = \det S$, this is the quadratic

$$\lambda^2 - (\operatorname{tr} S)\,\lambda + \det S = 0.$$

Once an eigenvalue λ is known, an eigenvector is any nonzero solution of $(S - \lambda I)\mathbf{v} = \mathbf{0}$; it is usually scaled to length 1.

Example: the eight runs

Here $\operatorname{tr} S = 6.8594 + 5.7500 = 12.6094$ and $\det S = 6.8594 \times 5.7500 - 5.4375^2 = 9.8750$. The quadratic formula gives

$$\lambda = \tfrac{1}{2}\Big(12.6094 \pm \sqrt{12.6094^2 - 4 \times 9.8750}\Big) = \tfrac{1}{2}\big(12.6094 \pm \sqrt{119.4963}\big) = \tfrac{1}{2}\big(12.6094 \pm 10.9314\big),$$

so $\lambda_1 = 11.7704$ and $\lambda_2 = 0.8390$. For λ_1, the first row of $(S - \lambda_1 I)\mathbf{v} = \mathbf{0}$ with $\mathbf{v} = (a, b)$ reads $(6.8594 - 11.7704)\,a + 5.4375\,b = 0$, so $b = (4.9110/5.4375)\,a = 0.9032\,a$. Taking $a = 1$ and dividing $(1,\ 0.9032)$ by its length $\sqrt{1 + 0.9032^2} = 1.3475$ gives the unit eigenvector $\mathbf{v}_1 = (0.7421,\ 0.6703)$, at $42.1°$ to the first axis. The second eigenvector is perpendicular to it, $\mathbf{v}_2 = (-0.6703,\ 0.7421)$. To check, $S\mathbf{v}_1 = (6.8594 \times 0.7421 + 5.4375 \times 0.6703,\ 5.4375 \times 0.7421 + 5.7500 \times 0.6703) \approx (8.735,\ 7.889)$, which is $11.7704 \times (0.7421,\ 0.6703)$: the direction is unchanged and the length is multiplied by λ_1. In the same way $S\mathbf{v}_2 \approx (-0.562,\ 0.623) = 0.8390\,\mathbf{v}_2$.

Why the eigenvectors give the largest and the smallest variance. Because $\mathbf{v}_1$ and $\mathbf{v}_2$ are perpendicular unit vectors, every unit vector $\mathbf{u}$ in the plane can be written as

$$\mathbf{u} = a\,\mathbf{v}_1 + b\,\mathbf{v}_2, \qquad a = \mathbf{v}_1^\top \mathbf{u}, \quad b = \mathbf{v}_2^\top \mathbf{u};$$

the formulas for a and b follow by taking the dot product of both sides with $\mathbf{v}_1$ and with $\mathbf{v}_2$, using $\mathbf{v}_1^\top \mathbf{v}_1 = \mathbf{v}_2^\top \mathbf{v}_2 = 1$ and $\mathbf{v}_1^\top \mathbf{v}_2 = 0$. The numbers a and b are the scores of $\mathbf{u}$ along the two eigenvectors, and by Pythagoras (Section 1.8, *Projections and the covariance matrix*) $a^2 + b^2 = \|\mathbf{u}\|^2 = 1$. Multiplying by S acts on each part separately, $S\mathbf{u} = a\,S\mathbf{v}_1 + b\,S\mathbf{v}_2 = a\lambda_1\mathbf{v}_1 + b\lambda_2\mathbf{v}_2$. Taking the dot product with $\mathbf{u}$ and multiplying out term by term, as for Pythagoras' theorem,

$$\begin{aligned}\mathbf{u}^\top S\mathbf{u} &= (a\,\mathbf{v}_1 + b\,\mathbf{v}_2)^\top (a\lambda_1\mathbf{v}_1 + b\lambda_2\mathbf{v}_2) \\ &= a^2\lambda_1\,\mathbf{v}_1^\top\mathbf{v}_1 + ab\lambda_2\,\mathbf{v}_1^\top\mathbf{v}_2 + ab\lambda_1\,\mathbf{v}_2^\top\mathbf{v}_1 + b^2\lambda_2\,\mathbf{v}_2^\top\mathbf{v}_2 \\ &= \lambda_1 a^2 + \lambda_2 b^2.\end{aligned}$$

This is the key formula. The variance along $\mathbf{u}$ is a *weighted average* of the two eigenvalues, with weights a^2 and b^2 that are nonnegative and add to 1. A weighted average can never exceed the largest of the numbers averaged nor fall below the smallest:

$$\lambda_2 = \lambda_2(a^2 + b^2) \le \lambda_1 a^2 + \lambda_2 b^2 \le \lambda_1(a^2 + b^2) = \lambda_1.$$

The right-hand inequality is an equality only when $b = 0$, that is, when $\mathbf{u} = \pm\mathbf{v}_1$, and the left-hand one only when $a = 0$, $\mathbf{u} = \pm\mathbf{v}_2$. Hence:

- the largest variance of the projected data is the largest eigenvalue λ_1, reached along its eigenvector $\mathbf{v}_1$;
- the smallest variance is the smallest eigenvalue λ_2, reached along $\mathbf{v}_2$;

- every other direction gives a variance between the two, closer to λ_1 the closer $\mathbf{u}$ is to $\mathbf{v}_1$.

If ψ is the angle between $\mathbf{u}$ and $\mathbf{v}_1$, the score of $\mathbf{u}$ along $\mathbf{v}_1$ is $a = \|\mathbf{u}\| \cos\psi = \cos\psi$ by the shadow formula and $b^2 = 1 - a^2 = \sin^2\psi$, so

$$s^2(\mathbf{u}) = \lambda_1 \cos^2\psi + \lambda_2 \sin^2\psi.$$

For the rising diagonal, $\psi = 45° - 42.1° \approx 2.9°$, $\cos^2\psi = 0.9974$ and $\sin^2\psi = 0.0026$, so $s^2 = 11.7704 \times 0.9974 + 0.8390 \times 0.0026 = 11.7400 + 0.0022 = 11.7422$, the value found directly in the previous section. The diagonal is only 2.9° away from $\mathbf{v}_1$, so its variance is just below $\lambda_1 = 11.7704$. The falling diagonal is the same angle away from $\mathbf{v}_2$, and its variance, 0.8672, is just above $\lambda_2 = 0.8390$. Figure 1.6b plots $s^2(\mathbf{u})$ for every direction.

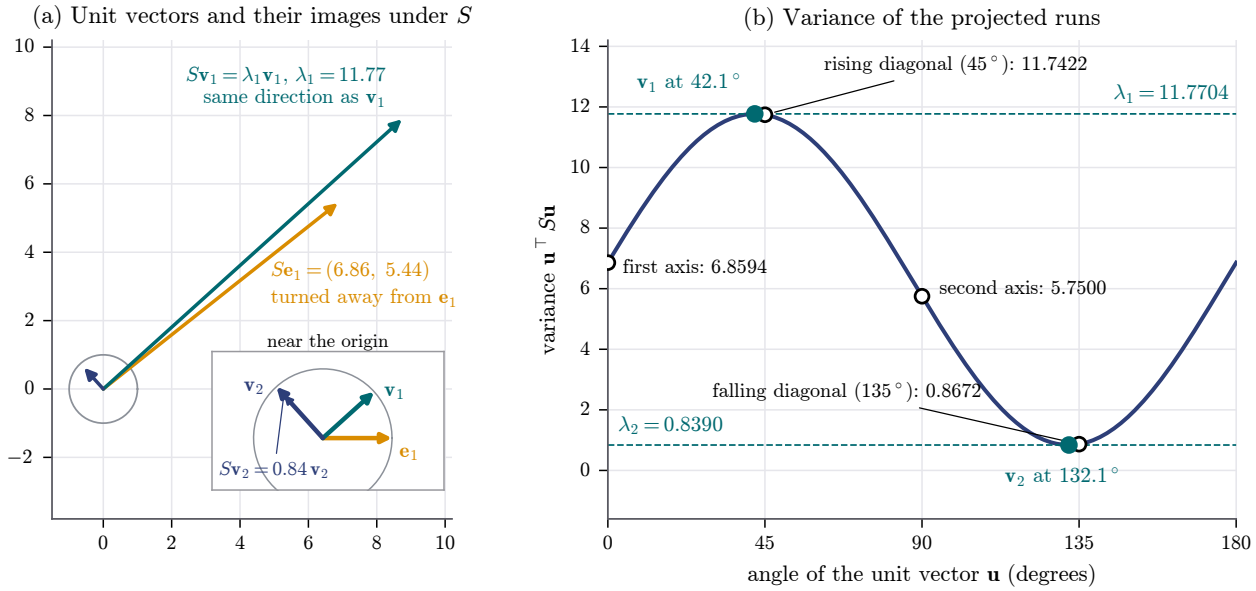


Figure 1.6: The covariance matrix S of the eight runs. (a) Multiplying by S turns most vectors: the first axis $\mathbf{e}_1$ is sent to $S\mathbf{e}_1$, which points at 38.4°. The eigenvector $\mathbf{v}_1$ is not turned, only stretched by λ_1, and $\mathbf{v}_2$ is only shrunk by λ_2 (inset). (b) The variance $\mathbf{u}^\top S\mathbf{u}$ of the projected runs as the unit vector $\mathbf{u}$ turns. Its maximum is λ_1, reached at $\mathbf{v}_1$, and its minimum is λ_2, reached at $\mathbf{v}_2$; the axes and the diagonals give values in between.

The spectral theorem. The argument above used three facts about the covariance matrix of the eight runs: its eigenvalues are real numbers, its two eigenvectors are perpendicular, and every vector can be split into parts along them. None of these holds for every square matrix. The spectral theorem says that all three hold for every *symmetric* matrix, of any size; covariance matrices are symmetric, so the argument works for any number of features.

Theorem (spectral theorem for symmetric matrices)

Let S be a real symmetric $d \times d$ matrix. Then

1. its d eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$ are real numbers;
2. it has d eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_d$ of length 1 that are perpendicular to one another, $\mathbf{v}_j^\top \mathbf{v}_k = 0$ for $j \neq k$ (an **orthonormal basis**);
3. S can be rebuilt from them as

$$S = \sum_{j=1}^{d} \lambda_j \mathbf{v}_j \mathbf{v}_j^\top = \lambda_1 \mathbf{v}_1 \mathbf{v}_1^\top + \cdots + \lambda_d \mathbf{v}_d \mathbf{v}_d^\top.$$

In words: *a symmetric matrix has d perpendicular directions along which it acts as a pure stretch, by the factor λ_j along $\mathbf{v}_j$.* Part 3 says exactly this. The matrix $\mathbf{v}_j\mathbf{v}_j^\top$ is $d \times d$, with entry $v_{jk}v_{jl}$ in row k and column l, and applying it to a vector gives

$$(\mathbf{v}_j\mathbf{v}_j^\top)\,\mathbf{u} = \mathbf{v}_j\,(\mathbf{v}_j^\top\mathbf{u}) = (\mathbf{v}_j^\top\mathbf{u})\,\mathbf{v}_j,$$

the projection of $\mathbf{u}$ onto the line of $\mathbf{v}_j$ (Section 1.8, *Projections and the covariance matrix*). Hence

$$S\mathbf{u} = \sum_{j=1}^{d} \lambda_j\,(\mathbf{v}_j^\top\mathbf{u})\,\mathbf{v}_j :$$

to multiply a vector by S, split it into its projections onto the eigenvectors, stretch the jth projection by λ_j, and add the results. In Figure 1.6a the vector $\mathbf{e}_1$ has a large part along $\mathbf{v}_1$, stretched by 11.7704, and a part along $\mathbf{v}_2$, shrunk by 0.8390; that is why $S\mathbf{e}_1$ is turned towards $\mathbf{v}_1$.

Why the theorem holds: two features. With two features every step can be checked directly.

1. *The eigenvalues are real.* They are the roots of $\lambda^2 - (S_{11}+S_{22})\lambda + (S_{11}S_{22} - S_{12}^2) = 0$, using $S_{21} = S_{12}$. The roots are real when the discriminant is nonnegative, and

$$\begin{aligned}(S_{11}+S_{22})^2 - 4(S_{11}S_{22} - S_{12}^2) &= S_{11}^2 + 2S_{11}S_{22} + S_{22}^2 - 4S_{11}S_{22} + 4S_{12}^2 \\ &= (S_{11}-S_{22})^2 + 4S_{12}^2 \;\geq\; 0,\end{aligned}$$

   a sum of two squares. For the eight runs it is $1.1094^2 + 4 \times 5.4375^2 = 1.2307 + 118.2656 = 119.4963$, the discriminant found in the example. Symmetry is what makes the last term a square: for a matrix that is not symmetric it is $4S_{12}S_{21}$, which can be negative, and then the eigenvalues are not real.

2. *Eigenvectors with different eigenvalues are perpendicular.* Let $S\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ and $S\mathbf{v}_2 = \lambda_2\mathbf{v}_2$ with $\lambda_1 \neq \lambda_2$, and compute the number $\mathbf{v}_2^\top S\mathbf{v}_1$ in two ways. First, $\mathbf{v}_2^\top(S\mathbf{v}_1) = \lambda_1\,\mathbf{v}_2^\top\mathbf{v}_1$. Second, because $S^\top = S$, $\mathbf{v}_2^\top S = \mathbf{v}_2^\top S^\top = (S\mathbf{v}_2)^\top = \lambda_2\mathbf{v}_2^\top$, so $\mathbf{v}_2^\top S\mathbf{v}_1 = \lambda_2\,\mathbf{v}_2^\top\mathbf{v}_1$. Subtracting,

$$(\lambda_1 - \lambda_2)\,\mathbf{v}_2^\top\mathbf{v}_1 = 0,$$

   and since $\lambda_1 - \lambda_2 \neq 0$, $\mathbf{v}_2^\top\mathbf{v}_1 = 0$. For the eight runs, $0.7421 \times (-0.6703) + 0.6703 \times 0.7421 = 0$. (If the two eigenvalues are equal, the discriminant in part 1 is zero, which forces $S_{11} = S_{22}$ and $S_{12} = 0$; then $S = \lambda I$ and every vector is an eigenvector, so any two perpendicular unit vectors will do.)

3. *S is rebuilt from its eigenvectors.* Both sides of $S = \lambda_1\mathbf{v}_1\mathbf{v}_1^\top + \lambda_2\mathbf{v}_2\mathbf{v}_2^\top$ send $\mathbf{v}_1$ to the same vector: the left side gives $S\mathbf{v}_1 = \lambda_1\mathbf{v}_1$, and the right side gives $\lambda_1\mathbf{v}_1(\mathbf{v}_1^\top\mathbf{v}_1) + \lambda_2\mathbf{v}_2(\mathbf{v}_2^\top\mathbf{v}_1) = \lambda_1\mathbf{v}_1 + 0$. In the same way both send $\mathbf{v}_2$ to $\lambda_2\mathbf{v}_2$. Every vector is a combination $a\mathbf{v}_1 + b\mathbf{v}_2$, and multiplying by a matrix respects such combinations, so the two sides agree on every vector and are the same matrix. For the eight runs, with $\mathbf{v}_1 = (0.7421,\ 0.6703)$ and $\mathbf{v}_2 = (-0.6703,\ 0.7421)$, the entries of the right side, written (row, column), are

$$\begin{aligned}(1,1)\text{ entry:}&\quad 11.7704 \times 0.7421^2 + 0.8390 \times (-0.6703)^2 = 6.4825 + 0.3769 = 6.8594,\\ (1,2)\text{ entry:}&\quad 11.7704 \times 0.7421 \times 0.6703 + 0.8390 \times (-0.6703) \times 0.7421\\ &\qquad = 5.8548 - 0.4173 = 5.4375,\\ (2,2)\text{ entry:}&\quad 11.7704 \times 0.6703^2 + 0.8390 \times 0.7421^2 = 5.2879 + 0.4621 = 5.7500,\end{aligned}$$

   which are the entries of S.

For d features the same two calculations show that the eigenvalues are real and that eigenvectors with different eigenvalues are perpendicular; that a full set of d perpendicular eigenvectors always exists takes an induction on d, given for example by Strang (*Introduction to Linear Algebra*, 5th ed., 2016, Section 6.4).

Two consequences. Multiplying $S\mathbf{v}_j = \lambda_j \mathbf{v}_j$ on the left by $\mathbf{v}_j^\top$ gives $\mathbf{v}_j^\top S\mathbf{v}_j = \lambda_j \mathbf{v}_j^\top \mathbf{v}_j = \lambda_j$, because $\mathbf{v}_j^\top \mathbf{v}_j = \|\mathbf{v}_j\|^2 = 1$. If S is positive semidefinite, therefore, every $\lambda_j = \mathbf{v}_j^\top S\mathbf{v}_j \geq 0$. Second, the eigenvalues add up to the **trace**, the sum of the diagonal entries: $\operatorname{tr} S = \sum_j \lambda_j$. To see this, write v_{jl} for the lth coordinate of $\mathbf{v}_j$. The lth diagonal entry of $\mathbf{v}_j\mathbf{v}_j^\top$ is v_{jl}^2, so $\operatorname{tr}(\mathbf{v}_j\mathbf{v}_j^\top) = \sum_l v_{jl}^2 = \|\mathbf{v}_j\|^2 = 1$; since the trace of a sum is the sum of the traces, $\operatorname{tr} S = \sum_j \lambda_j \operatorname{tr}(\mathbf{v}_j\mathbf{v}_j^\top) = \sum_j \lambda_j$.

For a covariance matrix, $\mathbf{v}_j^\top S\mathbf{v}_j = \lambda_j$ says that *each eigenvalue is the variance of the data along its own eigenvector*. Writing a unit vector as $\mathbf{u} = \sum_j a_j \mathbf{v}_j$ with $a_j = \mathbf{v}_j^\top \mathbf{u}$ and $\sum_j a_j^2 = 1$, the calculation above gives

$$\mathbf{u}^\top S\mathbf{u} = \sum_{j=1}^{d} \lambda_j a_j^2,$$

again a weighted average of the eigenvalues. It is largest, λ_1, along $\mathbf{v}_1$; and among directions perpendicular to $\mathbf{v}_1$ (those with $a_1 = 0$) it is largest, λ_2, along $\mathbf{v}_2$, and so on. The trace identity says that the eigenvalues share out the total variance $\operatorname{tr} S = S_{11} + \cdots + S_{dd}$: for the eight runs, $11.7704 + 0.8390 = 12.6094 = 6.8594 + 5.7500$.

Use in machine learning. This section is the mathematical core of principal component analysis. The principal directions are the eigenvectors of the covariance matrix, the variance each one keeps is its eigenvalue, and keeping the directions with the largest eigenvalues keeps as much of the total variance as possible (Section 4.2.2, *The first principal direction*, Section 4.2.4, *Several principal components*). The next section reaches the same eigenvector equation $S\mathbf{u} = \lambda\mathbf{u}$ by a second route, a Lagrange multiplier. The spectral theorem also gives the elliptical contours of a Gaussian component (Section 3.1.6, *General Gaussian mixtures*) and the Mahalanobis distance in principal coordinates (Section 5.3, *Two features: distance, shape and neighbours*).

1.10 Constrained optimisation with a Lagrange multiplier

To maximise a smooth function $f(\mathbf{u})$ subject to a constraint $g(\mathbf{u}) = 0$, form the **Lagrangian** $\mathcal{L}(\mathbf{u}, \lambda) = f(\mathbf{u}) - \lambda\, g(\mathbf{u})$. At a constrained maximum (where $\nabla g \neq 0$) the gradient of f is parallel to the gradient of g: $\nabla f = \lambda \nabla g$, which is $\partial\mathcal{L}/\partial\mathbf{u} = 0$. The condition holds for the following reason. If $\mathbf{u}$ moves along the constraint in a direction $\mathbf{h}$, then to first order g changes by $\nabla g^\top \mathbf{h}$, which must be zero, so the allowed directions are those perpendicular to ∇g. At a constrained maximum, f cannot increase to first order along $\mathbf{h}$ or along $-\mathbf{h}$, so $\nabla f^\top \mathbf{h} = 0$ for every $\mathbf{h}$ perpendicular to ∇g. Write $\nabla f = \lambda \nabla g + \mathbf{h}_0$ with $\lambda = \nabla g^\top \nabla f / \nabla g^\top \nabla g$, the projection of ∇f onto ∇g (Chapter 1, *Mathematical background*), so that $\mathbf{h}_0$ is perpendicular to ∇g. Taking $\mathbf{h} = \mathbf{h}_0$ gives $0 = \nabla f^\top \mathbf{h}_0 = \lambda\, \nabla g^\top \mathbf{h}_0 + \mathbf{h}_0^\top \mathbf{h}_0 = \|\mathbf{h}_0\|^2$, so $\mathbf{h}_0 = \mathbf{0}$ and $\nabla f = \lambda \nabla g$.

The gradient $\nabla_{\mathbf{u}}$ is the vector of partial derivatives $\partial/\partial u_m$. Writing $\mathbf{u}^\top S\mathbf{u} = \sum_j \sum_k u_j\, S_{jk}\, u_k$ and differentiating with respect to u_m by the product rule, the terms with $j = m$ give $\sum_k S_{mk} u_k$ and the terms with $k = m$ give $\sum_j u_j S_{jm}$, so

$$\frac{\partial}{\partial u_m}\, \mathbf{u}^\top S\mathbf{u} = \sum_k S_{mk} u_k + \sum_j S_{jm} u_j = (S\mathbf{u})_m + (S^\top \mathbf{u})_m = 2(S\mathbf{u})_m,$$

the last step because $S^\top = S$. Hence, for a symmetric matrix S, $\nabla_{\mathbf{u}}(\mathbf{u}^\top S\mathbf{u}) = 2S\mathbf{u}$, and taking $S = I$ gives $\nabla_{\mathbf{u}}(\mathbf{u}^\top \mathbf{u}) = 2\mathbf{u}$. For two coordinates this can be checked directly: $\mathbf{u}^\top S\mathbf{u} = S_{11}u_1^2 + 2S_{12}u_1u_2 + S_{22}u_2^2$ has $\partial/\partial u_1 = 2S_{11}u_1 + 2S_{12}u_2 = 2(S\mathbf{u})_1$.

Use in machine learning. A Lagrange multiplier finds the first principal direction, the unit vector that keeps the most variance (Section 4.2.2, *The first principal direction*), and the mixing probabilities of a mixture, which must sum to one (Section 3.1.6, *General Gaussian mixtures*).

1.11 Jensen's inequality for the logarithm

The logarithm is concave: its second derivative, $\frac{d^2}{da^2}\log a = -\frac{1}{a^2}$, is negative for every $a > 0$, so its slope decreases, and a chord between two points of its graph lies below the graph. For two points this is the inequality $\log(w_1 a_1 + w_2 a_2) \geq w_1 \log a_1 + w_2 \log a_2$ when $w_1 + w_2 = 1$. The form for K points follows by induction. If $w_K < 1$, put $b = \sum_{k<K} \frac{w_k}{1-w_K} a_k$, whose weights $\frac{w_k}{1-w_K}$ sum to one. Then

$$\log\Big(\sum_{k=1}^{K} w_k a_k\Big) = \log\big(w_K a_K + (1-w_K)\, b\big) \geq w_K \log a_K + (1-w_K)\log b \geq w_K \log a_K + \sum_{k<K} w_k \log a_k,$$

using the two-point form and then the form for $K-1$ points. In the form needed later: for positive numbers $a_1, \ldots, a_K$ and weights $w_k \geq 0$ with $\sum_k w_k = 1$,

$$\log\Big(\sum_k w_k\, a_k\Big) \;\geq\; \sum_k w_k \log a_k,$$

with equality when all the a_k with $w_k > 0$ are equal. For example, with $a = (1, 3)$ and equal weights, $\log 2 = 0.693 \geq \frac{1}{2}(\log 1 + \log 3) = 0.549$.

Use in machine learning. Jensen's inequality proves that the EM algorithm never decreases the log-likelihood (Section 3.1.5, *Why the log-likelihood never decreases (optional)*) and that the Kullback–Leibler divergence minimised by t-SNE is nonnegative (Section 4.4.1, *t-SNE*).

1.12 Median and robust measures of spread

The **median** of a set of numbers is the middle value after sorting (the mean of the two middle values when the count is even). The **median absolute deviation** (MAD) is the median of the distances $|x_j - \text{median}|$. Unlike the mean and standard deviation, these change little when a few values are moved far away. For normally distributed data with mean μ and standard deviation σ, the density is symmetric about μ, so the median is μ. The MAD is then the number m for which half of the values satisfy $|x - \mu| \leq m$. In standard units, with $Z = (x - \mu)/\sigma$, this reads $P(-m/\sigma \leq Z \leq m/\sigma) = \frac{1}{2}$. By symmetry each of the two tails beyond $\pm m/\sigma$ has probability $\frac{1}{4}$, so $P(Z \leq m/\sigma) = \frac{3}{4}$, and the table of the standard normal distribution gives $m/\sigma \approx 0.6745$. Hence $\text{MAD} \approx 0.6745\,\sigma$, so $0.6745\,(x - \text{median})/\text{MAD}$ is on the same scale as a standard score.

Use in machine learning. The median and the MAD give robust anomaly scores that a few extreme values cannot mask (Section 5.2.1, *Standard scores and robust scores*). With the Manhattan distance, the best centroid of a cluster is the coordinate-wise median, the basis of K-medians (Section 2.2, *The K-means objective*).

Chapter 2

Clustering with K-means

Companion notebook in Google Colab: Part A.

2.1 Eight training runs

The same model was trained eight times with different settings. For each run, two quantities were recorded: the training time x_1, in hours, and the peak GPU memory x_2, in gigabytes.

run i	1	2	3	4	5	6	7	8
x_1 (hours)	4	4	2	1	8	8	8	6
x_2 (GB)	4	3	3	3	8	6	9	8

No outcome was recorded, so the data contain no labels. The question is whether the runs fall into clusters and, if so, which run belongs to which. If the runs fall into a few kinds, it may be enough to repeat one run of each kind; runs of the same kind can be scheduled on the same hardware; and a run that belongs to no cluster may indicate an error. More generally, when a large collection of examples has no labels, clustering is often the first step in understanding what the collection contains. For eight points in two dimensions the clusters are visible by eye. For many points with many features they are not, and two things are needed: a quantity that measures the quality of a clustering, and a procedure that finds a clustering of high quality. Applied to the eight runs, the procedure recovers the clusters $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8\}$. Figure 2.1 plots the eight runs.

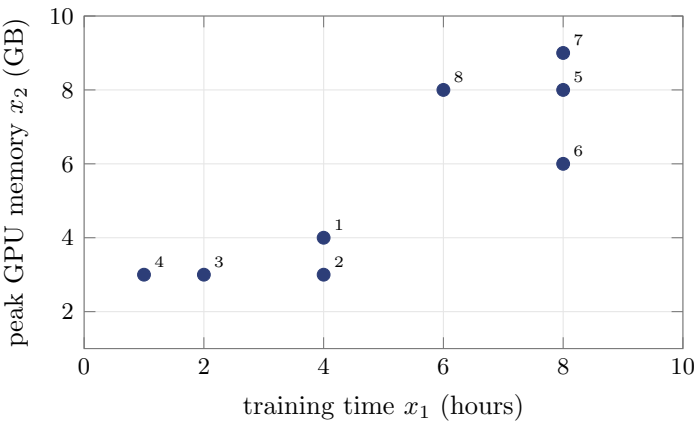


Figure 2.1: The eight training runs. Runs 1–4 have short training times and small memory; runs 5–8 have long training times and large memory.

2.2 The K-means objective

Intuition: pizza outlets

A pizza company wants to open K outlets in a town. Every household orders from one outlet, and the cost of a delivery grows with the distance travelled, more than proportionally for distant homes. A good plan places the outlets so that the total delivery cost over all households is small. In the language of this chapter, the households are the data points, the outlets are the centroids, each household's choice of outlet is its cluster label, and the total of squared delivery distances is the loss J.

2.2.1 Clusters and centroids

Suppose the n examples are to be divided into K clusters. A **partition** assigns to each example i a cluster label $g_i \in \{1, \dots, K\}$ (for *group*). Each cluster k is also given a representative point $\boldsymbol{\mu}_k$, its **centroid**. A cluster is compact when its examples lie close to its centroid, and we measure this by the squared distances from each example to the centroid of its own cluster. Summing over all examples gives the **within-cluster sum of squares**

$$J = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \boldsymbol{\mu}_{g_i} \right\|^2 .$$

The problem of choosing the labels and the centroids so as to minimise J is the **K-means problem**. A small value of J indicates compact clusters.

2.2.2 Three partitions of the eight runs

Three partitions of the eight runs are compared below, each centroid being the mean of its cluster.

Small runs and large runs: $\{1,2,3,4\}$ and $\{5,6,7,8\}$. The centroids are (2.75, 3.25) and (7.50, 7.75), and the squared distances are as follows.

run	point $\mathbf{x}_i$	centre $\boldsymbol{\mu}$	$(x_1-\mu_1)^2+(x_2-\mu_2)^2$	squared distance
1	$(4,4)$	$(2.75,3.25)$	$(1.25)^2+(0.75)^2$	2.1250
2	$(4,3)$	$(2.75,3.25)$	$(1.25)^2+(-0.25)^2$	1.6250
3	$(2,3)$	$(2.75,3.25)$	$(-0.75)^2+(-0.25)^2$	0.6250
4	$(1,3)$	$(2.75,3.25)$	$(-1.75)^2+(-0.25)^2$	3.1250
			cluster 1 total	**7.50**
5	$(8,8)$	$(7.50,7.75)$	$(0.50)^2+(0.25)^2$	0.3125
6	$(8,6)$	$(7.50,7.75)$	$(0.50)^2+(-1.75)^2$	3.3125
7	$(8,9)$	$(7.50,7.75)$	$(0.50)^2+(1.25)^2$	1.8125
8	$(6,8)$	$(7.50,7.75)$	$(-1.50)^2+(0.25)^2$	2.3125
			cluster 2 total	**7.75**

Hence $J = 7.50 + 7.75 = 15.25$.

A single cluster. The centroid is the mean of all eight runs, (5.125, 5.500), and the same computation gives $J = 100.875$.

A mixed split: $\{1,2,5,8\}$ and $\{3,4,6,7\}$. The centroids are $(\frac{4+4+8+6}{4}, \frac{4+3+8+8}{4}) = (5.50, 5.75)$ and $(\frac{2+1+8+8}{4}, \frac{3+3+6+9}{4}) = (4.75, 5.25)$. In the first cluster the squared distances are $1.5^2 + 1.75^2 = 2.25 + 3.0625 = 5.3125$ (run 1), $2.25 + 7.5625 = 9.8125$ (run 2), $6.25 + 5.0625 = 11.3125$ (run 5) and $0.25 + 5.0625 = 5.3125$ (run 8), with sum 31.75. In the second they are $7.5625 + 5.0625 = 12.625$ (run 3), $14.0625 + 5.0625 = 19.125$ (run 4), $10.5625 + 0.5625 = 11.125$ (run 6) and $10.5625 + 14.0625 = 24.625$ (run 7), with sum 67.5. Hence $J = 31.75 + 67.5 = 99.25$.

partition	clusters	number	J
one cluster	all eight runs	1	100.875
a mixed split	$\{1,2,5,8\}$, $\{3,4,6,7\}$	2	99.25
small and large runs	$\{1,2,3,4\}$, $\{5,6,7,8\}$	2	**15.25**

The mixed split reduces J only slightly below its value for a single cluster (99.25 against 100.875), because each of its clusters contains runs from both parts of the plot. The split into small and large runs reduces J to 15% of the single-cluster value. Figure 2.2 shows both splits.

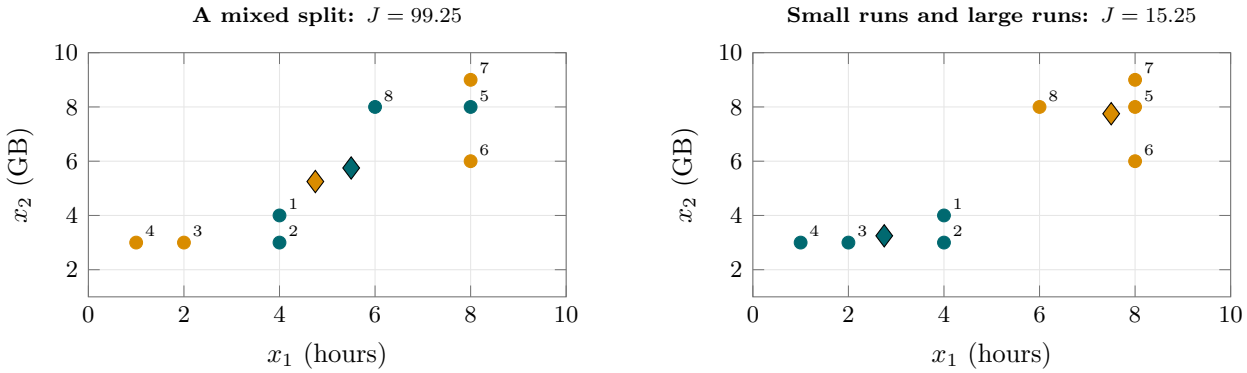


Figure 2.2: The mixed split and the split into small and large runs. Colours show the clusters and diamonds their centroids. In the mixed split each cluster contains runs from both parts of the plot, and J is almost as large as for a single cluster.

2.2.3 The choice of squared distance

Squaring the distances has two consequences. Examples far from their centroid are penalised heavily: doubling the distance multiplies the contribution by four. And, as proved below, the optimal centroid of a cluster is then its mean, which is cheap to compute. If distance is measured instead as the sum of absolute coordinate differences (the Manhattan distance), the optimal centroid is the coordinate-wise median, and the method is called K-medians; with unsquared Euclidean distance the optimal centroid is the *geometric median*, which has no closed form.

Definition: the K-means problem

Given points $\mathbf{x}_1, \ldots, \mathbf{x}_n$ and a number of clusters K, choose labels $g_1, \ldots, g_n \in \{1, \ldots, K\}$ and centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ that minimise $J = \sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2$.

Exhaustive search is impractical. There are K^n possible assignments of labels, because each of the n examples can receive any of the K labels independently of the others, giving $K \times K \times \cdots \times K$ (n factors) choices. This is $2^8 = 256$ for the eight runs, but 2^{1000} for a thousand points and two clusters, a number with 302 digits: $\log_{10} 2^{1000} = 1000 \log_{10} 2 \approx 1000 \times 0.30103 = 301.03$, so $10^{301} < 2^{1000} < 10^{302}$. Lloyd's algorithm instead exploits the fact that each of the two sets of unknowns is easy to optimise when the other is held fixed.

2.3 Lloyd's algorithm

2.3.1 Optimal centroids for a fixed partition

Intuition: where one outlet should stand

Suppose the households served by one outlet are already fixed. The outlet that makes the total squared delivery distance to those households smallest stands at their average position: moving the outlet towards any part of its neighbourhood shortens some deliveries and lengthens others, and the average is the point where the two effects balance exactly.

Hold the labels fixed. Then J is a sum over clusters, and the centroid $\boldsymbol{\mu}_k$ appears only in the terms for cluster k, so each centroid can be optimised separately. Moreover, a squared distance is a sum over coordinates, so each coordinate of the centroid can also be optimised separately. Consider one cluster with values $x_1, \dots, x_m$ in a given coordinate, and a candidate value c for that coordinate of the centroid:

$$f(c) = \sum_{j=1}^{m} (x_j - c)^2.$$

Setting the derivative to zero gives

$$f'(c) = \sum_{j=1}^{m} -2(x_j - c) = -2\Big(\sum_{j=1}^{m} x_j - mc\Big) = 0 \quad \Longrightarrow \quad \boxed{c = \frac{1}{m}\sum_{j=1}^{m} x_j}.$$

Since $f''(c) = 2m > 0$ and f is a quadratic, this is the unique minimum. Hence **the optimal centroid of a cluster is the mean of its points**, $\boldsymbol{\mu}_k = \frac{1}{n_k}\sum_{i:\, g_i = k} \mathbf{x}_i$, where n_k is the number of points in cluster k.

Example

For the cluster of small runs $\{1, 2, 3, 4\}$, the first coordinates are $4, 4, 2, 1$, with sum 11 and mean 2.75. For three candidate values,

$$\begin{aligned} f(2) &= (4-2)^2 + (4-2)^2 + (2-2)^2 + (1-2)^2 = 4 + 4 + 0 + 1 = 9.00, \\ f(2.75) &= 1.25^2 + 1.25^2 + (-0.75)^2 + (-1.75)^2 = 1.5625 + 1.5625 + 0.5625 + 3.0625 = 6.75, \\ f(3.5) &= 0.5^2 + 0.5^2 + (-1.5)^2 + (-2.5)^2 = 0.25 + 0.25 + 2.25 + 6.25 = 9.00. \end{aligned}$$

The mean gives the smallest value. At the mean the deviations 1.25, 1.25, -0.75 and -1.75 sum to zero, which is the condition $f'(c) = 0$.

2.3.2 Within-cluster and between-cluster sums of squares

The fact that deviations from a mean sum to zero gives a decomposition that explains what K-means achieves. Let $\bar{\mathbf{x}}$ be the mean of all n points, and define the **total**, **within-cluster** and **between-cluster** sums of squares

$$T = \sum_{i=1}^{n} \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2, \qquad W = \sum_{i=1}^{n} \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2, \qquad B = \sum_{k=1}^{K} n_k \, \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2,$$

where $\boldsymbol{\mu}_k$ is the mean of cluster k and n_k its number of points. T measures the spread of the whole data set about its mean, and W, the objective J, measures the spread that remains inside the clusters.

Why B is weighted by n_k. B is meant to measure the spread that the clustering itself accounts for. To isolate it, replace every point by the mean of its cluster, keeping one copy per point: the n_k

points of cluster k all move to $\boldsymbol{\mu}_k$. This simplified data set keeps the differences between clusters and removes the differences within them. Its total sum of squares about $\bar{\mathbf{x}}$ adds one term per point, and the n_k points of cluster k each contribute the same amount $\|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2$, so

$$\sum_{i=1}^{n} \|\boldsymbol{\mu}_{g_i} - \bar{\mathbf{x}}\|^2 = \sum_{k=1}^{K} \sum_{i:\, g_i = k} \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2 = \sum_{k=1}^{K} n_k \, \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2 = B.$$

The factor n_k is therefore the number of points that cluster k stands for. Without it, a cluster of two points far from $\bar{\mathbf{x}}$ would count as much as a cluster of two hundred points at the same distance, although the large cluster carries a hundred times as much of the spread T. The weights also keep $\bar{\mathbf{x}}$ as the mean of the simplified data: since $\sum_{i:\, g_i = k} \mathbf{x}_i = n_k \boldsymbol{\mu}_k$, the overall mean is $\bar{\mathbf{x}} = \frac{1}{n} \sum_k n_k \boldsymbol{\mu}_k$, the mean of the cluster means weighted by their sizes. The factor n_k appears again in the proof below, where the same term $\|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2$ is added once for each of the n_k points of the cluster.

Theorem (decomposition of the total sum of squares)

For every partition, with each $\boldsymbol{\mu}_k$ the mean of its cluster, $T = W + B$.

Proof. For a point i in cluster k, write $\mathbf{x}_i - \bar{\mathbf{x}} = (\mathbf{x}_i - \boldsymbol{\mu}_k) + (\boldsymbol{\mu}_k - \bar{\mathbf{x}})$ and expand the squared length. For any two vectors $\mathbf{a}$ and $\mathbf{b}$,

$$\|\mathbf{a} + \mathbf{b}\|^2 = (\mathbf{a} + \mathbf{b})^{\top}(\mathbf{a} + \mathbf{b}) = \mathbf{a}^{\top}\mathbf{a} + \mathbf{a}^{\top}\mathbf{b} + \mathbf{b}^{\top}\mathbf{a} + \mathbf{b}^{\top}\mathbf{b} = \|\mathbf{a}\|^2 + 2\,\mathbf{b}^{\top}\mathbf{a} + \|\mathbf{b}\|^2,$$

because $\mathbf{a}^{\top}\mathbf{b} = \sum_l a_l b_l = \mathbf{b}^{\top}\mathbf{a}$. With $\mathbf{a} = \mathbf{x}_i - \boldsymbol{\mu}_k$ and $\mathbf{b} = \boldsymbol{\mu}_k - \bar{\mathbf{x}}$ this gives

$$\|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 + 2(\boldsymbol{\mu}_k - \bar{\mathbf{x}})^{\top}(\mathbf{x}_i - \boldsymbol{\mu}_k) + \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2.$$

Sum over the n_k points of cluster k. The middle term becomes $2(\boldsymbol{\mu}_k - \bar{\mathbf{x}})^{\top} \sum_{i:\, g_i = k} (\mathbf{x}_i - \boldsymbol{\mu}_k)$, and the sum is the zero vector: since $\boldsymbol{\mu}_k = \frac{1}{n_k} \sum_{i:\, g_i = k} \mathbf{x}_i$, we have $\sum_{i:\, g_i = k} \mathbf{x}_i = n_k \boldsymbol{\mu}_k$, and therefore

$$\sum_{i:\, g_i = k} (\mathbf{x}_i - \boldsymbol{\mu}_k) = \sum_{i:\, g_i = k} \mathbf{x}_i - n_k \boldsymbol{\mu}_k = n_k \boldsymbol{\mu}_k - n_k \boldsymbol{\mu}_k = \mathbf{0}.$$

The last term, $\|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2$, is the same for every point of the cluster, so summing it over the n_k points gives $n_k \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2$. Hence

$$\sum_{i:\, g_i = k} \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \sum_{i:\, g_i = k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 + n_k \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2,$$

and summing over k gives $T = W + B$. □

Since T depends only on the data, *minimising the within-cluster sum of squares is the same as maximising the between-cluster sum of squares*: K-means looks for clusters that are internally tight and, equivalently, whose means are spread as far apart as possible, weighted by size.

Example

For the eight runs, $\bar{\mathbf{x}} = (5.125,\ 5.5)$ and $T = 100.875$. For the small and large runs, with means $(2.75,\ 3.25)$ and $(7.5,\ 7.75)$, the within-cluster sum is $W = 15.25$, the value of J found in Section 2.2, *The K-means objective*. The total is $T = \sum_i \tilde{x}_{i1}^2 + \sum_i \tilde{x}_{i2}^2 = 54.875 + 46.000 = 100.875$, from the deviations in Chapter 1, *Mathematical background*. The mean of the small runs differs from $\bar{\mathbf{x}}$ by $(2.75 - 5.125,\ 3.25 - 5.5) = (-2.375,\ -2.25)$, so its squared distance from $\bar{\mathbf{x}}$ is $2.375^2 + 2.25^2 = 5.640625 + 5.0625 = 10.703125$. The eight runs form two clusters of four, so $\bar{\mathbf{x}} = \frac{1}{8}(4\boldsymbol{\mu}_1 + 4\boldsymbol{\mu}_2) = \frac{1}{2}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)$ is the midpoint of the two means. The mean of the large runs therefore differs from $\bar{\mathbf{x}}$ by $(2.375,\ 2.25)$, and its squared distance is also 10.703125. Hence $B = 4 \times 10.703125 + 4 \times 10.703125 = 85.625$, and indeed $15.25 + 85.625 = 100.875$. The clusters

account for 84.9% of the total sum of squares.

Figure 2.3 draws the three sums for the eight runs: T from each run to the overall mean, W from each run to its own cluster mean, and B from each cluster mean to the overall mean.

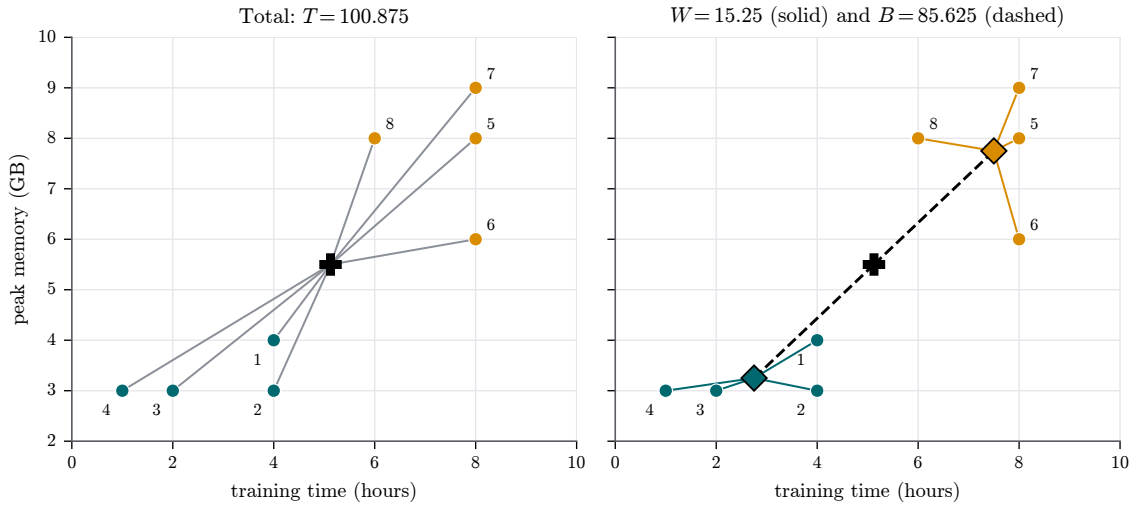


Figure 2.3: The decomposition $T = W + B$ for the eight runs. Left: the total sum of squares adds the squared lengths of the segments from each run to the overall mean (cross). Right: the same quantity split into the squared lengths from each run to its cluster mean (solid, W) and from each cluster mean to the overall mean, counted four times (dashed, B).

2.3.3 Optimal partition for fixed centroids

Intuition: ordering from the nearest outlet

Now suppose the outlets are already built. Each household simply orders from the outlet nearest to it; no other choice gives that household a shorter delivery, and its choice does not affect anyone else's.

Now hold the centroids fixed. J is a sum of one term for each example, and the term for example i, $\|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2$, depends only on its own label g_i. Each label can therefore be chosen separately, and the best choice minimises its own term:

$$g_i = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2.$$

In words, each example is assigned to the cluster with the nearest centroid. If two centroids are equally near, either choice gives the same value of J; by convention a point keeps its current label if that centroid is among the nearest, and otherwise takes the one with the smaller index. With this rule a point changes label only when a strictly nearer centroid exists.

2.3.4 The boundary between two clusters

Each centroid is assigned the points that are nearer to it than to any other centroid. A point $\mathbf{x}$ is nearer to $\boldsymbol{\mu}_a$ than to $\boldsymbol{\mu}_b$ when $\|\mathbf{x} - \boldsymbol{\mu}_a\|^2 < \|\mathbf{x} - \boldsymbol{\mu}_b\|^2$. For any centroid $\boldsymbol{\mu}$,

$$\|\mathbf{x} - \boldsymbol{\mu}\|^2 = (\mathbf{x} - \boldsymbol{\mu})^\top(\mathbf{x} - \boldsymbol{\mu}) = \mathbf{x}^\top\mathbf{x} - \mathbf{x}^\top\boldsymbol{\mu} - \boldsymbol{\mu}^\top\mathbf{x} + \boldsymbol{\mu}^\top\boldsymbol{\mu} = \|\mathbf{x}\|^2 - 2\boldsymbol{\mu}^\top\mathbf{x} + \|\boldsymbol{\mu}\|^2,$$

since $\mathbf{x}^\top\boldsymbol{\mu} = \boldsymbol{\mu}^\top\mathbf{x}$. Applying this to both sides,

$$\|\mathbf{x}\|^2 - 2\boldsymbol{\mu}_a^\top\mathbf{x} + \|\boldsymbol{\mu}_a\|^2 \;<\; \|\mathbf{x}\|^2 - 2\boldsymbol{\mu}_b^\top\mathbf{x} + \|\boldsymbol{\mu}_b\|^2.$$

The term $\|\mathbf{x}\|^2$ appears on both sides and cancels, leaving

$$2(\boldsymbol{\mu}_b - \boldsymbol{\mu}_a)^\top \mathbf{x} \;<\; \|\boldsymbol{\mu}_b\|^2 - \|\boldsymbol{\mu}_a\|^2.$$

This inequality is *linear* in $\mathbf{x}$. The boundary, where the two sides are equal, is a straight line in two dimensions and a hyperplane in general. Write the boundary as $\mathbf{w}^\top \mathbf{x} = \beta$, with $\mathbf{w} = 2(\boldsymbol{\mu}_b - \boldsymbol{\mu}_a)$ and $\beta = \|\boldsymbol{\mu}_b\|^2 - \|\boldsymbol{\mu}_a\|^2$. Any two points $\mathbf{x}$ and $\mathbf{x}'$ on it satisfy $\mathbf{w}^\top \mathbf{x} - \mathbf{w}^\top \mathbf{x}' = \beta - \beta = 0$, that is $\mathbf{w}^\top (\mathbf{x} - \mathbf{x}') = 0$, so every segment lying in the boundary is perpendicular to $\mathbf{w}$, and hence to $\boldsymbol{\mu}_b - \boldsymbol{\mu}_a$. The boundary passes through the midpoint $\frac{1}{2}(\boldsymbol{\mu}_a + \boldsymbol{\mu}_b)$: substituting the midpoint into the left-hand side gives $2(\boldsymbol{\mu}_b - \boldsymbol{\mu}_a)^\top \frac{1}{2}(\boldsymbol{\mu}_a + \boldsymbol{\mu}_b)$, and multiplying out, with the cross terms cancelling because $\boldsymbol{\mu}_b^\top \boldsymbol{\mu}_a = \boldsymbol{\mu}_a^\top \boldsymbol{\mu}_b$, $(\boldsymbol{\mu}_b - \boldsymbol{\mu}_a)^\top (\boldsymbol{\mu}_a + \boldsymbol{\mu}_b) = \boldsymbol{\mu}_b^\top \boldsymbol{\mu}_a + \boldsymbol{\mu}_b^\top \boldsymbol{\mu}_b - \boldsymbol{\mu}_a^\top \boldsymbol{\mu}_a - \boldsymbol{\mu}_a^\top \boldsymbol{\mu}_b = \|\boldsymbol{\mu}_b\|^2 - \|\boldsymbol{\mu}_a\|^2$. The boundary is therefore the **perpendicular bisector** of the two centroids. With K centroids, the region won by $\boldsymbol{\mu}_k$ is the set of points that satisfy $K-1$ such inequalities at once, an intersection of half-spaces (half-planes in two dimensions). A region is **convex** if, together with any two of its points $\mathbf{x}$ and $\mathbf{x}'$, it contains the whole segment joining them, that is, every point $t\,\mathbf{x} + (1-t)\,\mathbf{x}'$ with $0 \le t \le 1$. Each half-space $\mathbf{w}^\top \mathbf{x} < \beta$ is convex: if $\mathbf{w}^\top \mathbf{x} < \beta$ and $\mathbf{w}^\top \mathbf{x}' < \beta$, then for $0 \le t \le 1$

$$\mathbf{w}^\top \big(t\,\mathbf{x} + (1-t)\,\mathbf{x}'\big) = t\,\mathbf{w}^\top \mathbf{x} + (1-t)\,\mathbf{w}^\top \mathbf{x}' < t\,\beta + (1-t)\,\beta = \beta.$$

If two points lie in each of the $K-1$ half-spaces, the segment joining them lies in each of them as well, so their intersection is a convex region, called the **Voronoi cell** of $\boldsymbol{\mu}_k$.

Example

For the final centroids of the eight runs, $\boldsymbol{\mu}_{\text{small}} = (2.75,\ 3.25)$ and $\boldsymbol{\mu}_{\text{large}} = (7.5,\ 7.75)$, we have $2(\boldsymbol{\mu}_{\text{large}} - \boldsymbol{\mu}_{\text{small}}) = 2(7.5-2.75,\ 7.75-3.25) = 2(4.75,\ 4.5) = (9.5,\ 9)$, $\|\boldsymbol{\mu}_{\text{large}}\|^2 = 7.5^2 + 7.75^2 = 56.25 + 60.0625 = 116.3125$ and $\|\boldsymbol{\mu}_{\text{small}}\|^2 = 2.75^2 + 3.25^2 = 7.5625 + 10.5625 = 18.125$. The boundary is the line

$$9.5\,x_1 + 9\,x_2 = 116.3125 - 18.125 = 98.1875.$$

For run 1, $9.5 \times 4 + 9 \times 4 = 74 < 98.1875$, so run 1 lies on the side of the small runs.

With more than two centroids, every pair of centroids is separated by its own perpendicular bisector, and the plane is divided into one region, or *cell*, per centroid: the points nearer to that centroid than to any other. Figure 2.4 shows the cells for three centroids.

In practice this means that K-means can separate groups only by straight lines (hyperplanes in higher dimensions), which is why it fails on curved or elongated clusters (Section 2.5.4, *Dependence on cluster shape and size*).

2.3.5 The algorithm and its convergence

Intuition: outlets that follow their customers

Open the outlets anywhere. Each household orders from its nearest outlet. Each outlet then relocates to the average position of the households that ordered from it. With the outlets in new places, some households now find a different outlet nearer and switch. Repeat until no household switches. Every relocation and every switch shortens the total delivery distance or leaves it unchanged, so the process cannot go on forever.

Alternating the two optimisations gives the standard algorithm for the K-means problem.

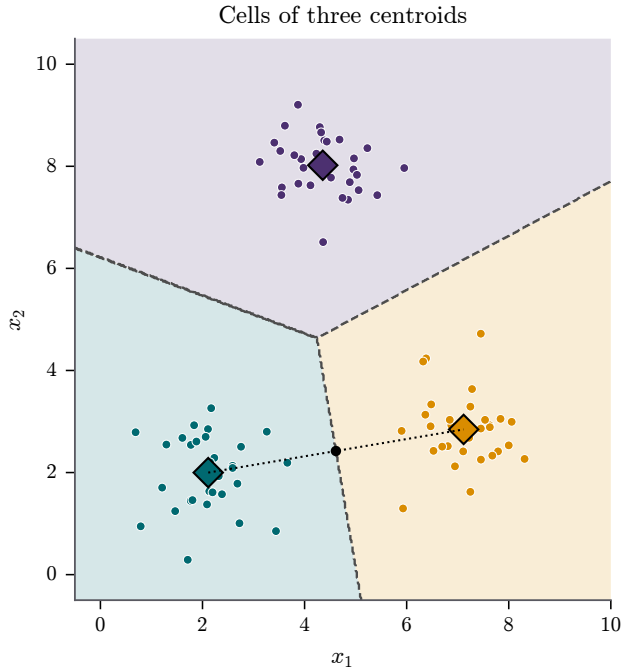


Figure 2.4: The three centroids found by K-means for 90 points from three clusters (Section 2.5.5, *Choosing the number of clusters*) and the cell of points nearest to each. Every boundary (dashed) lies on the perpendicular bisector of two centroids; the dotted segment joins two centroids and the dot marks its midpoint. Each cell is convex.

Algorithm 1: Lloyd's algorithm for K-means

Input: points $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$; number of clusters K; initial centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$
Output: labels $g_1, \dots, g_n$ and centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$

repeat
  for $i = 1, \dots, n$ **do** — assignment step
    $g_i \leftarrow \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$
  end for
  for $k = 1, \dots, K$ **do** — update step
    $n_k \leftarrow$ number of points with $g_i = k$
    $\boldsymbol{\mu}_k \leftarrow \frac{1}{n_k} \sum_{i:\, g_i = k} \mathbf{x}_i$ (if $n_k > 0$)
  end for
until no label g_i has changed

J does not increase. In the assignment step each point chooses the smallest of its possible terms, and its previous choice was among them, so J cannot increase. In the update step each centroid moves to the minimiser of its cluster's sum of squares, and its previous position was one of the candidates, so again J cannot increase.

The algorithm terminates. If some point changes cluster in an assignment step, it moves to a strictly nearer centroid and J strictly decreases. There are finitely many partitions, at most K^n (Section 2.2, *The K-means objective*). After the update step of an iteration every centroid is the mean of its cluster, so J then equals the within-cluster sum of squares $W(P)$ of the current partition P, a number fixed by P alone. Suppose the next assignment step changes the partition to P'. Then J strictly decreases in that assignment step and does not increase in the update step that follows, so $W(P') < W(P)$. For as long as the partition keeps changing, the values of W at the end of successive iterations therefore decrease strictly, and a partition that occurred earlier, having a larger value of W, cannot occur again, so after finitely many iterations the partition stops changing, and the centroids stop changing with it.

Cost. An assignment step computes nK squared distances in d dimensions, about nKd operations; an update step needs about nd. The number of iterations is usually small (tens), so the algorithm is practical for millions of points.

Remark: a local, not a global, minimum

The algorithm stops at a partition that neither step can improve. Such a partition need not minimise J. Section 2.5.1, *Dependence on initialisation* gives an example with four points in which the algorithm stops at $J = 16$, although $J = 4$ is attainable.

Remark: empty clusters

If no point is nearest to some centroid, that cluster becomes empty and its mean is undefined. Implementations either leave the centroid unchanged or move it to the point that is farthest from its own centroid.

2.4 A worked example

Intuition: two outlets opened in the same neighbourhood

Both outlets begin among the small runs, as if both pizza outlets had opened in the same corner of town. The households in the far neighbourhood all order from whichever outlet is slightly nearer, that outlet moves towards them, then further as it gains more of them, and after a few rounds each outlet sits in the middle of its own neighbourhood. Figure 2.5 shows the same process for a town with two neighbourhoods.

We apply Lloyd's algorithm to the eight runs with $K = 2$. To show that a poor initialisation can be corrected, both initial centroids are placed in the same cluster: $\mu_A = (2, 3)$, which is run 3, and $\mu_B = (1, 3)$, which is run 4.

2.4.1 First iteration

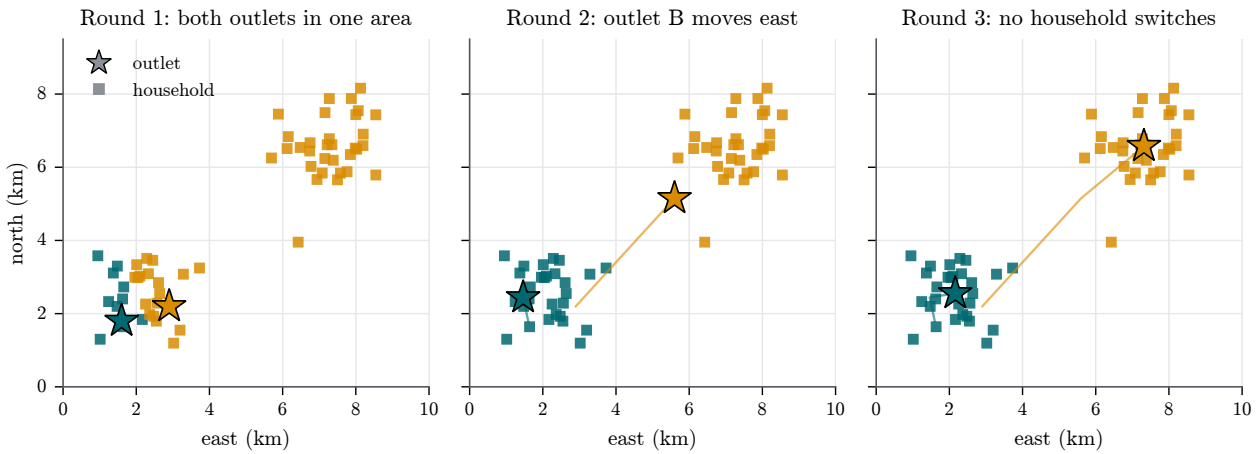


Figure 2.5: Two pizza outlets (stars) in a town with two neighbourhoods of households (squares), both opened in the south-west. Each household orders from its nearest outlet (colour); each outlet then moves to the average position of its customers. Outlet B follows the north-east households, and after three rounds no household switches.

Assignment step. For run 1, $\mathbf{x}_1 = (4, 4)$, the squared distance to μ_A is $(4-2)^2+(4-3)^2 = 4+1 = 5$, and to μ_B it is $(4-1)^2 + (4-3)^2 = 9+1 = 10$. Run 1 is therefore assigned to A. The table gives the squared distances for every run; the smaller value in each row is shown in bold.

run	$\mathbf{x}_i$	$\lVert\mathbf{x}_i - \mu_A\rVert^2$ $\mu_A = (2.000, 3.000)$	$\lVert\mathbf{x}_i - \mu_B\rVert^2$ $\mu_B = (1.000, 3.000)$	assigned to
1	$(4, 4)$	**5.00**	10.00	A
2	$(4, 3)$	**4.00**	9.00	A
3	$(2, 3)$	**0.00**	1.00	A
4	$(1, 3)$	1.00	**0.00**	B
5	$(8, 8)$	**61.00**	74.00	A
6	$(8, 6)$	**45.00**	58.00	A
7	$(8, 9)$	**72.00**	85.00	A
8	$(6, 8)$	**41.00**	50.00	A

Only run 4, which coincides with μ_B, is assigned to B. With these centroids, $J = 228.00$.

Update step. Cluster A contains runs 1, 2, 3, 5, 6, 7, 8. The mean of their first coordinates is $(4+4+2+8+8+8+6)/7 = 40/7$, and of their second coordinates $(4+3+3+8+6+9+8)/7 = 41/7$, so $\mu_A = (5.714, 5.857)$. Cluster B contains run 4 alone, and $\mu_B = (1, 3)$ is unchanged.

2.4.2 Second iteration

Assignment step. The centroid μ_A now lies among the runs with large values, and runs 2 and 3 are closer to μ_B.

run	$\mathbf{x}_i$	$\lVert\mathbf{x}_i - \mu_A\rVert^2$ $\mu_A = (5.714, 5.857)$	$\lVert\mathbf{x}_i - \mu_B\rVert^2$ $\mu_B = (1.000, 3.000)$	assigned to
1	$(4, 4)$	**6.39**	10.00	A
2	$(4, 3)$	11.10	**9.00**	B
3	$(2, 3)$	21.96	**1.00**	B
4	$(1, 3)$	30.39	**0.00**	B
5	$(8, 8)$	**9.82**	74.00	A
6	$(8, 6)$	**5.24**	58.00	A
7	$(8, 9)$	**15.10**	85.00	A
8	$(6, 8)$	**4.67**	50.00	A

Update step. Cluster A contains runs 1, 5, 6, 7, 8. Their first coordinates sum to $4+8+8+8+6 = 34$ and their second coordinates to $4 + 8 + 6 + 9 + 8 = 35$; dividing by 5 gives $\mu_A = (6.800, 7.000)$. Cluster B contains runs 2, 3, 4, and $\mu_B = (2.333, 3.000)$. Run 1 remains in cluster A.

2.4.3 Third and fourth iterations

In the third iteration run 1 is at squared distance 16.84 from $\mu_A = (6.800, 7.000)$ and 3.778 from $\mu_B = (2.333, 3.000)$, so it moves to cluster B, giving the clusters $\{5, 6, 7, 8\}$ and $\{1, 2, 3, 4\}$. In the fourth iteration the centroids are $\mu_A = (7.5,\ 7.75)$ and $\mu_B = (2.75,\ 3.25)$, and by Section 2.3.4, *The boundary between two clusters* a run is nearer μ_B exactly when $9.5\,x_1 + 9\,x_2 < 98.1875$. For runs 1 to 4 the left-hand side is $38 + 36 = 74$, $38 + 27 = 65$, $19 + 27 = 46$ and $9.5 + 27 = 36.5$, all below 98.1875. For runs 5 to 8 it is $76 + 72 = 148$, $76 + 54 = 130$, $76 + 81 = 157$ and $57 + 72 = 129$, all above. No assignment changes, and the algorithm terminates. Figure 2.6 shows the four iterations.

iteration	centroid A	centroid B	clusters A | B	J after assignment	J after update
1	$(2.00, 3.00)$	$(1.00, 3.00)$	1, 2, 3, 5, 6, 7, 8 | 4	228.00	74.29
2	$(5.71, 5.86)$	$(1.00, 3.00)$	1, 5, 6, 7, 8 | 2, 3, 4	51.22	33.47
3	$(6.80, 7.00)$	$(2.33, 3.00)$	5, 6, 7, 8 | 1, 2, 3, 4	20.40	15.25
4	$(7.50, 7.75)$	$(2.75, 3.25)$	5, 6, 7, 8 | 1, 2, 3, 4	15.25	15.25

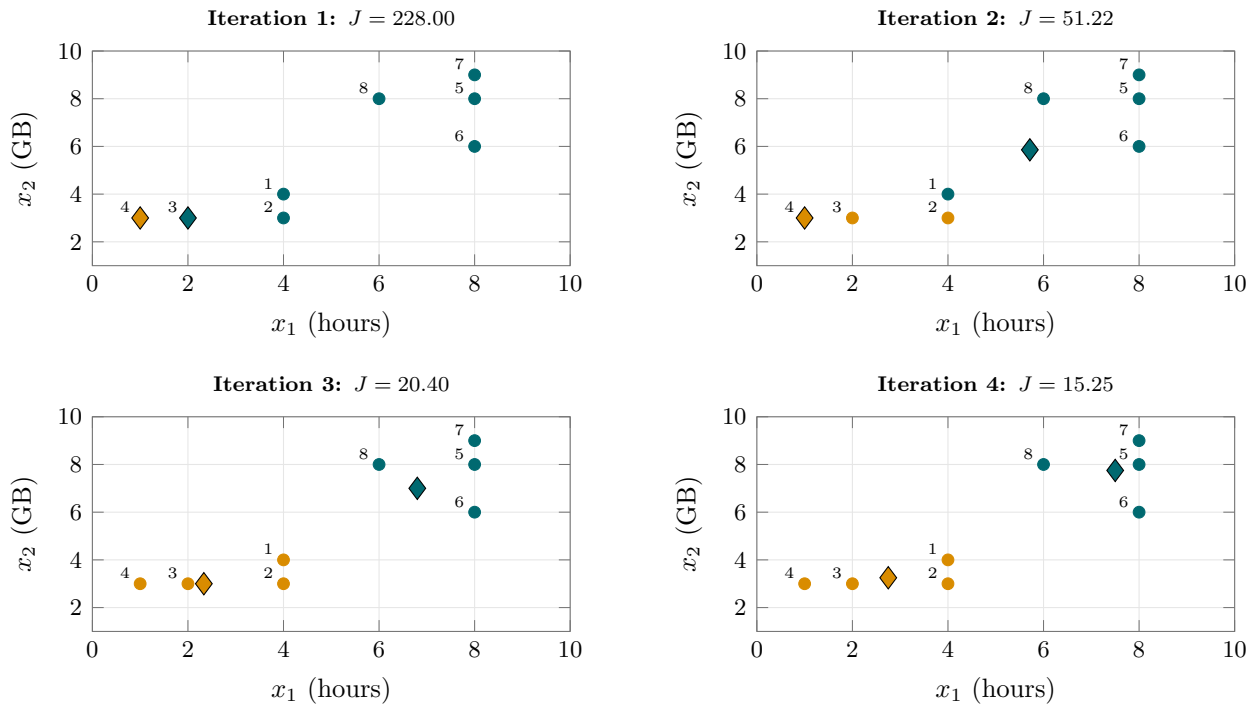


Figure 2.6: The four iterations of Lloyd's algorithm on the eight runs. Colours show the assignment made in each iteration; diamonds show the centroids used for that assignment. Centroid A (teal) begins among the small runs and ends at the mean of the large ones.

Reading the last two columns from left to right and from top to bottom, J never increases, as proved above. The final partition, $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8\}$ with $J = 15.25$, is the split into small and large runs that the scatter plot of the runs suggests. The labels A and B carry no meaning beyond identifying the two centroids. Figure 2.7 follows the two centroids from their starting points to the final means.

For this small example the result can be checked exhaustively. None of the 256 assignments of the eight runs to two clusters has J smaller than 15.25. Moreover, each of the 28 pairs of runs, taken as initial centroids with the lower-numbered run as the first centroid, leads to the same final partition.

2.5 Using K-means in practice

2.5.1 Dependence on initialisation

Intuition: a town with four housing blocks

A rectangular town has four housing blocks, one at each corner, and two outlets. If one outlet starts on the west side and one on the east, each serves the two blocks on its side. If they start on the north and south sides, each serves the two blocks on its side instead. In both cases no household wants to switch and no outlet wants to move, so the process stops, although one plan has much shorter deliveries than the other. Each step only improves the plan locally; it never considers moving an outlet across town.

Consider four points at the corners of a rectangle, $(0, 0)$, $(4, 0)$, $(0, 2)$ and $(4, 2)$, to be divided into two clusters.

With initial centroids $(0, 1)$ and $(4, 1)$, the two left points are assigned to the first centroid and the two right points to the second. The mean of each pair equals its centroid, so the update step changes nothing. Each pair contributes $1^2 + 1^2 = 2$, and $J = 4$.

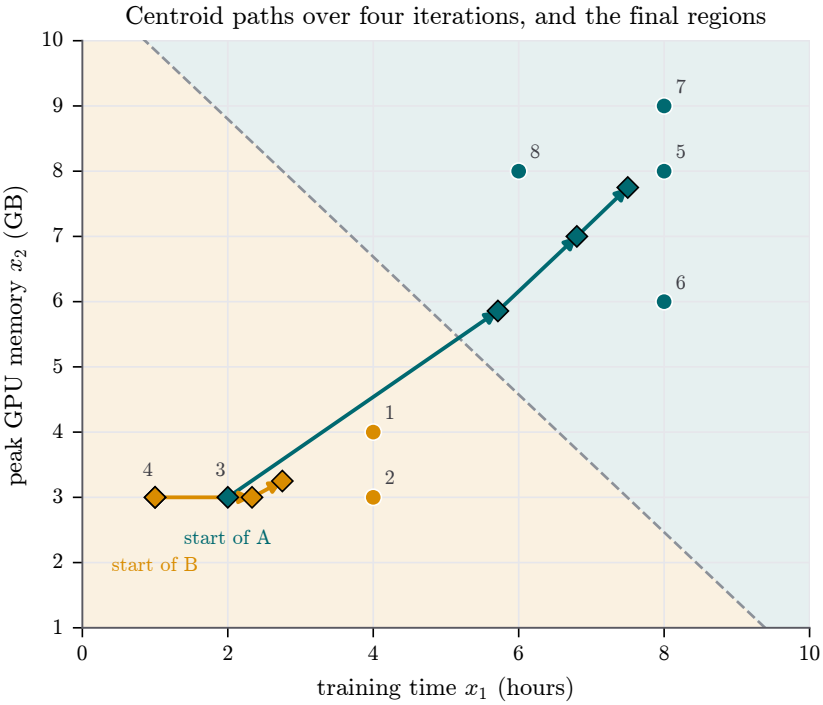


Figure 2.7: Paths of the two centroids over the four iterations above, from the poor start inside the small group. The shading shows which centroid is nearer at the end; the dashed line is the boundary between the clusters, the perpendicular bisector of the final centroids.

With initial centroids $(2, 0)$ and $(2, 2)$, the two lower points are assigned to the first centroid (squared distance 4, against 8) and the two upper points to the second. Again each mean equals its centroid and nothing changes. Each pair contributes $2^2 + 2^2 = 8$, and $J = 16$. Figure 2.8 shows both outcomes.

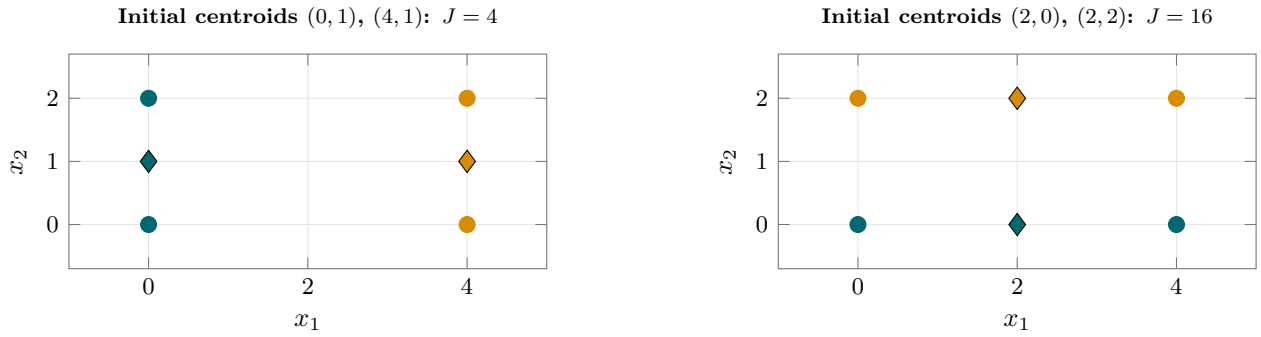


Figure 2.8: Two initialisations for the same four points. Both final partitions are fixed points of Lloyd's algorithm; only the first minimises J.

Both final partitions are fixed points of the algorithm, but only the first is optimal. Since neither step can increase J, the algorithm cannot leave a fixed point once it has reached one, and the result depends on where it starts. Figure 2.9 shows the same effect for the four numbers $0, 1, 2, 4$ on a line. Plotted against the two centroids, J has separate valleys, and the algorithm stops at the bottom of the valley in which it starts.

2.5.2 Choosing the initial centroids: K-means++

What goes wrong with a random start. The simplest start takes K of the data points, chosen at random, as the initial centroids. Consider the 90 points of Section 2.5.5, *Choosing the number*

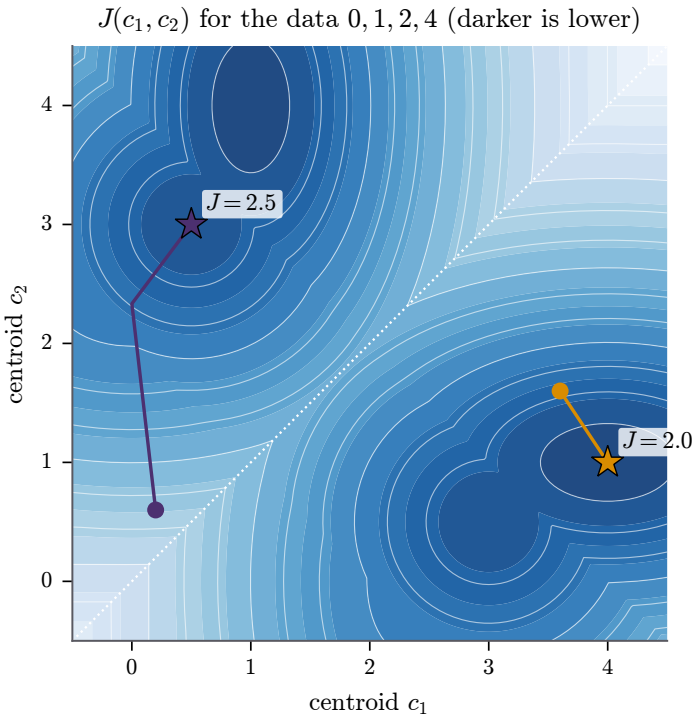


Figure 2.9: The objective $J(c_1, c_2) = \sum_i \min_k (x_i - c_k)^2$ for the one-dimensional data $0, 1, 2, 4$, as a function of the two centroids. It has separate valleys: Lloyd's algorithm started at the two dots stops at the two stars, one at the optimum $J = 2$ and one at $J = 2.5$. The dotted diagonal marks equal centroids; the picture is symmetric about it.

of clusters, three well-separated groups of 30, with $K = 3$. The three random points fall in three different groups only if the second lies outside the group of the first (60 of the remaining 89 points) and the third lies in the one group still unused (30 of the remaining 88), which happens with probability $\frac{60}{89} \times \frac{30}{88} = 0.230$. In the other 77% of random starts, at least two initial centroids lie in the same group. Lloyd's algorithm sometimes repairs such a start, but not always. Figure 2.10 shows a start with two centroids in the top group. The two share that group and each takes part of it, while the third centroid is left to serve both lower groups and settles in the empty space between them. Every step lowers J, yet the algorithm stops at $J = 453.1$, about 5.8 times the best value 77.6. No centroid can move across the gap to the lower groups, because the update step places each centroid at the mean of the points already assigned to it.

Two remedies. The first is to run the algorithm from several random starts and keep the result with the smallest J. The second is to choose the initial centroids so that they are spread out over the data, which is what **K-means++** does (Arthur and Vassilvitskii, 2007).

The idea of K-means++. Choose the first centroid at random among the points. Then choose each further centroid preferably *far from the centroids already chosen*, because a point far from every chosen centroid probably belongs to a group that has no centroid yet. The obvious rule, always taking the point farthest from the chosen centroids, is easily misled: a single stray point far from everything else would be chosen and become a cluster of its own. K-means++ therefore chooses at random, with probabilities that favour distant points. Let D_i be the squared distance from point i to the nearest centroid chosen so far; the next centroid is point i with probability $D_i / \sum_l D_l$. A whole group without a centroid contributes many large D_i and so is very likely to receive the next centroid, whereas a stray point contributes only one. The choice of the *squared* distance is not arbitrary: D_i is exactly what point i would contribute to J if the centroids chosen so far were the final ones, so K-means++ places the next centroid, with high probability, where the current value of J is concentrated, which is where a new centroid can lower J the most.

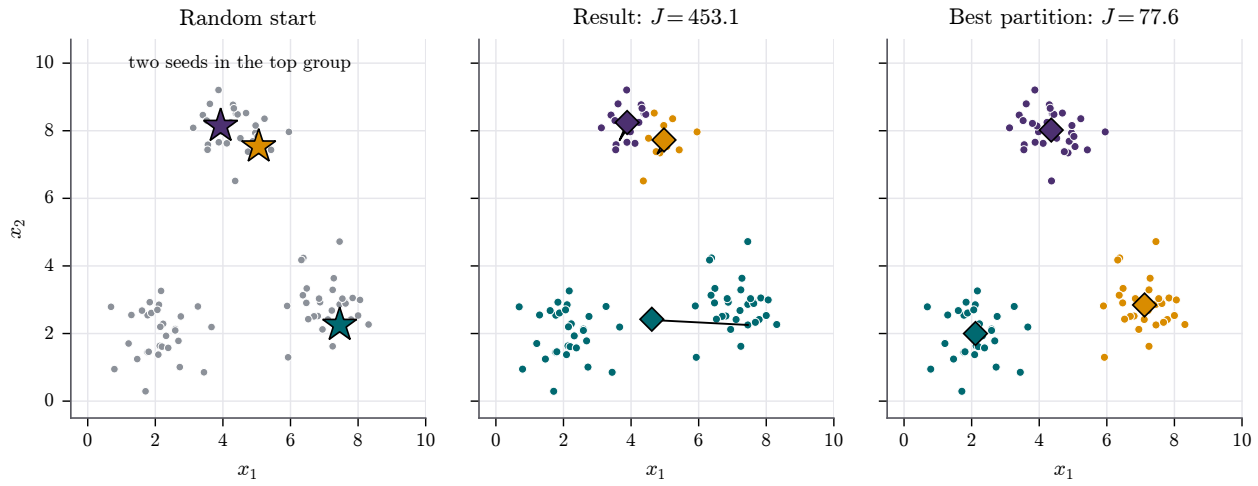


Figure 2.10: A poor random start on the 90 points of Section 2.5.5, *Choosing the number of clusters*. Left: two of the three initial centroids (stars) lie in the top group. Centre: Lloyd's algorithm splits the top group between them and leaves the third centroid (teal, its path drawn) between the two lower groups; diamonds are the final centroids. Right: the best partition, for comparison. Colours identify clusters within each panel only.

Example: the four corners

Take the rectangle of the previous example, $(0,0)$, $(4,0)$, $(0,2)$ and $(4,2)$, with $K = 2$, and suppose the first centroid is $(0,0)$. The squared distances from it are $D = 0,\ 16,\ 4,\ 20$, with sum 40, so K-means++ picks $(4,0)$ with probability $16/40 = 0.4$, $(0,2)$ with probability $4/40 = 0.1$ and $(4,2)$ with probability $20/40 = 0.5$; the first centroid itself has probability 0. If the second centroid is $(4,0)$ or $(4,2)$, the points split into the left and right pairs and Lloyd's algorithm ends at the optimum $J = 4$. Only the choice $(0,2)$ leads to the top and bottom pairs, with $J = 16$. By symmetry the same holds whichever corner is chosen first, so K-means++ reaches the optimum with probability $0.4 + 0.5 = 0.9$. Two corners chosen uniformly at random form one of 6 possible pairs, of which 4 lie on opposite sides; they reach the optimum with probability $4/6 \approx 0.67$.

Algorithm 2: K**-means++ initialisation**

Input: points $\mathbf{x}_1, \dots, \mathbf{x}_n$; number of clusters K
Output: initial centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$

1: choose $\boldsymbol{\mu}_1$ uniformly at random among the points; set $D_i \leftarrow \infty$ for every i
2: **for** $k = 2, \dots, K$ **do**
3: **for** $i = 1, \dots, n$ **do**
4: $D_i \leftarrow \min\big(D_i,\ \|\mathbf{x}_i - \boldsymbol{\mu}_{k-1}\|^2\big)$ squared distance to the nearest chosen centroid
5: **end for**
6: choose $\boldsymbol{\mu}_k = \mathbf{x}_i$ with probability $D_i / \sum_{l=1}^{n} D_l$
7: **end for**

Line 4 updates each D_i using only the newest centroid, since the nearest of the chosen centroids is either the previous nearest or the new one. A point that coincides with a chosen centroid has $D_i = 0$ and cannot be chosen again. Lloyd's algorithm is then started from the K centroids chosen. The initialisation costs about nKd operations, the same as one assignment step. Figure 2.11 shows the three choices on 90 points from three clusters.

On the 90 points of Section 2.5.5, *Choosing the number of clusters*, 400 random starts reached the best partition in 84% of cases, and 400 K-means++ starts in 97.8%; every random start that failed had two initial centroids in the same group. With ten restarts of either kind, the probability of missing the optimum becomes very small.

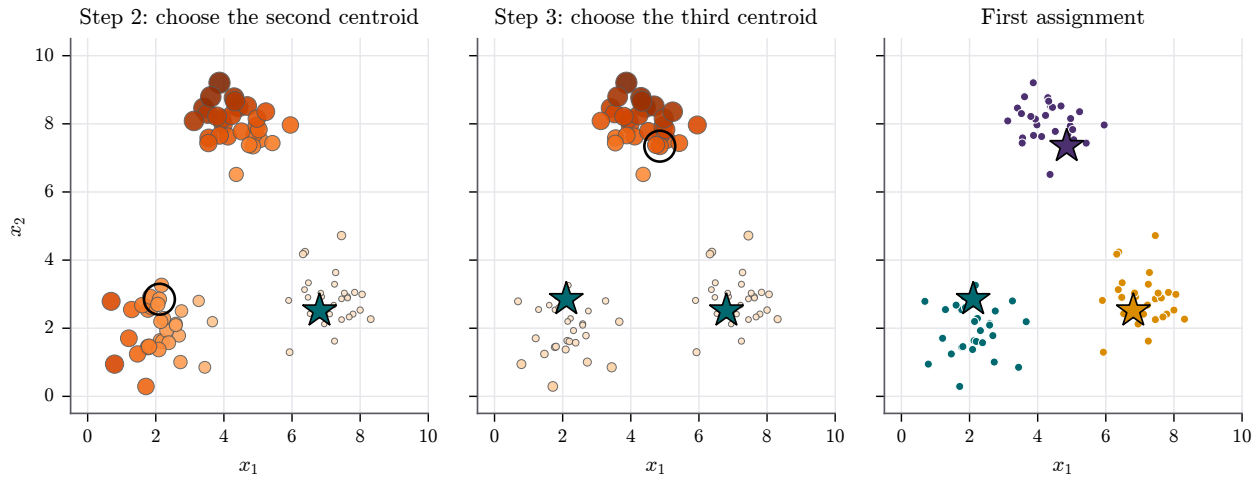


Figure 2.11: K-means++ on the 90 points of Section 2.5.5, *Choosing the number of clusters*. The first centroid (star) was a point chosen at random. Left: the probability of each point being chosen next, proportional to its squared distance from the nearest centroid already chosen (larger and darker discs are more likely); the circled point was chosen. Centre: the same after the second choice; points near either centroid are now unlikely to be chosen. Right: the three centroids and the first assignment of Lloyd's algorithm, already close to the three clusters.

Two results explain why such heuristics are used. Finding the partition with the smallest J is computationally hard, even for $K = 2$ when the number of features may grow (it is NP-hard; Aloise, Deshpande, Hansen and Popat, 2009): no known algorithm finds it for every data set in a time that grows only as a power of the size of the data. On the other hand, K-means++ comes with a guarantee: the expected value of J at its initial centroids is at most $8(\ln K + 2)$ times the optimal value (Arthur and Vassilvitskii, 2007), and Lloyd's iterations can only lower it further.

2.5.3 Dependence on the scale of the features

Intuition: rent in rupees, distance in kilometres

Compare hostel rooms by their distance from the department in kilometres and their monthly rent in rupees. A difference of 2 km and a difference of 2 rupees count equally in the distance formula, so rooms end up grouped almost entirely by rent, however different their locations. Unless the features are put on comparable scales, the feature with the largest numbers dominates the distance.

The squared distance is a sum of squared coordinate differences, so a feature measured on a large numerical scale dominates the distance regardless of its relevance.

Consider eighty runs, generated for illustration, of two kinds that differ in training time, about 2 hours against about 5 hours, but not in peak memory. Training time is recorded in hours (standard deviation 1.6) and memory in megabytes (standard deviation 3634). A difference of 3 hours contributes $3^2 = 9$ to a squared distance, whereas a difference in memory of half a standard deviation, about 1,817 MB, contributes about 3.3 million. The distances are therefore determined almost entirely by memory, which carries no information about the two kinds. Call the *agreement* between a clustering and known groups the fraction of points placed with their own group, after the clusters are named to match the groups as well as possible. On the raw features, the clusters found by K-means agree with the two kinds for 51% of the runs, which is no better than chance. After each feature is standardised, the agreement is 100%. Figure 2.12 shows both clusterings.

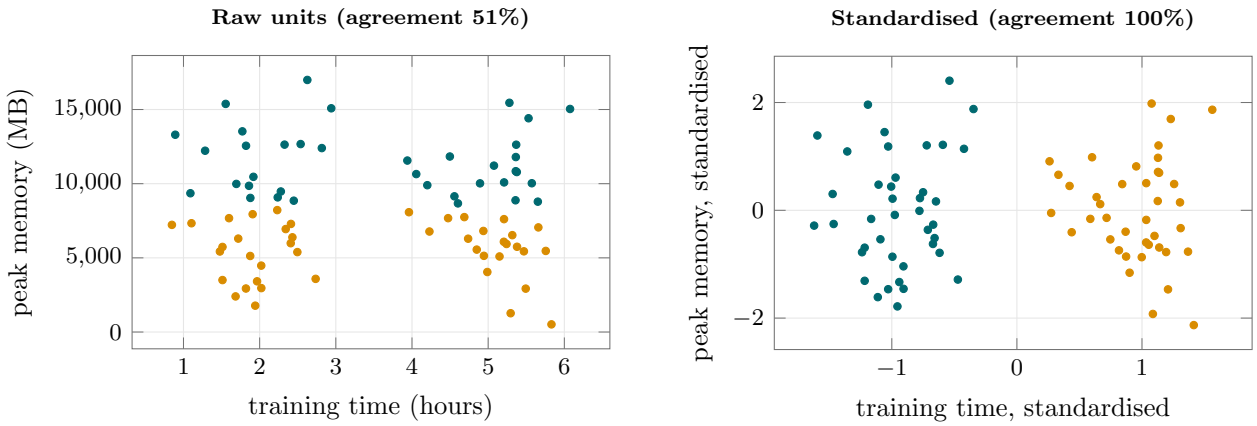


Figure 2.12: K-means with $K = 2$ on the same 80 runs, before and after standardisation. On the raw scale the clusters are separated by memory; after standardisation they are separated by training time.

Remark: standardisation is a modelling choice

After standardisation every feature has the same influence on the distance. If some features are known to be more important, or irrelevant, standardisation cannot take this into account, and the features should be selected or weighted accordingly. The means and standard deviations should be computed once, on the data being clustered, and the same values used for any new example.

2.5.4 Dependence on cluster shape and size

Intuition: a town along a highway

Houses stretch for many kilometres along a highway, and a small village sits just off its middle. With two outlets, the cheapest plan in total squared distance is to split the highway town in half and let the village be served by whichever half is nearer, not to give the village an outlet of its own. K-means prefers compact, round groups of similar size, even when the real groups are long or small.

For two centroids, the points equidistant from both lie on the perpendicular bisector of the segment joining them (Section 2.3.4, *The boundary between two clusters*). The boundary between two K-means clusters is therefore a straight line, and each cluster lies on one side of it. Combined with the update step, this means that K-means favours compact, roughly spherical clusters of similar extent.

Consider data, generated for illustration, consisting of an elongated cluster of 120 points and a small cluster of 25 points above its centre. Dividing the elongated cluster lowers J more than separating the small one, because the points at the two ends of the elongated cluster are far from any single centroid. The agreement with the true clusters is only 56%. Figure 2.13 compares the true clusters with those found by K-means. Two remedies are available for clusters of this kind: a model that allows each cluster its own shape, such as the Gaussian mixture of Section 3.1.6, *General Gaussian mixtures*, or a choice of features in which the clusters are approximately spherical. Curved clusters are harder still. For two interleaved half-moons (Section 3.3, *Density-based clustering: DBSCAN and HDBSCAN*), which no straight boundary can separate, K-means agrees with the true groups on only 77% of the points, and a mixture of ellipses does not help either: a density-based method is needed.

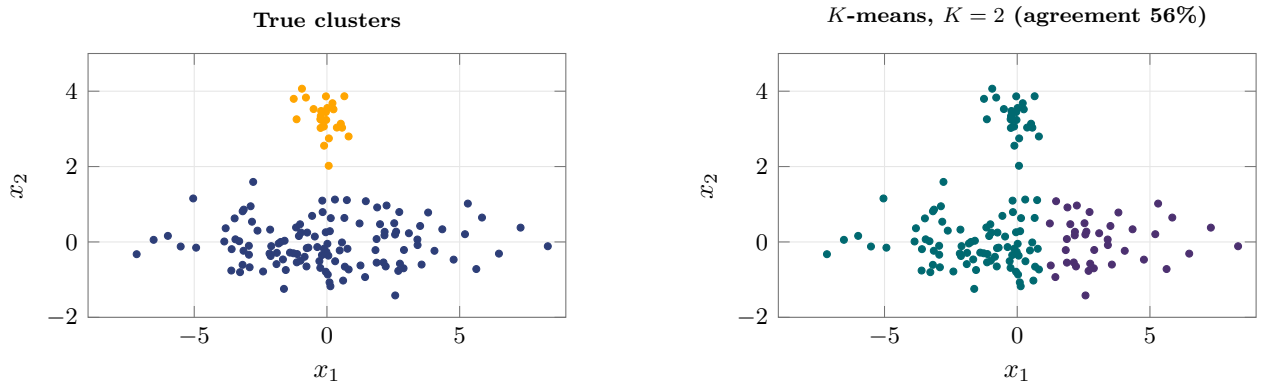


Figure 2.13: Left: the true clusters. Right: the partition found by K-means with $K = 2$, which divides the elongated cluster by a straight boundary and merges the small cluster with one half.

2.5.5 Choosing the number of clusters

Intuition: how many outlets to open

Every additional outlet shortens some deliveries, and with one outlet at every door the delivery distance is zero. The total distance alone therefore always argues for more outlets. The useful question is where the savings from one more outlet become small: the first few outlets cut delivery distances sharply, later ones barely. The value of K at that bend is called the elbow.

The number of clusters K must be specified in advance, and it cannot be chosen by minimising J. Indeed, an optimal partition into K clusters can be turned into a partition into $K + 1$ clusters by taking a single point from a cluster that has at least two points (one exists when $K < n$), placing it in a cluster of its own whose centroid is the point itself, and leaving all other centroids where they are. The moved point then contributes 0 instead of a nonnegative amount, and every other point keeps its centroid and its contribution, so J cannot increase; so the smallest attainable J never increases with K. For the eight runs, exhaustive search gives the following values.

K	1	2	3	4	5	6	7	8
best J	100.875	15.250	8.750	4.333	1.500	1.000	0.500	0.000

When each run forms its own cluster ($K = 8$), $J = 0$. Minimising J would therefore always select the largest possible K.

A common heuristic looks instead for the value of K beyond which J decreases only slowly, called the **elbow** of the curve. For the eight runs, J falls from 100.875 to 15.25 between $K = 1$ and $K = 2$ and only slowly thereafter, which suggests $K = 2$.

The second example consists of 90 points drawn from three well-separated clusters of 30. Here the smallest values of J are 1092.0, 463.9, 77.6 and 66.3 for $K = 1, \dots, 4$; the large decreases end at $K = 3$. The elbow is a matter of judgement, and many real data sets show no clear elbow. It should be supplemented by a second criterion, such as the silhouette of Section 3.4, *Evaluating a clustering*, and by the purpose of the analysis: if the runs are to be allocated to two types of machine, then $K = 2$ regardless of the shape of the curve. Figure 2.14 shows the data, the smallest J for each K, and the mean silhouette.

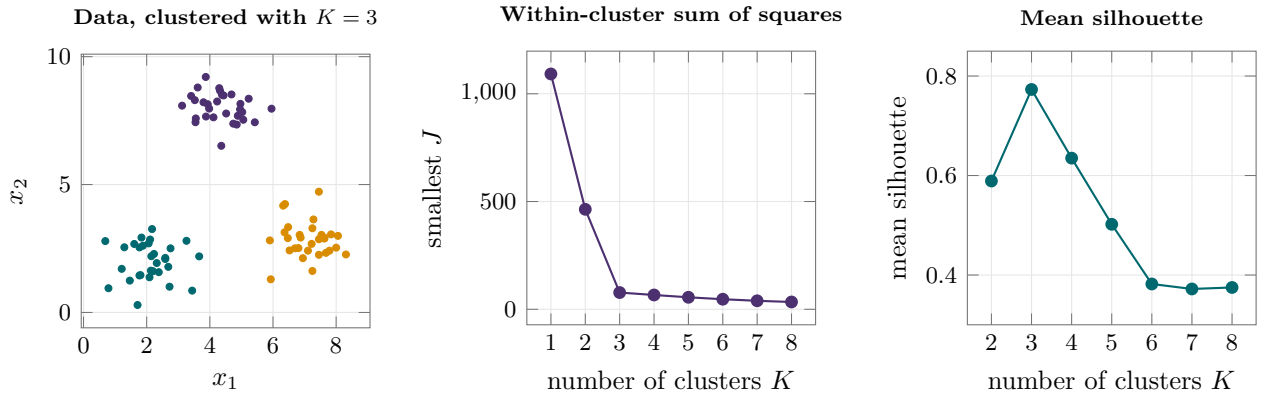


Figure 2.14: Choosing K for 90 points from three clusters. Left: the data, clustered with $K = 3$. Centre: the smallest J found for each K. Right: the mean silhouette (Section 3.4, *Evaluating a clustering*), which is largest at $K = 3$.

2.6 Where clustering is used

Intuition: a shop arranging its shelves

A supermarket that groups products bought together, a music service that sorts songs into moods, and a photo app that groups pictures of the same face all solve the same problem: put similar things together when nobody has supplied the categories.

Clustering is used wherever many unlabelled examples must be summarised or organised before anything else can be done with them.

- **Segmentation.** Customers, users or patients are grouped by behaviour or measurements, so that each group can be described and treated separately.
- **Science.** In single-cell biology, cells are clustered by the expression of thousands of genes to identify cell types; in astronomy, stars and galaxies are grouped by their spectra; in chemistry and materials science, compounds are grouped by numerical descriptions of their structure.
- **Engineering and operations.** The operating conditions of a machine are grouped from its sensor readings into regimes; the jobs on a computing cluster are grouped by their resource use, as the eight training runs were, to plan hardware.
- **Images and signals.** The pixels of an image are clustered by colour to reduce the palette or to divide the image into regions; recorded signals are grouped by shape.
- **Choosing representatives.** One example per cluster is selected to be labelled, inspected or repeated, which spreads a limited budget over the variety in the data.

K-means is also used inside systems that store and search very large collections of vectors, such as *embeddings*: vectors of numbers that a trained model assigns to each image, document or record. To save space, each stored vector can be replaced by the index of its nearest centroid. To save time, a query can be compared only with the vectors in the few clusters whose centroids are nearest to it. The same idea is used to remove near-duplicates from large collections, including the training data of large models, by comparing examples only within clusters.

In every case the result depends on the initialisation, on the scale of the coordinates, and on the choice of K.

2.7 Practice problems for Chapter 2

2.1 [Basic] Apply K-means with $K = 2$ to the one-dimensional data $1, 2, 9$, starting from centroids 1 and 2. Give the final clusters, centroids and J.

2.2 [Standard] The points $(1, 1)$, $(2, 1)$, $(4, 3)$ and $(5, 4)$ are to be divided into two clusters.

(a) Compute the centroids and J for the partition $\{(1, 1), (2, 1)\}$, $\{(4, 3), (5, 4)\}$.

(b) Run Lloyd's algorithm from the initial centroids $(1, 1)$ and $(2, 1)$ until it stops. Report the assignments, centroids and J at each iteration.

2.3 [Standard] Let $\mathbf{x}_1, \dots, \mathbf{x}_m$ be points with mean $\bar{\mathbf{x}}$. Prove that for every point $\mathbf{c}$,

$$\sum_{j=1}^{m} \|\mathbf{x}_j - \mathbf{c}\|^2 = \sum_{j=1}^{m} \|\mathbf{x}_j - \bar{\mathbf{x}}\|^2 + m\,\|\bar{\mathbf{x}} - \mathbf{c}\|^2.$$

Deduce, without calculus, that the mean is the unique optimal centroid.

2.4 [Standard] (a) Show that a partition is a fixed point of Lloyd's algorithm if and only if every point is nearest to the centroid of its own cluster, the centroids being the cluster means. (b) Show that the algorithm performs at most K^n iterations. (c) Is every fixed point a global minimum of J? Justify with an example.

2.5 [Basic] Two training runs differ by 3 hours in training time and by 2000 MB in peak memory. Across all runs, the standard deviations are 1.6 hours and 3634 MB. Compute the squared distance between the two runs on the raw features and on standardised features, and the share of each squared distance that comes from memory.

2.6 [Basic] One-dimensional data $0, 1, 10$; K-means++ has chosen 0 as the first centroid. With what probabilities does it choose 1, and 10, as the second?

2.7 [Standard] (a) For one-dimensional data $x_1, \dots, x_m$, show that $\sum_j |x_j - c|$ is minimised by any median of the data. (b) For the data $1, 2, 9$, compare $\sum_j |x_j - c|$ and $\sum_j (x_j - c)^2$ at $c =$ median and at $c =$ mean.

2.8 [Basic] One feature takes values between 0 and 1 and another between 0 and 1000. Describe what K-means does on the raw features, and give two remedies, stating what each assumes.

2.9 [Standard] Explain why the smallest attainable J cannot increase when K increases by one (assume $K < n$), and why this rules out choosing K by minimising J.

2.10 [Basic] One-dimensional data $2, 4, 6, 10, 12$, $K = 2$. Compute J for the partitions $\{2, 4, 6\}, \{10, 12\}$ and $\{2, 4\}, \{6, 10, 12\}$. Is the second a fixed point of Lloyd's algorithm?

2.11 [Standard] K-means++ is applied to the one-dimensional data $0, 1, 2, 10, 11$ with first centroid 0. Give the probability of each point being chosen as the second centroid. If 10 is chosen, give the probabilities for the third.

2.12 [Standard] Prove that the points equidistant from two centroids $\boldsymbol{\mu}_a \neq \boldsymbol{\mu}_b$ form the hyperplane $(\boldsymbol{\mu}_a - \boldsymbol{\mu}_b)^\top \mathbf{x} = \frac{1}{2}\big(\|\boldsymbol{\mu}_a\|^2 - \|\boldsymbol{\mu}_b\|^2\big)$, and that it is the perpendicular bisector of the segment joining them.

2.13 [Basic] For the 90 points with three clusters in Section 2.5.5, *Choosing the number of clusters*, the smallest values of J for $K = 1, \dots, 5$ are 1092.0, 463.9, 77.6, 66.3, 55.6. Compute the successive decreases and choose K by the elbow.

2.14 [Challenge] Prove that for a cluster C of m points with mean $\bar{\mathbf{x}}$, $\sum_{i \in C} \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \frac{1}{2m} \sum_{i \in C} \sum_{j \in C} \|\mathbf{x}_i - \mathbf{x}_j\|^2$. Verify it for runs 1–4 of the eight training runs, and explain why it shows that J measures the tightness of clusters without reference to centroids.

2.15 [Challenge] Find all partitions of the one-dimensional data $0, 1, 2, 4$ into two clusters that are fixed points of Lloyd's algorithm, and show that one of them is not optimal.

Chapter 3

Mixture models, hierarchical and density-based clustering

Companion notebook in Google Colab: Part B.

3.1 Gaussian mixtures and the EM algorithm

3.1.1 A probabilistic model for clustering

Intuition: soft K-means

In a mess hall, one group of friends sits by the window and another by the door. A student sitting between them belongs to neither group completely. K-means must put the student wholly in one group. A mixture model instead says, for example, 70% window group and 30% door group, and lets the student count 70% towards the window group's average and 30% towards the door group's. A Gaussian mixture whose components share one fixed spread behaves like K-means with such soft memberships.

In K-means each point belongs to exactly one cluster, and a point lying between two clusters is assigned entirely to one of them, which then moves its centroid as if the point belonged to it fully. A probabilistic model allows such a point to belong partly to each cluster.

Seven requests to a deployed model had these latencies, in milliseconds:

$$10,\ 12,\ 14,\ 22,\ 31,\ 33,\ 36.$$

They appear to come from two kinds of request, fast and slow, and the request of 22 ms lies between them.

Suppose that each latency is drawn from one of two normal distributions, corresponding to fast and slow requests, but we do not observe which distribution produced which latency. The two distributions have unknown means μ_1 and μ_2, a common known standard deviation $\sigma = 4$ ms, and each is chosen with probability $\frac{1}{2}$. The density of a latency x is then

$$p(x) = \tfrac{1}{2}\,\mathcal{N}(x \mid \mu_1, \sigma) + \tfrac{1}{2}\,\mathcal{N}(x \mid \mu_2, \sigma).$$

A model of this form is called a **Gaussian mixture model**, and the two normal distributions are its **components**. Fixing σ and the mixing probabilities keeps the calculations short; Section 3.1.6, *General Gaussian mixtures* removes both restrictions.

3.1.2 The E-step: responsibilities

Intuition: which counter did the order come from

A canteen has a fast counter, where orders usually take 10 to 15 minutes, and a slow counter, where they usually take 30 to 35. An order that took 22 minutes could have come from either: a slow order from the fast counter or a quick one from the slow counter. Comparing how typical 22 minutes is for each counter gives the probability of each. These probabilities are the responsibilities.

Suppose for the moment that the means are known. By Bayes' rule, the probability that request i was produced by component k, given its latency x_i, is

$$r_{ik} = \frac{\frac{1}{2}\,\mathcal{N}(x_i \mid \mu_k, \sigma)}{\frac{1}{2}\,\mathcal{N}(x_i \mid \mu_1, \sigma) + \frac{1}{2}\,\mathcal{N}(x_i \mid \mu_2, \sigma)} = \frac{e^{-(x_i-\mu_k)^2/(2\sigma^2)}}{e^{-(x_i-\mu_1)^2/(2\sigma^2)} + e^{-(x_i-\mu_2)^2/(2\sigma^2)}},$$

where the second form follows because the factors $\frac{1}{2}$ and $1/(\sigma\sqrt{2\pi})$ are common to every term. The quantities r_{ik} are called **responsibilities**; for each request, $r_{i1} + r_{i2} = 1$.

Example

Take $\mu_1 = 15$ and $\mu_2 = 25$, so that $2\sigma^2 = 32$. The responsibilities of the seven requests are given in the table below. For the request of 22 ms, $e^{-(22-15)^2/32} = e^{-1.531} = 0.2163$ and $e^{-(22-25)^2/32} = e^{-0.281} = 0.7548$, so

$$r_{41} = \frac{0.2163}{0.2163 + 0.7548} = 0.223.$$

This request is attributed about one fifth to the fast component and four fifths to the slow one; each of the other requests is attributed almost entirely to a single component.

latency x_i (ms)	$e^{-(x_i-15)^2/32}$	$e^{-(x_i-25)^2/32}$	r_{i1} (fast)	r_{i2} (slow)
10	0.4578	0.0009	0.998	0.002
12	0.7548	0.0051	0.993	0.007
14	0.9692	0.0228	0.977	0.023
22	0.2163	0.7548	0.223	0.777
31	0.0003	0.3247	0.001	0.999
33	0.0000	0.1353	0.000	1.000
36	0.0000	0.0228	0.000	1.000
		column sums	3.192	3.808

3.1.3 The M-step: updating the means

Intuition: an average with shared votes

To estimate each counter's typical time, average the orders it served. An order that is four fifths likely to be from the slow counter counts four fifths in the slow counter's average and one fifth in the fast counter's. This is the weighted average below.

Suppose now that the responsibilities are known, and choose the means to maximise the log-likelihood in which request i counts towards component k with weight r_{ik}. The terms involving μ_k are

$$\sum_i r_{ik} \log \mathcal{N}(x_i \mid \mu_k, \sigma) = \sum_i r_{ik} \Big(-\log(\sigma\sqrt{2\pi}) - \frac{(x_i - \mu_k)^2}{2\sigma^2} \Big).$$

The first term in the bracket does not depend on μ_k. Differentiating and setting the derivative to zero gives

$$\sum_i r_{ik}\,\frac{x_i-\mu_k}{\sigma^2}=0 \quad\Longrightarrow\quad \boxed{\mu_k=\frac{\sum_i r_{ik}\,x_i}{\sum_i r_{ik}}}.$$

Here the derivative of $-(x_i-\mu_k)^2/(2\sigma^2)$ with respect to μ_k is, by the chain rule, $-2(x_i-\mu_k)(-1)/(2\sigma^2)=(x_i-\mu_k)/\sigma^2$. To solve the equation, multiply it by σ^2 and split the sum: $\sum_i r_{ik}x_i-\mu_k\sum_i r_{ik}=0$, so $\mu_k\sum_i r_{ik}=\sum_i r_{ik}x_i$. Dividing by $\sum_i r_{ik}>0$ gives the boxed formula. The second derivative is $-\sum_i r_{ik}/\sigma^2<0$, so the function of μ_k being maximised is a downward-opening parabola, and its only stationary point is its maximum. The new mean is a weighted mean of the latencies, each weighted by its responsibility. When every responsibility is 0 or 1 it reduces to the ordinary mean of a cluster.

Example

From the table above, $\sum_i r_{i1}=0.998+0.993+0.977+0.223+0.001+0.000+0.000=3.192$. The products $r_{i1}x_i$, computed with the unrounded responsibilities, are 9.981, 11.920, 13.678, 4.899, 0.032, 0.010 and 0.002. So $\sum_i r_{i1}x_i=40.52$ and $\mu_1=40.52/3.192=12.69$. Since $r_{i2}=1-r_{i1}$, $\sum_i r_{i2}=7-3.192=3.808$, and $\sum_i r_{i2}x_i=\sum_i x_i-\sum_i r_{i1}x_i=158-40.52=117.48$, where $158=10+12+14+22+31+33+36$. So $\mu_2=117.48/3.808=30.85$. The two sums of responsibilities add to 7, the number of requests.

3.1.4 The EM algorithm

Intuition: guessing and correcting in turns

Neither the counter of each order nor the counters' typical times are known. Guess the typical times; work out which counter each order probably came from; recompute the typical times from those shares; and repeat. Each round uses the best current guess of one unknown to improve the other.

Alternating the two steps gives the expectation–maximisation (EM) algorithm.

Algorithm 3: EM for the two-component mixture

Choose initial means. Then repeat until the means no longer change:

1. **E-step.** Compute the responsibilities r_{ik} by Bayes' rule.
2. **M-step.** Replace each mean by the responsibility-weighted mean.

The E-step is the soft version of the assignment step of K-means, and the M-step the soft version of its update step.

step	mean fast μ_1	mean slow μ_2	r_{41} (22 ms)	log-likelihood
0	15.00	25.00	0.223	−28.985
1	12.69	30.85	0.436	−24.133
2	13.27	31.54	0.613	−23.980
3	13.70	32.04	0.730	−23.901
4	13.96	32.40	0.795	−23.868
5	14.10	32.61	0.827	−23.858
…	…	…	…	…
converged	14.21	32.80	0.852	−23.855

The last column is the log-likelihood $\sum_i \log p(x_i)$ of the seven latencies under the current means. It increases at every iteration and converges; it plays the same role for EM as J does for K-means. That

it can never decrease is proved in Section 3.1.5, *Why the log-likelihood never decreases (optional)*. Figure 3.1 shows the mixture density at the start and at convergence.

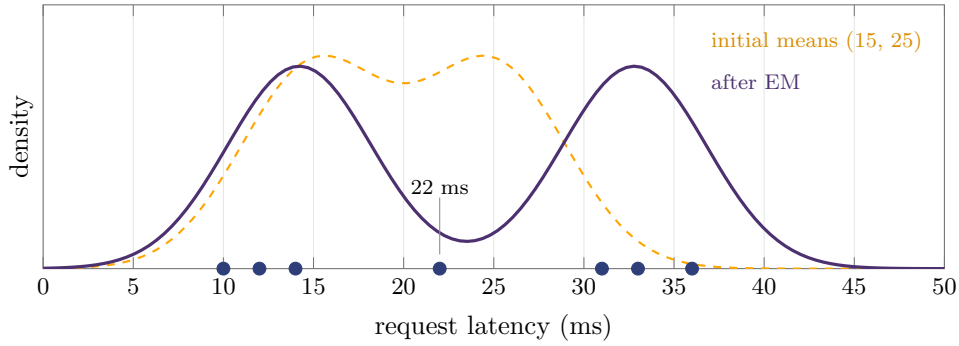


Figure 3.1: The mixture density with the initial means (15 and 25, dashed) and after convergence of EM (solid). The dots mark the seven latencies.

Comparison with K-means

K-means applied to the same data from the same initial values assigns the request of 22 ms entirely to the slow cluster, since 22 is closer to 25 than to 15. The slow centroid becomes the mean of $22 + 31 + 33 + 36$, which is $122/4 = 30.5$, and the fast centroid becomes 12.0. The fast centroid is $(10 + 12 + 14)/3 = 36/3 = 12.0$. The midpoint between the new centroids is $(12.0 + 30.5)/2 = 21.25$. The latencies 10, 12 and 14 lie below it and 22, 31, 33 and 36 lie above it, so each latency is still nearest to the centroid of its own cluster. The next assignment is therefore unchanged, and the algorithm stops. EM converges instead to means of 14.2 and 32.8. As the fast mean increases, the responsibility of the fast component for the 22 ms request rises from 0.223 to more than 0.8, so that this request no longer pulls the slow mean downwards.

Neither result is the true answer; both are local optima of their own criteria. The difference is that EM does not force an uncertain point into a single cluster, and its responsibilities quantify the uncertainty.

K-means as a limiting case

Take a point x_i. Let μ_a be the nearer of the two means and μ_b the other, at distances $d_a < d_b$. Divide the numerator and the denominator of r_{ia} by $e^{-d_a^2/(2\sigma^2)}$ and use $e^{-A}/e^{-B} = e^{-(A-B)}$:

$$r_{ia} = \frac{e^{-d_a^2/(2\sigma^2)}}{e^{-d_a^2/(2\sigma^2)} + e^{-d_b^2/(2\sigma^2)}} = \frac{1}{1 + e^{-(d_b^2 - d_a^2)/(2\sigma^2)}}.$$

The number $d_b^2 - d_a^2$ is positive and does not depend on σ. So as $\sigma \to 0$ the exponent $-(d_b^2 - d_a^2)/(2\sigma^2)$ tends to $-\infty$, the exponential tends to 0, $r_{ia} \to 1$, and $r_{ib} = 1 - r_{ia} \to 0$. For the request of 22 ms with means 15 and 25, the nearer mean is 25, $d_b^2 - d_a^2 = 49 - 9 = 40$, and $r_{42} = 1/(1 + e^{-40/(2\sigma^2)})$. This is $1/(1 + e^{-1.25}) = 0.777$ for $\sigma = 4$, as in the table; $1/(1 + e^{-5}) = 0.993$ for $\sigma = 2$; and $1/(1 + e^{-20})$, which differs from 1 by about 2×10^{-9}, for $\sigma = 1$. A point exactly midway between the means has $d_a = d_b$, and its responsibilities stay at $\frac{1}{2}$ and $\frac{1}{2}$ for every σ. In the M-step, $\sum_i r_{ik} x_i$ and $\sum_i r_{ik}$ are finite sums. Their weights tend to 1 for the points nearest to μ_k and to 0 for the others, so the two sums tend to the sum and the number of those points. Each point is then assigned wholly to the nearest mean, and the weighted mean becomes the ordinary mean of the assigned points. The E-step becomes the assignment step and the M-step the update step: K-means is the limit of EM for this mixture as $\sigma \to 0$. Figure 3.2 shows this limit: as σ shrinks, the smooth responsibility curve steepens into a step at the midpoint between the means.

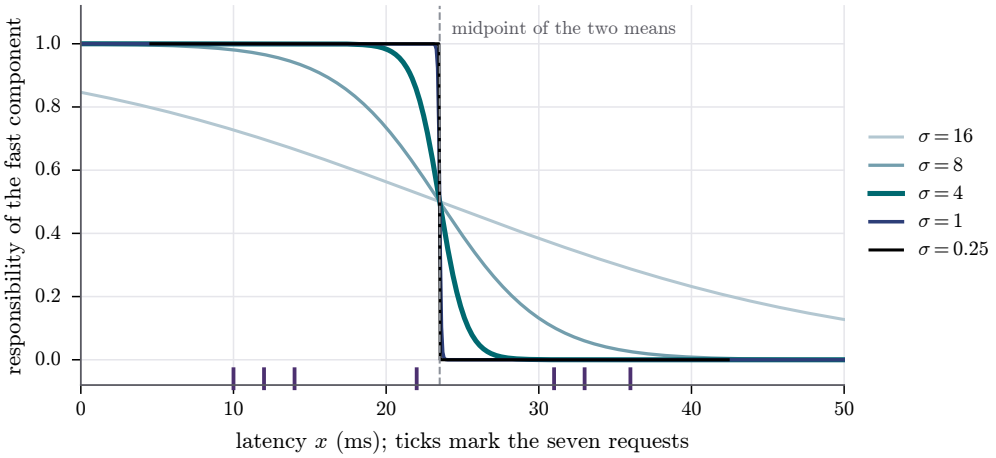


Figure 3.2: Responsibility of the fast component for a request of latency x, with the means found by EM (14.21 and 32.80 ms) and several values of σ; the thick teal curve is $\sigma = 4$, the value used in the example. As σ decreases, the soft assignment becomes a step at the midpoint between the means: the assignment rule of K-means.

3.1.5 Why the log-likelihood never decreases (optional)

This guarantee is what allows EM to be run until the log-likelihood stops changing, and the same argument underlies EM for other models with hidden variables, such as the hidden Markov models used in speech recognition and the topic models used for collections of documents.

We prove the property observed in the table above, for a mixture with any number of components and with parameters $\theta = (\pi_k, \mu_k, \sigma_k)_k$, where the mixing probabilities $\pi_k \geq 0$ sum to one. Write $p_k(x_i;\theta) = \pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k)$, so that the log-likelihood is $\ell(\theta) = \sum_i \log \sum_k p_k(x_i;\theta)$.

A lower bound. For any weights $q_{ik} > 0$ with $\sum_k q_{ik} = 1$ for each i, multiply and divide by q_{ik} and apply Jensen's inequality for the logarithm (Chapter 1, *Mathematical background*), with weights $w_k = q_{ik}$ and positive numbers $a_k = p_k(x_i;\theta)/q_{ik}$. The logarithm is concave, so the logarithm of a weighted average is at least the weighted average of the logarithms: $\log \sum_k q_{ik} a_k \geq \sum_k q_{ik} \log a_k$. Here $\sum_k q_{ik} a_k = \sum_k p_k(x_i;\theta)$. Summing over i,

$$\ell(\theta) = \sum_i \log \sum_k q_{ik} \frac{p_k(x_i;\theta)}{q_{ik}} \;\geq\; \sum_i \sum_k q_{ik} \log \frac{p_k(x_i;\theta)}{q_{ik}} \;=:\; F(q,\theta).$$

The E-step makes the bound tight. Take $q_{ik} = r_{ik} = p_k(x_i;\theta) / \sum_j p_j(x_i;\theta)$, the responsibilities at the current parameters. Then $p_k(x_i;\theta)/q_{ik} = \sum_j p_j(x_i;\theta)$ is the same for every k, so Jensen's inequality holds with equality. Directly, $\sum_k r_{ik} = 1$ because the numerators $p_k(x_i;\theta)$ add up to the common denominator, and therefore

$$F(r,\theta) = \sum_i \sum_k r_{ik} \log \sum_j p_j(x_i;\theta) = \sum_i \Big(\sum_k r_{ik}\Big) \log \sum_j p_j(x_i;\theta) = \sum_i \log \sum_j p_j(x_i;\theta) = \ell(\theta).$$

These weights are positive when every $\pi_k > 0$, as the lower bound requires.

The M-step raises the bound. With $q = r$ fixed, $F(r,\theta') = \sum_i \sum_k r_{ik} \log p_k(x_i;\theta') - \sum_i \sum_k r_{ik} \log r_{ik}$. The second sum does not depend on θ', and the first is the weighted log-likelihood that the M-step maximises. Hence $F(r,\theta_{\text{new}}) \geq F(r,\theta_{\text{old}})$.

Conclusion. Chaining the three facts,

$$\ell(\theta_{\text{new}}) \;\geq\; F(r,\theta_{\text{new}}) \;\geq\; F(r,\theta_{\text{old}}) \;=\; \ell(\theta_{\text{old}}).$$

The first inequality is the lower bound, valid for every θ; the second is the M-step; the equality is the E-step. As with Lloyd's algorithm, this guarantees only that the log-likelihood never decreases.

Under mild conditions the iterates approach a stationary point of the log-likelihood, which need not be its global maximum. Figure 3.3 shows one iteration of this argument on the seven latencies.

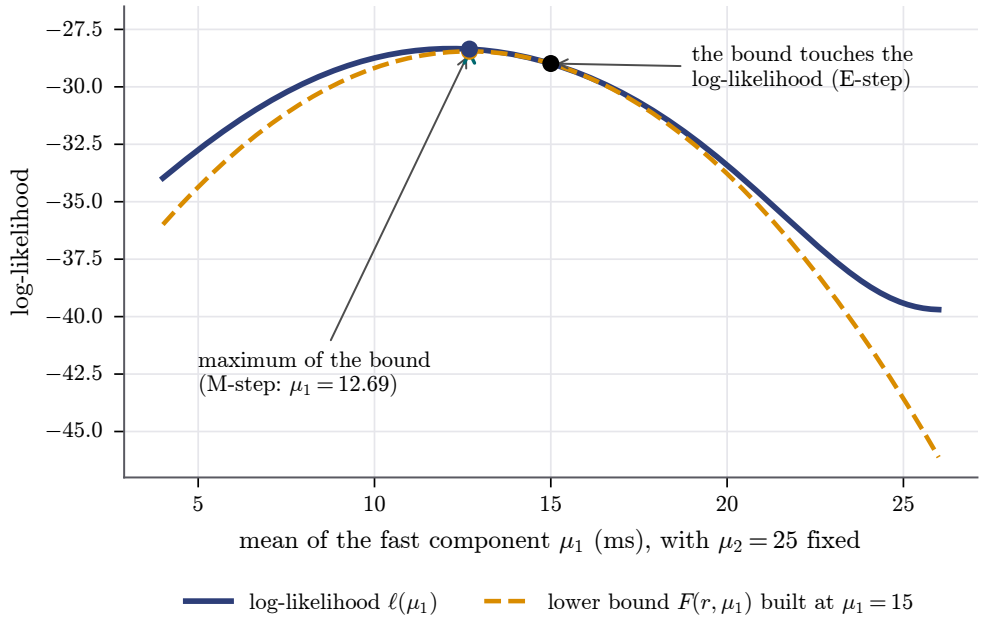


Figure 3.3: One EM iteration on the seven latencies, seen as a function of the fast mean μ_1 alone (the slow mean held at 25). The E-step at $\mu_1 = 15$ builds a lower bound that touches the log-likelihood there; the M-step moves to the maximum of the bound, $\mu_1 = 12.69$, where the log-likelihood is higher.

3.1.6 General Gaussian mixtures

Intuition: groups of different shapes

A crowd queuing along a street and a crowd gathered in a square are both groups, but one is long and thin and the other round. Allowing each component its own spread, and with several features its own covariance, lets the model describe each group by its own shape rather than forcing all of them to be equally round.

In general the mixing probabilities π_k (with $\sum_k \pi_k = 1$) and the standard deviations σ_k are also estimated, and with several features each component has its own covariance matrix, so that a cluster may be elongated and oriented in any direction. The E-step is again Bayes' rule, with π_k in place of $\frac{1}{2}$, and the M-step again consists of weighted averages:

$$\pi_k = \frac{1}{n}\sum_i r_{ik}, \qquad \mu_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}, \qquad \sigma_k^2 = \frac{\sum_i r_{ik}(x_i - \mu_k)^2}{\sum_i r_{ik}}.$$

These are the weighted analogues of a proportion, a mean and a variance.

Derivation of the M-step. With the responsibilities held fixed, the M-step maximises the weighted log-likelihood

$$\mathcal{Q} = \sum_{i=1}^{n}\sum_{k=1}^{K} r_{ik}\Big(\log \pi_k - \log \sigma_k - \frac{(x_i - \mu_k)^2}{2\sigma_k^2}\Big) + \text{const.}$$

Here $\log\big(\pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k)\big) = \log \pi_k - \log \sigma_k - \frac{1}{2}\log(2\pi) - \frac{(x_i-\mu_k)^2}{2\sigma_k^2}$. The bracket keeps every term except $-\frac{1}{2}\log(2\pi)$. Together those terms give the constant $-\frac{1}{2}\log(2\pi)\sum_i\sum_k r_{ik} = -\frac{n}{2}\log(2\pi)$, which involves no parameter. The means are found exactly as in Section 3.1.3, *The M-step: updating the means*. The derivative with respect to μ_k is $\sum_i r_{ik}(x_i - \mu_k)/\sigma_k^2$, and the factor $1/\sigma_k^2 > 0$ does not

affect where it vanishes. So $\mu_k = \sum_i r_{ik} x_i / \sum_i r_{ik}$ whatever the value of σ_k, and this value is used in the equation for σ_k. For the variances, differentiate with respect to σ_k, using $\frac{d}{d\sigma}(-\log\sigma) = -\frac{1}{\sigma}$ and, with $c = (x_i - \mu_k)^2$, $\frac{d}{d\sigma}\big(-\frac{c}{2\sigma^2}\big) = -\frac{c}{2}\cdot(-2)\sigma^{-3} = \frac{c}{\sigma^3}$. Set the derivative to zero:

$$\sum_i r_{ik}\Big(-\frac{1}{\sigma_k} + \frac{(x_i-\mu_k)^2}{\sigma_k^3}\Big) = 0 \quad\Longrightarrow\quad \sigma_k^2 = \frac{\sum_i r_{ik}(x_i-\mu_k)^2}{\sum_i r_{ik}}.$$

Multiplying the equation by $\sigma_k^3 > 0$ gives $-\sigma_k^2\sum_i r_{ik} + \sum_i r_{ik}(x_i-\mu_k)^2 = 0$, and dividing by $\sum_i r_{ik}$ gives the result. It is a maximum. The derivative equals $\big(\sum_i r_{ik}(x_i-\mu_k)^2 - \sigma_k^2\sum_i r_{ik}\big)/\sigma_k^3$, which is positive for σ_k^2 below the stationary value and negative above it, so $\mathcal{Q}$ rises and then falls as σ_k increases. The mixing probabilities must satisfy $\sum_k \pi_k = 1$, so we use a Lagrange multiplier λ. Collecting the terms of $\mathcal{Q}$ that contain a logarithm of a mixing probability, component by component, gives $\sum_i\sum_j r_{ij}\log\pi_j = \sum_j\big(\sum_i r_{ij}\big)\log\pi_j$. Only this term involves the mixing probabilities. Each $\log\pi_j$ is concave and the constraint is linear, so a stationary point of the Lagrangian is the constrained maximum, and

$$\frac{\partial}{\partial\pi_k}\Big[\sum_j\Big(\sum_i r_{ij}\Big)\log\pi_j - \lambda\Big(\sum_j\pi_j - 1\Big)\Big] = \frac{\sum_i r_{ik}}{\pi_k} - \lambda = 0 \quad\Longrightarrow\quad \pi_k = \frac{\sum_i r_{ik}}{\lambda}.$$

Summing over k, $1 = \sum_k \pi_k = \frac{1}{\lambda}\sum_i\sum_k r_{ik} = \frac{n}{\lambda}$, because each point's responsibilities sum to one. Hence $\lambda = n$ and $\pi_k = \frac{1}{n}\sum_i r_{ik}$.

Several features. With d features each component has a mean vector $\boldsymbol{\mu}_k$ and a $d\times d$ covariance matrix Σ_k, and its density is

$$\mathcal{N}(\mathbf{x}\mid\boldsymbol{\mu}_k,\Sigma_k) = \frac{1}{(2\pi)^{d/2}\,|\Sigma_k|^{1/2}}\exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top\Sigma_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)\Big).$$

The quadratic form $(\mathbf{x}-\boldsymbol{\mu}_k)^\top\Sigma_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)$ in the exponent is the squared *Mahalanobis distance* of $\mathbf{x}$ from $\boldsymbol{\mu}_k$, a distance measured in units of the component's own spread (Section 5.3, *Two features: distance, shape and neighbours* studies it in detail). The contours of each component are ellipsoids (ellipses when $d = 2$) whose axes lie along the eigenvectors of Σ_k, with half-lengths proportional to the square roots of the corresponding eigenvalues. By the spectral theorem, $\Sigma_k = \sum_j \lambda_j\mathbf{v}_j\mathbf{v}_j^\top$ with orthonormal $\mathbf{v}_j$. The eigenvalues satisfy $\lambda_j > 0$, because Σ_k is positive semidefinite and invertible. Then $\Sigma_k^{-1} = \sum_j \lambda_j^{-1}\mathbf{v}_j\mathbf{v}_j^\top$, as multiplying the two sums and using $\mathbf{v}_j^\top\mathbf{v}_l = 0$ for $j\neq l$ and $\mathbf{v}_j^\top\mathbf{v}_j = 1$ confirms. Let $y_j = \mathbf{v}_j^\top(\mathbf{x}-\boldsymbol{\mu}_k)$ be the coordinates of $\mathbf{x}-\boldsymbol{\mu}_k$ along the eigenvectors. Then

$$(\mathbf{x}-\boldsymbol{\mu}_k)^\top\Sigma_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k) = \sum_j \lambda_j^{-1}\big(\mathbf{v}_j^\top(\mathbf{x}-\boldsymbol{\mu}_k)\big)^2 = \sum_j \frac{y_j^2}{\lambda_j}.$$

The density decreases as this quantity increases, so it is constant where the quantity equals a constant c^2, that is, where $\sum_j y_j^2/(c\sqrt{\lambda_j})^2 = 1$. This is an ellipsoid whose j-th axis lies along $\mathbf{v}_j$, with half-length $c\sqrt{\lambda_j}$. The M-step for Σ_k is the weighted covariance matrix. To derive it, together with the M-step for $\boldsymbol{\mu}_k$, write $n_k = \sum_i r_{ik}$, $\bar{\mathbf{x}}_k = \frac{1}{n_k}\sum_i r_{ik}\mathbf{x}_i$ and $P = \Sigma_k^{-1}$. The terms of the weighted log-likelihood that involve $\boldsymbol{\mu}_k$ and Σ_k are

$$\mathcal{Q}_k = \sum_i r_{ik}\Big(-\tfrac{1}{2}\log|\Sigma_k| - \tfrac{1}{2}(\mathbf{x}_i-\boldsymbol{\mu}_k)^\top P(\mathbf{x}_i-\boldsymbol{\mu}_k)\Big).$$

The mean. Write $\mathbf{x}_i - \boldsymbol{\mu}_k = (\mathbf{x}_i - \bar{\mathbf{x}}_k) + (\bar{\mathbf{x}}_k - \boldsymbol{\mu}_k)$ and expand, using the symmetry of P:

$$\sum_i r_{ik}(\mathbf{x}_i-\boldsymbol{\mu}_k)^\top P(\mathbf{x}_i-\boldsymbol{\mu}_k) = \sum_i r_{ik}(\mathbf{x}_i-\bar{\mathbf{x}}_k)^\top P(\mathbf{x}_i-\bar{\mathbf{x}}_k) + 2\Big(\sum_i r_{ik}(\mathbf{x}_i-\bar{\mathbf{x}}_k)\Big)^\top P(\bar{\mathbf{x}}_k-\boldsymbol{\mu}_k) + n_k(\bar{\mathbf{x}}_k-\boldsymbol{\mu}_k)^\top P(\bar{\mathbf{x}}_k-\boldsymbol{\mu}_k).$$

The middle term is zero because $\sum_i r_{ik}(\mathbf{x}_i - \bar{\mathbf{x}}_k) = n_k\bar{\mathbf{x}}_k - n_k\bar{\mathbf{x}}_k = \mathbf{0}$. The last term is positive unless $\boldsymbol{\mu}_k = \bar{\mathbf{x}}_k$, because P is positive definite. The sum is therefore smallest, and $\mathcal{Q}_k$ largest, at $\boldsymbol{\mu}_k = \bar{\mathbf{x}}_k$, whatever Σ_k is.

The covariance. With $\boldsymbol{\mu}_k = \bar{\mathbf{x}}_k$, put $S_k = \frac{1}{n_k}\sum_i r_{ik}(\mathbf{x}_i - \bar{\mathbf{x}}_k)(\mathbf{x}_i - \bar{\mathbf{x}}_k)^\top$, and assume it is invertible. For a column vector $\mathbf{a}$, $\mathbf{a}^\top P\mathbf{a} = \operatorname{tr}(P\mathbf{a}\mathbf{a}^\top)$, and $\log|\Sigma_k| = -\log|P|$, so

$$\mathcal{Q}_k = \tfrac{n_k}{2}\big(\log|P| - \operatorname{tr}(PS_k)\big).$$

Let $S_k = \sum_j \lambda_j \mathbf{v}_j\mathbf{v}_j^\top$ with every $\lambda_j > 0$. Put $S_k^{1/2} = \sum_j \sqrt{\lambda_j}\,\mathbf{v}_j\mathbf{v}_j^\top$, which satisfies $S_k^{1/2}S_k^{1/2} = S_k$. Let $m_1, \dots, m_d$ be the eigenvalues of the symmetric matrix $M = S_k^{1/2}PS_k^{1/2}$. They are positive, because $\mathbf{v}^\top M\mathbf{v} = (S_k^{1/2}\mathbf{v})^\top P(S_k^{1/2}\mathbf{v}) > 0$ for $\mathbf{v} \neq \mathbf{0}$. Because $\operatorname{tr}(AB) = \operatorname{tr}(BA)$, we have $\operatorname{tr}(PS_k) = \operatorname{tr}(S_k^{1/2}PS_k^{1/2}) = \sum_j m_j$. The determinant of a product is the product of the determinants, and $|M|$ is the product of the eigenvalues of M, so $\log|P| = \log|M| - \log|S_k| = \sum_j \log m_j - \log|S_k|$. Hence

$$\tfrac{2}{n_k}\,\mathcal{Q}_k = \sum_{j=1}^{d}\big(\log m_j - m_j\big) - \log|S_k|.$$

The function $m - 1 - \log m$ has derivative $1 - 1/m$, which is negative for $m < 1$ and positive for $m > 1$. Its minimum is therefore 0, at $m = 1$. So $\log m_j - m_j \le -1$, with equality only when $m_j = 1$. The maximum is attained when every $m_j = 1$. By the spectral theorem that means $M = \sum_j \mathbf{w}_j\mathbf{w}_j^\top = I$, where the $\mathbf{w}_j$ are the orthonormal eigenvectors of M. Then $S_k^{1/2}PS_k^{1/2} = I$, so $P = S_k^{-1}$ and $\Sigma_k = S_k$: the weighted covariance matrix.

Algorithm 4: EM for a Gaussian mixture

Input: points $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$; number of components K; initial $\pi_k, \boldsymbol{\mu}_k, \Sigma_k$ (for example from K-means); a small $\tau > 0$

Output: parameters $\pi_k, \boldsymbol{\mu}_k, \Sigma_k$ and responsibilities r_{ik}

repeat

  $r_{ik} \leftarrow \pi_k\,\mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k) \big/ \sum_j \pi_j\,\mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \Sigma_j)$ for all i, k — E-step

  $n_k \leftarrow \sum_i r_{ik}$ for all k

  $\pi_k \leftarrow n_k/n, \quad \boldsymbol{\mu}_k \leftarrow \frac{1}{n_k}\sum_i r_{ik}\mathbf{x}_i$ — M-step

  $\Sigma_k \leftarrow \frac{1}{n_k}\sum_i r_{ik}(\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top + \tau I$

  $\ell \leftarrow \sum_i \log \sum_k \pi_k\,\mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k)$

until the increase in ℓ is below a tolerance

The term τI keeps each Σ_k invertible. The weighted covariance matrix $S_k = \frac{1}{n_k}\sum_i r_{ik}(\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top$ is positive semidefinite, since for every vector $\mathbf{v}$, $\mathbf{v}^\top S_k\mathbf{v} = \frac{1}{n_k}\sum_i r_{ik}\big(\mathbf{v}^\top(\mathbf{x}_i - \boldsymbol{\mu}_k)\big)^2 \ge 0$. It can still have an eigenvalue 0, for instance when the points carrying the component's weight lie on a line in the plane, and then it has no inverse. If $S_k\mathbf{v}_j = \lambda_j\mathbf{v}_j$, then $(S_k + \tau I)\mathbf{v}_j = (\lambda_j + \tau)\mathbf{v}_j$, so adding τI raises every eigenvalue by τ. All eigenvalues of $S_k + \tau I$ are then at least $\tau > 0$, and the matrix has the inverse $\sum_j (\lambda_j + \tau)^{-1}\mathbf{v}_j\mathbf{v}_j^\top$ (see also the remark below); with it the M-step is no longer an exact maximisation, but for small τ the log-likelihood still increases in practice. Each iteration costs about nKd^2 operations for the E-step and the covariance updates, plus Kd^3 to invert the covariance matrices. A component with its own covariance can describe the elongated cluster of Section 2.5.4, *Dependence on cluster shape and size*. Figure 3.4 shows Algorithm 4 fitting three such components to points in two dimensions.

Remark: practical difficulties

EM, like Lloyd's algorithm, converges to a local optimum, so several initialisations are advisable. In addition, if a component's mean coincides with a single data point, its variance can shrink towards zero and the likelihood increases without bound. Implementations therefore impose a

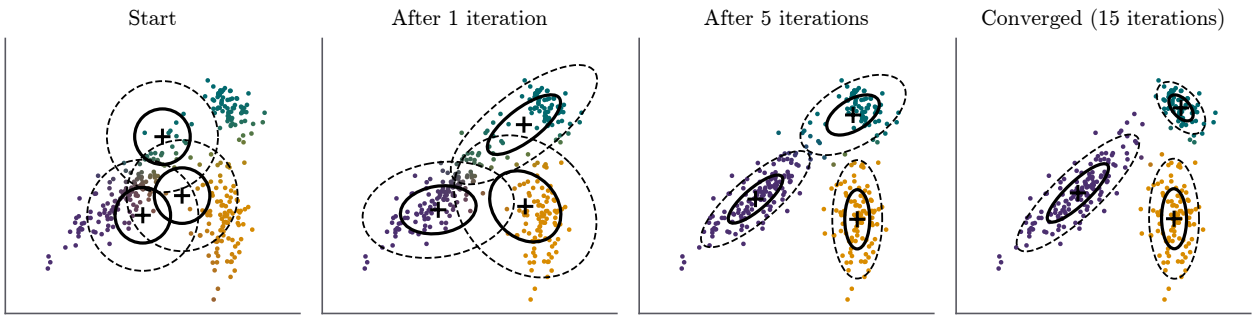


Figure 3.4: Algorithm 4 with three components on 320 points in two dimensions. Each component is drawn by its mean (cross) and its ellipses at Mahalanobis distance 1 (solid) and 2 (dashed); each point's colour mixes the three cluster colours in proportion to its responsibilities. The initial components are round and misplaced. After a few iterations each has taken the orientation and spread of one group, which K-means, with its straight boundaries and round clusters, cannot do. The log-likelihood increased at every iteration.

small lower bound on the variances.

3.2 Agglomerative hierarchical clustering

Intuition: a family tree

A family tree joins siblings first, then cousins, then second cousins, each level grouping people who are more distantly related. Hierarchical clustering builds such a tree for data: it joins the two most similar examples, then the next most similar groups, and so on until everything is joined. Cutting the tree at a chosen level gives the groups: near the leaves, many small families; near the root, a few large ones.

Agglomerative hierarchical clustering does not require K in advance. It begins with each point in a cluster of its own and repeatedly merges the two closest clusters until a single cluster remains. The sequence of merges is recorded in a tree called a **dendrogram**.

The distance between two clusters must be defined. This choice is called the **linkage**. Under *single* linkage it is the smallest distance between a point of one cluster and a point of the other; under *complete* linkage it is the largest such distance; under *average* linkage, which we use here, it is the average of all such distances.

Example

For the eight runs, the smallest pairwise distance is 1, attained by runs 1 and 2, runs 3 and 4, and runs 5 and 7; these pairs are merged first, at height 1. Run 8 is then merged with $\{5, 7\}$: its distances to runs 5 and 7 are 2 and 2.236, with average 2.118. This is the smallest average distance among the five clusters $\{1, 2\}$, $\{3, 4\}$, $\{5, 7\}$, $\{6\}$ and $\{8\}$ present after the first three merges. The nearest competitors are $\{5, 7\}$ and run 6, at $(2 + 3)/2 = 2.5$; $\{1, 2\}$ and $\{3, 4\}$, at 2.600; and runs 6 and 8, at $\sqrt{8} \approx 2.828$. Every pair made of a small-run cluster and a large-run cluster is farther apart than 4.7. Next, $\{1, 2\}$ and $\{3, 4\}$ are merged; the four distances between them are 2.236, 3.162, 2.000, 3.000, with average 2.600. This is just below the average distance between $\{5, 7, 8\}$ and run 6, which is $(2 + 3 + \sqrt{8})/3 \approx 2.609$. Every other pair of current clusters is farther apart than 4.7. Run 6 then joins $\{5, 7, 8\}$ at height $(2 + 3 + \sqrt{8})/3 \approx 2.609$, just above the previous merge, the smallest of the three average distances then present, since $\{1, 2, 3, 4\}$ lies at 5.949 from run 6 and at 6.927 from $\{5, 7, 8\}$. Finally the two remaining clusters are merged. The height of this merge is the mean of the 16 distances between $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8\}$. The four distances from run 1 sum to 21.004, from run 2 to 23.999, from run 3 to 29.407 and from

run 4 to 32.509, so the height is $(21.004 + 23.999 + 29.407 + 32.509)/16 = 106.919/16 = 6.682$.

cluster formed (runs)	height
1, 2	1.000
3, 4	1.000
5, 7	1.000
5, 7, 8	2.118
1, 2, 3, 4	2.600
5, 6, 7, 8	2.609
1, 2, 3, 4, 5, 6, 7, 8	6.682

A partition is obtained by cutting the dendrogram at a chosen height: the clusters are the subtrees below the cut. Any cut between 2.609 and 6.682 gives the two clusters found by K-means. The large gap between the last two merge heights indicates two well-separated clusters. Figure 3.5 shows the dendrogram of the eight runs with such a cut.

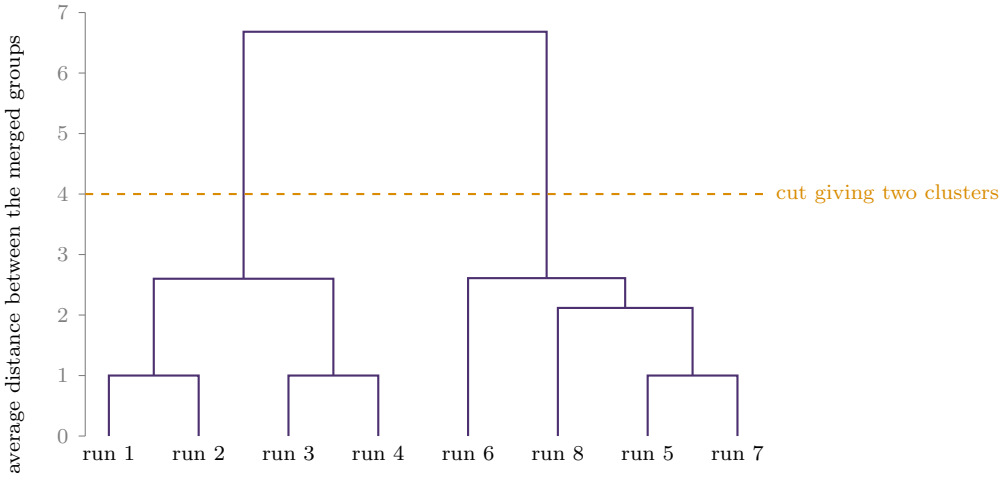


Figure 3.5: Dendrogram of the eight runs under average linkage. The height of each horizontal segment is the linkage distance at which the two clusters below it were merged. The dashed line shows a cut that produces two clusters.

3.2.1 Ward's linkage

A fourth linkage connects hierarchical clustering with K-means. **Ward's method** merges the pair of clusters whose union increases the within-cluster sum of squares W the least. The increase has a closed form. Apply the decomposition of Section 2.3.2, *Within-cluster and between-cluster sums of squares* to the points of $C_1 \cup C_2$ alone, divided into the two clusters C_1 and C_2. With $\boldsymbol{\mu}$ the mean of the union,

$$\sum_{\mathbf{x} \in C_1 \cup C_2} \|\mathbf{x} - \boldsymbol{\mu}\|^2 = \sum_{\mathbf{x} \in C_1} \|\mathbf{x} - \boldsymbol{\mu}_1\|^2 + \sum_{\mathbf{x} \in C_2} \|\mathbf{x} - \boldsymbol{\mu}_2\|^2 + n_1 \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}\|^2 + n_2 \|\boldsymbol{\mu}_2 - \boldsymbol{\mu}\|^2.$$

The merge therefore increases W by $\Delta(C_1, C_2) = n_1 \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}\|^2 + n_2 \|\boldsymbol{\mu}_2 - \boldsymbol{\mu}\|^2$. Since $\boldsymbol{\mu} = (n_1 \boldsymbol{\mu}_1 + n_2 \boldsymbol{\mu}_2)/(n_1 + n_2)$, we have

$$\boldsymbol{\mu}_1 - \boldsymbol{\mu} = \frac{(n_1 + n_2)\boldsymbol{\mu}_1 - n_1 \boldsymbol{\mu}_1 - n_2 \boldsymbol{\mu}_2}{n_1 + n_2} = \frac{n_2}{n_1 + n_2} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2),$$

$$\boldsymbol{\mu}_2 - \boldsymbol{\mu} = \frac{(n_1 + n_2)\boldsymbol{\mu}_2 - n_1 \boldsymbol{\mu}_1 - n_2 \boldsymbol{\mu}_2}{n_1 + n_2} = -\frac{n_1}{n_1 + n_2} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2).$$

Since $\|c\,\mathbf{v}\|^2 = c^2\|\mathbf{v}\|^2$ for a number c, the two terms of Δ are $n_1 \frac{n_2^2}{(n_1+n_2)^2}\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2$ and $n_2 \frac{n_1^2}{(n_1+n_2)^2}\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2$. Their numerators add to $n_1 n_2^2 + n_2 n_1^2 = n_1 n_2 (n_1 + n_2)$, and one factor $n_1 + n_2$ cancels with the denominator, so

$$\Delta(C_1, C_2) = \frac{n_1 n_2^2 + n_2 n_1^2}{(n_1+n_2)^2}\,\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 = \boxed{\frac{n_1 n_2}{n_1 + n_2}\,\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2}.$$

Ward's method thus prefers to merge clusters whose means are close and clusters that are small. Two single points at distance 1 cost $\frac{1}{2} \times 1 = 0.5$. Merging the small runs $\{1,2,3,4\}$ with the large runs $\{5,6,7,8\}$ costs $\frac{4\times 4}{8} \times 42.8125 = 85.625$, which is exactly the between-cluster sum of squares B in the example of Section 2.3.2, *Within-cluster and between-cluster sums of squares*: the last merge raises W from 15.25 to $T = 100.875$.

3.2.2 An algorithm for all linkages

The dissimilarity between a newly merged cluster and every other cluster can be computed from the dissimilarities already known, without returning to the points. When clusters i and j merge, the dissimilarity to any other cluster k is

$$\delta(i \cup j,\, k) = \alpha_i\,\delta(i,k) + \alpha_j\,\delta(j,k) + \beta\,\delta(i,j) + \gamma\,|\delta(i,k) - \delta(j,k)|,$$

the **Lance–Williams update** (Lance and Williams, 1967), where δ denotes the dissimilarity between clusters, with coefficients

linkage	α_i	α_j	β	γ
single	$1/2$	$1/2$	0	$-1/2$
complete	$1/2$	$1/2$	0	$1/2$
average	$n_i/(n_i+n_j)$	$n_j/(n_i+n_j)$	0	0
Ward ($\delta = \Delta$)	$(n_i+n_k)/n_{ijk}$	$(n_j+n_k)/n_{ijk}$	$-n_k/n_{ijk}$	0

Here n_i, n_j and n_k are the numbers of points in the clusters, and $n_{ijk} = n_i + n_j + n_k$. For single linkage the update is $\frac{1}{2}(a+b) - \frac{1}{2}|a-b| = \min(a,b)$ with $a = \delta(i,k)$ and $b = \delta(j,k)$, the smaller of the two distances, as the definition requires. The identity is checked case by case. If $a \le b$, then $|a-b| = b-a$ and $\frac{1}{2}(a+b) - \frac{1}{2}(b-a) = a$. If $a > b$, then $|a-b| = a-b$ and $\frac{1}{2}(a+b) - \frac{1}{2}(a-b) = b$. In both cases the result is $\min(a,b)$. This is the single-linkage distance of the merged cluster, because a minimum over the points of $i \cup j$ can be taken separately over the points of i and over those of j:

$$\delta(i \cup j, k) = \min_{\mathbf{x}\in i\cup j,\ \mathbf{y}\in k} \|\mathbf{x}-\mathbf{y}\| = \min\Big(\min_{\mathbf{x}\in i,\ \mathbf{y}\in k}\|\mathbf{x}-\mathbf{y}\|,\ \min_{\mathbf{x}\in j,\ \mathbf{y}\in k}\|\mathbf{x}-\mathbf{y}\|\Big) = \min(a,b).$$

For complete linkage $\gamma = +\frac{1}{2}$. The same two cases give $\frac{1}{2}(a+b) + \frac{1}{2}(b-a) = b$ when $a \le b$ and $\frac{1}{2}(a+b) + \frac{1}{2}(a-b) = a$ when $a > b$, that is, $\max(a,b)$. The same argument with maxima in place of minima shows that this is the largest distance between a point of $i \cup j$ and a point of k. For average linkage, the $(n_i + n_j)\,n_k$ distances between $i \cup j$ and k fall into two groups. The $n_i n_k$ distances from points of i have average $\delta(i,k)$, so their sum is $n_i n_k\,\delta(i,k)$. The $n_j n_k$ distances from points of j likewise sum to $n_j n_k\,\delta(j,k)$. Hence

$$\delta(i \cup j, k) = \frac{n_i n_k\,\delta(i,k) + n_j n_k\,\delta(j,k)}{(n_i+n_j)\,n_k} = \frac{n_i}{n_i+n_j}\,\delta(i,k) + \frac{n_j}{n_i+n_j}\,\delta(j,k),$$

which gives the coefficients in the table, with $\beta = \gamma = 0$. For Ward's method the dissimilarity of two single points is $\delta = \Delta = \frac{1}{2}\|\mathbf{x}_i - \mathbf{x}_j\|^2$, by the closed form with $n_1 = n_2 = 1$, since $\frac{1\cdot 1}{1+1} = \frac{1}{2}$. The

coefficients follow from the closed form of Δ. Write $n_{ij} = n_i + n_j$, $\mathbf{g} = \boldsymbol{\mu}_i - \boldsymbol{\mu}_k$ and $\mathbf{h} = \boldsymbol{\mu}_j - \boldsymbol{\mu}_k$, so that $\mathbf{g} - \mathbf{h} = \boldsymbol{\mu}_i - \boldsymbol{\mu}_j$. The mean of $i \cup j$ is $\boldsymbol{\mu}_{i\cup j} = (n_i\boldsymbol{\mu}_i + n_j\boldsymbol{\mu}_j)/n_{ij}$, and therefore

$$\boldsymbol{\mu}_{i\cup j} - \boldsymbol{\mu}_k = \frac{n_i\boldsymbol{\mu}_i + n_j\boldsymbol{\mu}_j - (n_i + n_j)\boldsymbol{\mu}_k}{n_{ij}} = \frac{n_i\mathbf{g} + n_j\mathbf{h}}{n_{ij}}.$$

Expanding the squared norm gives

$$\|n_i\mathbf{g} + n_j\mathbf{h}\|^2 = n_i^2\|\mathbf{g}\|^2 + n_j^2\|\mathbf{h}\|^2 + 2n_in_j\,\mathbf{g}^\top\mathbf{h}.$$

The identity $\|\mathbf{g} - \mathbf{h}\|^2 = \|\mathbf{g}\|^2 + \|\mathbf{h}\|^2 - 2\mathbf{g}^\top\mathbf{h}$ gives $2\mathbf{g}^\top\mathbf{h} = \|\mathbf{g}\|^2 + \|\mathbf{h}\|^2 - \|\mathbf{g} - \mathbf{h}\|^2$, which removes the cross term:

$$\|n_i\mathbf{g}+n_j\mathbf{h}\|^2 = (n_i^2+n_in_j)\|\mathbf{g}\|^2+(n_j^2+n_in_j)\|\mathbf{h}\|^2-n_in_j\|\mathbf{g}-\mathbf{h}\|^2 = n_{ij}\big(n_i\|\mathbf{g}\|^2+n_j\|\mathbf{h}\|^2\big)-n_in_j\|\mathbf{g}-\mathbf{h}\|^2.$$

The clusters $i \cup j$ (with n_{ij} points) and k (with n_k points) have $n_{ij} + n_k = n_{ijk}$ points together, so the closed form is $\Delta(i \cup j, k) = \frac{n_{ij}n_k}{n_{ijk}}\|\boldsymbol{\mu}_{i\cup j} - \boldsymbol{\mu}_k\|^2$. Substituting the last two displays,

$$\begin{aligned}\Delta(i \cup j, k) &= \frac{n_{ij}n_k}{n_{ijk}} \cdot \frac{n_{ij}\big(n_i\|\mathbf{g}\|^2 + n_j\|\mathbf{h}\|^2\big) - n_in_j\|\mathbf{g} - \mathbf{h}\|^2}{n_{ij}^2} \\ &= \frac{n_in_k\|\mathbf{g}\|^2 + n_jn_k\|\mathbf{h}\|^2}{n_{ijk}} - \frac{n_k}{n_{ijk}} \cdot \frac{n_in_j}{n_{ij}}\|\mathbf{g} - \mathbf{h}\|^2.\end{aligned}$$

Each term is a multiple of a dissimilarity already known. By the closed form, $\Delta(i, k) = \frac{n_in_k}{n_i+n_k}\|\mathbf{g}\|^2$, so $n_in_k\|\mathbf{g}\|^2 = (n_i + n_k)\,\Delta(i, k)$. In the same way $n_jn_k\|\mathbf{h}\|^2 = (n_j + n_k)\,\Delta(j, k)$, and $\frac{n_in_j}{n_{ij}}\|\mathbf{g} - \mathbf{h}\|^2 = \Delta(i, j)$. Hence

$$\Delta(i \cup j, k) = \frac{n_i + n_k}{n_{ijk}}\,\Delta(i, k) + \frac{n_j + n_k}{n_{ijk}}\,\Delta(j, k) - \frac{n_k}{n_{ijk}}\,\Delta(i, j),$$

which gives $\alpha_i = (n_i + n_k)/n_{ijk}$, $\alpha_j = (n_j + n_k)/n_{ijk}$, $\beta = -n_k/n_{ijk}$ and $\gamma = 0$, as in the table.

Algorithm 5: agglomerative hierarchical clustering

Input: points $\mathbf{x}_1, \ldots, \mathbf{x}_n$; a linkage (single, complete, average or Ward)
Output: the sequence of $n - 1$ merges and their heights (the dendrogram)

1: start with n clusters $\{\mathbf{x}_1\}, \ldots, \{\mathbf{x}_n\}$ and compute $\delta(i, j)$ for every pair: $\|\mathbf{x}_i - \mathbf{x}_j\|$ for single, complete and average linkage, $\frac{1}{2}\|\mathbf{x}_i - \mathbf{x}_j\|^2$ for Ward
2: **for** $t = 1, \ldots, n - 1$ **do**
3: find the pair of current clusters (i, j) with the smallest $\delta(i, j)$
4: record the merge of i and j at height $\delta(i, j)$
5: **for** every other current cluster k **do**
6: compute $\delta(i \cup j,\, k)$ by the Lance–Williams update
7: **end for**
8: replace clusters i and j by $i \cup j$
9: **end for**

For Ward's method the recorded heights are increases in W, not distances. The table of dissimilarities needs memory proportional to n^2, and a direct search for the smallest entry at each of the $n - 1$ steps makes the time proportional to n^3; keeping the entries in a priority queue reduces it to about $n^2 \log n$. Even n^2 is prohibitive for a million points, which is why hierarchical clustering is used on small and medium data sets, or on a sample.

Hierarchical clustering requires no choice of K in advance and no random initialisation, and the dendrogram describes the data at every level of granularity. On the other hand, merges are never undone, so an early poor merge persists. The linkage also affects the result: single linkage tends to join clusters through chains of intermediate points (Figure 3.6); complete linkage and Ward's method favour compact clusters.

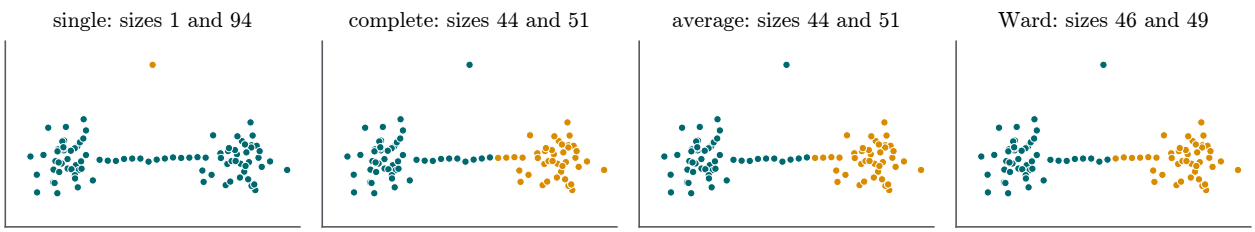


Figure 3.6: Two clusters requested from each linkage, for two groups of 40 points joined by a bridge of 14 points, with one isolated point above the bridge (95 points). Single linkage joins the two groups through the chain of the bridge and separates only the isolated point (cluster sizes 1 and 94). Complete, average and Ward linkage cut the bridge. Against the split at the middle of the bridge, with the isolated point counted on the left, they agree on 96.8%, 96.8% and 98.9% of the points.

3.3 Density-based clustering: DBSCAN and HDBSCAN

Intuition: crowds at a fair

At a fair, a person is part of a crowd if several others stand within arm's length; a crowd is everyone connected through such neighbours, so it can snake around the stalls in any shape; and a person wandering alone between crowds belongs to none. DBSCAN formalises this rule. A fixed arm's length suits crowds of one density; HDBSCAN considers all densities at once, so a packed queue and a loose picnic group can both be found.

K-means and Gaussian mixtures describe clusters by centres and spreads. A different definition does without centres: a *cluster is a region where points are densely packed*, separated from other clusters by regions of low density. Points in sparse regions belong to no cluster and are reported as *noise*. The number of clusters is not specified in advance.

Definition (DBSCAN; Ester, Kriegel, Sander and Xu, 1996)

Fix a radius ε and an integer minPts. The *ε-neighbourhood* of a point is the set of points within distance ε of it, the point itself included. A point is a **core point** if its neighbourhood contains at least minPts points. Two core points are connected if they lie in each other's neighbourhoods, or are linked by a chain of core points, each in the next one's neighbourhood. A **cluster** is a maximal set of connected core points together with the non-core points (**border points**) in their neighbourhoods. Points in no cluster are **noise**. A border point within reach of core points of two clusters may be assigned to either; Algorithm 6 below gives it to the first cluster that reaches it.

Figure 3.7 shows the three kinds of point.

Applied to the elongated and small clusters of Section 2.5.4, *Dependence on cluster shape and size* with $\varepsilon = 0.9$ and minPts $= 5$, DBSCAN finds 2 clusters and labels 10 points as noise. On the 135 points not labelled as noise it agrees with the true clusters on 100%; K-means, which labels no point as noise, agreed with them on 56% of all 145 points. Figure 3.8 shows the result.

On the two half-moons, DBSCAN with $\varepsilon = 0.2$ and minPts $= 5$ finds 2 clusters, labels no points as noise, and agrees with the true moons on 100% of the points, because each moon is a connected chain of densely packed points. Figure 3.9 compares K-means and DBSCAN on the moons.

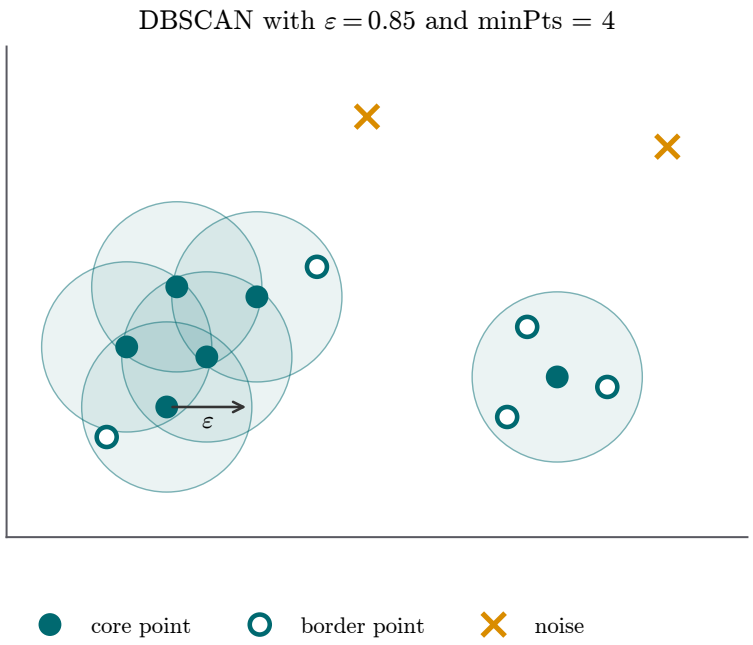


Figure 3.7: The three kinds of point in DBSCAN. Shaded discs are the ε-neighbourhoods of the core points, each containing at least minPts points; border points lie inside a core point's neighbourhood without being core points themselves; noise points lie in no such neighbourhood.

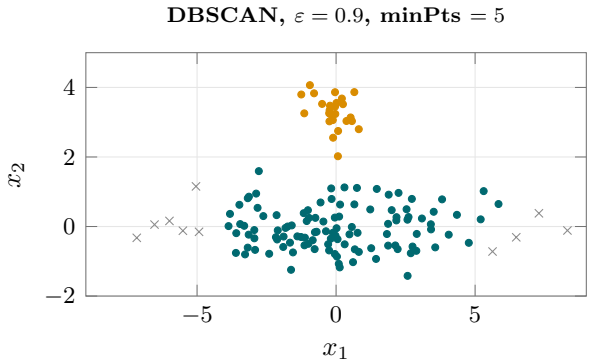


Figure 3.8: DBSCAN on the data of Section 2.5.4, *Dependence on cluster shape and size*: the elongated cluster and the small cluster are recovered; crosses are points labelled as noise.

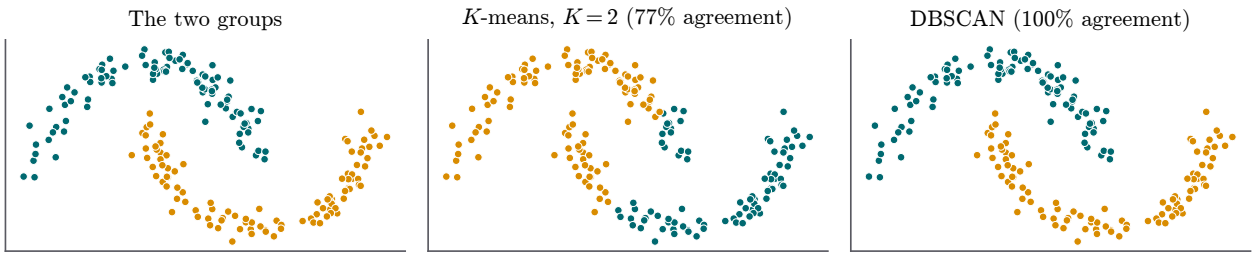


Figure 3.9: Two interleaved half-moons (200 points). K-means separates them with a straight boundary and cuts each moon in two; DBSCAN follows each moon as a chain of dense neighbourhoods and recovers both.

Algorithm 6: DBSCAN

Input: points $\mathbf{x}_1, \ldots, \mathbf{x}_n$; radius ε; integer minPts
Output: a label for every point: a cluster number, or -1 for noise

for $i = 1, \ldots, n$ **do**
    $N(i) \leftarrow \{\, j : \|\mathbf{x}_i - \mathbf{x}_j\| \le \varepsilon \,\}$; i is a core point if $|N(i)| \ge$ minPts
end for
set every label to -1; $c \leftarrow 0$
for each core point i whose label is -1 **do**
    $c \leftarrow c + 1$; give i label c; put i on a stack
    while the stack is not empty **do**
        take a point j from the stack
        if j is a core point **then**
            for each $p \in N(j)$ whose label is -1 **do**
                give p label c; put p on the stack
            end for
        end if
    end while
end for

Only core points pass the cluster on, so a border point joins a cluster without extending it, and a point that no core point reaches keeps the label -1. The neighbourhood queries dominate the cost: about n^2 distance computations in general, far fewer with a spatial index (such as a k-d tree, which partitions the space into boxes) when the dimension is low.

Choosing ε. Because the neighbourhood counts the point itself, a point is a core point exactly when its distance to its (minPts $-$ 1)-th nearest other point, its **k-distance** with $k =$ minPts $- 1$, is at most ε. The neighbourhood of a point contains the point itself and the other points within distance ε. It therefore has at least minPts members exactly when at least minPts $- 1$ other points lie within distance ε. List the other points in order of increasing distance: this happens exactly when the (minPts $- 1$)-th of them is within distance ε, since the points before it in the list are no farther away. Sorting the k-distances of all points in decreasing order gives the k-distance plot. Points inside clusters have small k-distances and isolated points large ones, so a value of ε at the bend of the curve makes most points core points while leaving the sparse ones as noise. For the half-moons with minPts $= 5$: at $\varepsilon = 0.1$ only 45.0% of the points are core points, and DBSCAN reports 8 clusters and 74 noise points; at $\varepsilon = 0.2$, 93.5% are core points and the two moons are recovered; at $\varepsilon = 0.5$ the neighbourhoods reach across the gap and the moons merge into a single cluster. Figure 3.10 shows the sorted distances and the clusters found at the three radii.

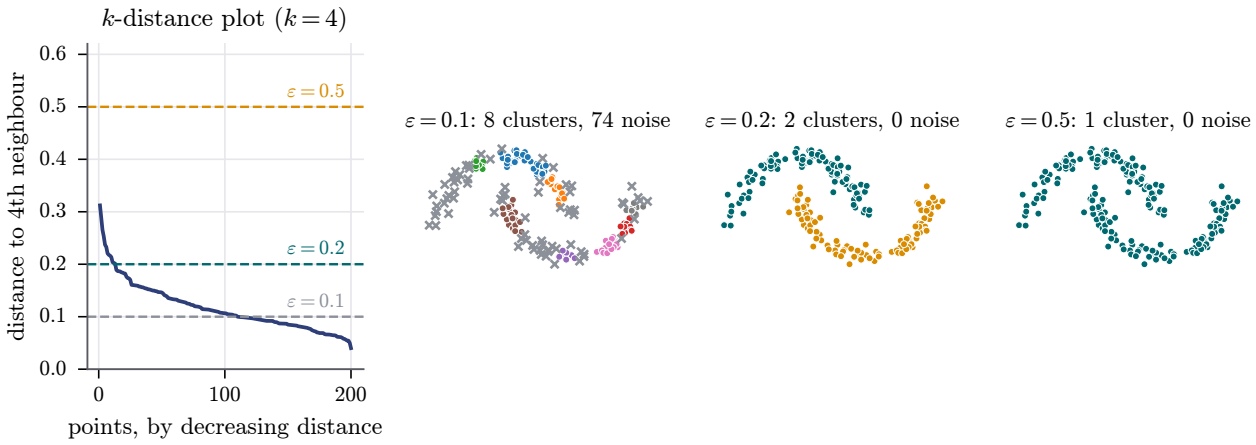


Figure 3.10: Choosing ε for the half-moons with minPts $= 5$. Left: the distance of every point to its fourth nearest other point, in decreasing order; a point is a core point when this distance is at most ε. Right: DBSCAN at the three radii marked on the left. Crosses are noise.

DBSCAN needs a single radius ε, which fails when clusters have different densities: a radius small enough to separate dense clusters breaks sparse ones into noise. **HDBSCAN** (Campello, Moulavi and Sander, 2013) removes this parameter. For an integer k, the *core distance* $\mathrm{core}_k(a)$ of a point is its distance to the k-th nearest other point (conventions differ on whether the point itself is counted; here it is not), and the *mutual reachability distance* is

$$d_{\mathrm{mreach}}(a,b) = \max\{\mathrm{core}_k(a),\ \mathrm{core}_k(b),\ d(a,b)\}.$$

This distance is never smaller than the ordinary one, and it is inflated for points in sparse regions. HDBSCAN builds the single-linkage hierarchy under d_{mreach} and then keeps, among the clusters of at least a minimum size, those that persist over the widest range of density levels, each weighted by its number of points. The radius ε is no longer needed, but the minimum cluster size and k remain. It returns clusters of varying density and shape together with a noise label, and it is widely used when the number of clusters is unknown.

3.4 Evaluating a clustering

Intuition: seeing faces in clouds

People see faces in clouds and animals in the stars. A clustering method is similar: it always reports groups, whether or not the data have any. Before trusting a grouping, check how well each point fits its group, whether the same groups appear in random data, whether they reappear when the data are resampled, and whether they mean anything.

Every clustering method returns clusters, whether or not the data contain any. In the absence of labels, a clustering must be evaluated by other means.

3.4.1 The silhouette coefficient

For a point i, let a_i be its mean distance to the other points of its own cluster, and b_i the smallest, over the other clusters, of its mean distance to the points of that cluster. For a point that is alone in its cluster, $s_i = 0$ by convention. The **silhouette coefficient** of point i is

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \qquad -1 \le s_i \le 1.$$

The bounds follow by cases. Both a_i and b_i are means of distances, so they are nonnegative; assume they are not both zero. If $a_i \le b_i$, then $\max(a_i, b_i) = b_i > 0$ and $s_i = 1 - a_i/b_i$ with $0 \le a_i/b_i \le 1$, so $0 \le s_i \le 1$. If $a_i > b_i$, then $\max(a_i, b_i) = a_i$ and $s_i = b_i/a_i - 1$ with $0 \le b_i/a_i < 1$, so $-1 \le s_i < 0$. A value close to 1 indicates that the point is much closer to its own cluster than to any other; a value close to 0 indicates that it lies near a boundary; a negative value indicates that it is closer, on average, to another cluster than to its own.

Example

Consider run 1, at $(4, 4)$, in the final partition $\{1,2,3,4\}$, $\{5,6,7,8\}$. Its distances to the other three runs of its cluster are $1.000, 2.236, 3.162$, so $a_1 = 2.133$. Its distances to the four runs of the other cluster are $5.657, 4.472, 6.403, 4.472$, so $b_1 = 5.251$. Hence

$$s_1 = \frac{5.251 - 2.133}{5.251} = 0.594.$$

The silhouette coefficients of all eight runs are as follows.

run	1	2	3	4	5	6	7	8	mean
s_i	0.594	0.667	0.763	0.706	0.766	0.561	0.735	0.596	**0.673**

If run 4 is moved from $(1, 3)$ to $(5, 5.5)$ but kept in the same cluster, its distances to runs 1, 2 and 3 become $\sqrt{1^2 + 1.5^2} = \sqrt{3.25} = 1.803$, $\sqrt{1^2 + 2.5^2} = \sqrt{7.25} = 2.693$ and $\sqrt{3^2 + 2.5^2} = \sqrt{15.25} = 3.905$, so $a_4 = 8.401/3 = 2.800$. Its distances to runs 5, 6, 7 and 8 are $\sqrt{3^2 + 2.5^2} = 3.905$, $\sqrt{3^2 + 0.5^2} = \sqrt{9.25} = 3.041$, $\sqrt{3^2 + 3.5^2} = \sqrt{21.25} = 4.610$ and $\sqrt{1^2 + 2.5^2} = 2.693$, so $b_4 = 14.249/4 = 3.562$. Since $b_4 > a_4$, its silhouette coefficient falls to $(3.562 - 2.800)/3.562 = 0.214$. A **silhouette plot** shows the coefficients of all points, sorted within each cluster, and so reveals which cluster is responsible for a low mean. The mean silhouette coefficient over all points summarises the whole clustering and can be used to compare values of K: for the three-cluster data of Section 2.5.5, *Choosing the number of clusters* it is 0.589 for $K = 2$, 0.773 for $K = 3$ and 0.635 for $K = 4$, largest at the true number of clusters. Figure 3.11 shows the silhouette plots for $K = 2$, 3 and 4.

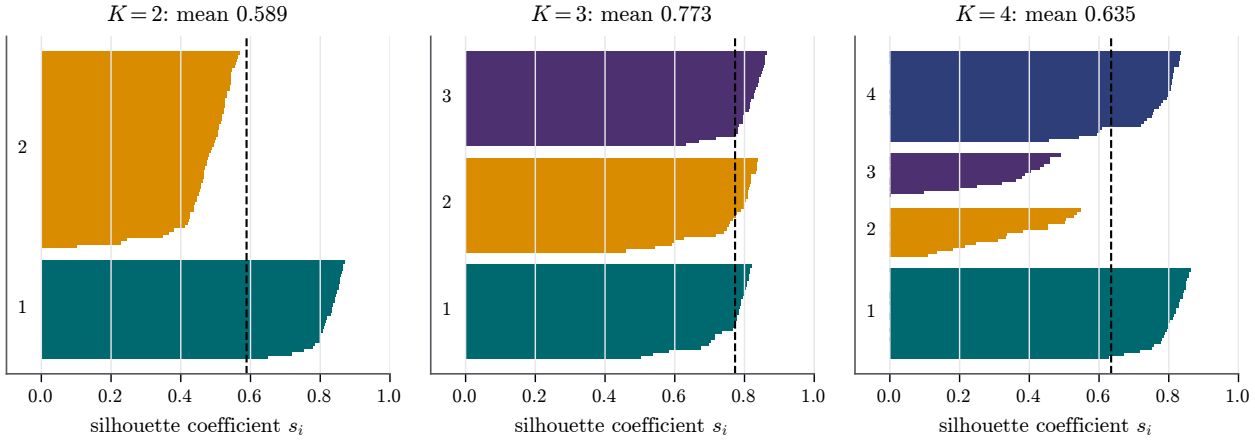


Figure 3.11: Silhouette plots for K-means with $K = 2, 3, 4$ on the 90 points from three clusters of Section 2.5.5, *Choosing the number of clusters*. Each bar is one point; the dashed line is the mean. With $K = 3$ every cluster has uniformly high values. With $K = 2$ one cluster contains two of the true groups, and its values are lower. With $K = 4$ one true group is split in two, and both halves have low values.

3.4.2 Comparison with data that have no clusters

Ninety points distributed uniformly over a square have no clusters. Nevertheless, K-means with $K = 3$ reduces J by 64% relative to a single cluster and gives a mean silhouette of 0.411. For the three-cluster data the corresponding values are 93% and 0.773. A substantial reduction in J, or a moderate silhouette value, is therefore not in itself evidence of clusters. A useful reference is obtained by applying the same method to data with the same ranges but no clusters and comparing the results. Figure 3.12 shows the two results side by side: three clusters are returned in both.

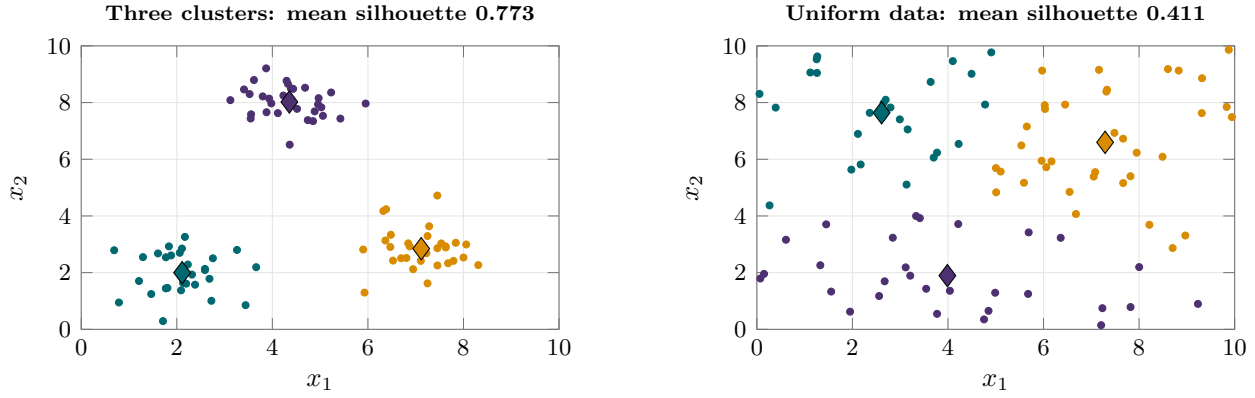


Figure 3.12: K-means with $K = 3$ applied to 90 points from three clusters (left) and to 90 points distributed uniformly (right). Three clusters are returned in both cases; diamonds mark the centroids.

3.4.3 Stability under resampling

Genuine clusters should not depend on the particular sample collected. To assess this, divide the data at random into two halves and cluster the first half. Label the points of the second half in two ways: by the nearest centroid from the first half, and by clustering the second half separately. If the clusters are genuine, the two labellings agree, up to a renaming of the clusters. Averaged over 60 random divisions, the agreement (the fraction of points given the same label, under the renaming of clusters that makes this fraction largest, as in Section 2.5.3, *Dependence on the scale of the features*) is 1.00 for the three-cluster data and 0.84 for the uniform data. Agreement is never below $1/K$ (Problem 3.15), and on data without clusters it can be well above that, so it must be judged against such data rather than against zero.

3.4.4 Interpretation

Finally, the clusters should admit an interpretation. This is assessed by examining the mean of each feature within each cluster and by inspecting representative examples. For the eight runs, the clusters are short runs with small memory requirements and long runs with large ones. A clustering that is useful for a subsequent task, such as allocating hardware or fitting a separate model to each cluster, is better supported than one that only scores well on the measures above.

3.5 Choosing a clustering method

The methods of Chapters 2 and 3 answer the same question under different assumptions about what a cluster is.

method	a cluster is	K given?	membership	noise label	suits
K-means	points near a centre; round, similar sizes	yes	hard	no	large data, compact groups; fast first attempt
Gaussian mixture	a bell-shaped cloud, each with its own spread	yes	soft (probabilities)	no	overlapping groups; uncertain members
hierarchical	a subtree of the merge tree	no (cut the tree)	hard	no	small data; groups at several scales
DBSCAN, HDBSCAN	a dense region, any shape	no	hard	yes	curved groups; data with outliers; unequal densities (HDBSCAN)

A common sequence is to standardise the features, reduce them with PCA if there are many (Chapter 4), try K-means as a baseline, and move to a mixture or a density-based method when the assumptions of K-means visibly fail; in every case the result is checked as in Section 3.4, *Evaluating a clustering*.

3.6 Practice problems for Chapter 3

3.1 [Basic] A mixture has two components with means 0 and 4, common standard deviation $\sigma = 1$ and equal weights. Compute the responsibility of the first component for $x = 1$ and for $x = 2$. Where is the boundary at which the two responsibilities are equal, and why?

3.2 [Standard] For a one-dimensional mixture with responsibilities r_{ik} fixed, and without consulting Section 3.1.6, *General Gaussian mixtures*, derive the M-step updates for (a) the variance σ_k^2 and (b) the weights π_k, using a Lagrange multiplier for the constraint $\sum_k \pi_k = 1$.

3.3 [Standard] In the proof of Section 3.1.5, *Why the log-likelihood never decreases (optional)*, show directly that $F(q, \theta) = \ell(\theta)$ when q_{ik} are the responsibilities at θ, and that for any other choice of q with $\sum_k q_{ik} = 1$, $F(q, \theta) \le \ell(\theta)$.

3.4 [Basic] One-dimensional data are clustered as $\{0, 1, 2\}$ and $\{6, 7\}$. Compute the silhouette coefficient of the point 2.

3.5 [Basic] For the points $0, 1, 3, 7$, give the sequence of merges and merge heights under single linkage and under complete linkage.

3.6 [Basic] Apply DBSCAN with $\varepsilon = 0.6$ and minPts $= 3$ (the point itself included) to the one-dimensional data 0, 0.5, 1, 1.5, 5, 5.4, 9. Classify every point as core, border or noise, and give the clusters.

3.7 [Standard] Show that the likelihood of a Gaussian mixture whose variances are estimated is unbounded: construct parameters for which it is arbitrarily large. What does this imply for implementations?

3.8 [Standard] For the points $0, 1, 2, 10$ and $k = 2$, compute the core distances (distance to the second nearest other point) and the mutual reachability distances $d_{\text{mreach}}(0, 1)$ and $d_{\text{mreach}}(2, 10)$. Which pair is pushed further apart relative to its ordinary distance, and why?

3.9 [Standard] A clustering of 500 runs into four clusters has mean silhouette 0.42. The same method on uniformly scattered data with the same ranges gives 0.38, and on random half-samples the agreement between labellings is 0.61, against 0.58 for the uniform data. What would you conclude, and what would you do next?

3.10 [Standard] In Problem 3.1, change the weights to $\pi_1 = 0.2$ and $\pi_2 = 0.8$. Compute the responsibility of the first component at $x = 2$, and find the boundary where the two responsibilities are equal. Which way does it move, and why?

3.11 [Standard] Data $0, 1, 5, 6$; two components with means 1 and 5, $\sigma = 1$, equal weights. Carry out one E-step and one M-step for the means.

3.12 [Basic] One-dimensional data are clustered as $\{0, 1, 4\}$ and $\{5, 6\}$. Compute the silhouette coefficient of the point 4 and interpret its sign.

3.13 [Basic] Repeat Problem 3.5 with average linkage.

3.14 [Challenge] For a mixture with equal weights and a common variance σ^2, prove that as $\sigma \to 0$ the responsibility r_{ik} tends to 1 if $\boldsymbol{\mu}_k$ is the unique nearest mean to $\mathbf{x}_i$ and to 0 otherwise.

3.15 [Challenge] Two labellings of the same n points into K clusters are compared by the best agreement over all renamings of the clusters. Prove that this best agreement is at least $1/K$, so that agreement must be judged against data that have no clusters, not against zero.

Chapter 4

Principal component analysis and maps for visualisation

Companion notebook in Google Colab: Part C.

4.1 Reducing the number of features

Intuition: height and arm span

A tailor who records a customer's height and arm span learns almost the same thing twice: the two are nearly equal for most people. A single number, a typical size, would carry most of the information, and a second number, the small difference between them, would carry the rest. PCA finds such combined numbers for any set of features.

Features are often redundant. In the eight training runs, a long run almost always uses much memory, so the two numbers largely repeat each other. The same is true much more widely: the logged metrics of a training run (losses at several checkpoints, throughput, memory, gradient norms), the expression levels of thousands of genes, the prices of related financial instruments, the pixels of an image, the readings of many sensors on one machine, and the embedding vectors produced by trained models all have many coordinates that are far from independent. Describing such data with fewer coordinates serves several purposes: data in two or three coordinates can be plotted; later models have fewer inputs to fit; storage and computation shrink; and coordinates that carry mostly noise can be discarded.

Many features also make distances less informative, an effect known as the **curse of dimensionality**. For 500 points scattered uniformly in the unit cube $[0,1]^d$, the table shows how much farther the farthest point is than the nearest, relative to the nearest, averaged over 50 reference points:

number of features d	2	10	100	1000
(farthest − nearest)/nearest	73.46	2.41	0.41	0.11

In two dimensions the farthest point is about 74.5 times as far away as the nearest; in a thousand dimensions all points are at nearly the same distance (the gap is 0.11 of the nearest distance), so "nearest" carries little meaning. Methods built on distances, including K-means and nearest-neighbour scores, suffer accordingly. Reducing the data to the few directions that carry most of the variation is one remedy.

Principal component analysis (PCA) finds new coordinates that are linear combinations of the old ones, chosen in order of how much of the variation of the data they retain. In genetics, the first two principal components of DNA measurements from many people reproduce the geography of their

origins; in finance, a few components describe most movements of interest rates; in manufacturing, a PCA model of normal operation flags faults by its reconstruction error (Chapter 5).

4.2 Principal components

4.2.1 Projection onto a direction and the variance it retains

Intuition: the shadow of an object

Shine a torch on a pencil and look at its shadow on the wall. Lit from the side, the shadow shows the full length of the pencil; lit from one end, the shadow is a dot. A projection of data onto a direction is such a shadow, and the variance retained measures how long the shadow of the data cloud is.

Centre the data by subtracting the mean $\bar{\mathbf{x}}$. A unit vector $\mathbf{u}$ defines a single new coordinate: the **score** $z_i = \mathbf{u}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$, the signed position of the projection of $\mathbf{x}_i$ along $\mathbf{u}$. The variance of the scores is $\mathbf{u}^\top S\mathbf{u}$. Different directions retain different amounts of variance; for the eight runs:

angle with the x_1 axis	unit direction $\mathbf{u}$	variance kept, $\mathbf{u}^\top S\mathbf{u}$
0.0°	(1.000, 0)	6.859
30.0°	(0.866, 0.500)	11.291
42.1°	(0.742, 0.670)	11.770 (first principal direction)
60.0°	(0.500, 0.866)	10.736
90.0°	(0, 1.000)	5.750
132.1°	(−0.670, 0.742)	0.839

The variance retained by the x_1 axis alone is $S_{11} = 6.8594$ and by the x_2 axis $S_{22} = 5.7500$; a diagonal direction retains much more than either, because the two features vary together. The direction at about 42.1° retains the most, and the perpendicular direction at 42.1° + 90° the least. Finding this direction exactly is an eigenvalue problem. Figure 4.1 shows the projections onto four directions and the retained variance for every direction.

4.2.2 The first principal direction

Intuition: photographing a long object

To photograph a pencil so that the picture shows as much of it as possible, turn the camera until the pencil lies across the frame, not pointing at the lens. The first principal direction is the camera angle that shows the data cloud at its longest.

The problem. Find the unit vector $\mathbf{u}$ that maximises the retained variance:

$$\max_{\mathbf{u}} \ \mathbf{u}^\top S\mathbf{u} \qquad \text{subject to} \qquad \mathbf{u}^\top\mathbf{u} = 1.$$

The constraint is necessary: without it, scaling $\mathbf{u}$ up would increase the objective without limit.

The solution. Using the gradients $\nabla_{\mathbf{u}}(\mathbf{u}^\top S\mathbf{u}) = 2S\mathbf{u}$ and $\nabla_{\mathbf{u}}(\mathbf{u}^\top\mathbf{u}) = 2\mathbf{u}$ derived in Chapter 1, *Mathematical background*, with the Lagrangian $\mathcal{L} = \mathbf{u}^\top S\mathbf{u} - \lambda(\mathbf{u}^\top\mathbf{u} - 1)$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}} = 2S\mathbf{u} - 2\lambda\mathbf{u} = 0 \quad \Longrightarrow \quad S\mathbf{u} = \lambda\mathbf{u}.$$

A maximiser exists: $\mathbf{u}^\top S\mathbf{u}$ is a continuous function, and the unit sphere $\{\mathbf{u} : \mathbf{u}^\top\mathbf{u} = 1\}$ is closed and bounded, so the function attains a largest value on it. At that maximiser the gradient of the constraint, $2\mathbf{u}$, is not zero because $\|\mathbf{u}\| = 1$, so the Lagrange condition applies and gives $S\mathbf{u} = \lambda\mathbf{u}$

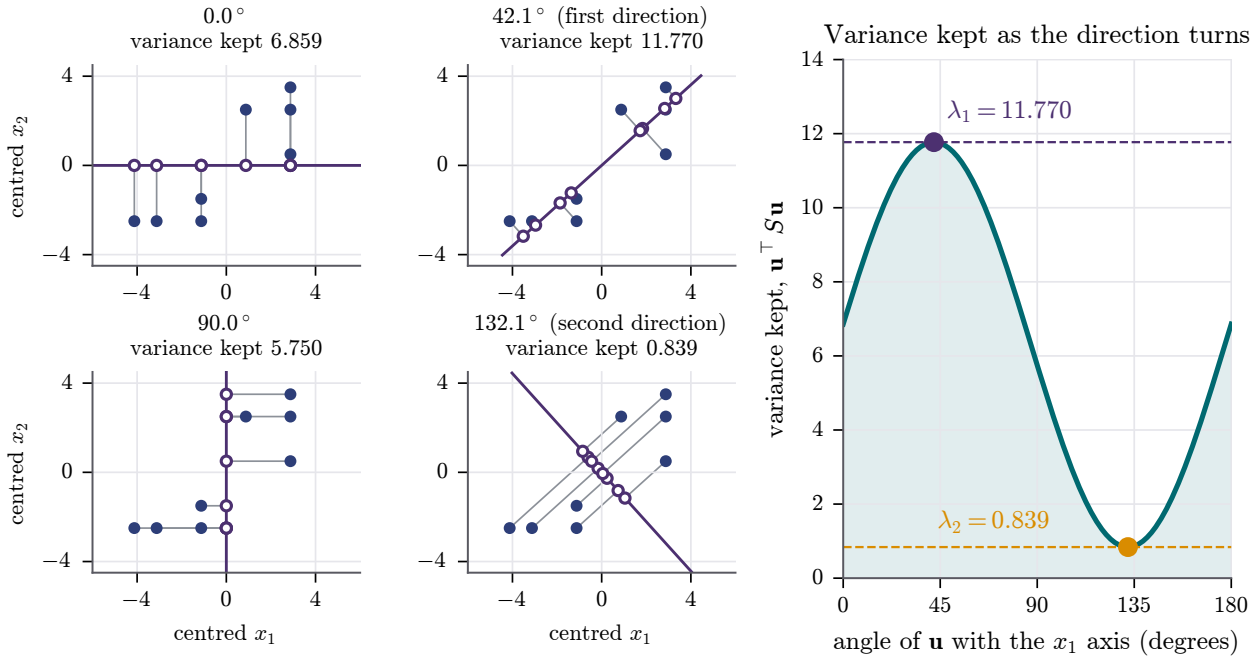


Figure 4.1: Left: the centred runs (filled) projected onto four directions (open circles), with the variance of the projections. Right: the retained variance $\mathbf{u}^\top S\mathbf{u}$ as the direction turns through 180°. Its maximum and minimum are the eigenvalues λ_1 and λ_2, attained at perpendicular directions.

with $\mathbf{u} \neq \mathbf{0}$. A constrained maximiser must therefore be an eigenvector of S, and the multiplier λ is its eigenvalue. At such a vector the retained variance equals the eigenvalue, since $\mathbf{u}^\top S\mathbf{u} = \lambda\,\mathbf{u}^\top\mathbf{u} = \lambda$. The value at the maximiser is thus one of the eigenvalues, so it is at most λ_1; and a unit eigenvector of λ_1 satisfies the constraint and attains λ_1. The same argument applied to the smallest value shows that the minimiser is a unit eigenvector of the smallest eigenvalue. With two features and $\lambda_1 > \lambda_2$, the spectral theorem makes the two eigenvectors perpendicular, which is why the least variance is retained at right angles to the direction that retains the most. The maximum is therefore attained at an eigenvector with the *largest* eigenvalue λ_1, which is called the **first principal direction** $\mathbf{v}_1$, and the maximum retained variance is λ_1.

Example: the eight training runs

The characteristic equation of S is $\lambda^2 - (\operatorname{tr} S)\lambda + \det S = 0$ with

$$\operatorname{tr} S = 6.8594 + 5.7500 = 12.6094, \qquad \det S = 6.8594 \times 5.7500 - 5.4375^2 = 9.8750.$$

The discriminant is $12.6094^2 - 4(9.8750) = 119.4963$, whose square root is 10.9314, so

$$\lambda_1 = \tfrac{1}{2}(12.6094 + 10.9314) = 11.7704, \qquad \lambda_2 = \tfrac{1}{2}(12.6094 - 10.9314) = 0.8390.$$

For $\mathbf{v}_1 = (a, b)$, the first row of $(S - \lambda_1 I)\mathbf{v}_1 = 0$ gives $(6.8594 - 11.7704)\,a + 5.4375\,b = 0$, so $b/a = 4.9110/5.4375 = 0.9032$. Normalising $(1,\ 0.9032)$ by its length 1.3475 gives

$$\mathbf{v}_1 = (0.7421,\ 0.6703), \qquad \mathbf{v}_2 = (-0.6703,\ 0.7421),$$

the second being perpendicular to the first. The normalising length is $\sqrt{1^2 + 0.9032^2} = 1.3475$, and dividing gives $1/1.3475 = 0.7421$ and $0.9032/1.3475 = 0.6703$. In the plane the unit vectors perpendicular to (a, b) are $\pm(-b, a)$, since $a(-b) + b\,a = 0$, and by the spectral theorem the eigenvector of λ_2 is perpendicular to $\mathbf{v}_1$; so $\mathbf{v}_2 = (-0.6703,\ 0.7421)$, the sign being a free choice. The first direction makes an angle of 42.1° with the x_1 axis, since $\tan\theta = b/a = 0.9032$, and retains variance $\lambda_1 = 11.770$, which is $11.7704/12.6094$, that is 93.3%, of the total variance $\operatorname{tr} S = 12.6094$.

4.2.3 Scores and reconstruction error

Intuition: locating a house by its distance along a highway

In a town built along a highway, a house can be described approximately by one number: how far along the highway it is. The description is exact for houses on the highway and off by the side distance for the others. The score is the distance along the highway, and the reconstruction error is the squared distance from the highway.

The **principal component scores** of run i are $z_{i1} = \mathbf{v}_1^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ and $z_{i2} = \mathbf{v}_2^\top(\mathbf{x}_i - \bar{\mathbf{x}})$. Keeping only the first, the run is **reconstructed** as the point $\hat{\mathbf{x}}_i = \bar{\mathbf{x}} + z_{i1}\mathbf{v}_1$ on the first principal line (Figure 4.2). For run 1, the deviation is $(-1.125,\ -1.500)$ and $z_{11} = 0.7421(-1.125) + 0.6703(-1.500) = -1.840$.

run	$\mathbf{x}_i$	$\mathbf{x}_i - \bar{\mathbf{x}}$	score $z_{i1} = \mathbf{v}_1^\top(\mathbf{x}_i - \bar{\mathbf{x}})$	$z_{i2} = \mathbf{v}_2^\top(\mathbf{x}_i - \bar{\mathbf{x}})$	squared distance z_{i2}^2
1	$(4, 4)$	$(-1.125,\ -1.500)$	-1.840	-0.359	0.129
2	$(4, 3)$	$(-1.125,\ -2.500)$	-2.511	-1.101	1.213
3	$(2, 3)$	$(-3.125,\ -2.500)$	-3.995	0.239	0.057
4	$(1, 3)$	$(-4.125,\ -2.500)$	-4.737	0.910	0.827
5	$(8, 8)$	$(2.875,\ 2.500)$	3.809	-0.072	0.005
6	$(8, 6)$	$(2.875,\ 0.500)$	2.469	-1.556	2.421
7	$(8, 9)$	$(2.875,\ 3.500)$	4.480	0.670	0.449
8	$(6, 8)$	$(0.875,\ 2.500)$	2.325	1.269	1.610
			sum of squared distances to the line		6.712

The variance of the first scores is $\frac{1}{8}\sum_i z_{i1}^2 = 11.770 = \lambda_1$, as it must be.

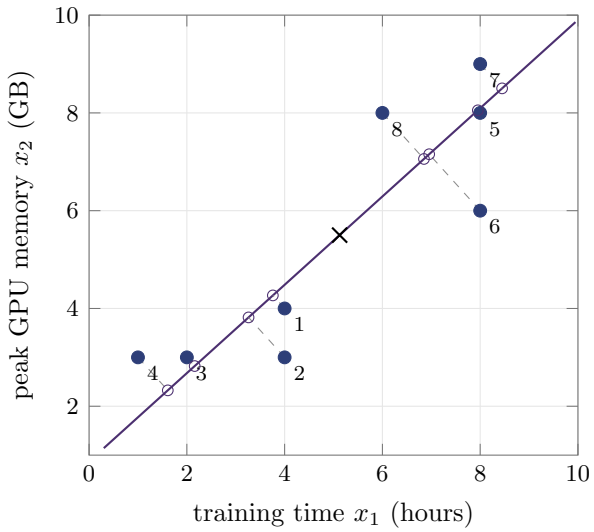


Figure 4.2: The eight runs (filled circles), their mean (cross), the first principal line, and the projection of each run onto it (open circles). The dashed segments are the reconstruction errors.

Maximum variance is minimum reconstruction error. Because $\mathbf{v}_1, \mathbf{v}_2$ form an orthonormal basis of the plane, every deviation can be written as $\mathbf{x}_i - \bar{\mathbf{x}} = a\,\mathbf{v}_1 + b\,\mathbf{v}_2$; the dot product with $\mathbf{v}_1$, using $\mathbf{v}_1^\top\mathbf{v}_1 = 1$ and $\mathbf{v}_1^\top\mathbf{v}_2 = 0$, gives $a = \mathbf{v}_1^\top(\mathbf{x}_i - \bar{\mathbf{x}}) = z_{i1}$, and in the same way $b = z_{i2}$. Thus $\mathbf{x}_i - \bar{\mathbf{x}} = z_{i1}\mathbf{v}_1 + z_{i2}\mathbf{v}_2$, and subtracting $\hat{\mathbf{x}}_i - \bar{\mathbf{x}} = z_{i1}\mathbf{v}_1$ leaves $\mathbf{x}_i - \hat{\mathbf{x}}_i = z_{i2}\mathbf{v}_2$. Expanding the squared lengths, $\|z_{i1}\mathbf{v}_1 + z_{i2}\mathbf{v}_2\|^2 = z_{i1}^2\,\mathbf{v}_1^\top\mathbf{v}_1 + 2z_{i1}z_{i2}\,\mathbf{v}_1^\top\mathbf{v}_2 + z_{i2}^2\,\mathbf{v}_2^\top\mathbf{v}_2 = z_{i1}^2 + z_{i2}^2$ and $\|z_{i2}\mathbf{v}_2\|^2 = z_{i2}^2\,\mathbf{v}_2^\top\mathbf{v}_2 = z_{i2}^2$. This is Pythagoras: for every run,

$$\|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = z_{i1}^2 + z_{i2}^2, \qquad \text{and} \qquad \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = z_{i2}^2.$$

The same split holds for any unit direction $\mathbf{u}$, not only $\mathbf{v}_1$: with $z_i = \mathbf{u}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$, the Pythagoras relation for a projection (Chapter 1, *Mathematical background*) gives $\|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = z_i^2 + \|\mathbf{x}_i - \bar{\mathbf{x}} - z_i\mathbf{u}\|^2$,

and the last term is the squared reconstruction error of run i on the line through $\bar{\mathbf{x}}$ in direction $\mathbf{u}$. Summing over i, the total sum of squares $\sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2$ is fixed by the data and does not depend on $\mathbf{u}$, so for every direction the retained part and the reconstruction error add to the same total; at $\mathbf{u} = \mathbf{v}_1$ they are $\sum_i z_{i1}^2$ and $\sum_i z_{i2}^2$, and *making the retained part $\sum_i z_{i1}^2$ as large as possible is the same as making the reconstruction error $\sum_i z_{i2}^2$ as small as possible.* For the eight runs the total is $\sum_i \tilde{x}_{i1}^2 + \sum_i \tilde{x}_{i2}^2 = 54.875 + 46.000 = 100.875$, which is also $n \operatorname{tr} S = 8 \times 12.6094$. The retained part is $\sum_i z_{i1}^2 = n\,\mathbf{v}_1^\top S \mathbf{v}_1 = n\lambda_1 = 8 \times 11.7704 = 94.163$, and the reconstruction error is $\sum_i z_{i2}^2 = n\,\mathbf{v}_2^\top S \mathbf{v}_2 = n\lambda_2 = 8 \times 0.8390 = 6.712$, the sum 6.712 of the last column of the table; indeed $94.163 + 6.712 = 100.875$. The total, 100.875, is the within-cluster sum of squares J of a single cluster: K-means with one cluster and PCA start from the same quantity.

4.2.4 Several principal components

Intuition: batting and bowling

A cricketer's ten statistics (runs, average, strike rate, wickets, economy, and so on) are largely summarised by two numbers: how good a batter and how good a bowler the player is. Further numbers add detail but little new information. Principal components are such summary numbers, ordered by how much of the variation they capture.

With d features we may keep $k < d$ coordinates. The natural generalisation chooses orthonormal directions $\mathbf{u}_1, \dots, \mathbf{u}_k$ to maximise the total retained variance $\sum_{j=1}^{k} \mathbf{u}_j^\top S \mathbf{u}_j$.

Theorem

The maximum of $\sum_{j=1}^{k} \mathbf{u}_j^\top S \mathbf{u}_j$ over orthonormal $\mathbf{u}_1, \dots, \mathbf{u}_k$ equals $\lambda_1 + \cdots + \lambda_k$, and it is attained by the eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ of the k largest eigenvalues.

Proof. Expand with the spectral theorem, $S = \sum_{i=1}^{d} \lambda_i \mathbf{v}_i \mathbf{v}_i^\top$:

$$\mathbf{u}_j^\top S \mathbf{u}_j = \sum_{i=1}^{d} \lambda_i \, (\mathbf{u}_j^\top \mathbf{v}_i)(\mathbf{v}_i^\top \mathbf{u}_j) = \sum_{i=1}^{d} \lambda_i (\mathbf{v}_i^\top \mathbf{u}_j)^2,$$

since $\mathbf{u}_j^\top \mathbf{v}_i = \mathbf{v}_i^\top \mathbf{u}_j$ (a dot product does not depend on the order of its factors). Summing over j and exchanging the order of the two finite sums,

$$\sum_{j=1}^{k} \mathbf{u}_j^\top S \mathbf{u}_j = \sum_{i=1}^{d} \lambda_i \, w_i, \qquad w_i = \sum_{j=1}^{k} (\mathbf{v}_i^\top \mathbf{u}_j)^2.$$

Each w_i is the squared length of the projection of the unit vector $\mathbf{v}_i$ onto the span of the orthonormal $\mathbf{u}_j$, so $0 \le w_i \le 1$. In detail, fix i, write $c_j = \mathbf{u}_j^\top \mathbf{v}_i$ and $\mathbf{p} = \sum_{j=1}^{k} c_j \mathbf{u}_j$. Because $\mathbf{u}_j^\top \mathbf{u}_l$ is 1 when $j = l$ and 0 otherwise,

$$\|\mathbf{p}\|^2 = \sum_{j=1}^{k} \sum_{l=1}^{k} c_j c_l \, \mathbf{u}_j^\top \mathbf{u}_l = \sum_{j=1}^{k} c_j^2 = w_i, \qquad \mathbf{u}_l^\top (\mathbf{v}_i - \mathbf{p}) = c_l - \sum_{j=1}^{k} c_j \, \mathbf{u}_l^\top \mathbf{u}_j = c_l - c_l = 0.$$

The remainder $\mathbf{v}_i - \mathbf{p}$ is therefore perpendicular to every $\mathbf{u}_l$, hence to $\mathbf{p}$, and Pythagoras gives $1 = \|\mathbf{v}_i\|^2 = \|\mathbf{p}\|^2 + \|\mathbf{v}_i - \mathbf{p}\|^2 \ge w_i$; and $w_i \ge 0$ because it is a sum of squares. Next, because $\mathbf{v}_1, \dots, \mathbf{v}_d$ is an orthonormal basis, each $\mathbf{u}_j$ can be written as $\mathbf{u}_j = \sum_{i=1}^{d} a_i \mathbf{v}_i$; the dot product with $\mathbf{v}_m$ gives $a_m = \mathbf{v}_m^\top \mathbf{u}_j$, and the same calculation as for $\|\mathbf{p}\|^2$ gives $\|\mathbf{u}_j\|^2 = \sum_{i=1}^{d} (\mathbf{v}_i^\top \mathbf{u}_j)^2$. Hence

$$\sum_{i=1}^{d} w_i = \sum_{j=1}^{k} \sum_{i=1}^{d} (\mathbf{v}_i^\top \mathbf{u}_j)^2 = \sum_{j=1}^{k} \|\mathbf{u}_j\|^2 = \sum_{j=1}^{k} 1 = k.$$

A weighted sum $\sum_i \lambda_i w_i$ with weights in $[0,1]$ summing to k is largest when the weight 1 is placed on the k largest λ_i, giving $\lambda_1 + \cdots + \lambda_k$. To see this, subtract that value and split the sum at k:

$$\sum_{i=1}^{d} \lambda_i w_i - \sum_{i=1}^{k} \lambda_i = \sum_{i=1}^{k} \lambda_i (w_i - 1) + \sum_{i=k+1}^{d} \lambda_i w_i.$$

For $i \le k$, $w_i - 1 \le 0$ and $\lambda_i \ge \lambda_k$, so $\lambda_i(w_i - 1) \le \lambda_k(w_i - 1)$; for $i > k$, $w_i \ge 0$ and $\lambda_i \le \lambda_k$, so $\lambda_i w_i \le \lambda_k w_i$. Therefore

$$\sum_{i=1}^{d} \lambda_i w_i - \sum_{i=1}^{k} \lambda_i \le \lambda_k \Big(\sum_{i=1}^{k} (w_i - 1) + \sum_{i=k+1}^{d} w_i \Big) = \lambda_k \Big(\sum_{i=1}^{d} w_i - k \Big) = 0.$$

With $\mathbf{u}_j = \mathbf{v}_j$ for $j = 1, \ldots, k$, orthonormality gives $\mathbf{v}_i^\top \mathbf{u}_j = 1$ when $i = j$ and 0 otherwise, so $w_i = 1$ for $i \le k$ and $w_i = 0$ for $i > k$, the bound is reached, and the eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ achieve this. □

The same Pythagoras argument shows that these directions also minimise the reconstruction error. For orthonormal $\mathbf{u}_1, \ldots, \mathbf{u}_k$, point i is reconstructed as $\hat{\mathbf{x}}_i = \bar{\mathbf{x}} + \sum_{j=1}^{k} (\mathbf{u}_j^\top \tilde{\mathbf{x}}_i)\, \mathbf{u}_j$: the projection of $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$ onto the span of the $\mathbf{u}_j$, added back to the mean. The residual $\mathbf{x}_i - \hat{\mathbf{x}}_i$ is perpendicular to every $\mathbf{u}_l$ (by the calculation used for w_i in the proof), and the projection has squared length $\sum_j (\mathbf{u}_j^\top \tilde{\mathbf{x}}_i)^2$, so Pythagoras gives

$$\|\tilde{\mathbf{x}}_i\|^2 = \sum_{j=1}^{k} (\mathbf{u}_j^\top \tilde{\mathbf{x}}_i)^2 + \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2.$$

Summing over i, with $\sum_i (\mathbf{u}_j^\top \tilde{\mathbf{x}}_i)^2 = n\, \mathbf{u}_j^\top S \mathbf{u}_j$ (n times the variance of the scores) and $\sum_i \|\tilde{\mathbf{x}}_i\|^2 = \sum_m \sum_i \tilde{x}_{im}^2 = n \sum_m S_{mm} = n \operatorname{tr} S = n(\lambda_1 + \cdots + \lambda_d)$ (spectral theorem),

$$\sum_{i=1}^{n} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = n(\lambda_1 + \cdots + \lambda_d) - n \sum_{j=1}^{k} \mathbf{u}_j^\top S \mathbf{u}_j \;\ge\; n(\lambda_1 + \cdots + \lambda_d) - n(\lambda_1 + \cdots + \lambda_k) = n(\lambda_{k+1} + \cdots + \lambda_d),$$

the inequality by the theorem. Equality holds for $\mathbf{u}_j = \mathbf{v}_j$, so the smallest reconstruction error is $n(\lambda_{k+1} + \cdots + \lambda_d)$. The fraction $(\lambda_1 + \cdots + \lambda_k)/(\lambda_1 + \cdots + \lambda_d)$ is the **proportion of variance explained** by k components. The theorem is the justification for keeping the first k principal components when the features are to be compressed: no other choice of k directions keeps more of the variance.

The scores are uncorrelated. Collect the first k directions as the columns of $V_k = [\mathbf{v}_1 \;\cdots\; \mathbf{v}_k]$, so that the score vector of point i is $\mathbf{z}_i = V_k^\top (\mathbf{x}_i - \bar{\mathbf{x}})$. Since $\sum_i (\mathbf{x}_i - \bar{\mathbf{x}}) = \mathbf{0}$, the scores have mean zero: $\frac{1}{n} \sum_i \mathbf{z}_i = V_k^\top \big(\frac{1}{n} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}}) \big) = \mathbf{0}$. Their covariance matrix is therefore the average of $\mathbf{z}_i \mathbf{z}_i^\top$. By the rule $(A\mathbf{b})^\top = \mathbf{b}^\top A^\top$, $\mathbf{z}_i^\top = (\mathbf{x}_i - \bar{\mathbf{x}})^\top V_k$, so $\mathbf{z}_i \mathbf{z}_i^\top = V_k^\top (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top V_k$; the fixed matrices $V_k^\top$ and V_k can be taken outside the sum, and, since row j of $V_k^\top$ is $\mathbf{v}_j^\top$ and column l of V_k is $\mathbf{v}_l$, the (j,l) entry of the result is $\mathbf{v}_j^\top S \mathbf{v}_l$. In matrix form the covariance matrix is

$$\frac{1}{n} \sum_{i=1}^{n} \mathbf{z}_i \mathbf{z}_i^\top = V_k^\top \Big(\frac{1}{n} \sum_{i=1}^{n} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \Big) V_k = V_k^\top S V_k.$$

Its (j,l) entry is $\mathbf{v}_j^\top S \mathbf{v}_l = \lambda_l\, \mathbf{v}_j^\top \mathbf{v}_l$, which is λ_j when $j = l$ and 0 otherwise. Hence $V_k^\top S V_k = \operatorname{diag}(\lambda_1, \ldots, \lambda_k)$: *the principal component scores are uncorrelated, and the variance of the j-th score is λ_j*. PCA thus replaces correlated features by uncorrelated ones, ordered by variance.

Algorithm 7: principal component analysis

Input: data matrix X ($n \times d$, one row per point); number of components k
Output: mean $\bar{\mathbf{x}}$, directions V_k, scores Z, reconstructions $\hat{X}$, proportion of variance explained

1: $\bar{\mathbf{x}} \leftarrow \frac{1}{n}\sum_i \mathbf{x}_i$; $X_c \leftarrow X$ with $\bar{\mathbf{x}}$ subtracted from every row
2: if the features are in different units, let sd_j be the standard deviation of column j, otherwise $\mathrm{sd}_j = 1$; $D_s \leftarrow \mathrm{diag}(\mathrm{sd}_1, \ldots, \mathrm{sd}_d)$; $X_s \leftarrow X_c D_s^{-1}$
3: $S \leftarrow \frac{1}{n} X_s^\top X_s$
4: find the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d$ and orthonormal eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_d$ of S or the SVD of X_s
5: $V_k \leftarrow [\mathbf{v}_1 \ \cdots \ \mathbf{v}_k]$; $Z \leftarrow X_s V_k$ scores, $n \times k$
6: $\hat{X} \leftarrow \mathbf{1}\bar{\mathbf{x}}^\top + Z V_k^\top D_s$ reconstructions, in the original units
7: proportion explained $\leftarrow (\lambda_1 + \cdots + \lambda_k)/(\lambda_1 + \cdots + \lambda_d)$

In Algorithm 7, row i of $Z = X_s V_k$ is $\mathbf{x}_{s,i}^\top V_k = (V_k^\top \mathbf{x}_{s,i})^\top = \mathbf{z}_i^\top$, where $\mathbf{x}_{s,i}^\top$ is row i of X_s. In line 6, $\mathbf{1}$ is the column of n ones, so $\mathbf{1}\bar{\mathbf{x}}^\top$ is the $n \times d$ matrix with $\bar{\mathbf{x}}^\top$ in every row. Row i of $Z V_k^\top$ is $\mathbf{z}_i^\top V_k^\top = (V_k \mathbf{z}_i)^\top = \left(\sum_{j=1}^k z_{ij} \mathbf{v}_j\right)^\top$, the reconstruction of the scaled deviation; multiplying on the right by D_s multiplies column m by sd_m, which undoes the division in line 2, and adding $\mathbf{1}\bar{\mathbf{x}}^\top$ undoes the centring in line 1. Forming S costs about nd^2 operations and its eigendecomposition about d^3. When only the first few components of a very large matrix are needed, iterative and randomised methods compute them without the full decomposition.

4.2.5 PCA by the singular value decomposition (optional)

Intuition: the same answer by a safer route

Two routes lead to the same principal components: through the covariance matrix, or directly from the data table by the singular value decomposition. The second avoids multiplying the table by itself, which squares its condition number and so magnifies small rounding errors.

Every $n \times d$ matrix can be written as $X_c = U \Sigma V^\top$, where (in the thin form, for $n \geq d$) U has orthonormal columns, V is orthogonal and Σ is diagonal with nonnegative entries $s_1 \geq s_2 \geq \ldots$, the **singular values**. Applied to the centred data matrix,

$$S = \frac{1}{n} X_c^\top X_c = \frac{1}{n} V \Sigma U^\top U \Sigma V^\top = V \frac{\Sigma^2}{n} V^\top.$$

In the middle step, the transpose of a product reverses the order of its factors, so $X_c^\top = (U \Sigma V^\top)^\top = (V^\top)^\top \Sigma^\top U^\top = V \Sigma U^\top$, using $\Sigma^\top = \Sigma$ for a diagonal matrix; and $U^\top U = I$, because the (j, l) entry of $U^\top U$ is the dot product of columns j and l of U, which is 1 when $j = l$ and 0 otherwise. Hence $X_c^\top X_c = V \Sigma I \Sigma V^\top = V \Sigma^2 V^\top$ with $\Sigma^2 = \mathrm{diag}(s_1^2, \ldots, s_d^2)$. Writing $\mathbf{v}_j$ for column j of V, orthonormality of the columns gives $V^\top \mathbf{v}_j = \mathbf{e}_j$, the j-th column of the identity, so

$$S \mathbf{v}_j = V \frac{\Sigma^2}{n} V^\top \mathbf{v}_j = V \frac{\Sigma^2}{n} \mathbf{e}_j = V \frac{s_j^2}{n} \mathbf{e}_j = \frac{s_j^2}{n} \mathbf{v}_j.$$

This is the spectral decomposition of S: the columns of V are the principal directions, and the eigenvalues are $\lambda_j = s_j^2/n$, in decreasing order because $s_1 \geq s_2 \geq \ldots$. The score matrix is $X_c V = U \Sigma V^\top V = U \Sigma$, since $V^\top V = I$. For the eight runs the singular values are 9.7038 and 2.5907, and $9.7038^2/8 = 11.770$, $2.5907^2/8 = 0.839$, the eigenvalues found above. Software computes PCA this way: forming the product $X_c^\top X_c$ squares the ratio of the largest to the smallest singular value, which magnifies rounding errors.

4.3 Using principal component analysis

4.3.1 How many components

Intuition: compressing a photograph

A photograph saved with a few components is recognisable, and adding more makes it sharper with diminishing visible gain. How many to keep depends on use: a thumbnail needs few, a print needs many. The scree plot shows where additional components stop adding much.

Three hundred training runs each record ten standardised metrics. The metrics are driven by two underlying factors (roughly, the size of the model and the aggressiveness of the learning rate) plus independent noise. The eigenvalues of the covariance matrix are

$$5.452, 3.341, 0.202, 0.180, 0.167, 0.153, 0.151, 0.128, 0.119, 0.107.$$

Figure 4.3 plots them in decreasing order, a *scree plot*, together with the cumulative proportion of variance explained. Two eigenvalues stand far above the rest. Because each standardised metric has variance 1, the total variance is $\operatorname{tr} S = 10 \times 1 = 10$, and by the spectral theorem the ten eigenvalues add to this total. To four decimals the first four eigenvalues are 5.4516, 3.3414, 0.2017 and 0.1797, so two components explain $(5.4516 + 3.3414)/10 = 8.7930/10$, that is 87.93% of the variance; three explain $(8.7930 + 0.2017)/10 = 8.9947/10$, that is 89.95%; and four explain $(8.9947 + 0.1797)/10 = 9.1744/10$, that is 91.74%. The three common rules do not all agree: the elbow of the scree plot suggests $k = 2$; the rule "keep components with eigenvalue above 1" (for standardised data, the variance of a single original feature) also gives 2; a rule "explain at least 90%" requires $k = 4$, because three components explain 89.95%. Thresholds of this kind are conventions. As with the number of clusters, the choice depends on the purpose: for a plot $k \leq 3$; for compression, the reconstruction error that can be tolerated.

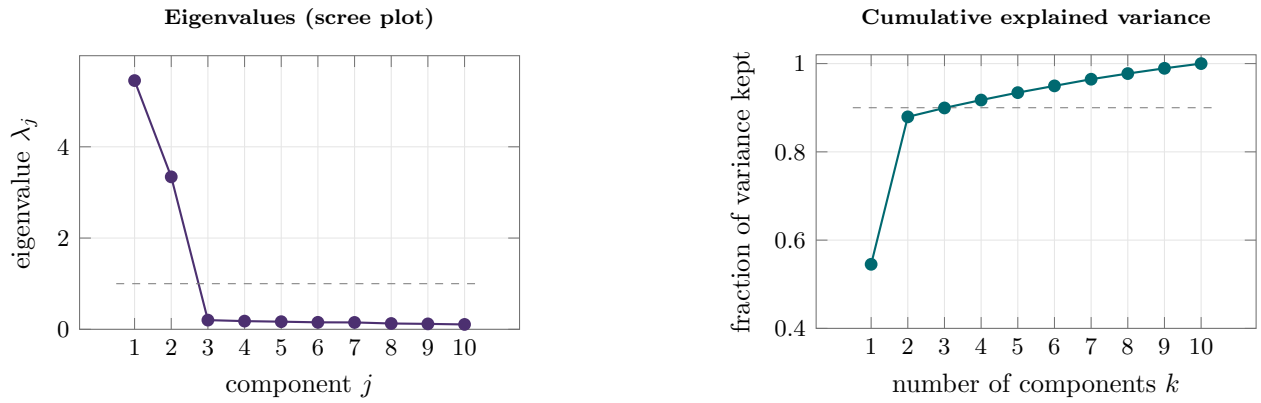


Figure 4.3: Left: eigenvalues in decreasing order (a *scree plot*); the dashed line marks 1. Right: cumulative proportion of variance explained; the dashed line marks 90%.

4.3.2 The effect of scaling on PCA

Intuition: population and area

Describe cities by population and by area. If area is entered in square metres, its numbers dwarf the population and PCA reports area as the main direction of variation; enter it in square kilometres and population dominates instead. The data are the same; only the units changed. PCA follows the largest numbers, so features must be put on comparable scales first.

PCA maximises variance, and variance depends on units. For the eighty runs of Section 2.5.3, *Dependence on the scale of the features*, recorded in hours and megabytes, the first principal direction on the raw features is $(0,\ 1.0000)$, the memory axis, and it explains 100.00% of the variance: the numbers in megabytes are simply larger. In numbers, the raw variances are $1.642^2 \approx 2.696$ for training time (hours^2) and $3634^2 \approx 1.3206 \times 10^7$ for memory (MB^2), and the covariance is only 8.09. For a 2×2 matrix the eigenvalues are $\frac{1}{2}\big(S_{11} + S_{22} \pm \sqrt{(S_{11} - S_{22})^2 + 4S_{12}^2}\big)$; here $4S_{12}^2 \approx 262$ is negligible beside $(S_{22} - S_{11})^2 \approx 1.744 \times 10^{14}$, so the eigenvalues are, to the precision shown, the two variances themselves, and the first explains $1.3206 \times 10^7/(1.3206 \times 10^7 + 2.696) = 1 - 2.04 \times 10^{-7}$ of the total. After standardisation both variances are 1 and the covariance becomes the correlation $r = 8.09/(1.642 \times 3634) \approx 0.0014$, so $S = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$, with eigenvalues $1 + r$ and $1 - r$ and directions $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$ (Problem 4.13). The two directions explain $(1 + r)/2$ and $(1 - r)/2$ of the total 2, that is 50.07% and 49.93%: the features are nearly uncorrelated, so no reduction is possible, and PCA gives the diagonal $(0.707,\ 0.707)$ only because the two variances are now equal.

Features in different units should normally be standardised before PCA. And a direction of large variance need not be a direction of interest: in these data the feature that separates the two kinds of run is training time, which has the *smaller* raw variance. PCA does not use any outcome, so it cannot know which variation matters.

4.3.3 Example: images of handwritten digits

Intuition: a sketch with a few strokes

A good sketch artist captures a face with a few strokes and adds detail stroke by stroke. Principal components play the role of strokes: each is a pattern of pixels, and an image is rebuilt by adding the patterns in the right amounts, the most important first.

The handwritten-digits data set of Alpaydin and Kaynak (1998) contains 1797 images of the digits 0 to 9, each 8×8 pixels with intensities from 0 to 16, so every image is a point with 64 features. PCA of these data (Figure 4.4) gives:

- two components explain 28.5% of the variance, ten explain 73.8%, and twenty explain 89.4%; 21 components are needed for 90%;
- the first principal directions, shown as images, are broad patterns of light and dark that distinguish families of digits (for example a loop at the top against a stroke at the bottom);
- an image rebuilt from ten or twenty components is already easily recognisable.

The two-dimensional view separates some digits (0, 4 and 6 sit apart) and mixes others (1 overlaps 7; 2, 5 and 8 overlap; 9 lies close to 3), which shows both the use and the limit of keeping only two components.

By the reconstruction-error result of Section 4.2.4, *Several principal components*, divided by n, the mean squared reconstruction error per image, $\frac{1}{n}\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$, equals the sum of the discarded eigenvalues, $\lambda_{k+1} + \cdots + \lambda_{64}$. For the digits the total variance is 1201.5; the error is 858.9 with two components (Figure 4.5), 314.5 with ten and 116.3 with 21. Since the error is the total minus the retained eigenvalues, $\lambda_{k+1} + \cdots + \lambda_{64} = (\lambda_1 + \cdots + \lambda_{64}) - (\lambda_1 + \cdots + \lambda_k)$, each error divided by the total is one minus the proportion explained: $858.9/1201.5 = 0.715 = 1 - 0.285$, $314.5/1201.5 = 0.262 = 1 - 0.738$ and $116.3/1201.5 = 0.097 = 1 - 0.903$, in agreement with the proportions listed above.

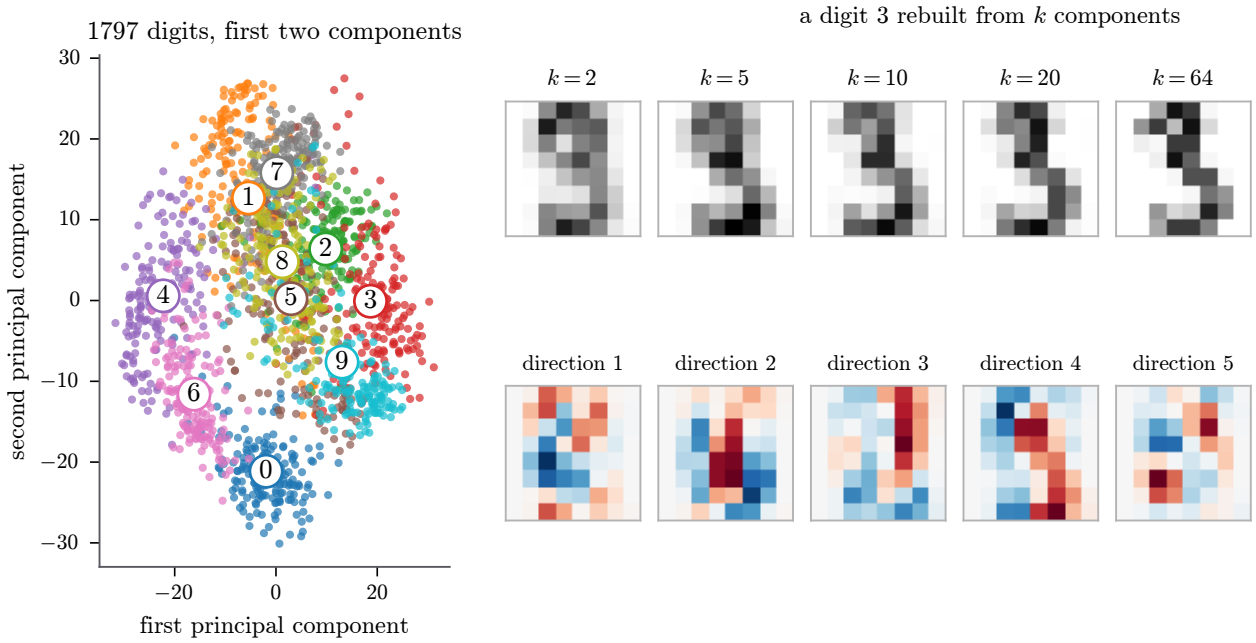


Figure 4.4: PCA of 1797 handwritten digits. Left: every image placed by its first two principal component scores, coloured by the digit it shows (the labels are used only for colouring); circled numbers mark each digit's average position. Top right: one image of a 3 rebuilt from k components. Bottom right: the first five principal directions shown as images (red positive, blue negative).

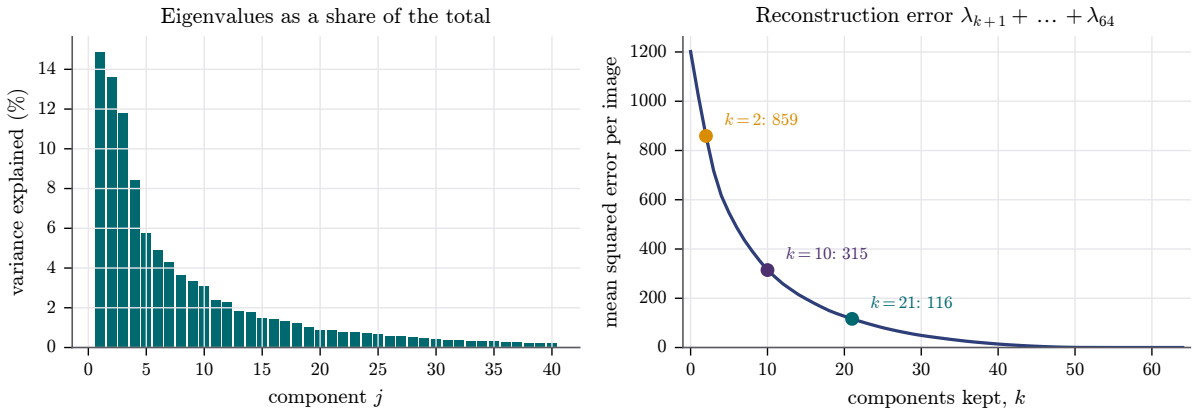


Figure 4.5: Principal components of the digits. Left: the share of the total variance carried by each of the first 40 components; the first few carry most of it and the rest decline slowly. Right: the mean squared reconstruction error per image when k components are kept, which equals the sum of the discarded eigenvalues; at $k = 21$, 90% of the variance is kept.

4.3.4 From PCA to clustering

PCA and K-means are often used in sequence: reduce the features first, then cluster the scores. On the digits, K-means with $K = 10$ was run on all 64 pixels, on the first ten principal component scores, and on the first two. The **purity** of a clustering is the fraction of images whose cluster's most common digit is their own digit. It is 79.1% on all 64 pixels, 78.7% on ten components and 59.3% on two. Ten components, a sixth of the original features, keep almost all the structure that K-means finds, at a fraction of the cost; two components lose much of it. For clustering, the number of components should be judged by the clustering it produces.

4.4 Nonlinear maps for visualisation

4.4.1 t-SNE

Intuition: a seating plan from friendships

To seat a class in a hall so that friends sit together, it matters that each student's friends are nearby; how far apart two students who are strangers end up matters little. t-SNE draws a map with this priority. A rolled carpet shows why a straight shadow is not enough: its shadow piles the layers on top of each other, while unrolling it keeps neighbouring threads together.

PCA is linear: the map from data to scores is a matrix. When the structure of interest is curved, or consists of many small groups, a two-dimensional linear projection can overlap them (Figure 4.6). **t-distributed stochastic neighbour embedding** (t-SNE; van der Maaten and Hinton, 2008) instead places points in the plane so that *neighbours remain neighbours*.

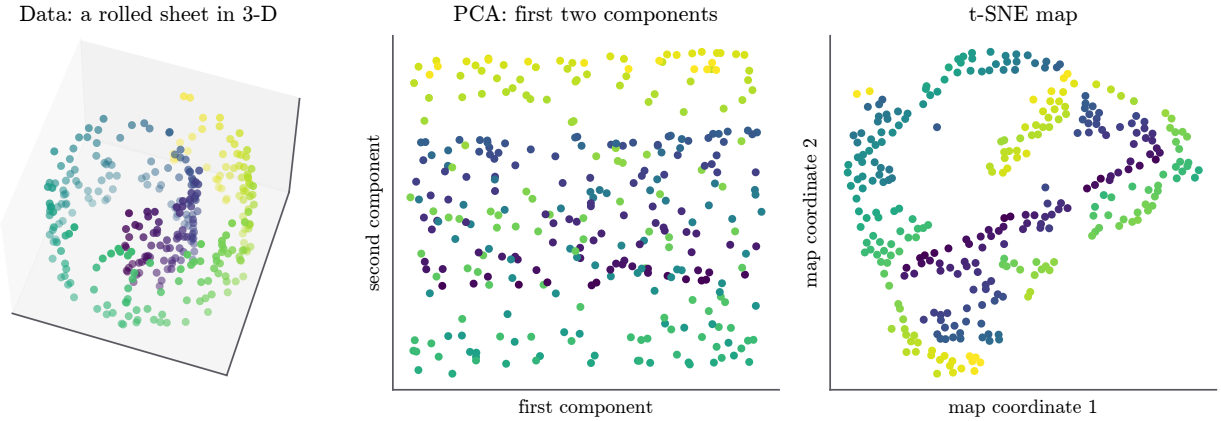


Figure 4.6: A sheet rolled up in three dimensions, coloured by position along the roll (320 points). The best linear projection (centre) superimposes distant layers of the roll, so the colours mix. The t-SNE map (right) keeps neighbouring points together, so the colours change gradually, although it tears the sheet into pieces and bends it.

Similarities in the data. For each point i, define the conditional probability that i would pick j as its neighbour,

$$p_{j|i} = \frac{\exp\bigl(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2\bigr)}{\sum_{k\neq i}\exp\bigl(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma_i^2\bigr)}.$$

The width σ_i is set separately for each point, by a one-dimensional numerical search, so that the *perplexity* 2^{H_i} of the distribution $p_{\cdot|i}$, where $H_i = -\sum_j p_{j|i}\log_2 p_{j|i}$ is its entropy, equals a value chosen by the user, typically between 5 and 50; it acts as an effective number of neighbours. If $p_{\cdot|i}$ put probability $1/m$ on each of m neighbours and none elsewhere, then $H_i = -\sum_{j=1}^{m}\frac{1}{m}\log_2\frac{1}{m} = \log_2 m$ and the perplexity would be $2^{\log_2 m} = m$; for any other distribution, 2^{H_i} is the number of equally likely neighbours that would give the same entropy. The search for σ_i works because the entropy increases steadily with σ_i. Write $\beta = 1/2\sigma_i^2$, $d_j = \|\mathbf{x}_i - \mathbf{x}_j\|^2$ and $Z = \sum_{k\neq i} e^{-\beta d_k}$, so that $p_{j|i} = e^{-\beta d_j}/Z$ and $\ln p_{j|i} = -\beta d_j - \ln Z$. Measured with natural logarithms (which changes H_i only by the constant factor $\ln 2$), the entropy is

$$H = -\sum_j p_{j|i}\ln p_{j|i} = \beta\sum_j p_{j|i}d_j + \ln Z\sum_j p_{j|i} = \beta D + \ln Z, \qquad D = \sum_j p_{j|i}d_j.$$

Differentiating with respect to β, $\frac{d\ln Z}{d\beta} = \frac{1}{Z}\sum_j(-d_j)e^{-\beta d_j} = -D$, so $\frac{d\ln p_{j|i}}{d\beta} = -d_j + D$ and

$\frac{dp_{j|i}}{d\beta} = p_{j|i}(D - d_j)$. Hence, expanding the square in the last step,

$$\frac{dD}{d\beta} = \sum_j d_j\, p_{j|i}(D - d_j) = D^2 - \sum_j p_{j|i} d_j^2 = -\sum_j p_{j|i}(d_j - D)^2,$$

$$\frac{dH}{d\beta} = D + \beta \frac{dD}{d\beta} - D = -\beta \sum_j p_{j|i}(d_j - D)^2 \le 0.$$

As σ_i grows, β falls and the entropy rises: from 0 (perplexity 1, all probability on the nearest point, if it is unique) when σ_i is very small, to $\log_2(n-1)$ (perplexity $n-1$, equal probability on every other point) when σ_i is very large. Each target perplexity between these limits is therefore reached at a single width, which bisection finds. With $p_{i|i} = 0$, the similarities are symmetrised, $p_{ij} = (p_{j|i} + p_{i|j})/2n$. The divisor $2n$ makes the p_{ij} a single probability distribution over pairs: for each i, $\sum_j p_{j|i} = 1$, so $\sum_i \sum_j p_{j|i} = n$, and in the same way $\sum_i \sum_j p_{i|j} = \sum_j \sum_i p_{i|j} = n$; hence $\sum_{i \ne j} p_{ij} = (n+n)/2n = 1$. The symmetrised values also satisfy $p_{ij} = p_{ji}$, a property the gradient of the objective relies on.

Similarities in the map. For map points $\mathbf{y}_i$ in the plane, with a heavy-tailed Student-t kernel,

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \ne l}(1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}.$$

The objective. The map minimises $C = \sum_{i \ne j} p_{ij} \log(p_{ij}/q_{ij})$, the *Kullback–Leibler divergence* $\mathrm{KL}(P \,\|\, Q)$ between the two distributions, which is nonnegative and is zero only when the two distributions agree. Both facts follow from Jensen's inequality for the logarithm (Chapter 1, *Mathematical background*). Terms with $p_{ij} = 0$ contribute nothing, so let the sums run over the pairs with $p_{ij} > 0$; their weights p_{ij} sum to 1, and every q_{ij} is positive. Then

$$-C = \sum_{p_{ij}>0} p_{ij} \log \frac{q_{ij}}{p_{ij}} \;\le\; \log\Big(\sum_{p_{ij}>0} p_{ij} \frac{q_{ij}}{p_{ij}} \Big) = \log\Big(\sum_{p_{ij}>0} q_{ij} \Big) \;\le\; \log 1 = 0,$$

the last step because the q_{ij} sum to 1 over all pairs. Hence $C \ge 0$. Equality in the first step requires all the ratios q_{ij}/p_{ij} to equal a common value c, and in the second requires $\sum_{p_{ij}>0} q_{ij} = 1$; then $1 = \sum_{p_{ij}>0} q_{ij} = c \sum_{p_{ij}>0} p_{ij} = c$, so $q_{ij} = p_{ij}$ for every such pair, and since every q_{ij} is positive and these already sum to 1, no pair with $p_{ij} = 0$ remains. It is minimised by gradient descent, and the gradient follows from the chain rule. Write $d_{kl} = \|\mathbf{y}_k - \mathbf{y}_l\|^2$, $w_{kl} = (1 + d_{kl})^{-1}$ and $Z = \sum_{k \ne l} w_{kl}$, so that $q_{kl} = w_{kl}/Z$; all three are symmetric in k and l, and so is p_{kl}. Since $\log(p_{kl}/q_{kl}) = \log p_{kl} - \log w_{kl} + \log Z = \log p_{kl} + \log(1 + d_{kl}) + \log Z$ and $\sum_{k \ne l} p_{kl} = 1$,

$$C = \sum_{k \ne l} p_{kl} \log p_{kl} + \sum_{k \ne l} p_{kl} \log(1 + d_{kl}) + \log Z.$$

The first sum does not involve the map. In the plane, $d_{ij} = (y_{i1} - y_{j1})^2 + (y_{i2} - y_{j2})^2$, whose partial derivatives with respect to y_{i1} and y_{i2} are $2(y_{i1} - y_{j1})$ and $2(y_{i2} - y_{j2})$; together, $\partial d_{ij}/\partial \mathbf{y}_i = 2(\mathbf{y}_i - \mathbf{y}_j)$. Only the pairs (i, j) and (j, i) involve $\mathbf{y}_i$, and they contribute equally because $p_{ij} = p_{ji}$ and $d_{ij} = d_{ji}$. With $\frac{d}{dt} \log(1 + t) = 1/(1 + t)$, the second sum contributes

$$\frac{\partial}{\partial \mathbf{y}_i} \sum_{k \ne l} p_{kl} \log(1 + d_{kl}) = 2 \sum_{j \ne i} p_{ij} \frac{2(\mathbf{y}_i - \mathbf{y}_j)}{1 + d_{ij}} = 4 \sum_{j \ne i} p_{ij}\, w_{ij} (\mathbf{y}_i - \mathbf{y}_j).$$

For the last term, $\partial w_{ij}/\partial \mathbf{y}_i = -(1 + d_{ij})^{-2}\, 2(\mathbf{y}_i - \mathbf{y}_j) = -2 w_{ij}^2 (\mathbf{y}_i - \mathbf{y}_j)$, and again each pair appears twice in Z, so

$$\frac{\partial \log Z}{\partial \mathbf{y}_i} = \frac{1}{Z} \frac{\partial Z}{\partial \mathbf{y}_i} = \frac{2}{Z} \sum_{j \ne i} \big(-2 w_{ij}^2 (\mathbf{y}_i - \mathbf{y}_j) \big) = -4 \sum_{j \ne i} \frac{w_{ij}}{Z}\, w_{ij} (\mathbf{y}_i - \mathbf{y}_j) = -4 \sum_{j \ne i} q_{ij}\, w_{ij} (\mathbf{y}_i - \mathbf{y}_j).$$

Adding the two contributions, and recalling $w_{ij} = (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$, gives the gradient

$$\frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij}) \, \frac{\mathbf{y}_i - \mathbf{y}_j}{1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2}.$$

Each term pulls $\mathbf{y}_i$ towards $\mathbf{y}_j$ when the pair is more similar in the data than in the map ($p_{ij} > q_{ij}$) and pushes it away otherwise. The divergence penalises heavily a pair that is close in the data but far apart in the map, and only lightly the reverse, so the map preserves neighbourhoods rather than large distances. The heavy tail of the t kernel (Figure 4.7) lets moderately distant points be placed far apart in the plane, which relieves the *crowding* that occurs when many points that are mutually distant in high dimension must share a small area.

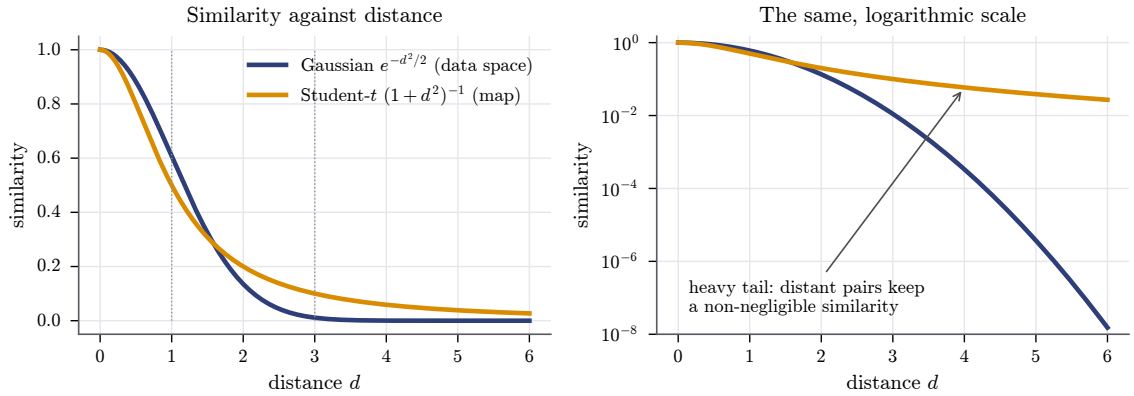


Figure 4.7: The Gaussian similarity used in the data space and the Student-t similarity used in the map; dotted lines in the left panel mark distances 1 and 3. On a logarithmic scale (right) the difference in the tails is plain: at distance 4 the Gaussian similarity is about 3×10^{-4} and the t similarity about 0.06.

Algorithm 8: t-SNE

Input: points $\mathbf{x}_1, \ldots, \mathbf{x}_n$; perplexity; number of iterations T; step size η; momentum α
Output: map points $\mathbf{y}_1, \ldots, \mathbf{y}_n$ in the plane

for $i = 1, \ldots, n$ **do**
    find σ_i by bisection so that the perplexity of $p_{\cdot|i}$ equals the chosen value
end for
$p_{ij} \leftarrow (p_{j|i} + p_{i|j})/2n$ for all pairs
draw each $\mathbf{y}_i^{(1)}$ at random close to the origin, and set $\mathbf{y}_i^{(0)} = \mathbf{y}_i^{(1)}$
for $t = 1, \ldots, T$ **do**
    compute all q_{ij} and the gradients $\partial C / \partial \mathbf{y}_i$
    $\mathbf{y}_i^{(t+1)} \leftarrow \mathbf{y}_i^{(t)} - \eta \, \partial C / \partial \mathbf{y}_i + \alpha\big(\mathbf{y}_i^{(t)} - \mathbf{y}_i^{(t-1)}\big)$ for all i
end for

During the first iterations the p_{ij} are usually multiplied by a factor of about 12 (*early exaggeration*), which makes the attractions dominate so that groups form and separate early. Each iteration as written costs about n^2 operations; the Barnes–Hut approximation, which treats distant groups of points as single points, reduces it to about $n \log n$.

The map depends on the perplexity (Figure 4.8): a small value makes each point attend only to its closest few neighbours, and a large one to many.

One hundred and twenty points in ten dimensions form three clusters of forty: A, tight (standard deviation 0.5 per coordinate); B, broad (3.0); and C, far away (1.0; its centre is 60 units from A along one axis, while B's is 12 units from A along the same axis). The *spread* of a cluster below is the root-mean-square distance of its points from their centre.

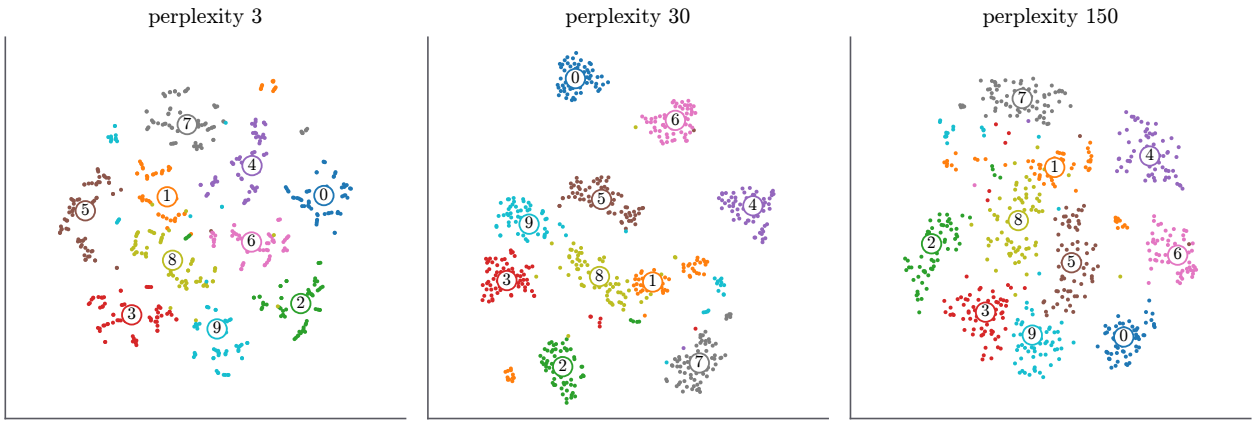


Figure 4.8: t-SNE maps of 700 of the handwritten digits (first reduced to 30 principal components) at three perplexities, from the same random start. At perplexity 3 the digits break into many small fragments. At 30 they form roughly ten compact groups. At 150 the groups are broader and closer together. Maps at several perplexities should be compared before any structure is interpreted.

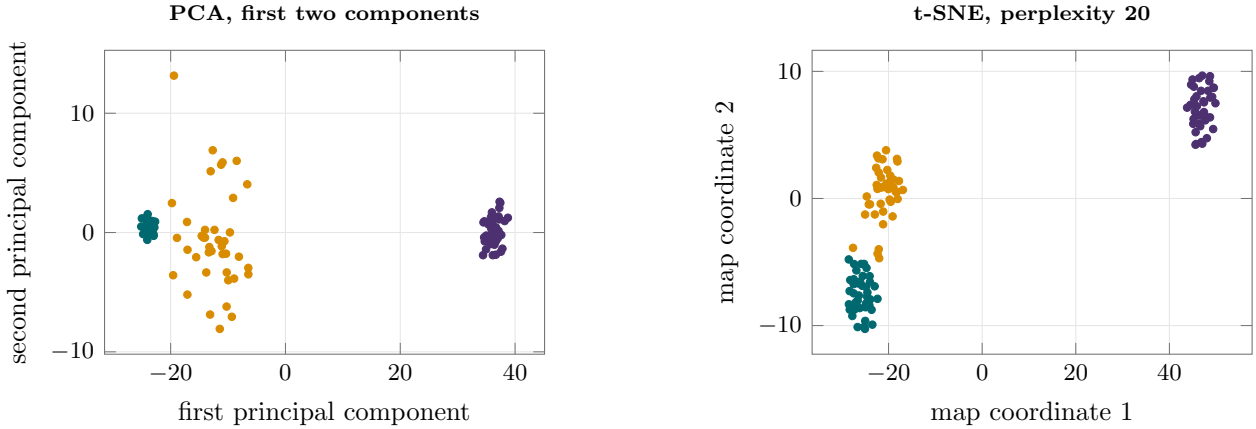


Figure 4.9: The same three clusters (teal A, orange B, purple C) shown by the first two principal components (left) and by t-SNE (right). The two axes of each panel are drawn to different scales.

	cluster A	cluster B	cluster C
spread in the original 10 dimensions	1.47	10.42	3.17
spread on the t-SNE map	2.20	3.06	2.22
	A to B	B to C	ratio
distance between centres, original	11.48	48.73	4.25
distance between centres, t-SNE map	9.27	68.21	7.36

The map (Figure 4.9) separates the clusters cleanly, but two features of the data are not reproduced. In ten dimensions cluster B is about 7.1 times as spread as cluster A; on the map, about 1.4 times. And the ratio of the distance B–C to the distance A–B changes from 4.25 to 7.36. In general, *on a t-SNE map, neighbourhoods are meaningful, but the sizes of clusters and the distances between clusters are not.*

4.4.2 UMAP, PaCMAP, and how to read such maps

Intuition: a metro map

A metro map shows stations in the right order and lines in the right connections, but the distances on it are not the distances on the ground and the empty areas between lines mean nothing. Maps from t-SNE, UMAP and PaCMAP should be read the same way: who is next to whom is meaningful, sizes and gaps are not.

UMAP (uniform manifold approximation and projection; McInnes, Healy and Melville, 2018) links each point to its nearest neighbours, with strengths that fall with distance, and arranges the map so that linked points are close and unlinked points are pushed apart. It is substantially faster than the original t-SNE on large data, can place new points into an existing map, and is widely used. **PaCMAP** (Wang, Huang, Rudin and Shaposhnik, 2021) attracts near neighbours and some moderately distant pairs and repels distant pairs, with weights that change during the optimisation; it is designed to keep both local neighbourhoods and the global arrangement.

The methods differ less than their derivations suggest: mainly in the balance between attraction and repulsion, and in how the map is started (Böhm, Berens and Kobak, 2022; Kobak and Linderman, 2021). Faster implementations of t-SNE have removed most of the difference in speed.

Remark: reading a neighbour-embedding map

- Neighbours in the data are usually close on the map; points close on the map need not be neighbours in the data.
- The size of a cluster on the map does not measure its spread in the data, and gaps between clusters do not measure their separation.
- A cluster on the map is not evidence of a cluster in the data; compare with the same method applied to data that have no clusters, as in Section 3.4, *Evaluating a clustering*, and with different random starting layouts and perplexities.
- Do not draw conclusions from distances, densities or clusters measured *on the map*; cluster in the original space or in a reduction to a moderate number of dimensions (by PCA, or by UMAP to a few tens of dimensions), and use the two-dimensional map only to look.

4.5 Unsupervised learning on learned representations

Intuition: a catalogue card for every item

A library describes each book on a catalogue card with a fixed set of fields. A trained model does the same for an image or a sentence, producing a fixed-length list of numbers, an embedding, in which similar items get similar numbers. Clustering and visualising those cards is then the same task as in the earlier chapters.

When the raw data are images, audio, text, molecules or long records, the features used for clustering and visualisation are now often **embeddings**: vectors produced by a model trained on large amounts of similar data. A typical sequence of steps for a large collection of documents, images or records is: compute embeddings; reduce them, by PCA or by UMAP to a few tens of dimensions; cluster with HDBSCAN (Section 3.3, *Density-based clustering: DBSCAN and HDBSCAN*); describe each cluster by its most typical members; and plot with a two-dimensional UMAP or PaCMAP map. As before, the result depends on the embedding model, on scaling, on the parameters of each step, and it must be checked for stability and meaning. When the features are meaningful measurements, as with the training runs and latencies, the classical sequence (standardise, reduce with PCA, cluster, check) remains the right starting point.

The representations themselves are now usually learned without labels by **self-supervised learning**: a model is trained on a task constructed from the data alone, such as predicting masked parts of an input or recognising two altered views of the same example. PCA can be regarded as the simplest such method: the best linear compression from which the data can be reconstructed.

Remark: sparse autoencoders

An autoencoder is a network that maps an input to a code and back, trained to reconstruct its input. With a linear encoder and decoder and squared error, the optimal code spans the same subspace as the first principal components. With k code units, an encoder E ($k \times d$) and decoder D ($d \times k$) reconstruct a centred point as $DE\tilde{\mathbf{x}}_i$, which lies in the column space W of D, of dimension at most k; enlarge W to dimension k if necessary and take an orthonormal basis $\mathbf{u}_1, \dots, \mathbf{u}_k$ of it. With $\mathbf{p}_i = \sum_j (\mathbf{u}_j^\top \tilde{\mathbf{x}}_i)\mathbf{u}_j$, any $\mathbf{w}$ in W gives $\tilde{\mathbf{x}}_i - \mathbf{w} = (\tilde{\mathbf{x}}_i - \mathbf{p}_i) + (\mathbf{p}_i - \mathbf{w})$, the first part perpendicular to W and the second in W, so $\|\tilde{\mathbf{x}}_i - \mathbf{w}\|^2 = \|\tilde{\mathbf{x}}_i - \mathbf{p}_i\|^2 + \|\mathbf{p}_i - \mathbf{w}\|^2 \geq \|\tilde{\mathbf{x}}_i - \mathbf{p}_i\|^2$. By the reconstruction-error result of Section 4.2.4, *Several principal components*,

$$\sum_{i=1}^{n} \|\tilde{\mathbf{x}}_i - DE\tilde{\mathbf{x}}_i\|^2 \;\geq\; \sum_{i=1}^{n} \|\tilde{\mathbf{x}}_i - \mathbf{p}_i\|^2 = n(\lambda_1 + \cdots + \lambda_d) - n\sum_{j=1}^{k} \mathbf{u}_j^\top S \mathbf{u}_j \;\geq\; n(\lambda_{k+1} + \cdots + \lambda_d),$$

and the bound is reached by $E = V_k^\top$ and $D = V_k$, whose code is the vector of the first k principal component scores. When $\lambda_k > \lambda_{k+1}$, equality in the theorem forces $w_i = 0$ for every $i > k$ (for such i, $\lambda_i w_i < \lambda_k w_i$ whenever $w_i > 0$), so every $\mathbf{u}_j$ is perpendicular to $\mathbf{v}_{k+1}, \dots, \mathbf{v}_d$ and W is the span of $\mathbf{v}_1, \dots, \mathbf{v}_k$. A *sparse* autoencoder instead uses many more code units than input dimensions and penalises codes with many nonzero entries, so that each input is reconstructed from a few active units. Applied to the *activations* of a large trained network (the vectors it computes at an intermediate layer), including language models, sparse autoencoders are used to study what such networks compute: individual code units often correspond to recognisable features of the input. Like PCA, the method rebuilds the data from directions learned without labels; unlike PCA, it uses many directions in total but only a few for any one input.

4.6 Practice problems for Chapter 4

4.1 [Standard] Centred data have covariance matrix $S = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$. (a) Find the eigenvalues and unit eigenvectors. (b) What proportion of the variance does the first component explain? (c) For the centred point $(1, 1)$, compute the first score and the squared reconstruction error when one component is kept.

4.2 [Standard] Prove that the variance of the scores $z_i = \mathbf{u}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ is $\mathbf{u}^\top S \mathbf{u}$, and that the scores on two different principal directions are uncorrelated.

4.3 [Basic] Prove that the total variance $\sum_j S_{jj}$ equals the sum of the eigenvalues, and interpret this for the proportion of variance explained.

4.4 [Basic] A centred data matrix with $n = 50$ rows has singular values 10, 5 and 1. Find the eigenvalues of S, the proportion of variance explained by each component, and the smallest k that explains at least 95%.

4.5 [Standard] In the proof of Section 4.2.4, *Several principal components*, justify the statement $0 \leq w_i \leq 1$ in detail.

4.6 [Standard] For the one-dimensional points $0, 1, 3$, with $\sigma_0 = 1$ for the point 0, compute $p_{1|0}$ and $p_{3|0}$. Then compare the ratio of a Gaussian kernel $e^{-d^2/2}$ at distances 1 and 3 with the ratio of the Student-t kernel $(1 + d^2)^{-1}$ at the same distances, and explain how the difference relieves crowding.

4.7 [Basic] Two features are recorded in hours and in megabytes. A colleague reports that "the first principal component explains 99.99% of the variance, so the data are essentially one-dimensional". Explain what is likely to be happening and what you would check.

4.8 [Standard] Show that PCA with all d components retained is a rotation of the centred data, possibly combined with a reflection (an orthogonal change of coordinates),, and conclude that it changes no Euclidean distance between points.

4.9 [Standard] A t-SNE map of customer records shows one cluster three times as wide as the others and far from them. A manager concludes that these customers are more varied and very different from the rest. What can and cannot be concluded from the map, and how would you check the claim?

4.10 [Basic] For the points $(1, 2)$, $(3, 4)$, $(5, 3)$ and $(3, 3)$, compute the mean, the covariance matrix (with $1/n$), its eigenvalues, and the proportion of variance explained by the first component.

4.11 [Basic] A covariance matrix has eigenvalues $4, 2, 1, 0.5, 0.5$. What proportion of the variance do two components explain, and how many are needed to explain at least 90%?

4.12 [Standard] Prove that when the first k principal components are kept, the mean squared reconstruction error $\frac{1}{n}\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$ equals $\lambda_{k+1} + \cdots + \lambda_d$. For the eigenvalues of Problem 4.11, compute it for $k = 2$ and express it as a fraction of the total variance.

4.13 [Standard] Show that for $S = \begin{pmatrix} a & b \\ b & a \end{pmatrix}$ with $b > 0$ the principal directions are $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$, with eigenvalues $a + b$ and $a - b$, whatever the value of a.

4.14 [Challenge] Two standardised features have covariance $\begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$. The first is then multiplied by 10. Compute the first principal direction before and after, and explain why PCA is not invariant to the scaling of individual features, while the Mahalanobis distance (Section 5.3, *Two features: distance, shape and neighbours*) is.

4.15 [Challenge] Show that if the conditional distribution $p_{\cdot|i}$ of t-SNE puts equal probability on k neighbours and none elsewhere, its perplexity is exactly k. Explain why perplexity is described as an effective number of neighbours.

Chapter 5

Anomaly detection

5.1 Anomalies and anomaly scores

Intuition: a card used in two cities

A bank notices that a card was used in Ahmedabad and, an hour later, in Delhi. Nothing labelled the second payment as fraud; it is suspicious because it does not fit the card's usual pattern. Anomaly detection formalises this: build a description of what usual looks like, and score each new example by how badly it fits.

An **anomaly** (or outlier) is an example that differs markedly from the bulk of the data. In machine-learning practice the bulk is "normal operation" and the anomalies are the cases worth a person's attention: a training run that diverges, a request whose latency spikes, a corrupted input, a sensor that fails, a fraudulent transaction. Anomalies are commonly classed as *point* anomalies (a single unusual example), *contextual* anomalies (unusual given their context, such as a latency normal at peak load but not at night) and *collective* anomalies (a group that is unusual only together, such as a sequence of small failures).

Outside machine learning, the same methods are used to detect fraudulent card transactions, intrusions in computer networks, incipient faults in machines from vibration and temperature sensors (predictive maintenance), unusual results in medical tests, and errors in recorded data.

The task is hard for three reasons. Anomalies are rare, so few or no labelled examples exist and the problem is usually unsupervised. "Unusual" must be defined by a model of what is usual, and different models disagree. And anomalies distort the very statistics used to find them. Two such effects are **masking**, when an anomaly inflates the estimated spread and so hides itself or others; and **swamping**, when it shifts the estimated centre so that normal points appear unusual.

Every method of anomaly detection assigns each example an **anomaly score** and flags examples whose score exceeds a threshold. The methods differ in what the score measures: distance from a centre, low density under a model, distance from a fitted subspace, distance to neighbours, or ease of isolation.

5.2 Scores for one feature

5.2.1 Standard scores and robust scores

Intuition: a billionaire walks into a café

Nine people in a café have ordinary incomes. A billionaire walks in, and the average income in the room jumps to several crores while the spread grows enormously. Measured against that inflated average and spread, the billionaire no longer looks extreme. The median income, by contrast, hardly changes, and against it the billionaire stands out at once. This effect, masking, motivates robust scores.

The **standard score** of a value is $z = (x - \bar{x})/s$, the number of standard deviations from the mean; a common rule flags $|z| > 3$. The rule has a limitation that is easy to prove.

Theorem (Samuelson's inequality)

For n numbers with mean $\bar{x}$ and standard deviation s (computed with $1/n$), every value satisfies $|x_i - \bar{x}| \le s\sqrt{n-1}$.

Proof. The deviations sum to zero, because $\sum_{j=1}^{n}(x_j - \bar{x}) = \sum_{j=1}^{n} x_j - n\bar{x} = n\bar{x} - n\bar{x} = 0$. Moving the term with $j = i$ to the other side gives $x_i - \bar{x} = -\sum_{j \ne i}(x_j - \bar{x})$. The Cauchy–Schwarz inequality states that for real numbers $a_1, \dots, a_m$ and $b_1, \dots, b_m$,

$$\Big(\sum_{k=1}^{m} a_k b_k\Big)^2 \le \Big(\sum_{k=1}^{m} a_k^2\Big)\Big(\sum_{k=1}^{m} b_k^2\Big).$$

To prove it, write $A = \sum_k a_k^2$, $B = \sum_k b_k^2$ and $C = \sum_k a_k b_k$. If $A = 0$, every a_k is zero and both sides are zero. Otherwise, for every real t,

$$0 \le \sum_k (a_k t + b_k)^2 = At^2 + 2Ct + B.$$

If the discriminant $4C^2 - 4AB$ of this quadratic were positive, it would have two distinct roots and be negative between them. Hence $4C^2 - 4AB \le 0$, that is, $C^2 \le AB$. Apply the inequality to the $m = n-1$ indices $j \ne i$, with $a_j = 1$ and $b_j = x_j - \bar{x}$. The left-hand side is $\big(\sum_{j \ne i}(x_j - \bar{x})\big)^2 = (x_i - \bar{x})^2$, and $\sum_{j \ne i} a_j^2 = n - 1$. By the definition of s (with $1/n$), $ns^2 = \sum_{j=1}^{n}(x_j - \bar{x})^2 = (x_i - \bar{x})^2 + \sum_{j \ne i}(x_j - \bar{x})^2$, so $\sum_{j \ne i}(x_j - \bar{x})^2 = ns^2 - (x_i - \bar{x})^2$. Together these give

$$(x_i - \bar{x})^2 \le (n-1)\sum_{j \ne i}(x_j - \bar{x})^2 = (n-1)\big(ns^2 - (x_i - \bar{x})^2\big).$$

Expanding the right-hand side, $(x_i - \bar{x})^2 \le (n-1)ns^2 - (n-1)(x_i - \bar{x})^2$. Adding $(n-1)(x_i - \bar{x})^2$ to both sides gives $n(x_i - \bar{x})^2 \le (n-1)ns^2$. Dividing by $n > 0$ gives $(x_i - \bar{x})^2 \le (n-1)s^2$, and taking non-negative square roots gives $|x_i - \bar{x}| \le s\sqrt{n-1}$. □

Dividing by s (assumed positive) gives $|z| = |x_i - \bar{x}|/s \le \sqrt{n-1}$ for every value. With $n \le 10$ values, $\sqrt{n-1} \le \sqrt{9} = 3$, so no standard score can exceed 3, however extreme a value is. The reason is masking: the extreme value inflates s.

Example: one slow request

Add a request of 95 ms to the seven request latencies of Section 3.1.4, *The EM algorithm*. The eight values $10, 12, 14, 22, 31, 33, 36, 95$ sum to 253, so their mean is $253/8 = 31.625$. For the variance, expanding the square gives $\frac{1}{n}\sum_j (x_j - \bar{x})^2 = \frac{1}{n}\sum_j x_j^2 - 2\bar{x}\cdot\frac{1}{n}\sum_j x_j + \bar{x}^2 = \frac{1}{n}\sum_j x_j^2 - \bar{x}^2$. The squares sum to $100 + 144 + 196 + 484 + 961 + 1089 + 1296 + 9025 = 13295$, so the variance is $13295/8 - 31.625^2 = 1661.875 - 1000.141 = 661.734$ and the standard deviation

is $\sqrt{661.734} = 25.724$, so $z = (95 - 31.625)/25.724 = 2.464$, below 3 (and necessarily below $\sqrt{7} \approx 2.646$). Computed from the other seven values alone, the mean is $158/7 = 22.571$ and the squares sum to $13295 - 9025 = 4270$. The variance is then $4270/7 - 22.571^2 = 610 - 509.469 = 100.531$, and the standard deviation is $\sqrt{100.531} = 10.026$. Against these, the score would be $(95 - 22.571)/10.026 = 72.429/10.026 = 7.22$.

The robust alternative uses the median and the MAD. The eight values sorted are $10, 12, 14, 22, 31, 33, 36, 95$. With an even count, the median is the mean of the fourth and fifth values, $(22+31)/2 = 26.5$; the absolute deviations from it are $16.5, 14.5, 12.5, 4.5, 4.5, 6.5, 9.5, 68.5$, which sorted are $4.5, 4.5, 6.5, 9.5, 12.5, 14.5, 16.5, 68.5$. The median of these eight is the mean of the fourth and fifth, $(9.5 + 12.5)/2$, so the MAD is 11.0. The robust score of 95 is $0.6745 \times (95 - 26.5)/11.0 = 0.6745 \times 68.5/11.0 = 46.203/11.0 = 4.20$, and that of 22 is $0.6745 \times (22 - 26.5)/11.0 = 0.6745 \times (-4.5)/11.0 = -0.28$. The slow request now stands out clearly, because the median and MAD are hardly affected by it.

Figure 5.1 shows the three bands for the eight latencies.

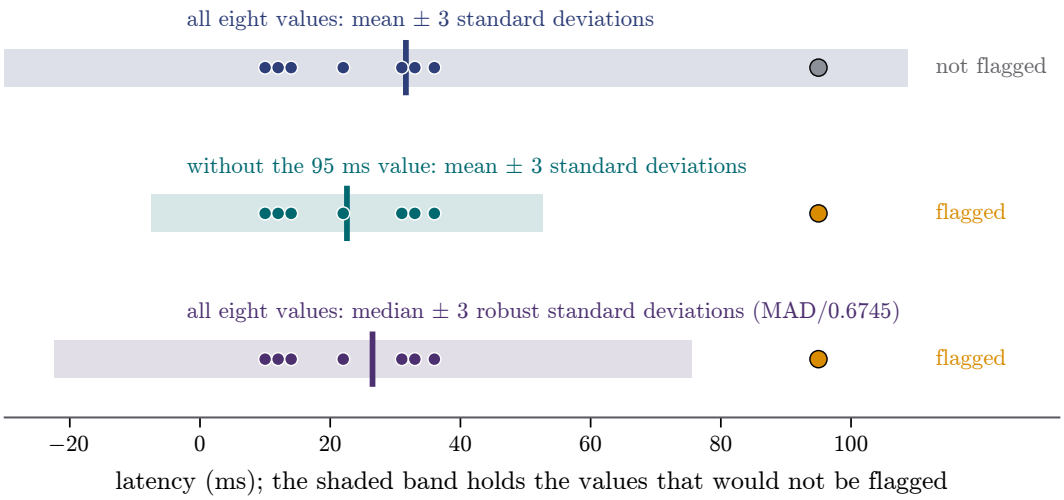


Figure 5.1: Masking. Each row shows the latencies, the centre used (vertical bar) and the band within which values are not flagged. With the ordinary mean and standard deviation of all eight values, the slow request widens the band enough to escape it. Without it, or with the median and the MAD, it is flagged.

5.2.2 Scoring by density under a fitted model

Intuition: a car on a quiet street at 3 a.m.

On a residential street, a car at 3 a.m. is unusual not because it is far from anything but because cars are rarely seen there at that hour. A density score measures exactly this: how often the model expects to see something like the example.

If a probability model of normal data has been fitted, a natural score is the density of an example under it: low density means unusual. With the two-component mixture fitted to the seven latencies in Section 3.1.4, *The EM algorithm* (means 14.21 and 32.80, $\sigma = 4$, equal weights), the densities are

$$p(12) = 4.279 \times 10^{-2}, \quad p(22) = 8.797 \times 10^{-3}, \quad p(33) = 4.981 \times 10^{-2}, \quad p(95) = 1.555 \times 10^{-54}.$$

Each density is $p(x) = \frac{1}{2}\mathcal{N}(x \mid \mu_1, 4) + \frac{1}{2}\mathcal{N}(x \mid \mu_2, 4)$, with the common factor $\frac{1}{2} \cdot \frac{1}{4\sqrt{2\pi}} = 0.049868$. For $x = 22$, with the fitted means to four decimals, $\mu_1 = 14.2126$ and $\mu_2 = 32.8004$, the exponents

are $(22-14.2126)^2/32 = 60.6436/32 = 1.8951$ and $(22-32.8004)^2/32 = 116.6486/32 = 3.6453$, so

$$\begin{aligned} p(22) &= 0.049868\left(e^{-1.8951} + e^{-3.6453}\right) \\ &= 0.049868 \times (0.15030 + 0.02611) = 0.049868 \times 0.17641 = 8.797 \times 10^{-3}. \end{aligned}$$

For $x = 95$, the exponents are $(95-14.2126)^2/32 = 203.96$ and $(95-32.8004)^2/32 = 3868.79/32 = 120.90$. The first term is $e^{-83.06}$ times the second, which changes $\ln p(95)$ by less than 10^{-35}, so

$$\ln p(95) = \ln 0.049868 - 120.90 = -2.998 - 120.90 = -123.90,$$

and $p(95) = e^{-123.90} = 1.555 \times 10^{-54}$. Working with log-densities (-3.15, -4.73, -3.00 and -123.90) avoids underflow. A threshold is usually set at a low quantile of the log-densities of the training data, for example the 1st percentile, which fixes the proportion of normal examples that will be flagged by mistake. The approach is only as good as the model: a single Gaussian fitted to data with two clusters assigns high density to the empty region between them.

5.3 Two features: distance, shape and neighbours

Intuition: height and weight

A person who is both tall and heavy is unremarkable, because height and weight usually go together. A person who is very tall and very light is unusual, although neither measurement alone is extreme. A score that ignores how the features vary together judges only the distance from the average person; the Mahalanobis distance measures the distance in units that respect the usual relationship between the features.

Two new runs are added to the eight training runs: $P = (2, 8)$, a short run with a large memory footprint, and $Q = (11, 11)$, a long run with a large footprint. Four scores, defined below, give the following.

score	run $P = (2,8)$	run $Q = (11,11)$
Euclidean distance from the mean	4.002	**8.048**
Mahalanobis distance	**4.316**	2.351
distance from the first principal line	**3.950**	0.144
mean distance to the 2 nearest runs	**4.236**	3.924

The mean of the eight runs is $\bar{\mathbf{x}} = (5.125,\ 5.500)$, so $P - \bar{\mathbf{x}} = (-3.125,\ 2.5)$ and $Q - \bar{\mathbf{x}} = (5.875,\ 5.5)$. The Euclidean distances from the mean are $\sqrt{3.125^2 + 2.5^2} = \sqrt{9.765625 + 6.25} = \sqrt{16.015625} = 4.002$ and $\sqrt{5.875^2 + 5.5^2} = \sqrt{34.515625 + 30.25} = \sqrt{64.765625} = 8.048$. Their ratio is $8.048/4.002 = 2.01$, so this distance ranks Q as twice as unusual as P. The other three scores reverse the ranking. Q continues the pattern of the data (a longer run that uses more memory), whereas P breaks it: its time and memory do not go together. The Mahalanobis distance and the distance from the principal line reverse the ranking decisively, because they account for the *shape* of the data; the nearest-neighbour score reverses it only narrowly. Figure 5.2 draws the Euclidean and the Mahalanobis distance from the mean as contours, with P and Q marked.

The Mahalanobis distance is

$$d_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \bar{\mathbf{x}})^\top S^{-1} (\mathbf{x} - \bar{\mathbf{x}})}.$$

A symmetric 2×2 matrix with $ac - b^2 \neq 0$ has the inverse

$$\begin{pmatrix} a & b \\ b & c \end{pmatrix}^{-1} = \frac{1}{ac - b^2} \begin{pmatrix} c & -b \\ -b & a \end{pmatrix},$$

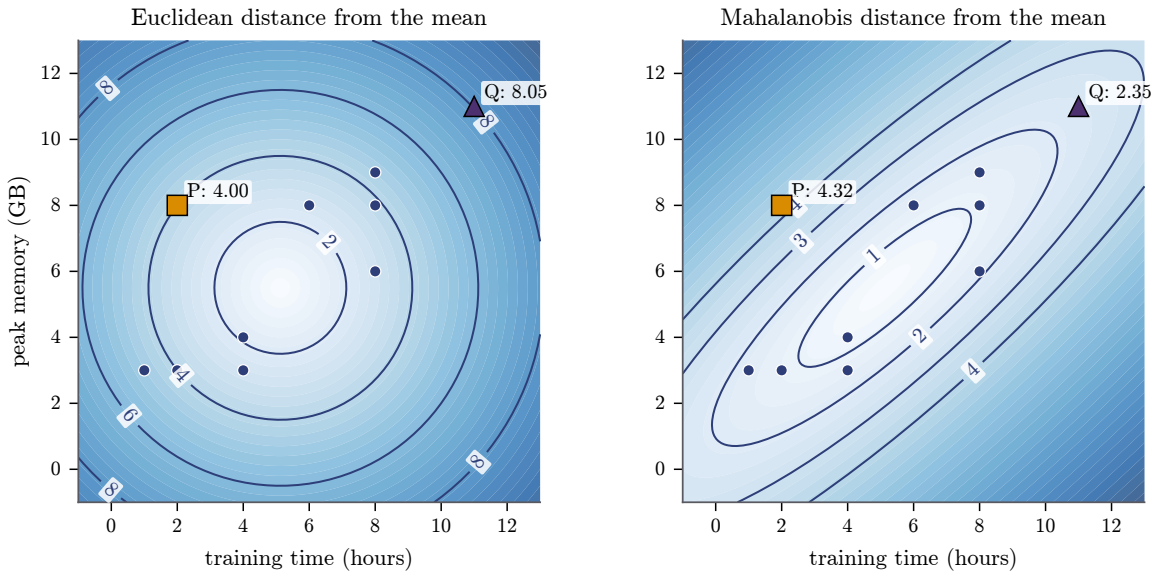


Figure 5.2: Distance from the mean of the eight runs, measured in two ways, with the new runs P (square) and Q (triangle). Euclidean distance (left) grows equally in every direction and ranks Q as the more unusual. Mahalanobis distance (right) grows slowly along the trend of the data and quickly across it, and ranks P as the more unusual.

because

$$\begin{pmatrix} a & b \\ b & c \end{pmatrix}\begin{pmatrix} c & -b \\ -b & a \end{pmatrix} = \begin{pmatrix} ac-b^2 & -ab+ba \\ bc-cb & -b^2+ca \end{pmatrix} = (ac-b^2)I.$$

For the eight runs, $S = \begin{pmatrix} 6.8594 & 5.4375 \\ 5.4375 & 5.7500 \end{pmatrix}$ (Chapter 1, *Mathematical background*), with $S_{11} = 54.875/8 = 6.859375$ exactly. Then $ac - b^2 = 6.859375 \times 5.75 - 5.4375^2 = 39.441406 - 29.566406 = 9.875$, and

$$S^{-1} = \frac{1}{9.875}\begin{pmatrix} 5.75 & -5.4375 \\ -5.4375 & 6.859375 \end{pmatrix} = \begin{pmatrix} 0.5823 & -0.5506 \\ -0.5506 & 0.6946 \end{pmatrix}.$$

Theorem (the Mahalanobis distance in principal coordinates)

If S has eigenvalues $\lambda_j > 0$ and orthonormal eigenvectors $\mathbf{v}_j$, and $z_j = \mathbf{v}_j^\top(\mathbf{x} - \bar{\mathbf{x}})$ are the principal component scores of $\mathbf{x}$, then

$$d_M(\mathbf{x})^2 = \sum_j \frac{z_j^2}{\lambda_j}.$$

Proof. By the spectral theorem $S = \sum_j \lambda_j \mathbf{v}_j \mathbf{v}_j^\top$, and the candidate inverse is $T = \sum_k \lambda_k^{-1} \mathbf{v}_k \mathbf{v}_k^\top$, which exists because every $\lambda_k > 0$. The product $\mathbf{v}_j^\top \mathbf{v}_k$ is a number, equal to 1 when $j = k$ and 0 otherwise; this is written δ_{jk}. Multiplying the two sums term by term,

$$ST = \sum_j \sum_k \lambda_j \lambda_k^{-1}\, \mathbf{v}_j (\mathbf{v}_j^\top \mathbf{v}_k) \mathbf{v}_k^\top = \sum_j \lambda_j \lambda_j^{-1}\, \mathbf{v}_j \mathbf{v}_j^\top = \sum_j \mathbf{v}_j \mathbf{v}_j^\top.$$

The last sum is the identity matrix. For any vector $\mathbf{w}$, expand it in the orthonormal basis as $\mathbf{w} = \sum_j c_j \mathbf{v}_j$. Then $\mathbf{v}_j^\top \mathbf{w} = c_j$, so $\left(\sum_j \mathbf{v}_j \mathbf{v}_j^\top\right)\mathbf{w} = \sum_j \mathbf{v}_j (\mathbf{v}_j^\top \mathbf{w}) = \sum_j c_j \mathbf{v}_j = \mathbf{w}$. Hence $ST = I$ and $S^{-1} = \sum_j \lambda_j^{-1} \mathbf{v}_j \mathbf{v}_j^\top$. Since a number equals its own transpose, $(\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{v}_j = \mathbf{v}_j^\top (\mathbf{x} - \bar{\mathbf{x}}) = z_j$, so each term $(\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{v}_j \mathbf{v}_j^\top (\mathbf{x} - \bar{\mathbf{x}})$ equals z_j^2. Substituting, $(\mathbf{x} - \bar{\mathbf{x}})^\top S^{-1} (\mathbf{x} - \bar{\mathbf{x}}) = \sum_j \lambda_j^{-1} \left(\mathbf{v}_j^\top (\mathbf{x} - \bar{\mathbf{x}})\right)^2$. □

The Mahalanobis distance is thus the Euclidean distance after rotating to principal coordinates and dividing each by its standard deviation $\sqrt{\lambda_j}$ (**whitening**). A step along a direction of small

variance counts for much more than a step along a direction of large variance. Figure 5.3 shows the rotation and the rescaling on the eight runs.

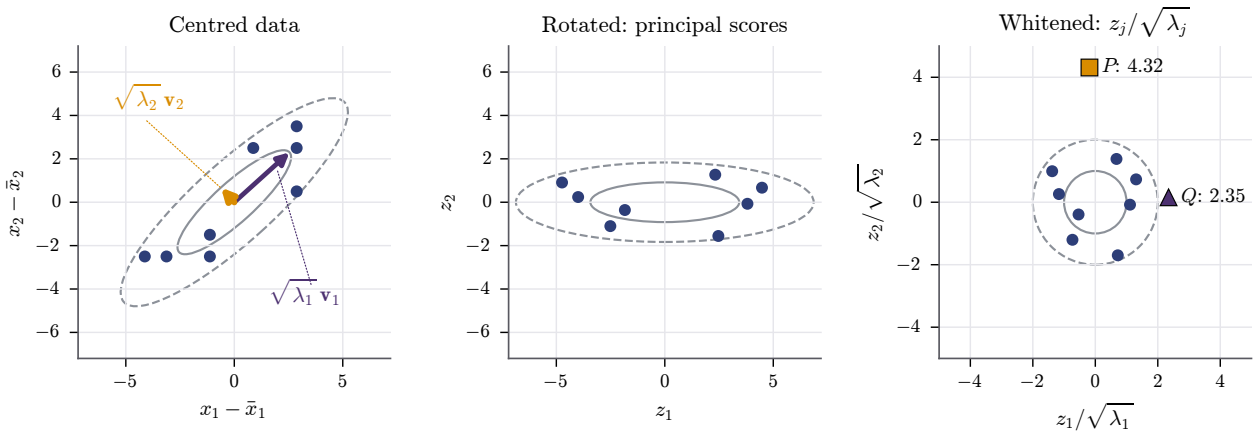


Figure 5.3: Whitening the eight runs. Left: the centred data with the ellipses of Mahalanobis distance 1 and 2 and the principal directions scaled by their standard deviations. Centre: the same points in principal coordinates. Right: after dividing each coordinate by its standard deviation the ellipses become circles, and the Mahalanobis distance is the ordinary distance from the origin, shown for P and Q.

Example

For P, the deviation from the mean $\bar{\mathbf{x}} = (5.125,\ 5.500)$ is $(2-5.125,\ 8-5.500) = (-3.125,\ 2.500)$. With $\mathbf{v}_1 = (0.7421,\ 0.6703)$ and $\mathbf{v}_2 = (-0.6703,\ 0.7421)$ (Section 4.2.2, *The first principal direction*), the principal scores are

$$z_1 = 0.7421 \times (-3.125) + 0.6703 \times 2.500 = -2.3191 + 1.6758 = -0.643,$$

$$z_2 = (-0.6703) \times (-3.125) + 0.7421 \times 2.500 = 2.0947 + 1.8553 = 3.950.$$

Then $d_M^2 = (-0.643)^2/11.770 + (3.950)^2/0.839 = 0.0352 + 18.5962 = 18.631$, so $d_M = 4.316$. Almost all of it comes from the second direction, along which the runs vary little. For Q, the deviation is $(11 - 5.125,\ 11 - 5.500) = (5.875,\ 5.500)$, so $z_1 = 0.7421 \times 5.875 + 0.6703 \times 5.500 = 4.3598 + 3.6867 = 8.046$. In $z_2 = (-0.6703) \times 5.875 + 0.7421 \times 5.500$ the two products nearly cancel, so $\mathbf{v}_2$ is needed to six decimals, $(-0.670267,\ 0.742120)$: $z_2 = -3.937819 + 4.081660 = 0.144$. Then $d_M^2 = z_1^2/\lambda_1 + z_2^2/\lambda_2 = 5.5007 + 0.0247 = 5.525$, so $d_M = \sqrt{5.525} = 2.351$. The definition with S^{-1} gives the same value for P:

$$\begin{aligned} &0.5823 \times (-3.125)^2 + 2 \times (-0.5506) \times (-3.125) \times 2.500 + 0.6946 \times 2.500^2 \\ &\quad = 5.6865 + 8.6031 + 4.3413 = 18.6309. \end{aligned}$$

For comparison, the Mahalanobis distances of the eight runs themselves are $0.664, 1.407, 1.193, 1.701, 1.113, 1.845, 1.497, 1.542$, at most 1.845.

Theorem (invariance to units)

If every example is transformed by $\mathbf{x} \mapsto A\mathbf{x} + \mathbf{b}$ with A invertible, the Mahalanobis distance of every example is unchanged.

Proof. The new mean is $\frac{1}{n}\sum_i (A\mathbf{x}_i + \mathbf{b}) = A\big(\frac{1}{n}\sum_i \mathbf{x}_i\big) + \mathbf{b} = A\bar{\mathbf{x}} + \mathbf{b}$. Each new deviation is therefore $(A\mathbf{x}_i + \mathbf{b}) - (A\bar{\mathbf{x}} + \mathbf{b}) = A(\mathbf{x}_i - \bar{\mathbf{x}})$: the shift $\mathbf{b}$ cancels. Using $(A\mathbf{u})^\top = \mathbf{u}^\top A^\top$, the new covariance is

$$\frac{1}{n}\sum_i A(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top A^\top = A\Big(\frac{1}{n}\sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top\Big)A^\top = ASA^\top.$$

Write $A^{-\top}$ for $(A^{-1})^\top$. It is the inverse of $A^\top$, because $A^\top(A^{-1})^\top = (A^{-1}A)^\top = I^\top = I$. Then $(ASA^\top)^{-1} = A^{-\top}S^{-1}A^{-1}$, because

$$(ASA^\top)(A^{-\top}S^{-1}A^{-1}) = AS(A^\top A^{-\top})S^{-1}A^{-1} = ASS^{-1}A^{-1} = AA^{-1} = I.$$

The new squared distance is therefore $(A(\mathbf{x}-\bar{\mathbf{x}}))^\top(ASA^\top)^{-1}A(\mathbf{x}-\bar{\mathbf{x}}) = (\mathbf{x}-\bar{\mathbf{x}})^\top A^\top A^{-\top}S^{-1}A^{-1}A(\mathbf{x}-\bar{\mathbf{x}}) = d_M(\mathbf{x})^2$. □

In particular, changing hours to minutes or gigabytes to megabytes does not change any Mahalanobis distance; the difficulty with units met by K-means (Section 2.5.3, *Dependence on the scale of the features*) does not arise.

Distance from the principal line, $|z_2|$, is the square root of the PCA reconstruction error z_2^2; it is the part of the Mahalanobis distance that comes from the minor direction, without the division by $\sqrt{\lambda_2}$. With many features, the error after keeping k components measures how far an example lies from the subspace in which normal data live.

Distance to nearest neighbours scores an example by its mean distance to its k nearest training examples. For $P = (2, 8)$, the two nearest runs are run 8 at $(6, 8)$, at distance $\sqrt{4^2 + 0^2} = 4$, and run 1 at $(4, 4)$, at distance $\sqrt{2^2 + 4^2} = \sqrt{20} = 4.4721$. With $k = 2$ the score is $(4 + 4.4721)/2 = 4.236$. For $Q = (11, 11)$, they are run 7 at $(8, 9)$, at distance $\sqrt{3^2 + 2^2} = \sqrt{13} = 3.6056$, and run 5 at $(8, 8)$, at distance $\sqrt{18} = 4.2426$, giving $(3.6056 + 4.2426)/2 = 3.924$. It makes no assumption of a single centre or an elliptical shape, so it also works when normal data form several clusters (Section 5.5, *Choosing a method, and evaluating with few labels*). Its cost is a nearest-neighbour search for every query.

5.4 Isolation forest

Intuition: twenty questions

In a game of twenty questions about the people in a room, a 2.1 m tall visitor is identified after one or two questions ("Is the person taller than 2 m?"), while a person of average height and build takes many. An isolation forest asks random yes/no questions about the features and scores each example by how few questions it takes to single it out.

The **isolation forest** (Liu, Ting and Zhou, 2008) scores examples by how easily they are separated from the rest by random cuts. An isolation tree is built on a sample of the data by repeating, until every point is alone: choose a feature at random and a cut point uniformly between that feature's minimum and maximum in the current group, and split the group. An unusual point tends to be cut off after few cuts, because it lies in a sparse region where a random cut is likely to fall next to it.

Let $h(x)$ be the depth at which x is isolated, averaged over many trees as $\bar{h}(x)$. To compare across sample sizes, it is divided by a normalising constant $c(m)$, the average isolation depth expected for a typical point among m:

$$c(m) = 2\big(\ln(m-1) + 0.5772\big) - \frac{2(m-1)}{m} \quad (m > 2), \qquad c(2) = 1, \qquad c(1) = 0, \qquad s(x) = 2^{-\bar{h}(x)/c(m)}.$$

Because $\bar{h}(x) \geq 0$ and 2^{-t} decreases as t increases, $0 < s(x) \leq 1$. A point isolated almost at once has $\bar{h}(x)$ close to 0 and $s(x)$ close to $2^0 = 1$. A point isolated at the typical depth, $\bar{h}(x) = c(m)$, has $s(x) = 2^{-1} = 0.5$, and a point that needs more cuts than typical has $s(x) < 0.5$. Scores close to 1 therefore indicate anomalies, scores of about 0.5 or less are unremarkable, and when every point scores close to 0.5 no point stands out. The formula for $c(m)$ comes from a simple model of splitting. Suppose a cut divides a group of m points into groups of k and $m - k$ points, with each $k = 1, \ldots, m - 1$ equally likely. This is also the model behind the average length of an unsuccessful search in a random binary search tree, from which Liu, Ting and Zhou took the constant. Let $D(m)$

be the expected sum of the isolation depths of all m points, so that $c(m) = D(m)/m$, with $D(1) = 0$. The first cut adds one to the depth of every point, and each part is then isolated on its own, so

$$D(m) = m + \frac{1}{m-1}\sum_{k=1}^{m-1}\big(D(k) + D(m-k)\big) = m + \frac{2}{m-1}\sum_{k=1}^{m-1} D(k),$$

because $m-k$ runs over $1, \dots, m-1$ as k does. Multiplying by $m-1$ gives $(m-1)D(m) - m(m-1) = 2\sum_{k=1}^{m-1} D(k)$, which also holds for $m = 1$. The same equation with $m+1$ in place of m is $mD(m+1) - (m+1)m = 2\sum_{k=1}^{m} D(k)$. Subtracting the first from the second,

$$mD(m+1) - (m-1)D(m) - 2m = 2D(m), \qquad \text{so} \qquad mD(m+1) = (m+1)D(m) + 2m.$$

Dividing by $m(m+1)$ gives $c(m+1) = c(m) + \dfrac{2}{m+1}$. Starting from $c(1) = 0$, this gives $c(2) = 1$ and in general

$$c(m) = \frac{2}{2} + \frac{2}{3} + \cdots + \frac{2}{m} = 2(H_m - 1), \qquad H_m = 1 + \frac{1}{2} + \cdots + \frac{1}{m}.$$

Since $H_m = H_{m-1} + \frac{1}{m}$, $c(m) = 2H_{m-1} + \frac{2}{m} - 2 = 2H_{m-1} - \frac{2(m-1)}{m}$. The harmonic number H_{m-1} is close to $\ln(m-1) + 0.5772$, where 0.5772 is Euler's constant, and this approximation gives the formula above, which is the one used in software. It is good for large m: $c(256)$ is 10.249 exactly and 10.245 approximately. It is rough for small m: 3.436 against 3.296 for $m = 8$. At $m = 2$ it would give 0.154 instead of 1, which is why $c(2)$ is stated separately. It is also added at a leaf that still contains more than one point when growth is stopped by a depth limit (Algorithm 9), as an estimate of the further depth those points would need.

Algorithm 9: isolation forest

Input: training points; number of trees t (for example 100); subsample size ψ (for example 256), which plays the role of m above

Output: an anomaly score $s(\mathbf{x})$ for any point $\mathbf{x}$

1: **for** each of the t trees **do**
2: draw ψ training points at random without replacement
3: split them recursively: choose a feature at random and a cut point uniformly between that feature's minimum and maximum in the current group; stop when a group has one point or the depth reaches $\lceil \log_2 \psi \rceil$
4: **end for**
5: for a point $\mathbf{x}$, in each tree follow the cuts to a leaf; its path length is the depth of the leaf plus $c(q)$, where q is the number of training points in the leaf
6: $\bar{h}(\mathbf{x}) \leftarrow$ the average path length over the t trees; $s(\mathbf{x}) \leftarrow 2^{-\bar{h}(\mathbf{x})/c(\psi)}$

The depth limit saves time without changing the ranking much: anomalies are isolated near the root, and the limit truncates only long paths, which belong to typical points. Building the forest costs about $t\,\psi \log \psi$ operations and scoring a point about $t \log \psi$, independent of the size of the training data, which makes the method fast on large tables. Figure 5.4 follows one random tree until an outlying point and a typical point are each isolated.

Example: the eight latencies

In one dimension, the first random cut isolates 95 exactly when it falls between 36 and 95, which happens with probability $(95 - 36)/(95 - 10) = 0.694$. Only the smallest value, 10, can also be isolated by the first cut, with probability $(12 - 10)/85 = 0.024$; no interior value can be isolated by a single cut. Averaged over 2000 random trees on all eight values, grown until every point is alone, ($m = 8$, $c(8) = 3.296$), the mean isolation depths and scores are as follows.

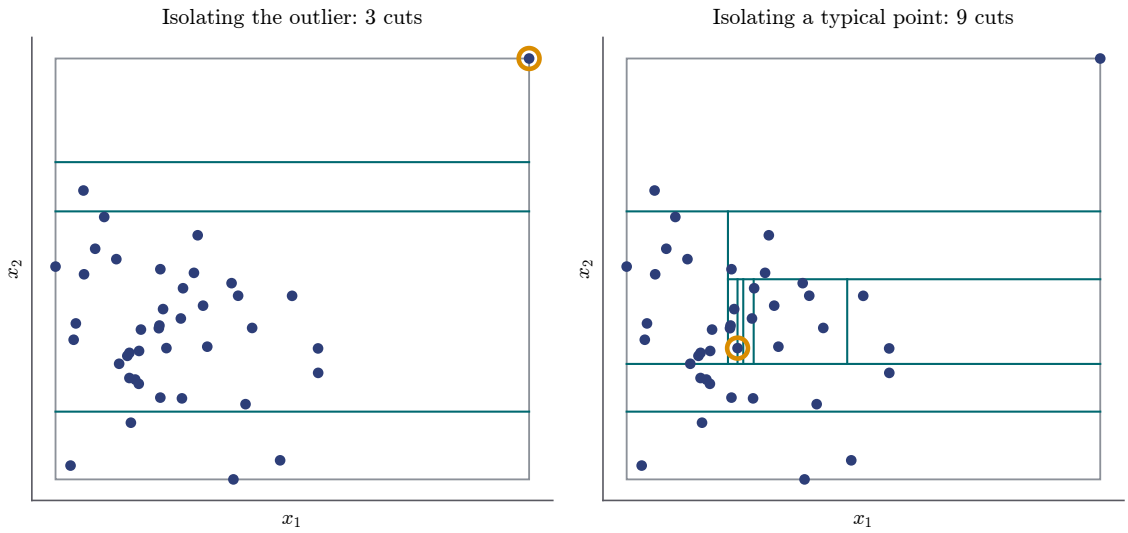


Figure 5.4: One random isolation tree on 41 points, followed until the circled point is alone. The outlying point (left) is separated by a few cuts; a point in the middle of the cloud (right) needs many more. The grey rectangle is the range of the data. Averaged over many trees, the depth at which a point is isolated determines its anomaly score.

latency (ms)	mean depth $\bar{h}(x)$	score $s(x)$
10	3.44	0.485
12	4.23	0.411
14	4.01	0.430
22	3.57	0.472
31	4.19	0.415
33	4.37	0.399
36	3.56	0.473
95	1.35	0.753

The request of 95 ms is isolated at mean depth 1.35 and receives score 0.753; the others need depths from 3.44 to 4.37 and score at most 0.485. The method needs no distances, no density model and no scaling of features, and it is fast; it is a good first method for tables of features.

5.5 Choosing a method, and evaluating with few labels

Intuition: a guard who can open twenty bags

A security guard has time to open only twenty bags out of five hundred. What matters is how many of the twenty contained something forbidden: that is precision at k. Different screening rules suit different threats; a rule that catches oversized items will miss small ones hidden among ordinary luggage.

Without labels, an anomaly detector cannot be evaluated directly. In practice a small set of known anomalies is usually collected (incidents found by hand, past failures), and the detectors are compared on how high they rank them. **Precision at** k is the fraction of anomalies among the k highest-scored examples; k is often set to the number of known anomalies, as in the table below. **Average precision** (AP) averages, over the known anomalies, the precision among all examples scored at least as high as that anomaly; in symbols, if a of the n examples are anomalies at ranks $r_1 < r_2 < \cdots < r_a$, the j-th of them has exactly j anomalies at or above it, so

$$\mathrm{AP} = \frac{1}{a} \sum_{j=1}^{a} \frac{j}{r_j}.$$

For a perfect ranking $r_j = j$, every term is 1 and AP $= 1$. For a random ranking, AP is close to the *base rate* a/n (the fraction of examples that are anomalies). To see this, take one anomaly and let its rank r be equally likely to be any of $1, \dots, n$. Each of the $r-1$ examples above it is one of the other $a-1$ anomalies with probability $(a-1)/(n-1)$, so the expected precision at its rank is

$$\frac{1 + (r-1)\frac{a-1}{n-1}}{r} = \frac{a-1}{n-1} + \frac{n-a}{n-1} \cdot \frac{1}{r}.$$

Averaging over r, with $\frac{1}{n}\sum_{r=1}^{n} \frac{1}{r} = H_n/n$ and $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$, the expected AP is $\frac{a-1}{n-1} + \frac{n-a}{n-1} \cdot \frac{H_n}{n}$. The second term vanishes as n grows, leaving about a/n. For data set A below ($n = 500$, $a = 20$) it is $0.038 + 0.013 = 0.051$, against a base rate of 0.040. For B ($n = 490$, $a = 10$) it is 0.032, against 0.020.

Two data sets show that no score is best in general. In A, 480 normal runs form one elongated, correlated cloud and 20 anomalies lie across it, in two tight groups of ten on either side, at moderate distance from the centre. Each such anomaly has other anomalies as its nearest neighbours, which masks it from the neighbour and isolation scores. In B, 480 normal runs form two clusters and 10 anomalies lie scattered in the gap between them, close to the overall mean. The base rates are 0.040 and 0.020. All scores are computed from the full data, anomalies included, as they would be when no labels are available. Figure 5.5 shows both data sets.

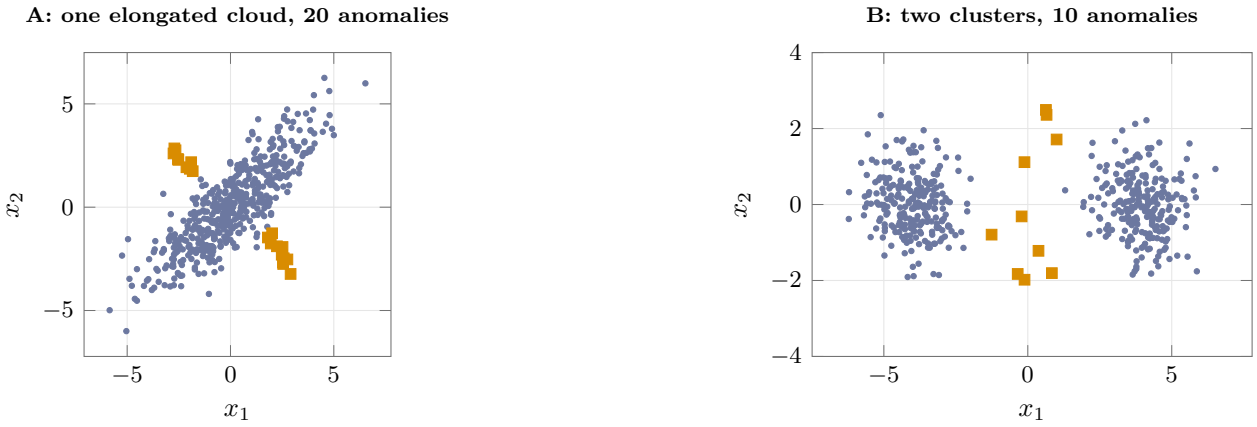


Figure 5.5: The two data sets; squares are the anomalies.

score	A: one elongated cloud precision at $k = 20$	A: one elongated cloud AP	B: two clusters precision at $k = 10$	B: two clusters AP
Euclidean distance from the mean	0.00	0.070	0.00	0.011
Mahalanobis distance	0.70	0.847	0.20	0.188
distance to 5th nearest neighbour	0.15	0.202	0.90	0.991
isolation forest score	0.10	0.207	0.50	0.580

In A the Mahalanobis distance ranks best (precision 0.70, AP 0.847), because the anomalies differ from normal data exactly in the direction of small variance. In B the distance to the fifth nearest neighbour ranks best (precision 0.90, AP 0.991), while the Mahalanobis distance falls to precision 0.20: a single ellipse fitted to two clusters places its centre in the gap, where the anomalies are. The isolation forest is intermediate in B (0.50). The Euclidean distance from the mean is the worst in both. The choice of score is a modelling decision about what "normal" looks like, and it should be checked on whatever labelled anomalies exist. Figure 5.6 shows why, by shading each score over the plane of data set B.

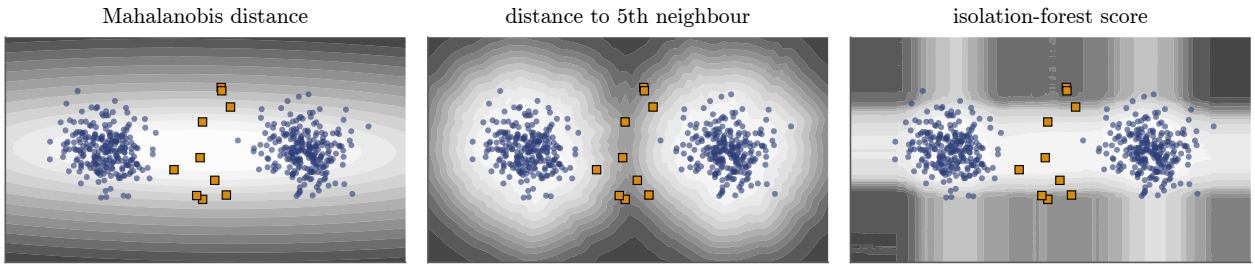


Figure 5.6: Three anomaly scores over the plane for data set B: the normal runs (dots) form two clusters and the anomalies (squares) lie in the gap. Darker shading means a higher score; the scores are shown by rank so that the three panels share one scale. The Mahalanobis distance fits one ellipse to both clusters and is lowest in the gap, exactly where the anomalies are. The distance to the fifth nearest neighbour is low only near the data, so the gap scores high. The isolation forest scores the gap only moderately, and its cuts parallel to the axes leave visible bands.

5.6 Anomaly detection in deployed machine-learning systems

Intuition: a shop after a new college opens

A shop near a new college finds its customers changing: younger, buying different things, at different hours. Its old sales forecasts quietly become wrong. Drift monitoring is the habit of checking whether today's inputs still look like the data a model learned from, before its predictions are trusted.

Monitoring for drift. A deployed model can fail silently when its inputs stop resembling its training data, a change called *drift*. Monitoring therefore treats each batch of recent inputs (through their features or, for text and images, their embeddings) as a possible anomaly relative to a reference set, measured in one of three ways: the fraction of recent inputs whose nearest-neighbour distance to the reference exceeds a high quantile of the reference's own distances; a statistical test of whether the two samples come from the same distribution; or the accuracy of a classifier trained to tell recent from reference inputs, which stays near one half (with equal sample sizes) when nothing has changed. The thresholds must be set with the cost of false alarms in mind, since frequent spurious alerts are soon ignored.

Nearest neighbours at scale. Distance-based scores require nearest-neighbour search over millions of stored vectors. Exact search is replaced by approximate search: indexes that link each stored vector to a few of its neighbours and search by walking along these links, and cluster-based indexes, which divide the stored vectors among K-means clusters and compare a query only with the vectors in the nearest few clusters. These return most of the true neighbours at a small fraction of the cost.

An unusual example is not necessarily an error. A flagged example may be an error, an attack, or simply a new and legitimate kind of input; the decision belongs to a person or to a decision rule that weighs the costs of errors.

5.7 Practice problems for Chapter 5

5.1 [Basic] For the data $4, 5, 5, 6, 30$: (a) compute the mean, the standard deviation (with $1/n$) and the standard score of 30; (b) compute the median, the MAD and the robust score of 30; (c) explain the difference using Samuelson's inequality.

5.2 [Standard] Typical (non-anomalous) data have mean $\mathbf{0}$ and covariance $S = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$ (as in Problem 4.1). Compute the Euclidean and Mahalanobis distances from the mean of $\mathbf{a} = (2, 1)$ and $\mathbf{b} = (-1, 2)$, and explain the result using the principal directions.

5.3 [Basic] One-dimensional data $1, 2, 3, 4, 20$. For the first random cut of an isolation tree (uniform between 1 and 20), find the probability that it isolates 20, that it isolates 1, and that it isolates 3.

5.4 [Basic] A detector ranks ten examples; the true labels in rank order are $1, 0, 1, 1, 0, 0, 1, 0, 0, 0$ (1 for an anomaly). Compute the precision at 3, the precision at 5, and the average precision.

5.5 [Standard] Prove that a single Gaussian fitted to data from two well-separated clusters of equal size assigns its highest density to the midpoint between the clusters. What does this imply for density-based anomaly scores?

5.6 [Standard] A drift monitor raises an alarm when more than 5% of a day's inputs have nearest-neighbour distance above the 95th percentile of the reference set's own distances. Without drift, about what fraction of inputs exceeds the threshold? Why might the alarm still fire on many days without drift, and how would you set the alarm level?

5.7 [Standard] Explain masking and swamping with an example of your own, and state which of the scores in this chapter are most and least affected by each.

5.8 [Standard] A team proposes to use reconstruction error from a PCA with k components as an anomaly score for readings from 200 sensors on a production line. Describe how you would choose k, how you would set the threshold, and one kind of anomaly the method would miss.

5.9 [Basic] Nineteen values equal 10 and one equals 30. Compute the standard score of 30 and compare it with Samuelson's bound.

5.10 [Basic] Show that in one dimension the Mahalanobis distance from the mean is the absolute standard score $|z|$.

5.11 [Basic] Typical (non-anomalous) data have mean $\mathbf{0}$ and covariance $\mathrm{diag}(4, 1)$. Compute the Mahalanobis distances of $(3, 0)$ and $(0, 3)$, and explain why they differ although the Euclidean distances are equal.

5.12 [Standard] For isolation forests with sub-samples of $m = 256$ points, compute $c(2)$ and $c(256)$, and the scores of points whose mean isolation depth is 2 and 10.

5.13 [Standard] Two detectors rank eight examples, three of which are anomalies. In rank order the labels are $1, 1, 0, 0, 0, 0, 0, 1$ for detector A and $0, 1, 1, 1, 0, 0, 0, 0$ for detector B. Compare their precision at 3 and their average precision, and say which you would prefer and why.

5.14 [Challenge] Typical (non-anomalous) data are drawn from a multivariate normal distribution with mean $\boldsymbol{\mu}$ and covariance Σ in d dimensions. Prove that the expected squared Mahalanobis distance $E\big[(\mathbf{x}-\boldsymbol{\mu})^{\top}\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\big]$ equals d, and explain what this implies for a fixed threshold such as $d_M > 3$ when d is large.

5.15 [Challenge] Prove that for a ranking chosen uniformly at random, the expected precision at k equals the proportion of anomalies in the data, for every k.

Chapter 6

Solutions and reference

6.1 Solutions to the practice problems

Chapter 2

2.1 In the first iteration, 1 is assigned to centroid 1 (squared distance 0 against 1), and 2 and 9 to centroid 2. The centroids become 1 and $(2+9)/2 = 5.5$. In the second iteration, 2 is at squared distance 1 from centroid 1 and 12.25 from 5.5, so it moves to the first cluster; 9 stays. The centroids become 1.5 and 9. The third iteration changes nothing. Clusters $\{1, 2\}$ and $\{9\}$, centroids 1.5 and 9, $J = 0.5^2 + 0.5^2 + 0 = 0.50$.

2.2 (a) Centroids (1.5, 1.0) and (4.5, 3.5); each point is at squared distance 0.25 or 0.5 from its centroid, and $J = 1.5$.
(b) *Iteration 1.* Squared distances to $(1, 1)$ and $(2, 1)$: $0, 1$; $1, 0$; $13, 8$; $25, 18$. Points 2, 3 and 4 go to the second centroid; $J = 26$. The centroids become $(1, 1)$ and (3.667, 2.667). *Iteration 2.* Squared distances: point 1: 0 and 9.889; point 2: 1.000 and 5.556; point 3: 13.000 and 0.222; point 4: 25.000 and 3.556. The first two points go to the first centroid and the last two to the second; $J = 4.778$. The centroids become (1.5, 1.0) and (4.5, 3.5). *Iteration 3.* No assignment changes and $J = 1.5$, the value of the partition in (a).

2.3 Write $\mathbf{x}_j - \mathbf{c} = (\mathbf{x}_j - \bar{\mathbf{x}}) + (\bar{\mathbf{x}} - \mathbf{c})$ and expand:

$$\sum_j \|\mathbf{x}_j - \mathbf{c}\|^2 = \sum_j \|\mathbf{x}_j - \bar{\mathbf{x}}\|^2 + 2(\bar{\mathbf{x}} - \mathbf{c})^\top \sum_j (\mathbf{x}_j - \bar{\mathbf{x}}) + m\|\bar{\mathbf{x}} - \mathbf{c}\|^2.$$

The middle sum is zero because the deviations from the mean sum to zero. The first term does not depend on $\mathbf{c}$ and the last is positive unless $\mathbf{c} = \bar{\mathbf{x}}$, so the mean is the unique minimiser.

2.4 (a) The assignment step leaves a partition unchanged exactly when each point is already assigned to its nearest centroid, and the update step leaves the centroids unchanged exactly when they are already the cluster means. A fixed point is a partition for which both hold. (b) Each iteration that changes the partition strictly lowers J, so no partition is visited twice. There are at most K^n partitions, so there are at most K^n iterations. (c) No. The rectangle of Section 2.5.1, *Dependence on initialisation* has a fixed point with $J = 16$ although $J = 4$ is attainable.

2.5 Raw: $3^2 + 2000^2 = 4{,}000{,}009$, of which memory contributes 99.9998%. Standardised: $(3/1.6)^2 + (2000/3634)^2 = 3.516 + 0.303 = 3.819$, of which memory contributes 7.9%. On raw features the distance is almost entirely memory; after standardisation it is mostly time.

2.6 The squared distances to the nearest chosen centroid (0) are 0, 1 and 100, so the probabilities are $1/101 = 0.0099$ for the point 1 and $100/101 = 0.9901$ for 10.

2.7 (a) The function $g(c) = \sum_j |x_j - c|$ is convex and piecewise linear. Away from the data points its slope is (number of points below c) minus (number above c). The slope is negative while fewer than half the points lie below c and positive once more than half do, so the minimum is attained at a median. (b) At the median 2: $\sum |x_j - c| = 8$ and $\sum (x_j - c)^2 = 50$. At the mean 4: $\sum |x_j - c| = 10$ and $\sum (x_j - c)^2 = 38$. Each centre is optimal for its own criterion.

2.8 On the raw features, squared distances are determined almost entirely by the second feature, so the first is effectively ignored. Remedies: standardise both features, which assumes they are equally important; or rescale them by weights chosen from knowledge of the problem, which assumes those weights are known.

2.9 Take an optimal partition into K clusters, and move one point from a cluster with at least two points into a cluster of its own, leaving all other centroids where they are. That point now contributes 0, every other contribution is unchanged, and so J does not increase. The optimum for $K+1$ clusters is at most this value. Since $J = 0$ for $K = n$, minimising J would always choose $K = n$.

2.10 First partition: centroids 4 and 11, $J = (4+0+4)+(1+1) = 10.0$. Second partition: centroids 3 and 9.333, $J = (1+1)+(3.333^2+0.667^2+2.667^2) = 20.667$. The second is not a fixed point: the point 6 is at distance 3 from the centroid 3 and 3.333 from 9.333, so it moves to the first cluster, and the algorithm reaches the first partition.

2.11 The squared distances to 0 are $0, 1, 4, 100, 121$, with sum 226, so the probabilities for $1, 2, 10, 11$ are $0.0044, 0.0177, 0.4425, 0.5354$. With centroids 0 and 10, the squared distances to the nearer centroid are $0, 1, 4, 0, 1$, with sum 6: the point 1 is chosen with probability 0.167, the point 2 with 0.667, and 11 with 0.167.

2.12 $\|\mathbf{x} - \boldsymbol{\mu}_a\|^2 = \|\mathbf{x} - \boldsymbol{\mu}_b\|^2$ expands to $-2\boldsymbol{\mu}_a^\top \mathbf{x} + \|\boldsymbol{\mu}_a\|^2 = -2\boldsymbol{\mu}_b^\top \mathbf{x} + \|\boldsymbol{\mu}_b\|^2$, that is, $(\boldsymbol{\mu}_a - \boldsymbol{\mu}_b)^\top \mathbf{x} = \frac{1}{2}(\|\boldsymbol{\mu}_a\|^2 - \|\boldsymbol{\mu}_b\|^2)$. This is a hyperplane with normal vector $\boldsymbol{\mu}_a - \boldsymbol{\mu}_b$, perpendicular to the segment joining the centroids. It contains the midpoint $(\boldsymbol{\mu}_a + \boldsymbol{\mu}_b)/2$, since $(\boldsymbol{\mu}_a - \boldsymbol{\mu}_b)^\top (\boldsymbol{\mu}_a + \boldsymbol{\mu}_b)/2 = \frac{1}{2}(\|\boldsymbol{\mu}_a\|^2 - \|\boldsymbol{\mu}_b\|^2)$.

2.13 The decreases are $628.1, 386.3, 11.3, 10.7$. The first two are large and the rest small, so the elbow is at $K = 3$.

2.14 Write $\mathbf{x}_i - \mathbf{x}_j = (\mathbf{x}_i - \bar{\mathbf{x}}) - (\mathbf{x}_j - \bar{\mathbf{x}})$ and expand: $\sum_i \sum_j \|\mathbf{x}_i - \mathbf{x}_j\|^2 = 2m \sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 - 2\left\|\sum_i (\mathbf{x}_i - \bar{\mathbf{x}})\right\|^2 = 2m \sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2$, because the deviations sum to zero. For runs 1–4 the sum over ordered pairs is 60, and $60/8 = 7.5$, the value found for the cluster of small runs. Hence $J = \sum_k \frac{1}{2n_k} \sum_{i,j \in k} \|\mathbf{x}_i - \mathbf{x}_j\|^2$: that is, $J = \sum_k \frac{n_k}{2} D_k$, where D_k is the mean squared distance over ordered pairs of points in cluster k, a quantity that involves no centroids.

2.15 A partition into two non-empty clusters is determined by the cluster that does not contain 0, which is one of the $2^3 - 1 = 7$ non-empty subsets of $\{1, 2, 4\}$. For each partition, take the two cluster means and look for a point strictly nearer the other mean (distances to its own mean and to the other mean in brackets): $\{0, 2, 4\}, \{1\}$, means 2 and 1: point 0 (2 against 1) moves; $\{0, 1, 4\}, \{2\}$, means $\frac{5}{3}$ and 2: point 4 (2.333 against 2) moves; $\{0, 4\}, \{1, 2\}$, means 2 and 1.5: point 0 (2 against 1.5) moves; $\{0, 2\}, \{1, 4\}$, means 1 and 2.5: point 2 (1 against 0.5) moves; $\{0\}, \{1, 2, 4\}$, means 0 and $\frac{7}{3}$: point 1 (1.333 against 1) moves; $\{0, 1, 2\}, \{4\}$, means 1 and 4: no point moves (0: 1 against 4; 1: 0 against 3; 2: 1 against 2; 4: 0 against 3), and $J = 1 + 0 + 1 + 0 = 2$; $\{0, 1\}, \{2, 4\}$, means 0.5 and 3: no point moves (0: 0.5 against 3; 1: 0.5 against 2; 2: 1 against 1.5; 4: 1 against 3.5), and $J = 0.25 + 0.25 + 1 + 1 = 2.5$. A global minimum is always a fixed point: if a point were strictly nearer another mean, moving it would strictly lower J, and the update step would not raise it again. Exactly two are fixed points: $\{0, 1, 2\}, \{4\}$ with centroids 1 and 4 and $J = 2$; and $\{0, 1\}, \{2, 4\}$ with centroids 0.5 and 3 and $J = 2.5$. In the second, the point 2 is at distance 1.5 from 0.5 and 1 from 3, so it stays. A global minimum of J is always a fixed point, so the minimum is $J = 2$, and the second fixed point is not optimal.

Chapter 3

3.1 At $x = 1$ the two exponentials are $e^{-1/2}$ and $e^{-9/2}$; their ratio is $e^4 = 54.60$, and the responsibility of the first component is $e^4/(1 + e^4) = 0.982$. At $x = 2$ the two distances are equal and the responsibility is 0.500. With equal weights and equal variances the responsibilities are equal where $(x - 0)^2 = (x - 4)^2$, that is at the midpoint $x = 2$.

3.2 (a) The terms of the weighted log-likelihood involving σ_k are $\sum_i r_{ik}\big(-\log\sigma_k - (x_i-\mu_k)^2/(2\sigma_k^2)\big)$. Setting the derivative $-\sum_i r_{ik}/\sigma_k + \sum_i r_{ik}(x_i - \mu_k)^2/\sigma_k^3$ to zero gives $\sigma_k^2 = \sum_i r_{ik}(x_i - \mu_k)^2/\sum_i r_{ik}$. (b) Maximise $\sum_i \sum_k r_{ik} \log \pi_k - \lambda(\sum_k \pi_k - 1)$. Setting the derivative to zero gives $\sum_i r_{ik}/\pi_k = \lambda$, so $\pi_k = \sum_i r_{ik}/\lambda$. Summing over k and using $\sum_k \pi_k = 1$ gives $\lambda = \sum_i \sum_k r_{ik} = n$, so $\pi_k = \frac{1}{n}\sum_i r_{ik}$.

3.3 With $q_{ik} = r_{ik}$, each ratio $p_k(x_i;\theta)/r_{ik}$ equals $\sum_j p_j(x_i;\theta)$, so $F = \sum_i \sum_k r_{ik} \log \sum_j p_j(x_i;\theta) = \sum_i \log \sum_j p_j(x_i;\theta) = \ell(\theta)$. For any q with $q_{ik} > 0$ and $\sum_k q_{ik} = 1$, write $\ell(\theta) = \sum_i\big(\sum_k q_{ik}\big) \log \sum_j p_j(x_i;\theta)$ and subtract F term by term:

$$\begin{aligned}\ell(\theta) - F(q,\theta) &= \sum_i \sum_k q_{ik}\Big(\log \sum_j p_j(x_i;\theta) - \log \frac{p_k(x_i;\theta)}{q_{ik}}\Big) \\ &= \sum_i \sum_k q_{ik} \log \frac{q_{ik} \sum_j p_j(x_i;\theta)}{p_k(x_i;\theta)} = \sum_i \sum_k q_{ik} \log \frac{q_{ik}}{r_{ik}},\end{aligned}$$

because $p_k(x_i;\theta)/\sum_j p_j(x_i;\theta) = r_{ik}$. Each inner sum $\sum_k q_{ik} \log(q_{ik}/r_{ik})$ is a Kullback–Leibler divergence, and it is nonnegative. The function $t - 1 - \log t$ has derivative $1 - 1/t$, so it decreases until $t = 1$, where it equals 0, and increases afterwards. Hence $\log t \le t - 1$, with equality only at $t = 1$, and

$$-\sum_k q_{ik} \log \frac{q_{ik}}{r_{ik}} = \sum_k q_{ik} \log \frac{r_{ik}}{q_{ik}} \le \sum_k q_{ik}\Big(\frac{r_{ik}}{q_{ik}} - 1\Big) = \sum_k r_{ik} - \sum_k q_{ik} = 1 - 1 = 0.$$

Equality requires $r_{ik}/q_{ik} = 1$ for every k, that is, $q = r$. Hence $F(q,\theta) \le \ell(\theta)$, with equality only when $q = r$.

3.4 Within its cluster, the point 2 has distances 2 and 1, so $a = 1.5$. To the other cluster it has distances 4 and 5, so $b = 4.5$. Then $s = (4.5 - 1.5)/4.5 = 0.667$.

3.5 Single linkage: $\{0, 1\}$ at height 1, then $\{0, 1, 3\}$ at 2, then all four points at 4. Complete linkage: $\{0, 1\}$ at 1, then $\{0, 1, 3\}$ at 3, then all four points at 7. Single linkage measures the closest pair and complete linkage the farthest, so complete linkage merges at greater heights.

3.6 The neighbourhoods have sizes 2, 3, 3, 2, 2, 2, 1. The core points are 0.5 and 1. Together with the border points 0 and 1.5 they form one cluster, $\{0, 0.5, 1, 1.5\}$. The points 5 and 5.4 are close to each other but neither has three points in its neighbourhood, so they are noise, as is 9.

3.7 Place the mean of component 1 on a data point, $\mu_1 = x_1$, and keep component 2 fixed with a positive variance so that every point has density bounded below. Because the exponent vanishes at $x_1 = \mu_1$, the term $\pi_1\mathcal{N}(x_1 \mid x_1, \sigma_1) = \pi_1/(\sigma_1\sqrt{2\pi})$, and it tends to infinity as $\sigma_1 \to 0$. In each sum $\pi_1\mathcal{N}(x_i \mid \mu_1, \sigma_1) + \pi_2\mathcal{N}(x_i \mid \mu_2, \sigma_2)$ both terms are positive, and the logarithm is increasing, so dropping a term lowers the log-likelihood. Keeping only the first component for $i = 1$ and only the second for $i \ge 2$,

$$\ell \;\ge\; \log \frac{\pi_1}{\sigma_1\sqrt{2\pi}} + \sum_{i\ge 2} \log\big(\pi_2\mathcal{N}(x_i \mid \mu_2, \sigma_2)\big) = -\log\sigma_1 + \log \frac{\pi_1}{\sqrt{2\pi}} + \sum_{i\ge 2} \log\big(\pi_2\mathcal{N}(x_i \mid \mu_2, \sigma_2)\big).$$

Only the first term on the right depends on σ_1, and $-\log\sigma_1 \to \infty$ as $\sigma_1 \to 0$. So the right-hand side, and with it ℓ, exceeds any given number once σ_1 is small enough. Implementations impose a lower bound on variances (or a prior), and treat solutions in which a component collapses onto one point as failures.

3.8 The core distances are 2 for 0, 1 for 1, 2 for 2 and 9 for 10. Then $d_{\text{mreach}}(0,1) = \max(2,1,1) = 2$ and $d_{\text{mreach}}(2,10) = \max(2,9,8) = 9$. Relative to its ordinary distance, the pair $(0,1)$ is stretched by a factor of 2 and the pair $(2,10)$ by a factor of 1.125. The mutual reachability distance never lets two points be closer than either one's core distance. This mainly affects pairs at short ordinary distance in which one point lies at the sparse edge of the data, and it keeps isolated points such as 10 far from everything.

3.9 The silhouette is barely above that of data with no clusters, and the agreement between half-samples, 0.61, is barely above the 0.58 found for data with no clusters. There is little evidence of genuine clusters. Do not report the four clusters as a finding. Try other numbers of clusters, a density-based method that may return few clusters and much noise, and other features or scalings, and check whether any resulting cluster has an interpretation that can be verified.

3.10 At $x = 2$ the two distances are equal, so the responsibility is the prior weight, $0.2/(0.2+0.8) = 0.2$. The responsibilities are equal where $\pi_1 e^{-x^2/2} = \pi_2 e^{-(x-4)^2/2}$, that is, after taking logarithms of both sides, $\ln \pi_1 - \frac{1}{2}x^2 = \ln \pi_2 - \frac{1}{2}(x-4)^2$. So $\ln(\pi_1/\pi_2) = \frac{1}{2}\big(x^2 - (x-4)^2\big) = \frac{1}{2}\big(x^2 - x^2 + 8x - 16\big) = 4x - 8$. Solving, $x = 2 + \frac{1}{4}\ln(\pi_1/\pi_2) = 2 + \ln(0.25)/4 = 2 - 1.3863/4 = 2 - 0.3466 = 1.653$. The boundary moves towards the component with the smaller weight: a point must be closer to a less common component before that component becomes the more probable source.

3.11 With $\sigma = 1$ and equal weights, $r_{i1} = e^{-(x_i-1)^2/2} / \big(e^{-(x_i-1)^2/2} + e^{-(x_i-5)^2/2}\big)$. Divide the numerator and the denominator by the larger exponential. At $x = 0$ the squared distances are 1 and 25, so $r_{11} = 1/(1+e^{-12})$. At $x = 1$ they are 0 and 16, so $r_{21} = 1/(1+e^{-8})$. At $x = 5$ they are 16 and 0, so $r_{31} = e^{-8}/(1+e^{-8})$. At $x = 6$ they are 25 and 1, so $r_{41} = e^{-12}/(1+e^{-12})$. To four decimals, the responsibilities of the first and second components are $x = 0$: 1.0000 and 0.0000; $x = 1$: 0.9997 and 0.0003; $x = 5$: 0.0003 and 0.9997; $x = 6$: 0.0000 and 1.0000. Since $\frac{1}{1+e^{-12}} + \frac{e^{-12}}{1+e^{-12}} = 1$, and likewise with e^{-8}, $\sum_i r_{i1} = 2$ exactly and $\sum_i r_{i2} = 4 - 2 = 2$. Then $\sum_i r_{i1} x_i = 0 \times 0.999994 + 1 \times 0.999665 + 5 \times 0.000335 + 6 \times 0.000006 = 1.00138$, and $\sum_i r_{i2} x_i = (0+1+5+6) - 1.00138 = 10.99862$. The new means are the weighted averages $1.00138/2 = 0.501$ and $10.99862/2 = 5.499$; with clusters this well separated, EM behaves almost exactly like K-means.

3.12 Within its cluster the point 4 has distances 4 and 3, so $a = 3.5$; to the other cluster, 1 and 2, so $b = 1.5$. Then $s = (1.5 - 3.5)/3.5 = -0.571$. The negative sign shows that the point is closer, on average, to the other cluster: it is probably misassigned.

3.13 First $\{0,1\}$ at height 1. The average distance from $\{0,1\}$ to 3 is $(3+2)/2 = 2.5$, from $\{0,1\}$ to 7 is 6.5, and from 3 to 7 is 4, so $\{0,1,3\}$ merges at 2.5. Finally the average distance from $\{0,1,3\}$ to 7 is $(7+6+4)/3 = 5.667$. The heights lie between those of single and complete linkage.

3.14 Write $d_j = \|\mathbf{x}_i - \boldsymbol{\mu}_j\|$. With equal weights and a common variance, every component density has the form $(2\pi\sigma^2)^{-d/2} e^{-d_j^2/(2\sigma^2)}$, so the weights and the factor $(2\pi\sigma^2)^{-d/2}$ cancel from Bayes' rule, leaving $r_{ik} = e^{-d_k^2/(2\sigma^2)} / \sum_j e^{-d_j^2/(2\sigma^2)}$. Divide the numerator and the denominator by $e^{-d_k^2/(2\sigma^2)}$ and use $e^{-d_j^2/(2\sigma^2)} / e^{-d_k^2/(2\sigma^2)} = e^{-(d_j^2 - d_k^2)/(2\sigma^2)}$. The responsibility is then $r_{ik} = \big(1 + \sum_{j \neq k} \exp(-(d_j^2 - d_k^2)/2\sigma^2)\big)^{-1}$. If $d_k < d_j$ for every $j \neq k$, each exponent tends to $-\infty$ as $\sigma \to 0$, so $r_{ik} \to 1$. If some $d_j < d_k$, the corresponding term tends to $+\infty$, so $r_{ik} \to 0$.

3.15 Consider the K cyclic renamings $c \mapsto c + s \pmod K$, $s = 0, \ldots, K-1$. Each point agrees under exactly one of them (the one that maps its first label to its second), so the agreements under the K renamings sum to n. One of them therefore has agreement at least n/K, and the best renaming has agreement at least $1/K$.

Chapter 4

4.1 (a) $\operatorname{tr} S = 7$, $\det S = 6$, so $\lambda^2 - 7\lambda + 6 = 0$ and $\lambda_1 = 6$, $\lambda_2 = 1$. From $(5-6)a + 2b = 0$, $b = a/2$, so $\mathbf{v}_1$ is a multiple of $(2,1)$, whose length is $\sqrt{2^2+1^2} = \sqrt{5}$; hence $\mathbf{v}_1 = (2,1)/\sqrt{5}$, and $\mathbf{v}_2 = (1,-2)/\sqrt{5}$, perpendicular to it since $2\times 1 + 1\times(-2) = 0$. (b) $\lambda_1/(\lambda_1+\lambda_2) = 6/7 = 85.7\%$. (c) The score is $\mathbf{v}_1^\top(1,1) = (2\times 1 + 1\times 1)/\sqrt{5} = 3/\sqrt{5} \approx 1.342$. The second score is $\mathbf{v}_2^\top(1,1) = (1\times 1 - 2\times 1)/\sqrt{5} = -1/\sqrt{5}$, and the squared reconstruction error is its square (Section 4.2.3, *Scores and reconstruction error*); as a check, $\|(1,1)\|^2 - (3/\sqrt{5})^2 = 2 - 9/5 = 1/5$. The squared reconstruction error is $\big((1-2)/\sqrt{5}\big)^2 = 0.200$.

4.2 The scores have mean zero, since $\sum_i (\mathbf{x}_i - \bar{\mathbf{x}}) = \mathbf{0}$, so their variance is $\frac{1}{n}\sum_i (\mathbf{u}^\top(\mathbf{x}_i - \bar{\mathbf{x}}))^2 = \frac{1}{n}\sum_i \mathbf{u}^\top(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top\mathbf{u} = \mathbf{u}^\top S\mathbf{u}$. For principal directions $\mathbf{v}_j \neq \mathbf{v}_k$, the covariance of the scores is $\frac{1}{n}\sum_i z_{ij}z_{ik} = \mathbf{v}_j^\top S\mathbf{v}_k = \lambda_k \mathbf{v}_j^\top\mathbf{v}_k = 0$.

4.3 $\operatorname{tr} S = \operatorname{tr}\sum_j \lambda_j \mathbf{v}_j\mathbf{v}_j^\top = \sum_j \lambda_j \operatorname{tr}(\mathbf{v}_j\mathbf{v}_j^\top) = \sum_j \lambda_j \|\mathbf{v}_j\|^2 = \sum_j \lambda_j$. The proportion of variance explained is therefore the share of the total variance, summed over the original features, that the retained components account for.

4.4 $\lambda_j = s_j^2/n$ gives eigenvalues $2.00, 0.50, 0.02$. The proportions explained are $79.4, 19.8, 0.8$ per cent. Two components explain $99.2\% \geq 95\%$, so $k = 2$.

4.5 The projection of $\mathbf{v}_i$ onto the span of the orthonormal $\mathbf{u}_1, \ldots, \mathbf{u}_k$ is $\sum_j (\mathbf{u}_j^\top\mathbf{v}_i)\mathbf{u}_j$, with squared length $\sum_j (\mathbf{u}_j^\top\mathbf{v}_i)^2 = w_i$. By Pythagoras, $\|\mathbf{v}_i\|^2$ equals this squared length plus the squared length of the perpendicular remainder, so $w_i \leq \|\mathbf{v}_i\|^2 = 1$. Clearly $w_i \geq 0$.

4.6 $p_{1|0} = e^{-1/2}/(e^{-1/2} + e^{-9/2}) = 0.982$ and $p_{3|0} = 0.018$. The Gaussian kernel falls by a factor of $0.607/0.0111 = 54.6$ between distances 1 and 3; the t kernel falls by $0.5/0.1 = 5.0$. Because the t kernel decays slowly, pairs that are moderately similar in the data can be placed well apart in the map at little cost. This leaves room in the plane and relieves crowding.

4.7 The megabyte feature has a variance millions of times larger, so the first component is essentially the memory axis, whatever the structure. Standardise the features and recompute the eigenvalues before drawing any conclusion about dimension (Section 4.3.2, *The effect of scaling on PCA*).

4.8 With all components, the scores are $V^\top(\mathbf{x} - \bar{\mathbf{x}})$ with V orthogonal. Here $V = [\mathbf{v}_1 \ \cdots \ \mathbf{v}_d]$ has orthonormal columns forming a basis, so $VV^\top\mathbf{x} = \sum_{j=1}^d \mathbf{v}_j(\mathbf{v}_j^\top\mathbf{x}) = \mathbf{x}$ for every $\mathbf{x}$ (the expansion of $\mathbf{x}$ in that basis), that is $VV^\top = I$; and $V^\top V = I$ because its (j,l) entry is $\mathbf{v}_j^\top\mathbf{v}_l$. For two points $\mathbf{a}$ and $\mathbf{b}$, the mean cancels in the difference of their scores, $V^\top(\mathbf{a} - \bar{\mathbf{x}}) - V^\top(\mathbf{b} - \bar{\mathbf{x}}) = V^\top(\mathbf{a} - \mathbf{b})$, and

$$\|V^\top(\mathbf{a}-\mathbf{b})\|^2 = \big(V^\top(\mathbf{a}-\mathbf{b})\big)^\top V^\top(\mathbf{a}-\mathbf{b}) = (\mathbf{a}-\mathbf{b})^\top VV^\top(\mathbf{a}-\mathbf{b}) = (\mathbf{a}-\mathbf{b})^\top(\mathbf{a}-\mathbf{b}) = \|\mathbf{a}-\mathbf{b}\|^2,$$

so all distances are preserved. Taking determinants in $V^\top V = I$ gives $(\det V)^2 = 1$, so $\det V = \pm 1$: the change of coordinates is a rotation when $\det V = 1$ and a rotation combined with a reflection when $\det V = -1$.

4.9 The map shows that these customers are neighbours of one another, and are not neighbours of the others. It does not show how varied they are or how different, since neither cluster size nor between-cluster distance is preserved (Section 4.4.1, *t-SNE*). To check, compute the spread of the cluster and its distances to the other clusters in the original feature space, and confirm that the cluster is stable across random seeds and perplexities.

4.10 The mean is $(3,3)$; the deviations are $(-2,-1), (0,1), (2,0), (0,0)$. Then $S_{11} = 2.00$, $S_{12} = 0.50$, $S_{22} = 0.50$, so $\operatorname{tr} S = 2.5$ and $\det S = 0.75$, and $\lambda = \frac{1}{2}(2.5 \pm 1.803)$, giving 2.151 and 0.349. The first component explains 86.1%.

4.11 The cumulative proportions are $50.00, 75.00, 87.50, 93.75, 100.00$ per cent. Two components explain 75%. Three explain 87.5%, so four are needed for 90%.

4.12 In the orthonormal basis $\mathbf{v}_1, \dots, \mathbf{v}_d$, $\mathbf{x}_i - \bar{\mathbf{x}} = \sum_j z_{ij}\mathbf{v}_j$ and $\hat{\mathbf{x}}_i - \bar{\mathbf{x}} = \sum_{j \le k} z_{ij}\mathbf{v}_j$, so $\|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \sum_{j>k} z_{ij}^2$ by Pythagoras. Averaging over i, and using that the variance of the j-th score is λ_j, the mean squared error is $\sum_{j>k} \lambda_j$. For the eigenvalues $4, 2, 1, 0.5, 0.5$ and $k = 2$ it is $1 + 0.5 + 0.5 = 2$, which is $2/8 = 25\%$ of the total variance 8.

4.13 $S(1,1)^\top = (a+b)(1,1)^\top$ and $S(1,-1)^\top = (a-b)(1,-1)^\top$. Since $b > 0$, $a + b$ is the larger eigenvalue, and the first principal direction is $(1,1)/\sqrt{2}$ for every such a and b.

4.14 Before rescaling, Problem 4.13 gives eigenvalues 1.5 and 0.5 and first direction $(1,1)/\sqrt{2}$, at 45°. After rescaling, the deviations become $D\tilde{\mathbf{x}}_i$ with $D = \mathrm{diag}(10, 1)$, so the covariance becomes $\frac{1}{n}\sum_i D\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^\top D = DSD$: the variance of the first feature is multiplied by $10^2 = 100$, the covariance by 10, and the variance of the second is unchanged, giving $S = \begin{pmatrix} 100 & 5 \\ 5 & 1 \end{pmatrix}$. Here $\mathrm{tr}\, S = 101$ and $\det S = 100 - 25 = 75$, so $\lambda = \frac{1}{2}\big(101 \pm \sqrt{101^2 - 4 \times 75}\big) = \frac{1}{2}(101 \pm \sqrt{9901}) = \frac{1}{2}(101 \pm 99.504)$, giving eigenvalues 100.252 and 0.748. The first row of $(S - \lambda_1 I)\mathbf{v} = 0$ gives $(100 - 100.252)a + 5b = 0$, so $b/a = 0.252/5 = 0.0504$; dividing $(1,\ 0.0504)$ by its length $\sqrt{1 + 0.0504^2} = 1.0013$ gives the first direction $(0.9987,\ 0.0503)$, with $\tan\theta = 0.0504$, at 2.88°: almost the first axis. With $D = \mathrm{diag}(10, 1)$, rescaling changes S to DSD, whose eigenvectors are not D times the old ones: PCA maximises a variance, and variances depend on units. For example, $D(1,1)^\top = (10,1)^\top$, but $DSD\,(10,1)^\top = (100 \times 10 + 5 \times 1,\ 5 \times 10 + 1 \times 1)^\top = (1005,\ 51)^\top$, which is not a multiple of $(10,1)^\top$, since $1005/10 = 100.5$ while $51/1 = 51$. By contrast, the inverse of a product is the product of the inverses in reverse order, $(DSD)^{-1} = D^{-1}S^{-1}D^{-1}$, and $D^\top = D$, so

$$(D\mathbf{x})^\top (DSD)^{-1}(D\mathbf{x}) = \mathbf{x}^\top D\, D^{-1}S^{-1}D^{-1}D\,\mathbf{x} = \mathbf{x}^\top S^{-1}\mathbf{x},$$

and the Mahalanobis distance is unchanged (Section 5.3, *Two features: distance, shape and neighbours*).

4.15 With $p_{j|i} = 1/k$ on k points, $H_i = -\sum_j \frac{1}{k}\log_2\frac{1}{k} = \log_2 k$, so the perplexity is $2^{H_i} = k$. For a non-uniform distribution, 2^{H_i} is the number of equally likely neighbours that would have the same entropy: an effective number of neighbours.

Chapter 5

5.1 (a) The mean is $(4 + 5 + 5 + 6 + 30)/5 = 50/5 = 10.0$. The deviations are $-6, -5, -5, -4, 20$, with squares $36, 25, 25, 16, 400$ summing to 502, so the variance is $502/5 = 100.4$ and the standard deviation is $\sqrt{100.4} = 10.020$. The standard score of 30 is $(30 - 10)/10.020 = 1.996$. (b) The median of the sorted values $4, 5, 5, 6, 30$ is the third, 5. The absolute deviations $1, 0, 0, 1, 25$, sorted $0, 0, 1, 1, 25$, have median 1, so the MAD is 1 and the robust score is $0.6745 \times 25/1 = 16.86$. (c) With $n = 5$, Samuelson's inequality bounds every standard score by $\sqrt{4} = 2$, however large the value 30 becomes, because the value inflates the standard deviation. The median and MAD are not inflated.

5.2 $S^{-1} = \frac{1}{6}\begin{pmatrix} 2 & -2 \\ -2 & 5 \end{pmatrix}$, so $d_M^2 = (2x^2 - 4xy + 5y^2)/6$. Here $\det S = 5 \times 2 - 2 \times 2 = 6$, which with the inverse of a symmetric 2×2 matrix (Section 5.3, *Two features: distance, shape and neighbours*) gives S^{-1}. The quadratic form is $\frac{1}{6}\big(x(2x - 2y) + y(-2x + 5y)\big) = \frac{1}{6}(2x^2 - 4xy + 5y^2)$. Both points have Euclidean distance $\sqrt{2^2 + 1^2} = \sqrt{(-1)^2 + 2^2} = \sqrt{5} \approx 2.236$. For $\mathbf{a} = (2, 1)$: $d_M^2 = (2 \cdot 4 - 4 \cdot 2 \cdot 1 + 5 \cdot 1)/6 = (8 - 8 + 5)/6 = 5/6$, so $d_M = \sqrt{5/6} = 0.913$. For $\mathbf{b} = (-1, 2)$: $d_M^2 = \big(2 \cdot 1 - 4 \cdot (-1) \cdot 2 + 5 \cdot 4\big)/6 = (2 + 8 + 20)/6 = 5$, so $d_M = \sqrt{5} = 2.236$. The point $\mathbf{a}$ lies along $\mathbf{v}_1$ (variance 6) and $\mathbf{b}$ along $\mathbf{v}_2$ (variance 1); by the theorem of Section 5.3, *Two features: distance, shape and neighbours*, $d_M = \sqrt{5}/\sqrt{6}$ and $\sqrt{5}/1$ respectively.

5.3 20 is isolated if the cut falls in $(4, 20)$: probability $16/19 = 0.842$. 1 is isolated if the cut falls in $(1, 2)$: $1/19 = 0.053$. 3 cannot be isolated by one cut, because every cut leaves it together with 2 or with 4: probability 0.

5.4 Precision at 3 is $2/3 = 0.667$ and at 5 is $3/5 = 0.600$. The anomalies are at ranks 1, 3, 4 and 7, where the numbers of anomalies at or above them are $1, 2, 3, 4$, so the precisions are $1/1,\ 2/3,\ 3/4,\ 4/7$, that is, $1.000, 0.667, 0.750, 0.571$. Their mean, the average precision, is $\frac{1}{4}\big(1 + \frac{2}{3} + \frac{3}{4} + \frac{4}{7}\big) = \frac{1}{4} \cdot \frac{84+56+63+48}{84} = \frac{251}{336} = 0.747$.

5.5 Let each cluster contain n points, with means $\mathbf{m}_1$ and $\mathbf{m}_2$. The fitted mean of a single Gaussian is the average of all $2n$ points (Section 3.1.3, *The M-step: updating the means*, with every responsibility equal to 1), which is $\frac{1}{2n}(n\mathbf{m}_1 + n\mathbf{m}_2) = \frac{1}{2}(\mathbf{m}_1 + \mathbf{m}_2)$, the midpoint between the clusters. A Gaussian density with mean $\boldsymbol{\mu}$ and covariance Σ is a positive constant times $\exp\big(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\big)$. By the theorem of Section 5.3, *Two features: distance, shape and neighbours*, the quadratic form equals $\sum_j z_j^2/\lambda_j$, which is at least 0 and equals 0 only when every $z_j = 0$, that is, at $\mathbf{x} = \boldsymbol{\mu}$. So the density is largest at its mean. The highest density is therefore assigned to the empty region between the clusters, and points there look the most normal. A density score needs a model that fits the shape of the data, such as a mixture with one component per cluster.

5.6 By the definition of the threshold, about 5% of inputs exceed it when there is no drift. A day's fraction fluctuates around 5% by sampling variability (binomially, with standard deviation $\sqrt{0.05 \times 0.95/m}$ for m inputs), and for large m it is approximately normal with mean 0.05. A normal quantity exceeds its mean with probability $\frac{1}{2}$, so an alarm level of exactly 5% would fire on about half of all days. With $m = 1000$ inputs a day, for instance, the standard deviation is $\sqrt{0.05 \times 0.95/1000} = 0.0069$, and a level three standard deviations above 5% is $0.05 + 3 \times 0.0069 = 0.071$, about 7.1%. Set the level several standard deviations above 5%, or calibrate it on a period known to be free of drift, choosing the acceptable rate of false alarms first.

5.7 *Masking*: two extreme values inflate the standard deviation so that neither has a large standard score. *Swamping*: a large value shifts the mean, so that ordinary values at the other end acquire large negative scores. Most affected are scores built on the mean and the ordinary covariance (standard scores, Mahalanobis distance with the ordinary covariance). Least affected are the median and MAD. The isolation forest, trained on small sub-samples, resists single anomalies, but like nearest-neighbour scores it is masked by tight groups of anomalies, which are each other's neighbours (data set A of Section 5.5, *Choosing a method, and evaluating with few labels*).

5.8 Choose k on normal data, for example by the cumulative explained variance or by the reconstruction error on held-out normal examples. Set the threshold at a high quantile of the reconstruction errors of held-out normal examples, chosen for an acceptable false-alarm rate. The method misses anomalies that lie inside the retained subspace: an example far out along a principal direction has a small reconstruction error, as Q in Section 5.3, *Two features: distance, shape and neighbours* shows (distance 0.144 from the line). Combining the error with a score within the subspace, such as the Mahalanobis distance of the retained scores, covers this case.

5.9 The mean is 11. The deviations are -1 nineteen times and 19 once, so the variance is $(19 + 361)/20 = 19$ and $s = \sqrt{19}$. The standard score of 30 is $19/\sqrt{19} = \sqrt{19} \approx 4.359$, which equals Samuelson's bound $\sqrt{n-1}$: the bound is attained when all values but one are equal.

5.10 In one dimension $S = s^2$, so $d_M = \sqrt{(x - \bar{x})^2/s^2} = |x - \bar{x}|/s = |z|$.

5.11 $d_M(3, 0) = \sqrt{9/4} = 1.5$ and $d_M(0, 3) = \sqrt{9/1} = 3$. A step of 3 is one and a half standard deviations along the first axis, where the standard deviation is 2, but three standard deviations along the second.

5.12 $c(2) = 1$ by definition, and it also follows from $c(2) = c(1) + 2/2$ with $c(1) = 0$ (Section 5.4, *Isolation forest*). For $m = 256$, $\ln 255 = 5.5413$, so $c(256) = 2(5.5413 + 0.5772) - 2 \times 255/256 = 12.2370 - 1.9922 = 10.245$. A point isolated at mean depth 2 scores $2^{-2/10.245} = 2^{-0.1952} = 0.873$, clearly anomalous; one at depth 10 scores $2^{-10/10.245} = 2^{-0.9761} = 0.508$, close to 0.5 and unremarkable.

5.13 Both have precision at 3 equal to 2/3. For A the anomalies are at ranks 1, 2 and 8, so its average precision is $(1 + 1 + 3/8)/3 = 0.792$. For B they are at ranks 2, 3 and 4, so it is $(1/2 + 2/3 + 3/4)/3 = 0.639$. Average precision prefers A because it puts two anomalies first. But if four cases can be investigated, B finds all three (precision at 4 is 0.75 against 0.5 for A). The right comparison depends on how many flagged cases can be reviewed.

5.14 Write $\mathbf{d} = \mathbf{x} - \boldsymbol{\mu}$. The number $\mathbf{d}^\top \Sigma^{-1} \mathbf{d} = \sum_{j,k} d_k (\Sigma^{-1})_{kj} d_j$ is also the sum of the diagonal entries of the matrix $\Sigma^{-1} \mathbf{d}\mathbf{d}^\top$, whose (k, k) entry is $\sum_j (\Sigma^{-1})_{kj} d_j d_k$. So $\mathbf{d}^\top \Sigma^{-1} \mathbf{d} = \operatorname{tr}(\Sigma^{-1} \mathbf{d}\mathbf{d}^\top)$. The trace is a sum of entries and the expectation of a sum is the sum of the expectations, so the expectation can be taken inside the trace. The constant matrix Σ^{-1} can be taken outside the expectation, and $E[\mathbf{d}\mathbf{d}^\top] = \Sigma$ is the definition of the covariance matrix. Hence $E\big[(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})\big] = E\big[\operatorname{tr}\big(\Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top\big)\big] = \operatorname{tr}(\Sigma^{-1}\Sigma) = \operatorname{tr} I = d$. This holds for any distribution with covariance Σ, not only the normal. A typical normal point therefore has $d_M \approx \sqrt{d}$; with $d = 100$ that is about 10, so a threshold of 3 would flag almost everything. Thresholds must grow with d, for example quantiles of the χ^2_d distribution, which d_M^2 follows for normal data.

5.15 Let a of the n examples be anomalies. Under a uniformly random ranking, each position holds an anomaly with probability a/n, by symmetry. By linearity of expectation, the expected number of anomalies among the first k is ka/n, so the expected precision at k is a/n.

6.2 Common questions

Is K-means related to K-nearest neighbours?
Only in name. K-nearest neighbours is a supervised method that predicts a label from the labels of the K nearest training examples. In K-means, K is the number of clusters, and no labels are used.

Does K-means find the optimal partition?
Not necessarily; it converges to a fixed point (Section 2.5.1, *Dependence on initialisation*). Finding the partition with the smallest J is NP-hard in general, which is why multiple initialisations and K-means++ are standard practice.

How is a new example assigned to a cluster?
In K-means, to the cluster with the nearest centroid; in a Gaussian mixture, by computing its responsibilities. Hierarchical clustering provides no such rule directly; a common choice is the cluster whose points are nearest on average.

How are categorical features handled?
Squared distance requires numerical features. A categorical feature can be replaced by one binary indicator per category (one-hot encoding), or a method designed for categorical data can be used, such as K-modes, which replaces each mean by the most frequent category in the cluster.

Are there methods for clusters of arbitrary shape?
Density-based methods, of which DBSCAN is the best known, define clusters as regions of high point density; they can find clusters of arbitrary shape and label isolated points as noise. Gaussian mixtures with full covariance matrices handle elongated and oriented clusters.

Why does the log-likelihood never decrease under EM?
The E-step constructs a lower bound on the log-likelihood that equals it at the current parameters, and the M-step maximises this bound. The new log-likelihood is therefore at least as large as the

old one. The full proof, using Jensen's inequality, is in Section 3.1.5, *Why the log-likelihood never decreases (optional)*.

Should features always be standardised?
Usually, when they are measured in different units. Not when the units are already comparable and meaningful, as with pixel intensities, and not when some features are known to deserve more weight.

Is the elbow criterion reliable?
Often not, since many data sets show no sharp elbow. It is best used together with the silhouette, stability under resampling, and the purpose of the clustering.

Should dimension be reduced before clustering?
Often, yes. With many correlated or noisy features, distances are dominated by noise, and clustering the first few principal components (or a UMAP reduction to a few tens of dimensions, for embeddings) usually gives more stable clusters. Cluster in the reduced space, not on a two-dimensional visualisation map.

How does PCA differ from fitting a regression line?
Least squares minimises vertical distances to the line, treating one feature as the response. The first principal line minimises perpendicular distances (Section 4.2.3, *Scores and reconstruction error*) and treats all features symmetrically. The two lines generally differ; for the eight runs they do.

Is UMAP better than t-SNE?
For speed on large data and for placing new points, UMAP is usually more convenient. In the quality of the map the difference is small, and largely a matter of the balance between attraction and repulsion and of the initialisation (Section 4.4.2, *UMAP, PaCMAP, and how to read such maps*). Whichever is used, read only neighbourhoods from the map.

What proportion of anomalies should be assumed?
If it is unknown, it should not be fixed in advance. Choose the threshold from the number of flagged examples that can be investigated, and report how the number of flagged examples changes with the threshold.

Is an anomaly the same as an error?
No. An anomaly score finds what is unusual relative to a model of normal data. A flagged example may be an error, an attack, or a new and legitimate kind of input.

6.3 Glossary

Anomaly score A number assigned to each example, larger for examples less like normal data.

Average precision The mean, over known anomalies, of the precision among all examples scored at least as high.

Centroid The representative point of a cluster; in K-means, the mean of its points.

Cluster A group of examples that are similar to each other and dissimilar to examples in other groups.

Clustering The division of unlabelled examples into clusters.

Base rate The fraction of examples that are anomalies.

Core point In DBSCAN, a point with at least minPts points (itself included) within distance ε.

DBSCAN, HDBSCAN Density-based clustering with a fixed radius; its hierarchical extension over all density levels.

Dendrogram The tree recording the merges of hierarchical clustering, with the distance at which each merge occurred.

Drift A change in the distribution of a deployed model's inputs relative to its training data.

Elbow The value of K beyond which the smallest attainable J decreases only slowly.

EM algorithm Expectation–maximisation: alternation of the E-step (responsibilities) and the M-step (weighted estimates).

Embedding A vector representation of an example produced by a trained model.

Fixed point A partition that neither step of Lloyd's algorithm changes.

Gaussian mixture model A model in which each example is drawn from one of several normal distributions, the choice of distribution being unobserved.

Hierarchical clustering (agglomerative) Clustering by repeatedly merging the two closest clusters, starting from single points.

Isolation forest Anomaly detection by the average depth at which random cuts isolate an example.

K-means The problem of minimising J; also Lloyd's algorithm for it.

K-means++ Initialisation in which each new centroid is a data point chosen with probability proportional to its squared distance from the nearest existing centroid.

Linkage The definition of the distance between two clusters: single, complete or average.

Log-likelihood $\sum_i \log p(x_i)$, the logarithm of the probability density of the data under a model.

MAD Median absolute deviation from the median.

Mahalanobis distance $\sqrt{(\mathbf{x}-\bar{\mathbf{x}})^\top S^{-1}(\mathbf{x}-\bar{\mathbf{x}})}$; Euclidean distance after whitening.

Masking, swamping An anomaly hiding itself or others by inflating the spread; making normal points look unusual by shifting the centre.

Partition An assignment of each example to exactly one cluster.

Perplexity In t-SNE, the effective number of neighbours of each point, 2^{H_i}.

Precision at k The fraction of true anomalies among the k highest-scored examples.

Principal direction, principal component An eigenvector of the covariance matrix; the scores along it.

Proportion of variance explained $(\lambda_1 + \cdots + \lambda_k)/(\lambda_1 + \cdots + \lambda_d)$.

Reconstruction error The squared distance from an example to its reconstruction from k principal components.

Responsibility r_{ik} The probability that example i was generated by component k, given the current parameters.

Robust score $0.6745\,(x - \text{median})/\text{MAD}$, a standard score that uses the median and the MAD, which outliers hardly move.

Scree plot A plot of the eigenvalues in decreasing order.

Silhouette coefficient s_i $(b_i - a_i)/\max(a_i, b_i)$.

Singular value decomposition $X_c = U\Sigma V^\top$; the columns of V are the principal directions and $\lambda_j = s_j^2/n$.

Standard score $z = (x - \bar{x})/s$, the number of standard deviations from the mean.

Standardisation Subtraction of the mean and division by the standard deviation, feature by feature.

t-SNE, UMAP, PaCMAP Nonlinear neighbour-embedding methods that place points in the plane so that neighbours remain neighbours.

Unsupervised learning Learning from features without labels.

Whitening Rotating to principal coordinates and dividing each by its standard deviation.

Within-cluster sum of squares J $\sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2$.

6.4 Summary of formulas and results

Formulas

- Objective: $J = \sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2$, where $\|\mathbf{x} - \boldsymbol{\mu}\|^2 = \sum_j (x_j - \mu_j)^2$.
- Assignment step: $g_i = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$. Update step: $\boldsymbol{\mu}_k = \frac{1}{n_k} \sum_{i:\, g_i = k} \mathbf{x}_i$.
- J does not increase under either step; Lloyd's algorithm converges to a fixed point, not necessarily the global minimum.
- Boundary between centroids $\boldsymbol{\mu}_a, \boldsymbol{\mu}_b$: $2(\boldsymbol{\mu}_b - \boldsymbol{\mu}_a)^\top \mathbf{x} = \|\boldsymbol{\mu}_b\|^2 - \|\boldsymbol{\mu}_a\|^2$, the perpendicular bisector.
- $T = W + B$: $\sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2 + \sum_k n_k \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2$.
- K-means++: choose the next centroid with probability proportional to $D_i = \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$.
- Standardisation: $z = (x - \bar{x})/s$.
- Responsibility: $r_{ik} = \pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k) / \sum_j \pi_j \mathcal{N}(x_i \mid \mu_j, \sigma_j)$.
- M-step: $\pi_k = \frac{1}{n} \sum_i r_{ik}$, $\mu_k = \sum_i r_{ik} x_i / \sum_i r_{ik}$, $\sigma_k^2 = \sum_i r_{ik} (x_i - \mu_k)^2 / \sum_i r_{ik}$.
- Ward's merge cost: $\Delta(C_1, C_2) = \frac{n_1 n_2}{n_1 + n_2} \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2$. Lance–Williams: $\delta(i \cup j, k) = \alpha_i \delta(i,k) + \alpha_j \delta(j,k) + \beta \delta(i,j) + \gamma |\delta(i,k) - \delta(j,k)|$.
- DBSCAN: a point is a core point when its distance to its (minPts $-$ 1)-th nearest other point is at most ε.
- Silhouette coefficient: $s_i = (b_i - a_i) / \max(a_i, b_i)$.
- PCA: maximise $\mathbf{u}^\top S \mathbf{u}$ subject to $\|\mathbf{u}\| = 1$ gives $S\mathbf{u} = \lambda \mathbf{u}$; k components retain $\lambda_1 + \cdots + \lambda_k$; reconstruction error $n(\lambda_{k+1} + \cdots + \lambda_d)$; $\lambda_j = s_j^2 / n$ for the singular values s_j of the centred data; scores uncorrelated, $V_k^\top S V_k = \mathrm{diag}(\lambda_1, \ldots, \lambda_k)$.
- t-SNE: $p_{j|i} \propto e^{-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2}$, $q_{ij} \propto (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$, minimise $\sum p_{ij} \log(p_{ij}/q_{ij})$.
- Samuelson: $|x_i - \bar{x}| \le s\sqrt{n-1}$, with s computed with $1/n$. Robust score: $0.6745\,(x - \text{median})/\text{MAD}$.
- Mahalanobis: $d_M^2 = (\mathbf{x} - \bar{\mathbf{x}})^\top S^{-1} (\mathbf{x} - \bar{\mathbf{x}}) = \sum_j z_j^2 / \lambda_j$, unchanged by $\mathbf{x} \mapsto A\mathbf{x} + \mathbf{b}$ with A invertible.
- Isolation forest: $s(x) = 2^{-\bar{h}(x)/c(m)}$, $c(m) = 2(\ln(m-1) + 0.5772) - 2(m-1)/m$ for $m > 2$, $c(2) = 1$, $c(1) = 0$, with m the number of points each tree is built on.

Key numbers from the examples

- Eight runs: $J = 100.875$ for one cluster, 99.25 for a mixed split, 15.25 for the small and large runs (centroids (2.75, 3.25) and (7.50, 7.75)).
- Lloyd's algorithm from $(2, 3)$ and $(1, 3)$: $J = 228.00,\ 51.22,\ 20.40,\ 15.25$ over four iterations.
- Four points at the corners of a rectangle: fixed points with $J = 4$ and $J = 16$.

- Hours against megabytes: agreement 51% on raw features and 100% after standardisation.
- Latencies, $\sigma = 4$, initial means 15 and 25: $r_{22,\mathrm{fast}} = 0.223$; means after one M-step 12.69 and 30.85; EM converges to 14.2 and 32.8; K-means stops at 12.0 and 30.5.
- Eight runs: $T = 100.875 = W + B = 15.25 + 85.625$; boundary $9.5x_1 + 9x_2 = 98.1875$; Ward cost of the last merge 85.625.
- Average-linkage merge heights: 1, 1, 1, 2.118, 2.600, 2.609, 6.682.
- Silhouette coefficient of run 1: 0.594; mean over the eight runs: 0.673.
- $K = 3$ on uniform and on three-cluster data: mean silhouette 0.411 and 0.773; reduction in J 64% and 93%; stability 0.84 and 1.00.
- PCA of the eight runs: $\lambda_1 = 11.770$, $\lambda_2 = 0.839$ (93.3% explained), $\mathbf{v}_1 = (0.7421, 0.6703)$ at 42.1°; total sum of squares $100.875 = n(\lambda_1 + \lambda_2)$; reconstruction error $6.712 = n\lambda_2$.
- Slow request (95 ms): standard score 2.464 (masked), robust score 4.20, isolation score 0.753.
- New runs $P = (2, 8)$, $Q = (11, 11)$: Euclidean 4.002, 8.048; Mahalanobis 4.316, 2.351.
- Evaluation of anomaly scores: Mahalanobis best on one elongated cloud (precision 0.70); fifth-neighbour distance best on two clusters (precision 0.90).